代码随想录知识星球精华（最强八股文）第五版（Java篇））

代码随想录知识星球精华（最强八股文）第五版为九份PDF，分别是：

- 代码随想录知识星球八股文概述
- C++篇
- go篇
- Java篇
- 前端篇
- 算法题篇
- 计算机基础篇
- 问答精华篇
- 面经篇

本篇为最强八股文之Java篇。

面试题

> 绝大部分内类来自 代码随想录知识星球 ，部分参考有 JavaGuide、 牛客网，还有一些博客资料，标注如有疏漏，敬请谅解。

JavaSE

Java概述

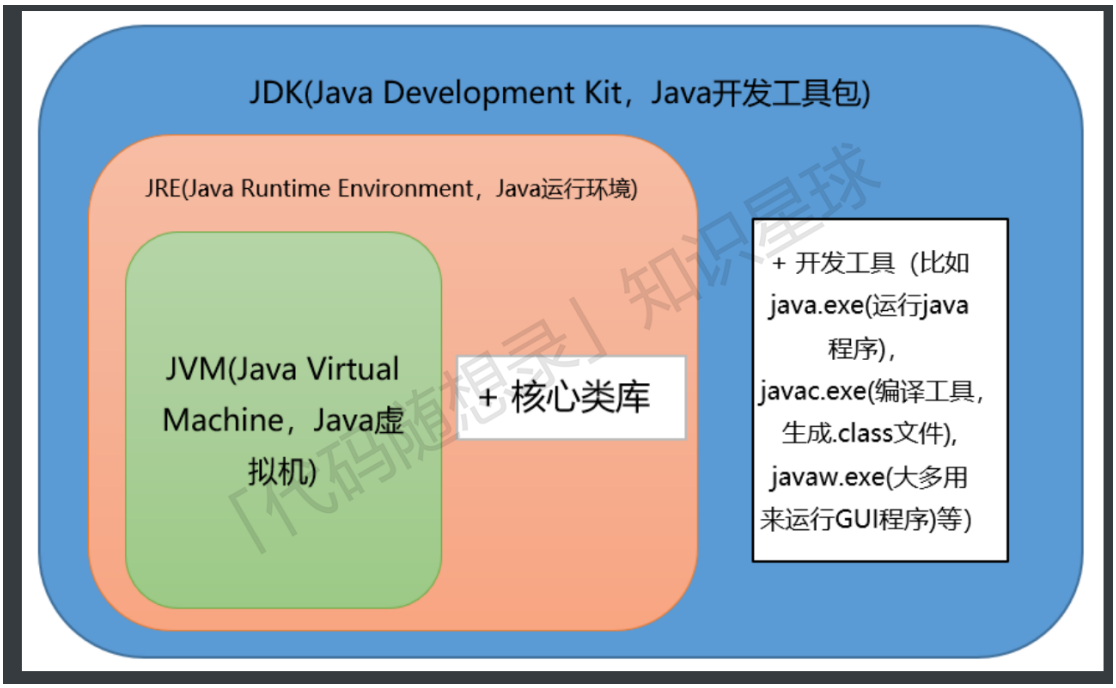
1、JVM、JRE、JDK之间的关系

JVM负责将Java字节码翻译成机器码，以便在特定平台上执行Java程序。

JDK是Java的开发工具包，它包含了开发和编译Java程序所需的工具，以及运行Java程序的JRE

JRE是Java应用程序运行的环境，它包含了运行Java程序所需的库、Java虚拟机（JVM）以及其他支持文件。

简而言之，JVM提供了Java程序运行的虚拟环境，JDK是用于开发Java程序的工具包，而JRE是用于运行Java程序的环境。在开发阶段，你需要JDK；而在部署和运行阶段，你只需要JRE。



Java基础数据类型

Java有几种数据类型

- 整型: byte 、short、int、long (int默认)
- 浮点型: float 、double (double默认)
- 布尔型: boolean
- 字符型: char

对应的包装类为`Byte`、`Short`、`Integer`、`Long`、`Float`、`Double`、`Character`、`Boolean`

基础数据类型的转换方式

- 自动类型转换:小–>大 byte --> short --> int --> long --> float --> double
- 强制类型转换:大–>小 小类型 变量名 = (大类型) 值

注意:自增/自减运算符、复合赋值运算符底层做了优化，内部自动强制类型转换; 如:++, --, +=, -=, ...

基本类型和包装类型的区别

为了在对象环境中使用基本类型，Java提供了对应的包装类型。

- 基本类型直接存储数据值，在栈上分配内存，占用固定的内存空间，
- 包装类型是对象动态分配内存，在堆上创建，包含额外的信息，例如方法和字段，同时也占用更多的内存空间。

不过Java提供了自动装箱（autoboxing）和拆箱（unboxing）的功能，使得基本类型和包装类型之间的转换更加方便。

什么是自动装箱和自动拆箱

自动装箱是指将基本类型转换为对应的包装类型，而自动拆箱是指将包装类型转换为对应的基本类型。这些转换通常由编译器自动处理。

```
// 自动装箱
Integer intValue = 42; // 将基本类型int自动转换为Integer包装类型

// 自动拆箱
int primitiveValue = intValue; // 将Integer包装类型自动转换为基本类型int
```

装箱其实就是调用了 包装类的`valueOf()`方法，拆箱其实就是调用了 `xxxValue()`方法。

注意：**所有整型包装类对象之间值的比较，全部使用 equals 方法比较。**

变量和方法

成员变量和局部变量的区别

1. 声明位置：

- **成员变量：** 声明在类中，但在方法、构造函数或块之外。它们是类的一部分，因此可以被整个类的方法访问。
- **局部变量：** 声明在方法、构造函数或块中。它们只在声明它们的方法、构造函数或块中可见。

```
public class Example {
    // 成员变量
    private int memberVariable;
    public void exampleMethod() {
        // 局部变量
        int localVar = 10;
        // ...
    }
}
```

2. 作用域

- **成员变量：** 具有类的整个实例的生命周期，可以在整个类中使用。
- **局部变量：** 仅在声明它们的方法、构造函数或块中可见，并且只在该方法、构造函数或块的执行期间存在。

3. 生命周期

- **成员变量：** 随着对象的创建而创建，随着对象的销毁而销毁。
- **局部变量：** 在声明时分配内存，在方法调用结束时销毁。

4. 默认值

- **成员变量：** 如果不显式初始化，会有默认值（例如，整数默认为0，对象引用默认为null等）。
- **局部变量：** 不会自动初始化，必须在使用之前显式初始化。

静态变量是什么?

静态变量是被 `static` 关键字修饰的变量，被类的所有实例共享，无论一个类创建了多少个对象，它们都共享同一份静态变量。静态变量通过类名来访问。

值传递和引用传递的区别

参数传递方式主要有值传递和引用传递两种，但需要注意的是 Java 中的参数传递是始终按值传递的。

- 值传递：在值传递中，传递给函数的是实际参数的值的副本。当在函数内修改这个副本时，不会影响到原始值
- 引用传递：方法接收的直接是实参所引用的对象在堆中的地址，不会创建副本，对形参的修改将影响到实参。

但在 Java 中，虽然传递的是引用的值（对象的地址），但仍然是按值传递。实际上，传递的是引用的副本，因此在函数内对引用的修改会影响到原始的引用，但无法修改引用指向的对象。

类和对象

面向对象和面向过程有什么区别

面向对象编程（OOP）和面向过程编程是两种不同的编程范式。

- 面向对象会先抽象出具有状态、行为的对象，然后用对象执行方法的方式解决问题，强调封装、继承和多态，更容易扩展和维护，因为修改一个对象不会影响到其他对象，适合处理复杂的系统。
- 面向过程将系统视为一系列的过程或函数，通过调用这些过程来完成任务。强调的是算法和流程， 如果需要修改一个过程，可能会影响到调用这个过程的其他地方，更适合简单的、线性的任务。

面向对象的三大特征

1. 封装

为了提高代码的安全性，隐藏对象的内部细节，封装将对象的内部状态（字段、属性）隐藏起来，并通过定义公共的方法（接口）来操作对象，外部代码只需要知道如何使用这些方法而无需了解内部实现。

2. 继承

允许一个类（子类）继承另一个类（父类）的属性和方法的机制。子类可以重用父类的代码，并且可以通过添加新的方法或修改（重写）已有的方法来扩展或改进功能，提高了代码的可重用性和可扩展性。

3. 多态

多态是指相同的操作或方法可以在不同的对象上产生不同的行为，通过方法的重载和重写实现。多态允许以一致的方式处理不同类型的对象，提高了代码的灵活性。

说一说你对多态的理解

因为子类其实是一种特殊的父类，因此Java允许把一个子类对象直接赋给一个父类引用变量，无须任何类型转换，或者被称为向上转型，向上转型由系统自动完成。

当把一个子类对象直接赋给父类引用变量时，例如`BaseClass obj = new SubClass();`，这个`obj`引用变量的编译时类型是`BaseClass`，而运行时类型是`SubClass`，当运行时调用该引用变量的方法时，其方法行为总是表现出子类方法的行为特征，而不是父类方法的行为特征，这就可能出现：相同类型的变量、调用同一个方法时呈现出多种不同的行为特征，这就是多态。

编译时多态和运行时多态的区别

1. 编译时多态

编译时多态也被称为静态多态或早期绑定。它是指在编译阶段，编译器就能够确定调用哪个方法，这是通过方法的重载来实现的。编译时多态主要依赖于方法的签名（方法名和参数列表），根据调用时提供的参数类型和数量来确定具体调用的方法。

```
public class Example {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public static void main(String[] args) {
        Example example = new Example();
        int resultInt = example.add(2, 3);           // 编译时确定调用add(int, int)
        double resultDouble = example.add(2.0, 3.0); // 编译时确定调用add(double, double)
    }
}
```

2. 运行时多态

运行时多态也被称为动态多态或晚期绑定。指的是在程序运行时，根据实际对象的类型来确定调用的方法，这是通过方法的重写来实现的。运行时多态主要依赖于对象的实际类型，而不是引用类型。

```
class Animal {
    void makeSound() {
        System.out.println("sound");
    }
}
class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Bark");
    }
}
public class PolymorphismExample {
    public static void main(String[] args) {
        Animal myDog = new Dog(); // 运行时多态，myDog的实际类型是Dog
        myDog.makeSound(); // 运行时确定调用Dog类的makeSound方法
    }
}
```

3. 总结

- 编译时多态是通过方法的重载在编译阶段确定调用的方法。
- 运行时多态是通过方法的重写在程序运行时确定调用的方法，实现动态绑定。

接口和抽象类有什么区别

1. 从定义上来说

- 接口是一种抽象类型，它定义了一组方法（方法签名）但没有实现任何方法的具体代码。接口中的方法默认是抽象的，且接口中只能包含常量（static final变量）和抽象方法。
- 抽象类是一个类，可以包含抽象方法和具体方法。抽象类中的抽象方法是没有实现的方法，而具体方法则包含实现代码。抽象类不能直接实例化，通常需要子类继承并实现其中的抽象方法。

2. 继承

- 接口支持多继承，一个类可以实现多个接口
- Java中不支持多继承，一个类只能继承一个抽象类。如果一个类已经继承了一个抽象类，就不能再继承其他类。

3. 构造器

- 接口不能包含构造器，因为接口不能被实例化。类实现接口时，必须实现接口中定义的所有方法。
- 抽象类可以包含构造器，用于初始化抽象类的实例。当子类实例化时，会调用父类的构造器。

4. 访问修饰符

- 接口中的方法默认是`public abstract`的。接口中的变量默认是`public static final`的。
- 抽象类中的抽象方法默认是`protected`的，具体方法的访问修饰符可以是`public`、`protected`或`private`。

5. 实现限制

- 类可以同时实现多个接口，实现接口的类必须提供接口中定义的所有方法。
- 一个类只能继承一个抽象类，继承抽象类的子类必须提供抽象类中定义的所有抽象方法的实现。

构造方法能不能重写

构造方法不能重写。因为构造方法需要和类保持同名，而重写的要求是子类方法要和父类方法保持同名。如果允许重写构造方法的话，那么子类中将会存在与类名不同的构造方法，这与构造方法的要求是矛盾的。

说一说你对Java访问权限的了解

在Java中，访问权限通过访问修饰符来控制。主要的访问修饰符有四个：`public`、`protected`、`default`（默认修饰符）和`private`。这些修饰符可以用于类、变量、方法和构造方法。

- 修饰符`public`表示对所有类可见。一个类如果被声明为`public`，那么它可以被其他任何类访问。
- 修饰符`protected`表示对同一包内的类和所有子类可见。子类可以访问父类中声明为`protected`的成员，而不管子类与父类是否在同一包中。
- 如果没有使用任何访问修饰符（即没有写`public`、`protected`、`private`），则默认为包级别访问。这意味着只有同一包中的类可以访问。
- 修饰符`private`表示对同一类内可见。私有成员只能在声明它们的类中访问。

修饰符	当前类	同一包内	子孙类(同一包)	子孙类(不同包)	其他包
public	Y	Y	Y	Y	Y
protected	Y	Y	Y	Y/N（说明）	N
default	Y	Y	Y	N	N
private	Y	N	N	N	N

「代码随想录」知识星球

你对static关键字有哪些认识

`static` 是Java中的关键字，用于声明静态成员（静态变量或静态方法）

1. 静态变量

使用 `static` 关键字声明的变量称为静态变量，也叫类变量。它们属于类而不是实例，因此所有实例共享相同的静态变量。

2. 静态方法

使用 `static` 关键字声明的方法称为静态方法。静态方法属于类而不属于实例，可以通过类名调用，而不需要创建类的实例。

3. 静态代码块

使用 `static` 关键字声明的代码块称为静态代码块。它在类加载时执行，并且只执行一次。

4. 静态内部类

在类中使用 `static` 关键字声明的内部类称为静态内部类

静态成员可以被类直接访问，而不需要创建类的实例。然而，静态成员无法直接访问非静态成员，因为非静态成员依赖于类的实例。

static和final有什么区别

- `static` 用于修饰成员时，该成员成为类级别的，而不是实例级别的。静态成员属于类，而不是属于类的实例。

```
public class MyClass {
    static int staticVariable; // 静态变量
    static void staticMethod() { // 静态方法
        // ...
    }
}
int value = MyClass.staticVariable;
MyClass.staticMethod();
```

- 当 `final` 用于修饰变量、方法或类时，表示它是不可变的。对于变量，一旦赋值后就不能再修改；对于方法，表示方法不能被子类重写；对于类，表示类不能被继承。

```
public class Example {
    final int constantValue = 42; // 不可变的变量
    final void finalMethod() { // 不可变的方法
        // ...
    }
}

final class FinalClass { // 不可变的类
    // ...
}
```

final、finally、finalize的区别

（1）final就是不可变的意思，可以修饰变量、方法和类。修饰变量时，这个变量必须初始化，所以也称为常量。

（2）finally是异常处理的一部分，只能用在try/catch中，并且附带一个语句块表示这段语句一定会被执行，无论是否抛出异常。

（3）finalize是java.lang.Object中的方法，也就是每一个对象都有这个方法，一个对象的finalize方法只会调用一次，调用了不一定被回收，因为只有对象被回收的时候才会被回收，就会导致前面调用，后面回收的时候出现问题，不推荐使用。

方法的重载和重写有什么区别

1. 重载

方法的重载指的是在同一个类中，可以有多个方法具有相同的名称，但是它们的参数列表不同（参数的类型、个数、顺序），具有以下特点。

- 方法的返回类型可以相同也可以不同。
- 方法的重载与方法的访问修饰符和返回类型无关。
- 编译器根据方法的参数列表来区分不同的重载方法

2. 重写

方法的重写指的是在子类中重新定义父类中已经定义的方法，方法名、参数列表和返回类型都必须相同。

- 重写是实现多态的一种方式，提供了一种子类可以提供自己特定实现的机制。
- 重写的方法不能比被重写的方法有更低的访问权限。
- 子类方法不能抛出比父类方法更多的异常（子类方法可以不抛出异常或抛出父类异常的子类）。

3. 总结

- 重载是在同一个类中定义多个方法，方法名相同但参数列表不同。
- 重写是在子类中重新定义父类中已有的方法，方法名和参数列表必须相同。
- 重载与返回类型和访问修饰符无关，而重写要求方法签名相同。
- 重载是编译时多态，而重写是运行时多态。

> - 重载(overloading) 是在一个类里面，方法名字相同，而参数不同。返回类型可以相同也可以不同。每个重载的方法（或者构造函数）都必须有一个独一无二的参数类型列表。最常用的地方就是构造器的重载。

- 重写发生在子类与父类之间，重写方法返回值和形参都不能改变，与方法返回值和访问修饰符无关，即重载的方法不能根据返回类型进行区分。**即外壳不变，核心重写！**

深拷贝和浅拷贝有什么区别

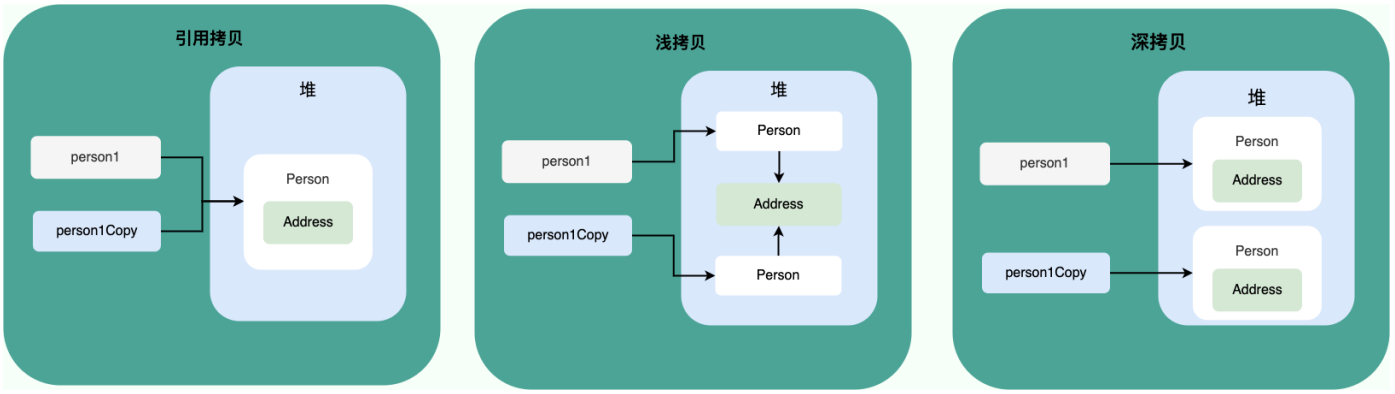
- 浅拷贝：浅拷贝创建一个新对象，然后将原对象的非静态字段复制到新对象。如果字段是基本数据类型，那么就复制其值；如果字段是引用类型，复制的就是引用而不是实际对象。
- 深拷贝：创建一个新对象，并递归复制原对象中的所有引用类型的字段指向的对象，而不是共享引用。因此，新对象和原对象中的引用类型字段引用的是两组不同的对象。

```
// 浅拷贝
class Person implements Cloneable {
    String name;
    Address address;

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

class Address {
    String city;
}

public class ShallowCopyExample {
    public static void main(String[] args) throws CloneNotSupportedException {
        Person person1 = new Person();
        person1.name = "John";
        person1.address = new Address();
        person1.address.city = "New York";

        Person person2 = (Person) person1.clone();

        System.out.println(person1 == person2); // false
        System.out.println(person1.address == person2.address); // true, 引用对象共享
    }
}
```

```
// 深拷贝
import org.apache.commons.lang3.SerializationUtils;

class Person implements Serializable {
    String name;
    Address address;

    // 通过序列化和反序列化实现深拷贝
    public Person deepCopy() {
```

```
        return SerializationUtils.clone(this);
    }
}

class Address implements Serializable {
    String city;
}

public class DeepCopyExample {
    public static void main(String[] args) {
        Person person1 = new Person();
        person1.name = "John";
        person1.address = new Address();
        person1.address.city = "New York";

        Person person2 = person1.deepCopy();

        System.out.println(person1 == person2); // false
        System.out.println(person1.address == person2.address); // false，引用对象不共享
    }
}
```

> 图源：javaguide



总结：

- 浅拷贝复制对象，包括对象的引用，两者共享引用对象。
- 深拷贝复制对象及其引用的对象，两者不共享引用对象，即使引用对象也会被复制。
- 在Java中，浅拷贝通常通过`clone`方法实现，深拷贝可以通过手动实现`clone`方法、序列化和反序列化，或者使用第三方库来实现。

面向对象【笔记】

> 本部分为面向对象的一些笔记整理，如不需要可以跳过。

面向对象程序设计概述

面向过程：

先确定如何操作数据，再决定数据的结构。适用于**小规模**问题;

面向对象OOP：

先决定数据的结构，在考虑操作数据的算法。适用于**大规模**问题;

1. 类

1、类是构造对象的模板或蓝图

2、封装是处理对象的一个重要概念

就是将数据和行为组合在一个包中，并对对象的使用者隐藏具体的实现方式

1. 实例字段：对象中的数据数据;
2. 方法：操作数据的过程;
3. 对象的状态：特定对象有一组特定的实例字段值，这些值的集合就是这个对象的当前状态。

3、OOP原则：

1. 封装：绝对不能让类中的方法直接访问其他类的实例字段;
2. 扩展：可以通过扩展其他类扩展来构建新类。

4、类、超类和子：

1. 继承：基于已有的类创建新的类。继承一个类就是复用（继承）这些类的方法，而且可以增加一些新的方法和字段
2. 继承：is-a 关系

5、定义子类

1. extends 关键字
2. 父类，又叫基类、超类
3. 子类，又叫派生类、孩子类
4. 子类比父类拥有的功能更多
    1. 通过扩招父类定义子类时，只用指出子类与父类的不同之处
    2. 一般的方法放在父类中，更特殊的方法放在子类中

6、覆盖方法

```
1. 超类中的有些方法对子类并不一定适用，此时需要在子类中覆盖（重写）父类方法

2. 使用 super 关键字可以调用父类方法，避免覆盖父类方法时调用父类方法造成不必要的递归

3. super 不是一个对象的引用，不能将super 赋给另一个对象变量
```

7、子类构造器

1. super(...) 调用父类构造器

2. 必须放在子类构造器的第一条
3. 若子类构造器没有显示调用父类构造器，将自动调用父类的无参构造器；
4. 若父类没有无参构造器，必须要在子类构造器中明确指明调用父类哪个构造器；否则，Java编译器就会报错

一个对象变量可以指示多种实际类型的现象成为多态。

在运行时能够自动选择适当的方法，成为动态绑定。

8、this 与 super 关键字

1. this 关键字
    - 隐式参数的引用
    - 调用该类的其他构造器
2. super 关键字
    - 调用父类方法
    - 调用父类构造器

注意：this 可以作为当前对象的引用，但是 super 却不可以作为父类对象的引用

2. 对象

1. 对象的状态改变必须通过调用方法实现;（如果不经过方法改变对象状态，说明破坏了封装性）
2. 对象的状态不能完全描述一个对象，每个对象有一个唯一标识;
3. 作为同一个类的实例，每个对象的标识总是不同的，状态往往也存在着差异。

3. 类之间的关系

- 依赖(uses-a)：一个类的**方法使用**另一个类的**对象**;
  ------->
- 聚合(has-a)： 类A的**对象包含**类B的**对象**;
  ◇———
- 继承(is-a) ： 一个**更特殊的类**和一个更**一般的类**之间的关系;
  ——▷

4. 继承与多态

1、继承层次

1. 由一个公共超类派生出来的所有类的集合成为继承层次。
2. 在继承层次中，从某个特定的类到其祖先的路径成为该类的继承链（子到父）

Java中不支持多继承，但是可以通过接口来实现

2、Java中，对象变量是多态的

1. 父类类型的变量既可以引用自身类型的变量，还可以引用子类类型的变量
2. 但是，子类类型的变量不可以引用父类类型的变量

3、子类可以调用父类public、protected、包权限的方法

但是父类不可以调用子类的特有方法（子类的特殊性）

1. 纯父类变量（左右都是父类）不可以调用子类的任何方法
2. 上转性变量（左边是父类，右边是子类）可以调用子类重写父类的方法（多态），但是仍然不能调用子类独有

的方法

```
public class Manager extends Employee {
    private double bonus;

    public void setBonus(double bonus) { // Manger 中独有的方法，Employee 类中没有
        this.bonus = bonus;
    }
}

Manager boss = new Manager(); // 纯子类变量（引用、对象）
Employee staff = boss; // 上转型对象。staff与boss引用同一个对象，但是编译器将 staff 看成一个 Employee 对象
boss.setBonus(100);   // OK。子类调用子类方法，没毛病
staff.setBonus(100);  // ERROR。setBonus 方法是子类独有的，不能通过上转型对象和父类对象调用，只能通过子类对象调用
```

「代码随想录」知识星球

posper

4、理解方法调用

以调用 x.f(arg) 为例，隐式参数 x 为类 C 的一个对象：

（1）编译器查看对象的声明类型和方法名

编译器查找 C 类中所有名为 f 的方法和父类中名为 f 且可访问的方法（父类 private 方法不可访问）

此时，编译器知道所有可能被调用的候选方法

（2）编译器确定方法调用中提供的参数类型

重载解析： 在所有名为 f 的方法中，找到一个与所提供参数类型完全匹配的方法

此时，编译器已经知道需要调用的方法名字和参数类型

（3）**静态绑定**(编译阶段绑定)

如果是 private 方法、static 方法、final 方法或者构造器， 那么编译器将可以准确地知道应该调用哪个方法。

此时调用方法只用考虑 x 的类型（若在类 C 中找不到 f 方法，则向其父类中找），不需要考虑类 C 的子类（因为这几种修饰的方法都不能被继承）

只有静态绑定成功，即编译通过了，才能进入运行阶段

（4）**动态绑定**(运行阶段绑定)

如果调用的方法依赖于隐式参数的实际类型，那么必须在运行时使用动态绑定。

虚拟机必须调用与 x 所引用对象的实际类型对应的那个方法

例如：x 的实际类型是 D，它是 C 类的子类。

如果：D 类定义了方法 f(String)，就直接调用它；
否则：将在 D 类的超类中寻找 f(String)，以此类推。

在覆盖一个方法的时候，子类方法不能低于父类方法的可见性。
比如，父类方法为 public，子类必须为 public。
如果子类遗漏了 public，编译器就会报错。

5、阻止继承：final 类和方法

（1）**final 关键字**

```
1. final 修饰字段

   * 基本类型：不可更改
   * 引用类型：不可指向新的引用，但是对象状态可能会改变

2. final 修饰类：该类不可以被继承

3. final 修饰方法：子类不能覆盖这个方法
```

（2）**将方法或类声明为 final 主要原因**

确保它们不会在子类中改变语义

6、强制类型转换

7、向上转型（upcasting）

- 子类型 -> 父类型
- 又被称为：自动类型转换

8、向下转型（downcasting）

- 父类型 -> 子类型
- 又被称为：强制类型转换

进行强制类型转换的唯一原因是：

在暂时忽视对象的实际类型之后， 使用对象的全部功能。

无论是向上转型还是向下转型，这两种类型之间必须要有继承关系；否则，编译器报错。

```
Employee[] staff = new Employee[3]; // 统一定义为父类变量，暂时忽视子类的实际类型

staff[0] = new Manager(); // 上转型对象
staff[1] = new Employee(); // 父类对象
staff[2] = new Employee();

Manager boss = (Manager) staff[0]; // 强制类型转换。使用对象的全部功能（因为上转型对象 staff[0] 不能使用 Manager 类中的 setBonus 方法）
```

- **可以**将上转型对象强制转化为子类变量，但是**不能**将父类对象强制类型转换为子类对象

```
// 同上代码
Manager boss = (Manager) staff[0]; // OK。上转型 to 子类
Manager boss = (Manager) staff[1]; // ERROR。父类 不可to 子类。此时，会产生 ClassCastException 异常

// 在将超类转换成子类之前，应该使用instanceof 进行检查
if (staff[1] instanceof Manager) {
    boss = (Manager) staff[1];
}
```

posper

「代码随想录」知识星球

小结：

1. 只能在继承层次内进行类型转换
2. 在将超类转换成子类之前，应该使用 instanceof 进行检查。

通过类型转换调整对象的类型并不是一种好的做法。在一般情况下，应该尽量少用强制类型转换和 instanceof 运算符。

大多情况下，因为多态性的动态绑定机制，我们不用将 Employee 转换为 Manager 也能正确调用方法；

只有在使用 Manager 特有方法才需要进行强制类型转换。例如，setBonus 方法。

但是，此时应该自问超类设计是否合理。是否需要重新设计超类，并添加 setBonus 方法，这才是更合适的选择。

抽象类

abstract 关键字：

1、abstract 修饰方法

1. 抽象方法，不用实现，在具体的子类中实现
2. 若子类未实现抽象方法，仍然要定义为抽象类

2、abstract 修饰类

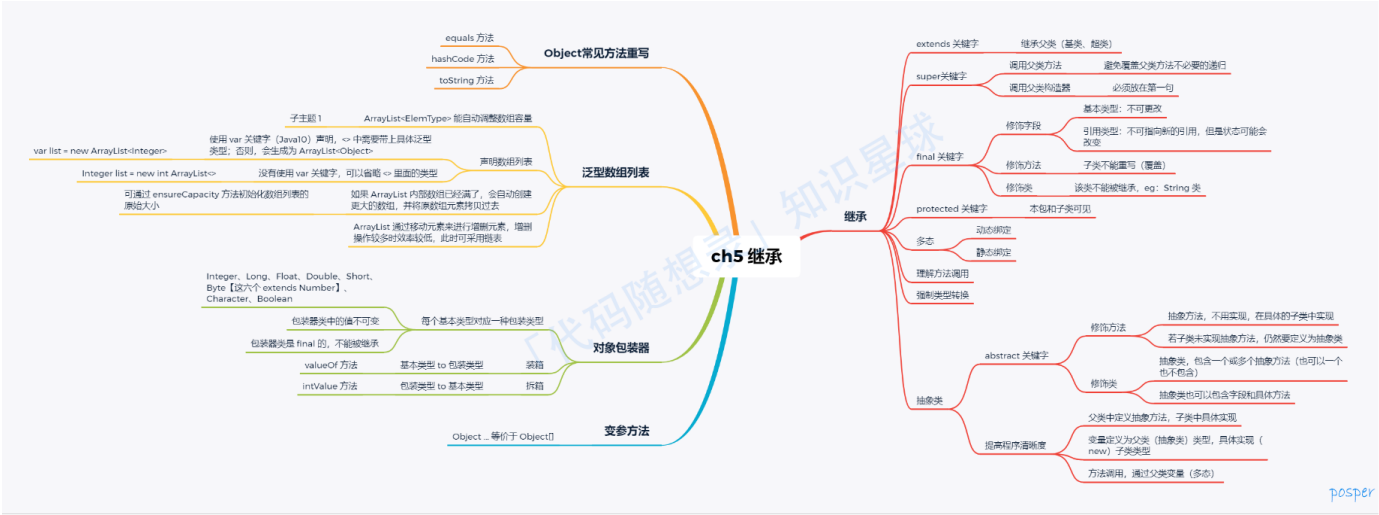
1. 抽象类，包含一个或多个抽象方法（也可以一个也不包含）
2. 抽象类也可以包含字段和具体方法

3、提高程序清晰度

1. 父类中定义抽象方法，子类中具体实现
2. 变量定义为父类（抽象类）类型，具体实现（new）子类类型（上转型）
3. 方法调用，通过父类变量（多态，动态绑定）

思维导图小结：

原图链接：「代码随想录」知识星球



使用预定义的类

1. 对象与对象变量

1. 对象变量并没有时间包含一个对象，它**只是引用**一个对象
2. 在Java中，任何对象变量的值**都是对存储在另一个地方某个对象的引用**
3. new操作符的返回值也是一个引用
4. Java对象都存储在**堆**中，当一个对象包含另一个对象变量时，它只是包含着另一个堆对象的指针

5. Java中如果使用一个没有初始化的指针，运行时系统将会产生一个运行时错误，同时不必担心内存管理问题，垃圾回收器会处理相关的事宜
6. Java中必须使用clone方法获得对象的完整副本

2. Java中的LocalDate类

1、Java类库包含两种保存时间的类

1. Date类：表示时间点
2. LocalDate类：日历表示法表示日期

2、一般通过这三种静态工厂方法构造一个LocalDate

```
LocalDate.of(2020,4,20);
LocalDate.now();
LocalDate.parse("2021-04-20");
```

3、使用方法

```
isLeapYear();                  //此LocalDate是否闰年
minusWeeks(); minusDays(); //减去星期/天之后的表示
plusWeeks(); plusDays(); //加上星期/天之后的表示
getMonthValue();          //获得月的值
getDayOfMonth();          //获得该LocalDate的day
getDayOfWeek(); getMonth(); //返回枚举值，获得该天的枚举值
//（1-7），获得该月的枚举值（1~12）
```

1. 访问器方法：只访问对象而不修改对象的方法;（如plusDays返回一个新的对象，原对象还在）
2. 更改器方法：调用方法之后修改对象的内容。

在C++中，带有const后缀的方法是访问器方法，没有的是更改器方法。而在Java中，这两种方法没有明面上的语法区别

用户自定义类

1. 主力类：通常没有main方法，却有自己的实例字段和实例方法;（例如：自定义User类）
2. 一个源文件中只能有一个公共类，但是可以有任意数量的非公共类。

1. 多个源文件的使用

将两个类存放在一个单独的类文件中

例如：Employee类存放在文件EmployeeTest.java中，然后键入以下指令javac EmployeeTest.java 此时并没有显示的编译Employee.java，不过编译器会发发现需要使用这两个类时，会自动搜索Employee.java，并编译。

如果编译器发现Employee.java较已有的Employee.java已有更新，那么Java编译器会自动的编译这个文件。

2. 构造器方法

构造器与类名同名，构造器总是结合new运算符来调用，不能对一个已存在的对象调用构造器来达到重新设置实例字段的目的。

3. 用var声明局部变量（在Java10中）

在Java10中，如果可以从变量的初始值推导出他们的类型，那么可以使用var关键字声明局部变量，而无需指定类型

var关键字只能用于方法中的局部变量，参数和字段的类型必须声明

4. 使用null引用

定义一个类时，最好清楚哪些字段可能为null，对于初始化时字段可能null，一般有两种解决方案:

1、宽容型

将null参数转换成一个非null值

```
if(n == null) name = "unknown"; else name = n;
/* Java9中有一个方法
name = Object.requireNonNull(n,"unknown");
达到同样的效果 */
```

2、严格型

直接拒绝null参数

```
Object.requireNonNull(n, "the name cannot be null");
```

5. 隐式参数和显式参数

1、隐式参数：

出现在方法名前的对象的参数;

2、显示参数：

位于方法名后面括号里的数值;

每个方法中，用this关键字指示隐式参数，这样可以将 实例字段 和 局部变量 明显的区分开来。

6. 封装的优点

注意不要编写返回可变对象引用的访问器方法（getter方法）

例如返回一个Date类：

因为Date类中有setDate()方法，所以Date对象可变，也就意味着破坏了封装性，如果需要返回一个可变对象的引用，首先应该对他进行clone，然后再返回克隆的对象副本。

7. 私有方法

只要方法是私有的，类设计者就可以确信它不会在别处使用，所以可以删去，而如果是公共的，那么可能会因为其他其他代码依赖这个方法。

8. final字段

1、final修饰符对于 基本类型 或者不可变类

类中所有方法都不会改变其对象，如String类的字段尤其有用

2、对于可变的类

使用final会造成混乱，final关键字只表示存储在该 变量 中的 对象引用 不会再指向另一个不同的对象，而这个对象还是可以更改

静态字段与静态方法

static的含义

1. 静态字段

1、静态字段：

静态字段属于**类**(即使没有创建该类的对象，静态字段也存在)，每个类只有一个这样的字段。创建的多个对象共享这同一个静态字段;

2、非静态字段：

每个对象都有自己的一个副本。

2. 静态常量

静态常量：设置为公共没问题，因为不允许再将它赋值为另一个值（System类中的setOut()方法可以将System.out设置为不同流，这是因为setOut方法是一个原生方法，不是在Java语言中实现的，因此可以绕开Java的访问控制机制。）

3. 静态方法

静态方法是一个没有this参数的方法;（可直接通过类名调用）

可以使用对象来调用静态方法，不过很容易造成混乱，因为方法计算的结果与该对象无关。建议使用类名直接调用静态方法;

两种情况下使用静态方法：

1. 方法不需要访问对象状态，如Math.pow
2. 方法只需要访问类的静态字段

4. 工厂方法

静态方法还有一种，类似LocalDate和NumberFormat 类使用静态工厂方法来构造对象。

使用这种方式构造对象的原因有两个:

1、无法命名构造器：

构造器名字必须与类名相同，但是有事希望有两个不同的名字，分别得到货币实例和百分比实例

2、使用构造器时：

无法改变所构造对象的类型。静态工厂方法可以，如工厂方法实际将返回DecimalFormat类对象，这是NumberFormat的一个子类

5. 方法参数

Java中，方法参数：

1. 方法不能修改基本数据类型的参数;（数值型或布尔型）
2. 方法可以改变对象参数的状态;
3. **方法不能让一个对象参数引用一个新的对象。**

对象构造

1. 重载

多个方法有相同的名字，不同的参数，便出现了重载

重载解析： 编译器必须挑选出具体调用那一个方法，它用各个方法首部中的 参数类型 和 值类型来匹配

方法签名：方法名 + 参数类型 注意返回类型不是方法签名的一部分

2. 默认字段初始化

如果在构造器中没有显式的为字段设置初始值，会自动赋为初始值（0，false，null）

方法中的局部变量必须要明确的初始化，否则会报错

类中的字段，没有初始化会默认初始化

3. 无参数的构造器

无参构造默认将所有字段设置为默认值

当且仅当类没有任何其他构造器的时候，才会得到一个默认的无参构造器

4. 显示字段初始化

可以在类定义时，为每个实例字段设置一个有意义的初始值（如果一个类的所有构造器希望把某个特定的实例字段设置为同一个值，这个就有用）

初始值也可以不是常量值，利用方法调用初始化一个字段，使用类中的静态变量，每次调用改变这个静态变量的值即可。

5. 参数名

参数变量会遮蔽同名的实例字段，但是给实例字段加上this就可以避免这种情况。

6. 调用另一个构造器

如果构造器的第一个语句形如`this(...)`，这个构造器将会调用另一个构造器，采用这种方式使用this关键字非常有用，对公共的构造器代码只需要编写一次即可。

C++中一个构造器不能调用另一个构造器。

7. 初始化块

一个类的生命中，可以包含任意多个代码块，只要构造这个类的对象，这些块就会被执行

（这种机制不常见，通常使用构造器初始化对象。同时，为了避免混淆，初始化块放在字段定义之后，避免重复定义带来的麻烦）

而静态初始化在类的第一次加载的时候，会进行初始化。

包

Java 允许使用包（ package） 将类组织起来

使用包的主要原因是确保**类名的唯一性**

包名通常是域名逆序来写，为了保证包名的绝对唯一性，将公司的因特网域名（这显然是独一无二的） 以逆序的形式作为包名

1. 类的导入

1、一个类可以使用所属包中的所有类，以及其他包中的公共类

2、两种方法访问另一个包中的公共类

（1）完全限定名（比较繁琐）

包名之后跟着类名`java.time.LocalDate today = java.time.LocalDate.now();`

（2）import导入（简单常用）

import应该位于源文件的顶部，package语句的后面

只能使用星号（*) 导入一个包， 而不能使用 import java.* 导入以java 为前缀的所有包

3、如果 import 两个包中包含了同名的类，则会发生命名冲突。

主要有以下两种解决方案：

1. 如果只是使用用一个包中的同名类时，可以考虑增加一个特定的 import 语句来解决这个问题
2. 如果两个个包中的同名类都被使用时，则需要在每个类名前加上完整的包名

其中，方法2通用，但是相对可能繁琐一些

2. 静态导入

使用import语句导入静态方法和静态字段，而不只是类

1. `import static java.lang.System.*;`
2. 就可以使用System 类的静态方法和静态域，而不必加类名前缀：
3. `out.println("Goodbye, World!");` // i.e., System.out

但是，这种编写形式不利于代码的清晰度。最好少用或者不用

3. 在包中增加类

1. 要想将一个类放入包中，就必须将包的名字放在源文件的开头，包中定义类的代码之前
2. 如果没有在源文件中放置 package 语句，这个源文件中的类就属于无名包
3. 编译器在编译源文件的时候不检查目录结构
4. 如果包与目录不匹配，或许可以成功编译（如果不依赖于其他包时），但是无法成功运行（虚拟机就找不到类）

1、包作用域：

1. 标记为 public 的部分可以被任意的类使用；
2. 标记为 private 的部分只能被定义它们的类使用
3. 如果没有指定public 或 private, 这个部分（类、方法或变量）可以被同一个包中的所有方法访问

注意：变量必须显式地标记为 private, 不然的话将默认为包可见。显然，这样做会破坏封装性。

2、类路径：

（1）类路径是所有包含类文件的路径的集合。

（2）javac 编译器总是在当前的目录中查找文件，

但 Java 虚拟机仅在类路径中有『.』目录的时候才查看当前目录

1. 如果没有设置类路径，默认的类路径包含『.』目录
2. 然而如果设置了类路径却忘记了包含 『.』 目录，则程序仍然可以通过编译， 但却不能运行（因为此时JVM找不到当前目录的文件）

4. Jar文件

在将应用程序打包时，希望**只向用户提供一个单独的文件**，而不是一个包含大量文件的目录结构。

JAR文件便是为此而设计的，一个JAR 文件可以包含文件、图像、声音等其他类型的文件。

JAR文件是压缩的，它使用了我们熟悉的zip压缩格式。

5. 清单文件

除了类文件，图像的其他资源外，每个JAR文件还包含一个清单文件(manifest) ，用于描述文档的特殊特性

清单文件位于JAR文件的 META-INF 子目录中

6. 可执行JAR文件

jar命令中的e指定程序的入口点，指定执行jar文件时启动的类。

执行命令：`java -jar jarFileName.jar`

文档注释

JDK包含一个很有用的工具，javadoc，它可以由源文件生成一个HTML文档

ch3中联机API 文档就是通过对标准Java 类库的源代码运行javadoc 生成的

1. 注释的插入

1、javadoc从以下项中抽取信息

1. 模块
2. package
3. public 的 class 和 interface
4. public 和 protected 的字段，构造器，方法

2、应该为以上各个部分添加注释`/** */`

注释中的内容是 自由格式文本，标记以@开始，如@param

第一句为概要性句子，javadoc将抽取第一句生成概要页

自由格式文本中可以使用HTML标签，如`<strong></strong>`表强调，`<img>`插入图片

其他文件图片的文件，应该放到包含源文件目录的一个子目录doc-files中

2. 类注释

1. @param 可以占据多行，可以使用HTML，一个方法的param必须放在一起
2. @return 可以多行，使用HTML
3. @throws 添加异常抛出注释

3. 字段注释

只需要对公共字段（静态常量static final）建立文档。

4. 通用注释

1. @since 引入该条目 始于版本;
2. @author 可用多个;
3. @version 当亲版本。

类设计技巧

1、保证数据私有

1. 绝对不要破坏封装性
2. 当数据保持私有时， 它们的表示形式的变化不会对类的使用者产生影响， 即使出现bug 也易于检测。

2、对数据初始化（避免错误）

1. Java 不对局部变量进行初始化， 但是会对对象的实例域进行初始化
2. 最好不要依赖于系统的默认值， 而是应该显式地初始化所有的数据

3、不要再类中使用过多基本类型（不易重复利用）

1. 就是说，用其他的类代替多个相关的基本类型的使用
2. 这样会使类更加易于理解且易于修改

4、不是所有的字段都需要单独的getter和setter

字段访问器和字段更改器（有一些实例字段不希望其他人修改）

在对象中，常常包含一些不希望别人获得或设置的实例域

5、分解职责过多的类

6、类名和方法名要充分体现职责（见名知意）

命名类名的良好习惯是采用一个名词（ Order)、前面有形容词修饰的名词（ RushOrder)或动名词（有“ -ing” 后缀）修饰名词（例如， BillingAddress)。

对于方法来说，习惯是访问器方法用小写get 开头, 更改器方法用小写的set 开头

7、优先使用不可变的类（避免多线程并发同时更新一个对象带来的麻烦）

更改对象的问题在于，如果多个线程试图同时更新一个对象，就会发生并发更改。其结果是不可预料的。如果类是不可变的，就可以安全地在多个线程间共享其对象。

因此，要尽可能让**类是不可变的**

当然，并不是所有类都应当是不可变的。

内存中对象

前言

“对象在内存中是怎么布局的？”

“对象头具体包括什么？”

“锁在对象的哪里？”

“对象怎么定位？”

面试过程中，你是否被问到过这些问题，如果不能完全回答出来，那么这篇文章一定要看下去。

对象的内存布局

对象实例中主要包含三部分的内容，分别为对象头、示例数据和对齐填充padding。下面分别进行介绍。

1、对象头

对象头中的内容主要是**运行时元数据和类型指针**。

其中运行时元数据主要存储的是以下六个内容：

1. 哈希值
2. GC分代年龄
3. 锁状态标志
4. 线程持有的锁
5. 偏向线程ID
6. 偏向时间戳

所以我们知道，锁是在对象头中的。

那么类型指针又是什么呢?

实际上，类型指针是指向元数据类型的InstanceKlass，用来确定该对象所属的类型。比如通过getInstance()得到对象实例时就需要用到该指针。这里需要注意的是，不是所有对象都会保留该指针的。

以上我们说的对象指的是自定义对象，如果是数组对象，那么还需要记录数据的长度。

2、示例数据

示例数据中是对象真正存储的有效信息，包括程序中定义的各种字段，这里的字段也包括从父类、Object类中定义的各种字段。

那么这么多字段，是怎么放在一起的呢？所以就不得不提存放规则了。

示例数据的存放规则遵循以下三个原则

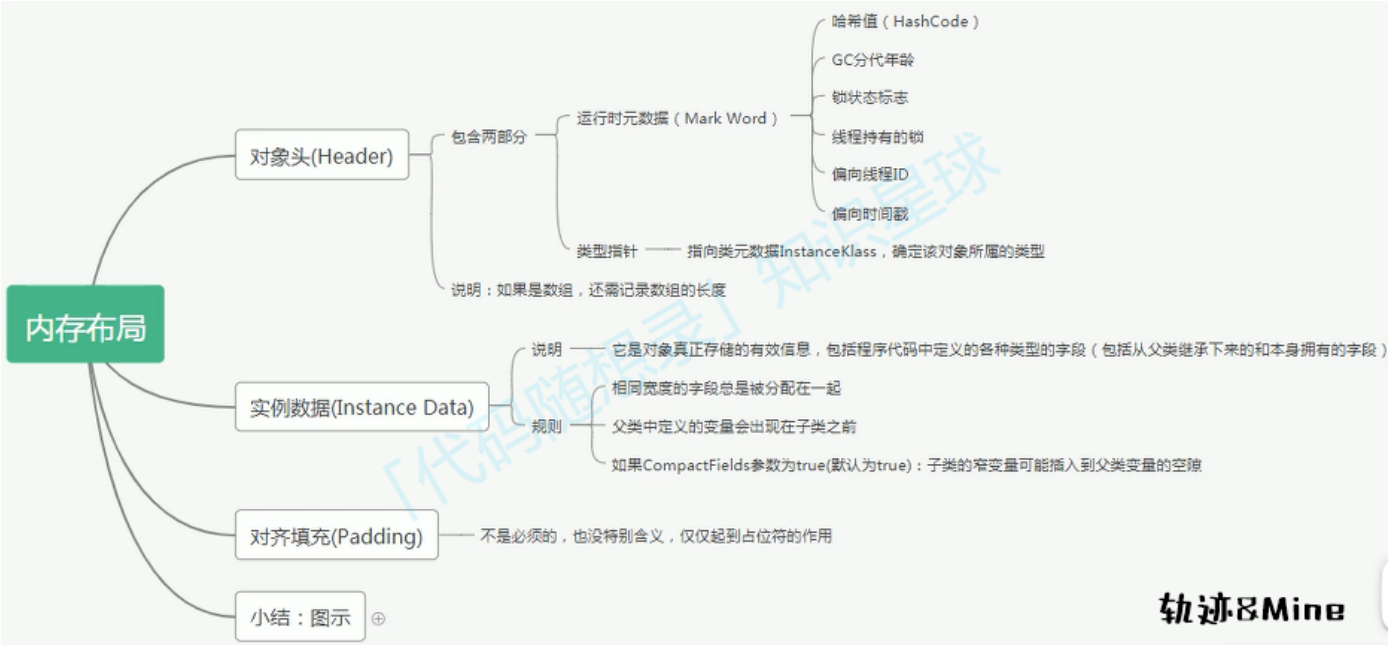
1. 父类中定义的变量在子类之前。
2. 同一个类中，相同宽度的字段被分配在一起。如四个字节的数据一起。
3. 若 CompareFileds参数为true（默认为true）,子类的窄变量可能插入到父类变量的空隙。

对齐填充padding

由于HotSpot虚拟机的自动内存管理系统要求对象起始地址必须是8字节的整数倍，换句话说就是任何对象的大小都必须是8字节的整数倍。对象头部分已经被精心设计成正好是8字节的倍数（1倍或者2倍），因此，如果对象实例数据部分没有对齐的话，就需要通过对齐填充来补全。

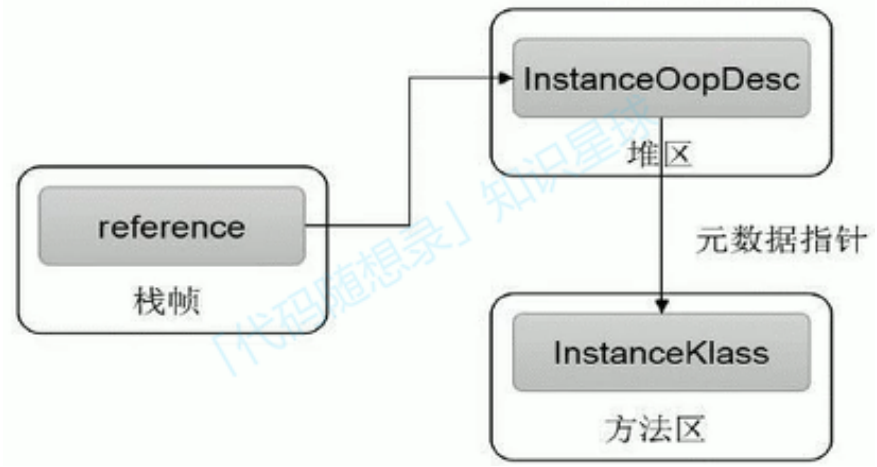
总的一句话来说，“数据项仅仅能存储在地址是数据项大小的整数倍的内存位置上（分别为偶地址、被4整除的地址、被8整除的地址）”比如int类型占用4个字节，地址仅仅能在0，4，8等位置上。

想讲好对齐填充，其实还是得去了解c++实现，这里不过多介绍。



对象的访问定位

堆中的对象是通过元数据指针找到方法区中的元数据信息的，那么栈帧中的对象引用又是如何访问找到对应的对象呢?



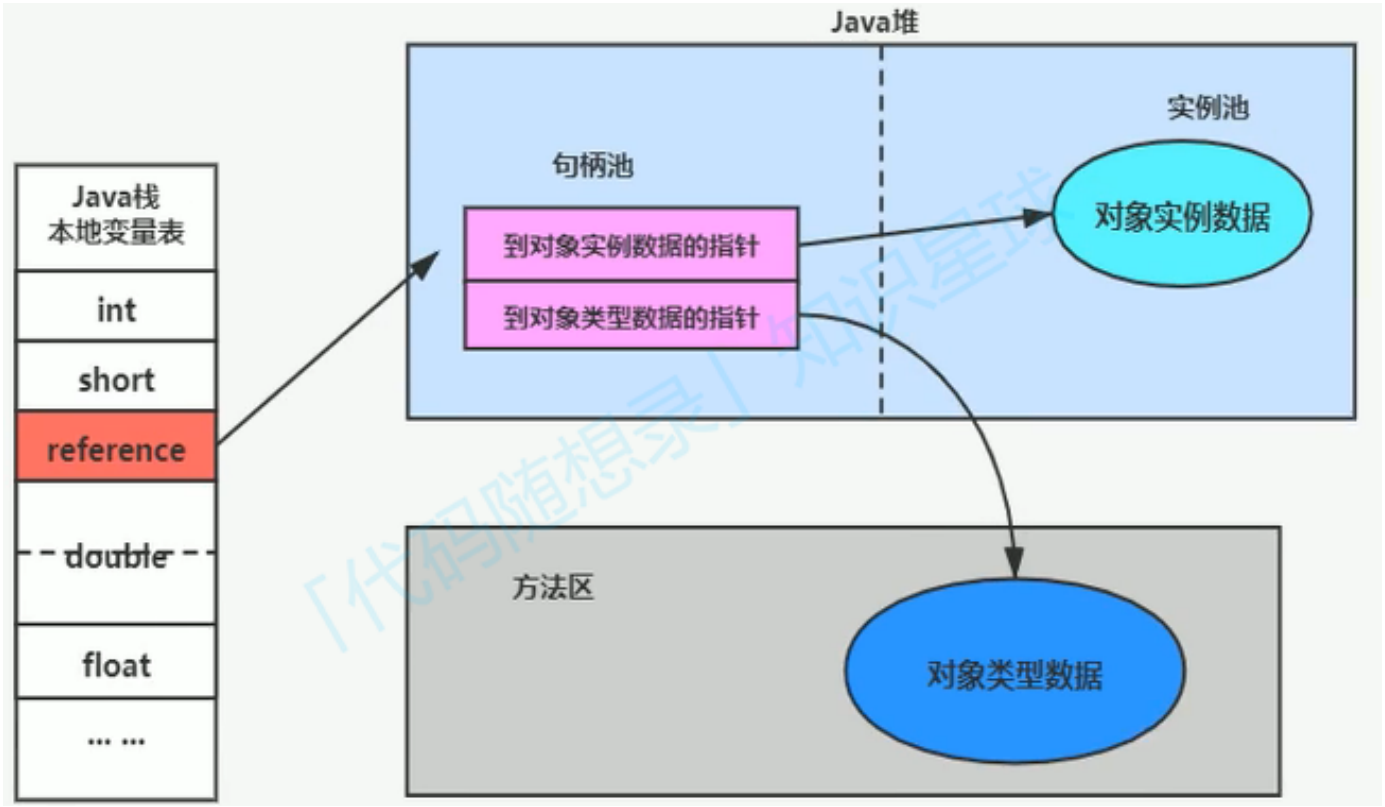
对象的访问定位方式主要有两种：

1. 句柄访问
2. 直接访问

句柄访问方式：

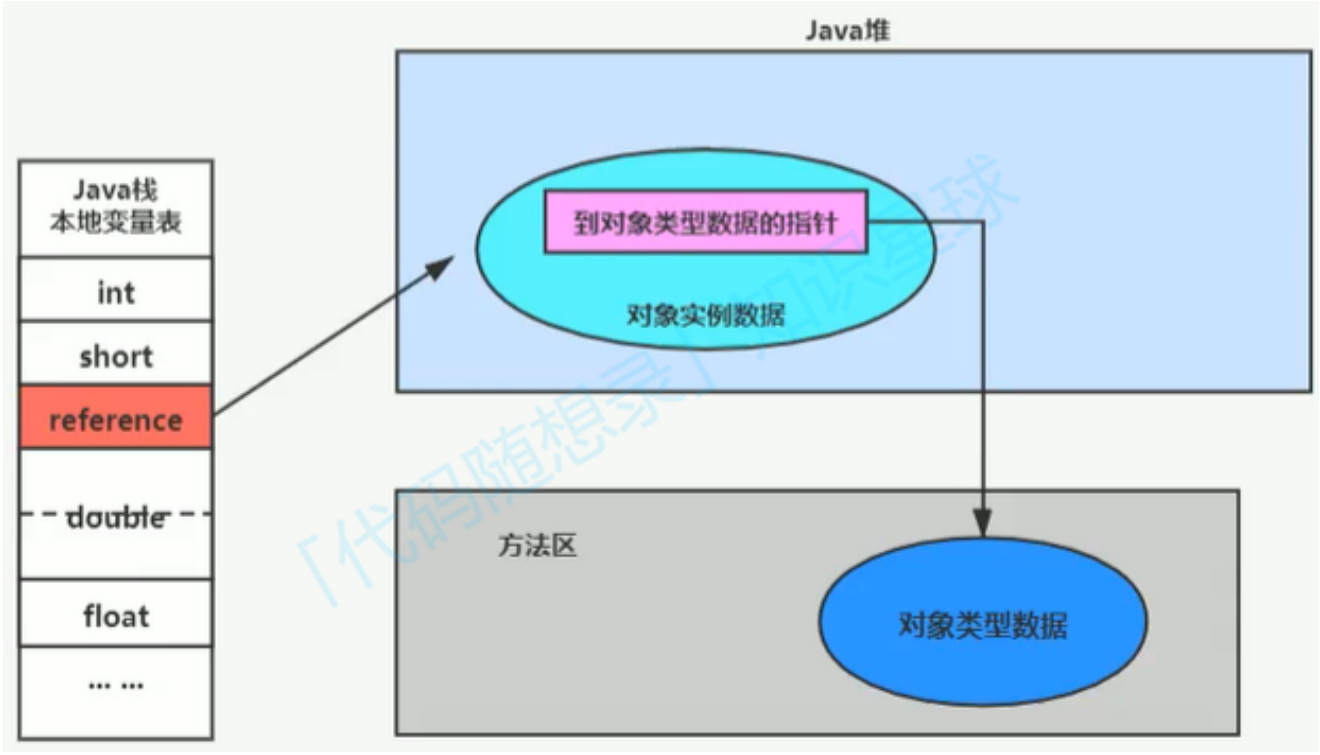
堆区中开辟了一块空间用于存储句柄，该空间称为句柄池。句柄池分别存储着两个句柄，分别是到对象实例数据的指针和到对象类型数据的指针。前者指向对象实例数据，后者指向对象类型数据。

栈帧的局部变量表中存储着到对象实例数据指针的位置，即指向句柄池中的第一个句柄。



直接访问方式：

直接访问方式相比句柄访问就简单多了，局部变量表中直接存储着对象实例数据的地址，而对象实例数据存储着方法区中对象类型数据的地址。这也是Hotspot虚拟机中的访问方式。



总结

关于对象的内部结构、内存布局和访问定位，值得深入的点还有很多，这里只是做一个初步的介绍，待深入学习后，会及时更新相应的内容。

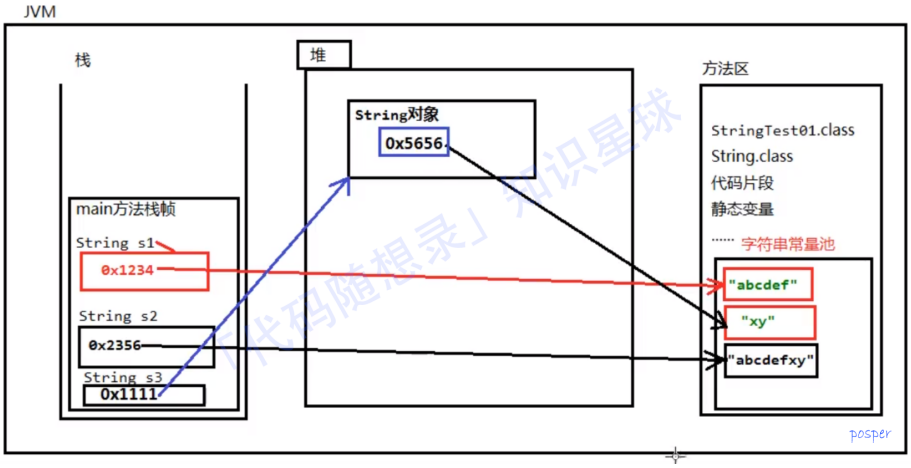
String

String存储原理

- String 类型是不可变的
- Java 中用双引号括起来的字符串，例如："abc"、"def"，都是直接存储在“方法区”的“字符串常量池”当中的。
- 为什么把字符串存储在一个“字符串常量池”当中呢?
    - 因为字符串在实际的开发中使用太频繁
    - 为了提高执行效率，所以把字符串放到了方法区的“字符串常量池”当中

```
public class StringTest02 {
    public static void main(String[] args) {
        /**
         * 这里会创建 3 个对象，均在“字符串常量池”中。
         * "abcdef"仍然在“字符串常量池”中，且只会存一份。
         * “字符串常量池”中还有 "xy"，以及新结果 "abcefxy"
         */
        String s1 = "abcdef";
        String s2 = "abcdef" + "xy";
    }
}
```

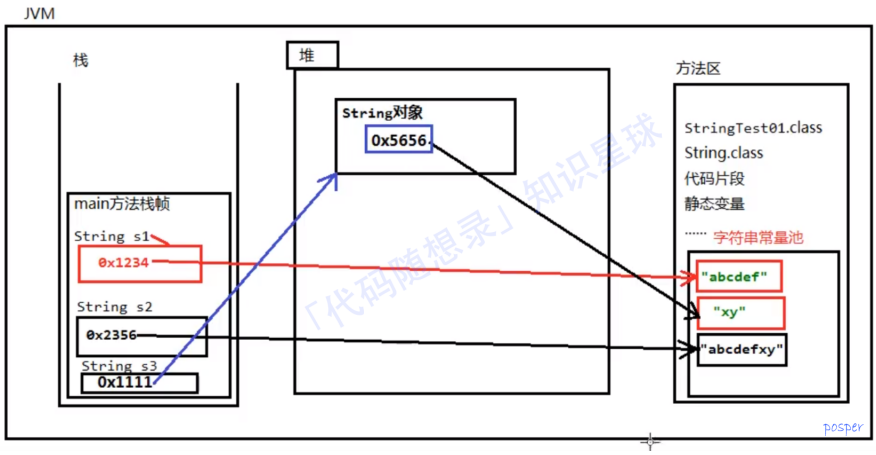
以上代码在 JVM 中加载过程如下：




```java
public class StringTest01 {
    public static void main(String[] args) {
        /**
         * "abcef"仍然在“字符串常量池”中。
         * 但是，new 的话，会在堆中开辟空间，其中存的是指向“字符串常量池”的内存地址。
         * 然后，在栈中存的是指向堆的引用地址。
         */
        /**
         * 这里一共创建了 3 个对象：
         *  方法区“字符串常量池”中有一个："abc"
         *  堆内存中有两个 String 对象。
         *  一共 3 个。
         */
        String s3 = new String("abcdef"); //
        String s4 = new String("abcdef"); //
        System.out.println(s3 == s4);       // false，比较地址
        System.out.println(s3.equals(s4));  // true，比较内容
    }
}
```

posper

以上代码在 JVM 中加载过程如下：



JDK1.8 中 String 部分源码解读

```java
// jdk1.8 中 String 部分源码解读
public final class String implements ... {
    private final char value[];
    /** Cache the hash code for the string */
    private int hash; // Default to 0

    public String() {
        this.value = "".value;
    }

     public String(String original) {
        this.value = original.value;
        this.hash = original.hash;
    }
}
```

posper

JDK1.8 中 StringBuilder 部分源码解读

```java
// jdk1.8 中 StringBuilder 部分源码解读
package java.lang;

public final class StringBuilder extends AbstractStringBuilder implements ... {
    // 字段 value 和 count 使用的是父类 AbstractStringBuilder 中的...

    // 无参构造器
    public StringBuilder() {
        super(16);
    }

    public StringBuilder(int capacity) {
        super(capacity);
    }

    // 待参数 string 的构造器
    public StringBuilder(String str) {
        super(str.length() + 16);
        append(str);
    }
}
```

posper

JDK1.8 中 AbstractStringBuilder 部分源码解读

```java
// jdk1.8 中 AbstractStringBuilder 部分源码解读
abstract class AbstractStringBuilder implements Appendable, CharSequence {
    char[] value; // 没有用 final修饰，可变。
                  // 包访问权限，AbstractStringBuilder 和 StringBuilder 都在 java.lang 包中
    int count;

    // 无参构造器
    AbstractStringBuilder() { }

    AbstractStringBuilder(int capacity) {
        value = new char[capacity];
    }

    @Override
    public int length() {
        return count;
    }

    @Override
    public StringBuilder append(String str) {
        super.append(str);
        return this;
    }
}
```

posper

「代码随想录」知识星球

JDK1.8 中 StringBuffer 部分源码解读

```
// jdk1.8 中 StringBuffer 部分源码解读
package java.lang;

public final class StringBuffer extends AbstractStringBuilder implements ... {
    private transient char[] toStringCache;

    // 无参构造器
    public StringBuffer() {
        super(16);
    }

    // 参数为 String的构造器
    public StringBuffer(String str) {
        super(str.length() + 16);
        append(str);
    }

    @Override
    public synchronized int length() { // 使用 synchronized 修饰
        return count; // 这里 count 是父类 AbstractStringBuilder 中的
    }
}
```

「代码随想录」知识星球
posper

String类能被继承吗

不能被继承，因为String类有final修饰符，而final修饰的类是不能被继承的。

Java中对String类的定义

```
public final class String
    implements java.io.Serializable, Comparable<String>, CharSequence {}
```

final修饰的特点

1. 修饰类: final修饰的类不可被继承,是最终类.
2. 修饰方法: 明确禁止该方法在子类中被覆盖的情况下才将方法设置为 **final**
3. 修饰变量:

final修饰 基本数据类型的变量，其数值一旦在初始化之后便不能更改, 称为常量;
final修饰 引用类型的变量，则在对其初始化之后便不能再让其指向另一个对象。虽然不能再指向其他对象，但是它指向的对象的内容是可变的。

String底层使用的什么类型

- 在JDK1.8之前，底层使用的是`char`类型的数组

```
public final class String
    implements java.io.Serializable, Comparable<String>, CharSequence {
    /** The value is used for character storage. */
    private final char value[];
```

- 在JDK1.9以后，底层是`byte`类型的数组

```
@Stable
private final byte[] value;
```

String为什么是不可变的

1. 保存字符串的数组被 `final` 修饰且为私有的，并且`String` 类没有提供/暴露修改这个字符串的方法。
2. `String` 类被 `final` 修饰导致其不能被继承，进而避免了子类破坏 `String` 不可变。

String/StringBuffer/StringBuilder的区别

三者的区别主要集中在不可变性、线程安全和性能方面

1. String（不可变，线程安全）

- `String` 是不可变的，一旦创建了字符串对象，其值就不能被修改。对字符串的任何修改都会创建一个新的字符串对象。
- `String` 是线程安全的，因为它的不可变性使得多个线程可以同时访问同一个字符串对象而不会发生竞争条件。
- 由于不可变性，对字符串的修改操作（例如连接、截取）可能会导致创建新的字符串对象，影响性能。

2. StringBuffer（可变、线程安全）

- `StringBuffer` 是可变的，它允许在同一对象上执行修改操作，而不创建新的对象。
- `StringBuffer` 是线程安全的，它的方法是同步的，可以安全地在多个线程中使用。
- 由于可变性， `StringBuffer` 适用于在多线程环境中进行字符串操作，但在单线程环境中，相比 `StringBuilder`，可能会有一些性能开销。

3. StringBuilder(可变、非线程安全)

- `StringBuilder` 也是可变的，类似于 `StringBuffer`
- `StringBuilder` 是非线程安全的，它的方法没有同步，因此在多线程环境中使用时需要注意同步问题。
- 由于可变性， `StringBuilder` 适用于在单线程环境中进行字符串操作，且通常比 `StringBuffer` 性能更好，因为它不需要同步。

综合来说， `String` 适用于不经常修改的情况，而 `StringBuffer` 和 `StringBuilder` 适用于需要频繁修改字符串的情况，具体选择取决于是否需要线程安全以及性能的考虑。

String类的常见方法有哪些

> 列举几个常用的即可

- **`char charAt(int index)`**： 返回字符串指定位置的字符。
- **`String substring(int beginIndex, int endIndex)`**： 返回从指定索引开始到指定索引结束的子字符串。
- **`String concat(String str)`**： 将指定字符串连接到此字符串的末尾。
- **`boolean equals(Object anObject)`**： 比较字符串的内容是否相等
- **`boolean contains(CharSequence sequence)`**： 判断字符串是否包含指定的字符序列。
- **`int indexOf(String str)`**： 返回指定子字符串第一次出现的索引。
- **`String replace(char oldChar, char newChar)`**： 替换字符串中的字符。
- **`String[] split(String regex)`**： 根据正则表达式切割字符串，返回字符串数组。
- **`char[] toCharArray()`**： 将字符串转换为字符数组。
- **`String toLowerCase()`**： 将字符串转换为小写。
- **`String toUpperCase()`**： 将字符串转换为大写。
- **`String trim()`**： 返回去除字符串两端空白的新字符串。

什么是字符串常量池

Java中，字符串是不可变的，这就意味着一旦创建了一个字符串对象，它的值就不能被修改。为了提高性能和减少内存占用，Java使用了字符串常量池的概念。

当程序中创建字符串常量时，这些字符串常量会被存储在字符串常量池中。如果后续的代码中创建了相同内容的字符串常量，Java 不会再创建新的对象，而是直接引用已经存在的字符串常量。这就意味着多个引用可以指向相同的字符串。

```
String s1 = "Hello";
String s2 = "Hello";
String s3 = new String("Hello").intern(); //  使用了 intern() 方法，指向了相同的字符串常量

System.out.println(s1 == s2); // true，引用相同的字符串常量
System.out.println(s1 == s3); // true，引用相同的字符串常量
```

字符串拼接发生了什么

在字符串拼接的过程中，涉及到字符串的不可变性和字符串常量池的概念。

1. 使用`+`运算符拼接

```
String str1 = "Hello";
String str2 = "World";
String result = str1 + ", " + str2;
```

在上面的例子中，字符串拼接的过程可以可以分为以下几步：

- 创建两个字符串常量：`"Hello"`和`"World"`。它们被存储在字符串常量池中。
- 执行第一次拼接：`str1 + ", "`，这将创建一个新的字符串对象（`"Hello, "`），并将其存储在字符串常量池中。
- 执行第二次拼接：`(str1 + ", ") + str2`，这将创建另一个新的字符串对象（`"Hello, World"`），并将其存储在字符串常量池中。
- 将结果引用赋给`result`变量：`String result = "Hello, World"`。

如果拼接的都是字符串直接量，则在编译时编译器会将其直接优化为一个完整的字符串，和你直接写一个完整的字符串是一样的，所以效率非常的高。

如果拼接的字符串中包含变量，则在编译时编译器采用StringBuilder对其进行优化，即自动创建StringBuilder实例并调用其append()方法，将这些字符串拼接在一起，效率也很高。但如果这个拼接操作是在循环中进行的，那么每次循环编译器都会创建一个StringBuilder实例，再去拼接字符串，相当于执行了 new StringBuilder().append(str)，所以此时效率很低。

2. 使用`concat()`

```
String str1 = "Hello";
String str2 = "World";
String result = str1.concat(", ").concat(str2);
```

`concat()`方法在每次调用时都会创建一个新的字符串对象，而不修改原有的字符串对象。

3. 使用`StringBuilder/StringBuffer`

```
StringBuilder stringBuilder = new StringBuilder();
stringBuilder.append("Hello");
stringBuilder.append(" ");
stringBuilder.append("World");
String result = stringBuilder.toString();
```

`StringBuilder`对象`stringBuilder`可以通过`append()`方法多次追加字符串。这样的操作是在同一个可变对象上进行的，不会产生大量的中间字符串对象。最后，通过`toString()`方法将`StringBuilder`对象转换为不可变的字符串对象。

new String("abc")和String a = "abc"的区别

- 使用`new String("abc")`会在堆内存中创建一个新的字符串对象，即使字符串常量池中已经存在相同内容的字符串。
- 使用`String a = "abc"`会检查字符串常量池中是否已经存在值为`"abc"`的字符串。如果存在，直接引用该字符串，如果不存在，则在常量池中创建一个新的字符串。

通常推荐使用`String a = "abc"`这种方式，因为它更符合字符串的常用特性，避免了不必要的对象创建。

Object

Object类的常见方法有哪些

```
String toString();  // 返回一个由类名和该对象的哈希码组成的字符串。通常重写这个方法以提供更有用的信息

boolean equals(Object obj); // 比较对象的引用是否相等

int hashCode(); // 返回对象的哈希码值，哈希码用于在使用哈希表的集合类中快速查找对象。通常，应该重写hashCode()方法，以便在重写equals()方法的同时保持一致性。

Class<?> getClass(); // 返回对象的运行时类。通常，用于获取对象的类信息。

protected Object clone() throws CloneNotSupportedException; // 创建一个浅拷贝

protected void finalize() throws Throwable; // 垃圾回收器销毁对象之前调用

// 实现线程之间的通信和同步
final void notify(); // 唤醒一个等待的线程
final void notifyAll(); // 唤醒所有等待的线程
final void wait() throws InterruptedException; // 使当前线程等待，直到被唤醒。
final void wait(long timeout) throws InterruptedException;
final void wait(long timeout, int nanos) throws InterruptedException;
```

===和equals()的区别

1. `==` 操作符比较的是对象的引用，即两个对象是否指向内存中的相同位置。如果两个引用指向相同的对象，则返回 `true`，否则返回 `false`，通常用于比较基础数据类型和对象的引用。
2. `equals()` 方法用于比较对象的内容。默认情况下，`equals()` 方法在 `Object` 类中的实现是比较对象的引用，但很多类都重写了这个方法，提供了对内容的比较。

```
String str1 = new String("Hello");
String str2 = new String("Hello");

System.out.println(str1 == str2); // false，不同的引用
System.out.println(str1.equals(str2)); // true，重写了equals方法，比较内容
```

说一说hashCode()和equals()的关系

`hashCode()` 方法返回对象的哈希码值，用于确定对象在哈希表中的存储位置。在集合类中，如`HashMap`，对象的哈希码决定了它在集合中的存储位置。如果两个对象被认为是相等的（通过`equals()`方法），那么它们的哈希码应该相同。

但是**哈希码相等的对象不一定相等：** 两个对象的哈希码相等并不意味着它们通过 `equals()` 方法相等

即遵守如下规定：

- 如果两个对象相等，则它们必须有相同的哈希码。
- 如果两个对象有相同的哈希码，则它们未必相等。

为什么要重写hashCode()和equals()

`equals()` 方法用于比较两个对象的内容是否相等。在Java中，默认实现是比较对象的引用，即比较两个对象是否指向内存中的相同位置。但通常，我们希望比较对象的内容是否相等。鉴于这种情况，Object类中`equals()`方法的默认实现是没有实用价值的，所以通常都要重写。

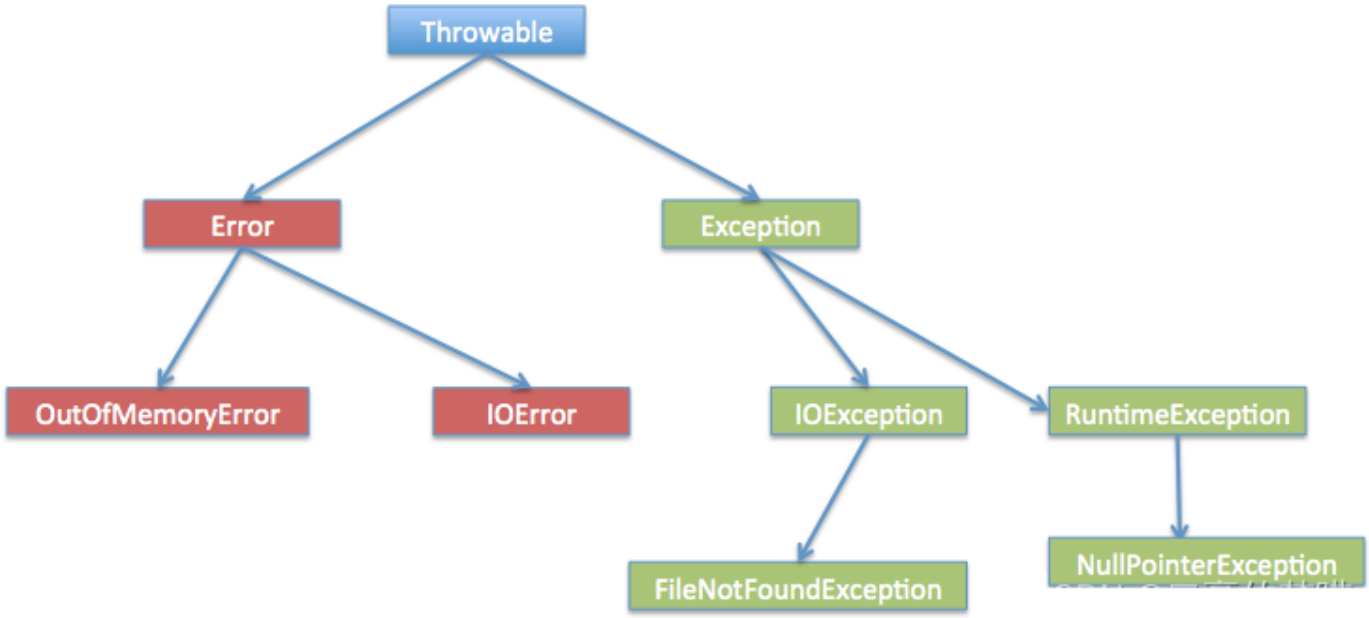
而由于hashCode()与equals()具有联动关系（如果两个对象相等，则它们必须有相同的哈希码），所以equals()方法重写时，通常也要将hashCode()进行重写，使得这两个方法始终保持一致性。

异常

> 前面是对Java异常处理机制的知识点讲解，面试题从 Exception 和 Error 有什么区别？ 开始

Java的异常都是派生于Throwable类的一个实例，所有的异常都是由Throwable继承而来的。

Throwable有分为了`Error`类和`Exception`类。



Error（错误）

Error 类层次结构描述了 Java 运行时系统的内部错误和资源耗尽错误。

Error表示比较严重的问题，一般是JVM运行时出现了错误，如没有内存可分配抛出OOM错误、栈资源耗尽抛出StackOverflowError错误、Java虚拟机运行错误Virtual MachineError、类定义错误NoClassDefFoundError。

如果出现了这样的内部错误，除了通告给用户，并尽力使程序安全地终止之外，再也无能为力了。

Exception（异常）

异常又分为RuntimeException和其他异常：

由程序错误导致的异常属于RuntimeException，而程序本身没有问题，但由于像 I/O 错误这类问题导致的异常属于其他异常。

运行时异常RuntimeException：

顾名思义，运行时才可能抛出的异常，编译器不会处理此类异常。比如数组索引越界、使用的对象为空、强制类型转换错误、除0等等。出现了运行时异常，一般是程序的逻辑有问题，是程序自身的问题而非外部因素。

其他异常：

Exception中除了运行时异常之外的，都属于其他异常。也可以称之为编译时异常，这部分异常编译器要求必须处置。这部分异常常常是因为外部运行环境导致，因为程序可能运行在各种环境中，如打开一个不存在的文件，此时抛出FileNotFoundException。编译器要求Java程序必须捕获或声明所有的编译时异常，强制要求程序为可能出现的异常做准备工作。

不要被运行时异常的名称所迷惑，理论上所有的错误都是运行时发生的。包括Error、RuntimeException、编译时异常等等。所有的这些都只能在程序运行的过程中才能碰到。编译时异常指的是编译器要求必须处理的异常，并不是代码编译间发生的错误。

受检异常和非受检异常

字面理解，接受检查的异常和不接受检查的异常。

根据上面的信息，Error和RuntimeException运行时异常都不能被检查。他们都是程序运行过程中所产生的，只是Error的错误比较严重，一般是JVM产生，而RuntimeException一般是程序逻辑自身的问题。

受检异常就是上面的编译时异常，这些异常在编译时被强制要求捕获或者声明。编译器将会为所有的受检异常提供异常处理器。

PS:其实异常发生了，除了更改程序或者配置等，没有其他的方法。只是对引起程序不正常工作的原因进行分类，就出现了Error和Exception（运行时异常和编译时异常）。

这种分类能让程序员更好的定位因此错误的原因，更方便更高效的进行开发，写出健壮性更好的代码。但是异常并不能改变当前运行的结果，因为从程序开始运行的那一刻，整个逻辑和数据都已经固定了。

异常处理机制

主要是try-catch-finally和throw、throws关键字。

异常都是派生于Throwable类的一个实例，这个实例可以由JVM产生，也可以在程序中手动创建，用throw手动抛出。throw的对象必须是派生于Throwable类的实例，其他类型无法通过编译。

受检异常只有两种选择。要么被捕获处理，要么被抛出，让调用者处理。

而非受检异常没有此强制要求。

throws是用来声明异常，只要是派生于Throwable类的都可以被声明。

方法应该在其首部声明所有可能抛出的受检异常，这是强制要求的。非受检异常不要求必须通过throws声明，因为Error发生后对其无能为力，而如果有运行时异常，那么就是程序自身的问题，应该把时间花在修正程序的错误上，而不是说明程序发生的可能性上。

所以编写程序的时候throws关注点是受检异常，但是非受检异常也可以通过throws声明，只是不强制要求而已。若有多个受检异常，必须在throws中全部声明，如果方法没有声明所有可能的受检异常，编译器就会发出错误提示。

try-catch-finally用来捕获异常。

所有派生于Throwable类都可以通过 catch 捕获，try 中放可能存在异常的方法，如果在 try语句块中的任何代码抛出了一个在 catch 子句中说明的异常类，那么程序将跳过 try 语句块的其余代码，并且执行 catch 子句中的处理器代码。如果在 try 语句块中的代码没有抛出任何异常，那么程序将跳过 catch 子句，如果方法中的任何代码抛出了一个在 catch 子句中没有声明的异常类型，那么这个方法就会立刻退出。

在一个 try 语句块中可以捕获多个异常类型，并对不同类型的异常做出不同的处理。可以为每个异常类型使用一个单独的 catch 子句。需要注意，如果多个catch中的异常非继承关系，那么catch顺序不影响结果，如果catch异常存在类继承关系，那么子类的catch应该放在前面，父类的在后面。

如下图，catch的判断是从上到下，如果父类在前，那么派生于父类的都会被捕获，执行父类的处理代码，这样导致后面子类的处理代码永远不会被执行。

```
try
{
   code that might throw exceptions
}
catch (FileNotFoundException e)
{
   emergency action for missing files
}
catch (UnknownHostException e)
{
   emergency action for unknown hosts
}
catch (IOException e)
{
   emergency action for all other I/O problems
}
```

finally一般用来关闭所占用的资源。如果代码抛出异常，就会终止剩余代码的处理，并且退出这个方法。

这样可能会导致一些程序占用的系统并不能被正确的释放。而不管是否有异常被捕获，finally中子句的代码都会被执行，可以在这里正确的释放资源。

抓抛模型

所有的异常，一定是先有一个实例然后抛出。这个可以JVM完成，也可以手动执行。

这个实例是整个异常处理的源头。然后调用类可以利用try-catch-finally对异常进行捕获，也可以在方法首部通过throws继续向上层抛出。

try-catch-finally是抓，throw/throws是抛。一个类遇到异常，要么捕获后处理，要么继续向上抛出，让调用者进行处理。

catch中可以为空，这样程序就会忽略掉哪些异常。不进行处理本身就是一种处理方式。在catch中也可能抛出异常，也可以手动抛出。这样可以改变异常的类型，使用异常的包装技术，原先的异常时新异常产生的原因，可以让用户抛出子系统中的高级异常，而不会丢失原始异常的细节。

try-catch-finally可以灵活组合。

可以try-catch、try-finally或者try-catch-finally。可以分为以下几种情况

1、try中执行正常，忽略catch，最后执行finally。

2、try中出现异常，且异常在catch中声明，执行catch,最后执行finally.

3、try中出现异常，但catch中未声明，不执行catch，最后执行finally.

4、try中出现异常，且异常在catch中声明，执行catch时出现异常，停止catch中代码，执行finally并且抛出catch中新出现的异常。

```
try {
    System.out.println(1);
    IOException e = new IOException();
    throw e;
    } catch (IOException e){
        System.out.println(2);
        RuntimeException e1 = new  ArrayIndexOutOfBoundsException();
        throw e1;
    } finally {
        System.out.println(3);
    }
```

执行结果：

1
2
3
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException

finally

finally不管是try或者catch中没有由异常，最后大概率都会执行。

这是因为编译器会讲 finally 块中的代码复制两份并分别添加在 try 和 catch 的后面。

但是如果try或者catch中出现了 System.exit()语句，则会直接退出，并不会执行finally模块。

finally中没有return语句 最后返回值为2

```
int i = 1;
try {
    i = 2;
    return i;
} catch (Exception e) {

} finally {
    i = 3;
}
```

finally中有return语句 最后返回值为3

```
int i=1;
try {
    i = 2;
    return i;
} catch (Exception e) {
} finally {
    i = 3;
    return i;
}
```

上面两个代码只是在finally中是否存在return语句的区别，但直接结果却并不相同。

可以看到如果finally中没有return语句，程序就会把finally中的操纵数据忽略掉。

其实finally中的数据操作也是执行了的，但是并没有返回。这是因为在return语句返回之前，虚拟机会将待返回的值压入操作数栈，等待返回。即使 finally 语句块对 i 进行了修改，但是待返回的值已经确实的存在于操作数栈中了，所以不会影响程序返回结果。

在try中的数据处理完，检测到有return语句时，会先将数据压入操作数栈等待返回，然后去执行finally语句，如果finally语句没有将新的结果压入操作数栈，那么只可能返回原先的结果。

从这个角度理解，即使finally中对数据处理，但是返回的依旧时try中的“脏数据”。

finally并不是没有执行，而是执行了却没有没返回。添加return语句则会将finally处理过的数据压入操作数栈返回，原先的“脏数据”失效。

注意这里指的是基本变量，如果是引用类型不受此影响。因为不管在哪里进行运算，处理的都是引用背后的“实体”。

Throwable

Throwable是顶层的异常类，下面是 Throwable 类的主要方法：

1、public String getMessage()

返回关于发生的异常的详细信息。这个消息在Throwable 类的构造函数中初始化了

2、public Throwable getCause()

返回一个Throwable 对象代表异常原因

3、public String toString()

使用getMessage()的结果返回类的串级名字

4、public void printStackTrace()

打印toString()结果和栈层次到System.err，即错误输出流

5、public StackTraceElement [] getStackTrace()

返回一个包含堆栈层次的数组。下标为0的元素代表栈顶，最后一个元素代表方法调用堆栈的栈底

6、public Throwable fillInStackTrace()

用当前的调用栈层次填充Throwable 对象栈层次，添加到栈层次任何先前信息中

自定义异常

Java 的异常机制中所定义的所有异常不可能预见所有可能出现的错误，某些特定的情境下，则需要我们自定义异常类型来向上报告某些错误信息。

一般地，用户自定义异常类都是RuntimeException的子类

自定义异常类通常需要编写几个重载的构造器

自定义异常需要提供 serialVersionUID 序列化唯一ID，方便调试

自定义异常最重要的是异常类的名字，当异常出现时，可以根据 名字判断异常类型

注意事项

当子类重写父类带有throws声明的函数时，其声明的异常范围必须要在父类的支持范围内，即范围不能比父类大。只能保持范围不变或者更精确，不能变大。如果父类没有throws，那么子类也不能有throws，受检异常必须在子类内部捕获处理。

Java程序可以是多线程的。每一个线程都是一个独立的执行流，独立的函数调用栈。如果程序只有一个线程，那么没有被任何代码处理的异常 会导致程序终止。如果是多线程的，那么没有被任何代码处理的异常仅仅会导致异常所在的线程结束。也就是说，Java中的异常是线程独立的，线程的问题应该由线程自己来解决，而不要委托到外部，也不会直接影响到其它线程的执行。

异常处理不能代替简单的测试。捕获异常所花费的时间要远远的超过了测试的时间，所以在可能的情况下，先对执行条件进行判断，而不要等执行出错之后捕获异常。

Exception 和 Error 有什么区别?

`Exception` 和 `Error` 都是派生自 `Throwable` 类的子类

1. `Exception` 类及其子类主要用于表示程序可以处理的异常情况。异常分为两种类型：可检查异常（Checked Exception）和不可检查异常（Unchecked Exception）。程序员可以选择捕获并处理异常，也可以通过在方法签名中使用 `throws` 关键字声明方法可能抛出的异常。
2. `Error` 类及其子类通常表示虚拟机无法恢复的严重错误。错误不应该由应用程序捕获和处理，因为它们通常表示虚拟机或系统本身的问题。 一般情况下，程序员不会直接捕获和处理 `Error`。这些错误的发生通常意味着程序无法继续正常执行。

Checked Exception 和 Unchecked Exception 有什么区别?

1. 可检查异常是在编译时由编译器检查的异常，程序员必须显式处理或声明这些异常。如果不处理，编译器将报错。通常派生自 `Exception` 类及其子类，但不派生自 `RuntimeException`。
2. 不可检查异常是在运行时才会被检测到的异常，通常是由编程错误导致的。这些异常通常派生自 `RuntimeException` 或其子类。程序员可以选择处理这些异常，但不是强制的。

如何处理异常

在Java中，处理异常的语句由try、catch、finally三部分组成。其中，try块用于包裹业务代码，catch块用于捕获并处理某个类型的异常，finally块则用于回收资源。当业务代码发生异常时，系统会创建一个异常对象，然后由JVM寻找可以处理这个异常的catch块，并将异常对象交给这个catch块处理。若业务代码打开了某项资源，则可以在finally块中关闭这项资源，因为无论是否发生异常，finally块一定会执行。

```
try {
    // 代码块，可能会抛出异常
    // 可能抛出异常的代码
} catch (ExceptionType1 e1) {
    // 处理 ExceptionType1 类型的异常
} catch (ExceptionType2 e2) {
    // 处理 ExceptionType2 类型的异常
} finally {
    // 无论是否发生异常，都会执行的代码块
}
```

finally总是会被执行嘛

一般来说，`finally` 块都会在 `try` 或 `catch` 块执行完毕后被执行，即使发生了异常。然而，有一些情况下 `finally` 块可能不会执行，主要是在以下情况：

1. **在 `try` 或 `catch` 块中调用了 `System.exit()`：** 调用 `System.exit()` 会导致Java虚拟机（JVM）退出，此时 `finally` 块中的代码不会被执行。

```
try {
    // 一些代码
    System.exit(0); // 这会导致JVM退出，finally块不会执行
} finally {
    // 这里的代码不会执行
}
```

2. **在 `try` 块中发生了死循环：** 如果在 `try` 块中发生了无限循环或者其他永远不会结束的操作，`finally` 块可能无法执行。

```
try {
    while (true) {
        // 一些代码
    }
} finally {
    // 这里的代码可能无法执行
}
```

泛型

说一说你对泛型的理解

在Java中，集合存在一个缺点，即当将对象添加到集合中后，集合会失去对该对象的具体数据类型的记忆，导致在取出对象时，集合将其视为 `Object` 类型。这是因为集合的设计者在创建时无法确定集合将被用来存储哪种类型的对象，因此选择设计成能够存储任何类型的对象，以保持通用性。

然而，这种设计带来了两个问题：

- 集合对元素类型没有限制，可能导致一些问题。例如，如果想创建一个只能保存 `Dog` 对象的集合，但程序却

可以轻松地将`Cat`对象添加进去，可能引发异常。

- 由于集合在存储对象时丢失了对象的具体类型信息，取出对象后通常需要进行强制类型转换。这不仅增加了编程的复杂性，还可能引发`ClassCastException`异常。

为了解决这些问题，Java 5引入了泛型（Generic）的概念，允许在创建集合时指定集合元素的具体类型，例如`List<String>`表示该列表只能保存字符串类型的对象。使用泛型后，程序员不能轻易地将不同类型的对象添加到集合中，而且在取出元素时无需进行强制类型转换，使得代码更加安全和简洁。

泛型的使用

1. 泛型类

在类的声明中使用尖括号引入类型参数，然后在类中使用这个类型参数。

```
public class Box<T> {
    private T value;

    public void setValue(T value) {
        this.value = value;
    }

    public T getValue() {
        return value;
    }
}
```

在实例化时，指定具体的类型：

```
Box<Integer> integerBox = new Box<>();
integerBox.setValue(42);

Box<String> stringBox = new Box<>();
stringBox.setValue("Hello");
```

2. 泛型方法

在方法声明中使用尖括号引入类型参数，然后在方法中使用这个类型参数。调用泛型方法时，编译器通常可以从上下文中推断出类型参数，无需显式指定。

```
public <T> T genericMethod(T[] array) {
    // 实现泛型方法的逻辑
}
```

3. 泛型接口

类似于泛型类，泛型接口在声明时引入类型参数，然后在接口的方法中使用这个类型参数。

```
public interface List<T> {
    void add(T element);
    T get(int index);
}
```

介绍一下泛型擦除

Java 泛型擦除是指在编译时期，Java 编译器会将泛型信息从泛型代码中移除，以便保持与 Java 早期版本的兼容性。这个过程称为类型擦除。

泛型擦除的主要影响包括：

- **类型参数的擦除：** 在编译时，泛型类、接口和方法中的类型参数会被擦除为它们的上界（对于有上界的情况）。例如，`List<T>` 在编译时会被擦除为 `List<Object>`。
- **桥方法的生成：** 在泛型类或接口中，如果有泛型方法并且涉及到泛型类型参数的类型转换，编译器会生成桥方法（bridge method）来保持类型安全。这是为了解决擦除后可能导致的类型不匹配问题。
- **泛型数组的限制：** 由于泛型擦除，不能直接创建泛型数组。例如，`new T[]` 是不合法的，因为在运行时无法确定 `T` 的实际类型。

反射

反射机制是什么?

反射的核心类是`Class`类，它是Java反射机制的入口。通过`Class`类，可以获取类的构造函数、方法、字段等信息，而不需要在编译时确定这些信息。

反射的基本步骤包括获取`Class`对象、获取类的信息（构造函数、方法、字段等），然后通过`Constructor`、`Method`、`Field`等类的实例进行具体的操作。

其优点在于程序可以在运行时动态地获取和操作类的信息，这使得代码更加灵活，能够适应不同的场景和需求。

但是正因为 反射允许绕过编译时的类型检查，这可能导致在运行时出现类型安全性问题，并且它涉及到在运行时进行类型检查和动态分派。这可能导致性能开销。

Java反射在实际项目中有哪些应用场景?

Spring/Spring Boot、MyBatis 等等框架中都大量使用了反射机制。**这些框架中也大量使用了动态代理，而动态代理的实现也依赖反射。**

- 多数框架都支持注解/XML配置，从配置中解析出来的类是字符串，需要利用反射机制实例化；
- Spring框架就是通过反射和注解来实现依赖注入。

注解

注解以`@`符号开头，通常紧跟着一个标识符，后面可以跟一些参数，用于给类、方法、字段等添加元数据信息，这些息可以在编译时、运行时被读取和处理。

有一些常见的Java注解，比如

- **`@Override`**： 用于标识方法重写父类的方法。
- **`@Deprecated`**： 用于标识已经过时的方法、类等。

- `@FunctionalInterface`： 用于标识函数式接口。

此外，开发者还可以定义自己的注解，通过 `@interface` 关键字进行定义。

```
@interface MyAnnotation {
    String value();
}

@MyAnnotation(value = "My Custom Annotation")
public class MyClass {
    // code
}
```

集合

导图前言

前言

何集合框架都包含三大块内容：对外的接口、接口的实现和对集合运算的算法。

接口：

是代表集合的抽象数据类型。例如 Collection、List、Set、Map 等。之所以定义多个接口，是为了以不同的方式操作集合对象

实现（类）：

是集合接口的具体实现。从本质上讲，它们是可重复使用的数据结构，例如：ArrayList、LinkedList、HashSet、HashMap。

算法：

是实现集合接口的对象里的方法执行的一些有用的计算，例如：搜索和排序。这些算法被称为多态，那是因为相同的方法可以在相似的接口上有着不同的实现。

总述

理解Java的集合体系可以从三个层次，分别是最上层的接口、中间的抽象类、最后的实现类。

最上层接口：

表示不同的类型集合，Collection、Map、List、Set、Queue、Deque等，以及辅助性质的接口Iterator、LinkIterator、Comparator、Comparable这些接口是为迭代和比较元素而准备。

中层抽象类：

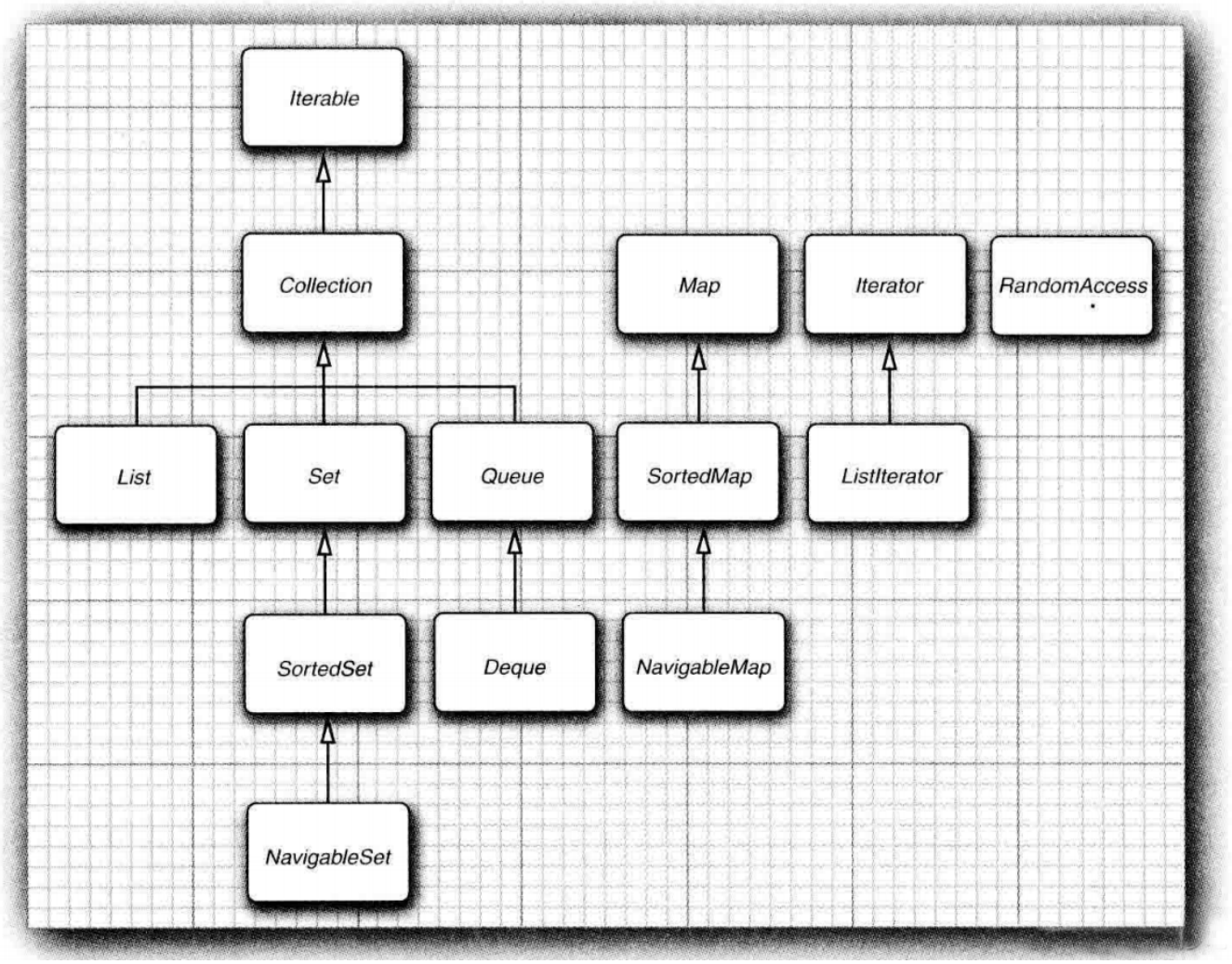
在这里实现大多数的接口方法，继承类只需要根据自身特性重写部分方法或者实现接口方法即可。

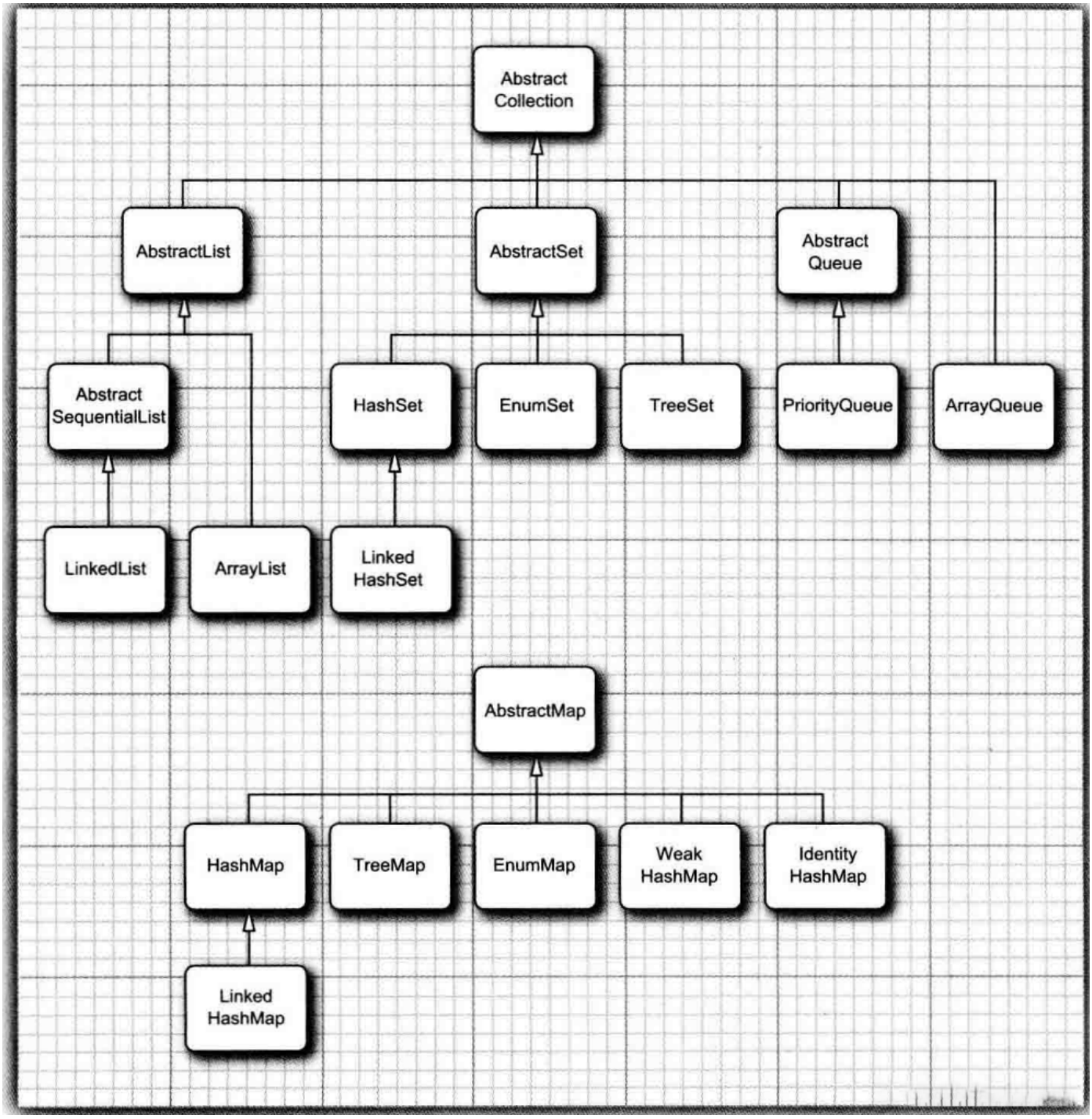
最底层实现类：

对接口的具体实现，主要常用的有：ArrayList、LinkedList、HashMap、HashSet、TreeMap等等。

除此之外，还有Collections和Arrays类用来提供各种方法，方便开发。

接口和实现类都比较熟悉，在敲代码时经常碰到，但是中间的抽象类一般看不到，但是各种各样的抽象类却有着承上启下的作用。

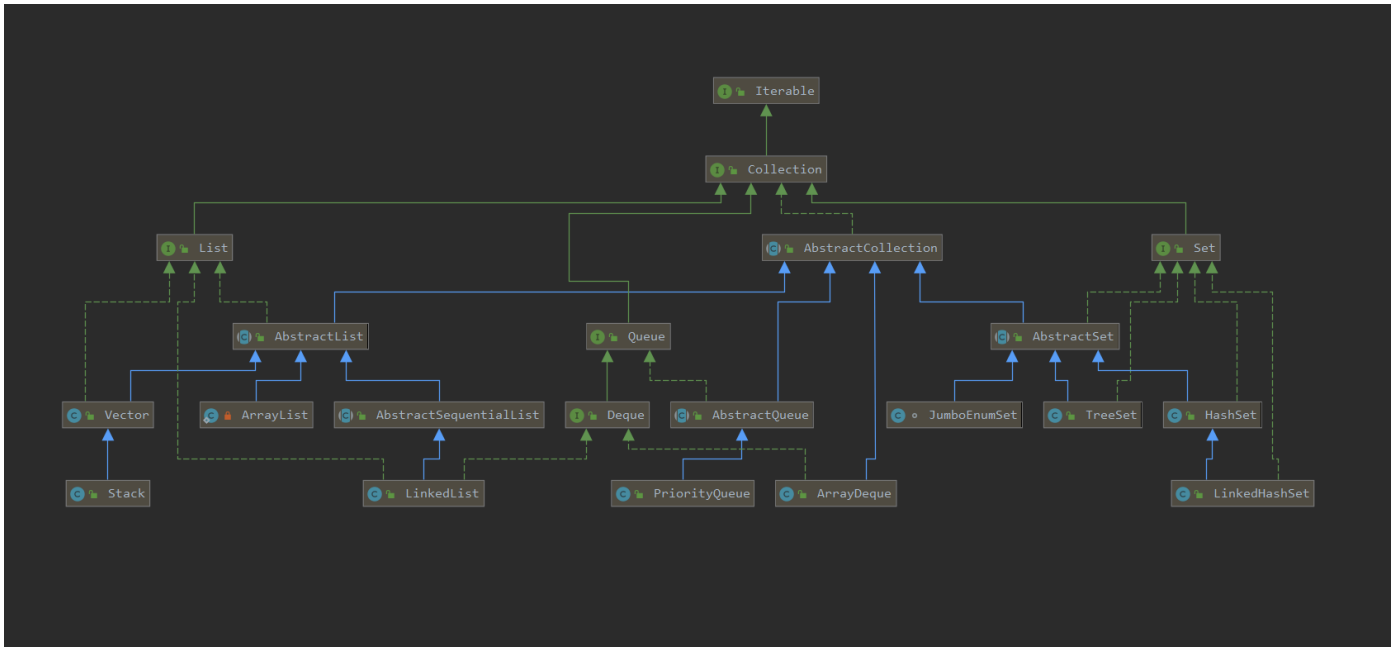




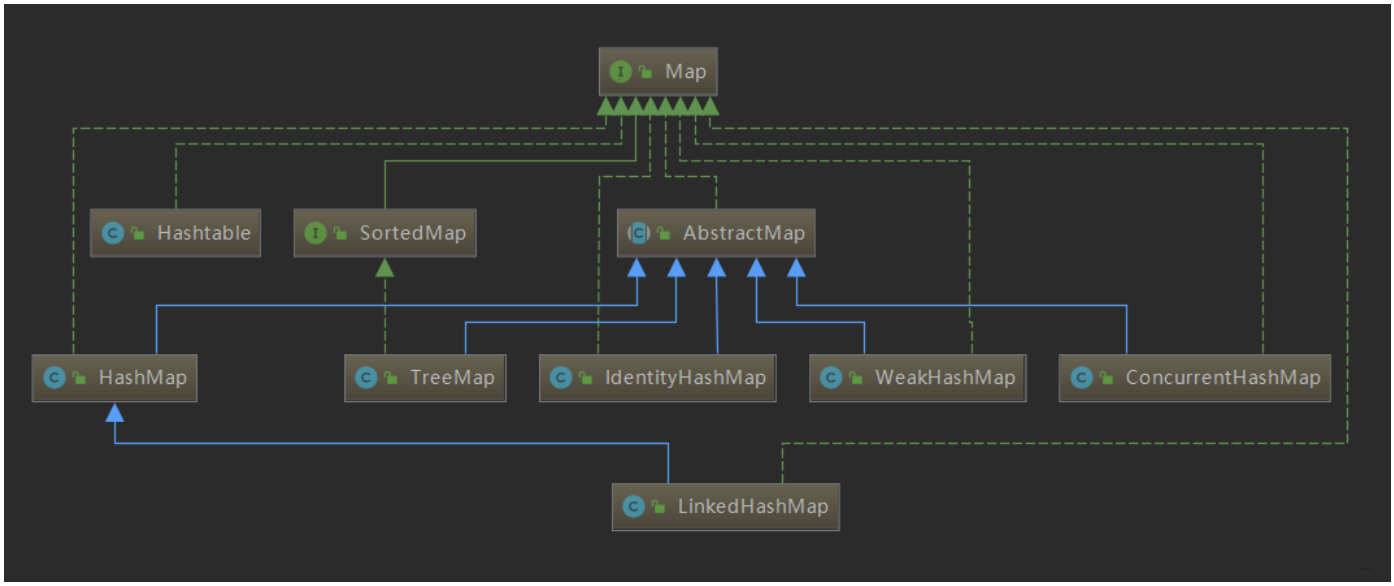
所有的集合都位于java.util包下，Java集合的基本接口是Collection接口和Map接口，所有的实现类都是这两个接口派生出来的。

Collection一次存一个元素，是单列集合，Map一次存一对元素，是双列集合。Map存储的一对元素：键-值，键（key）与值(value)间有对应(映射)关系。

单列集合继承关系图：

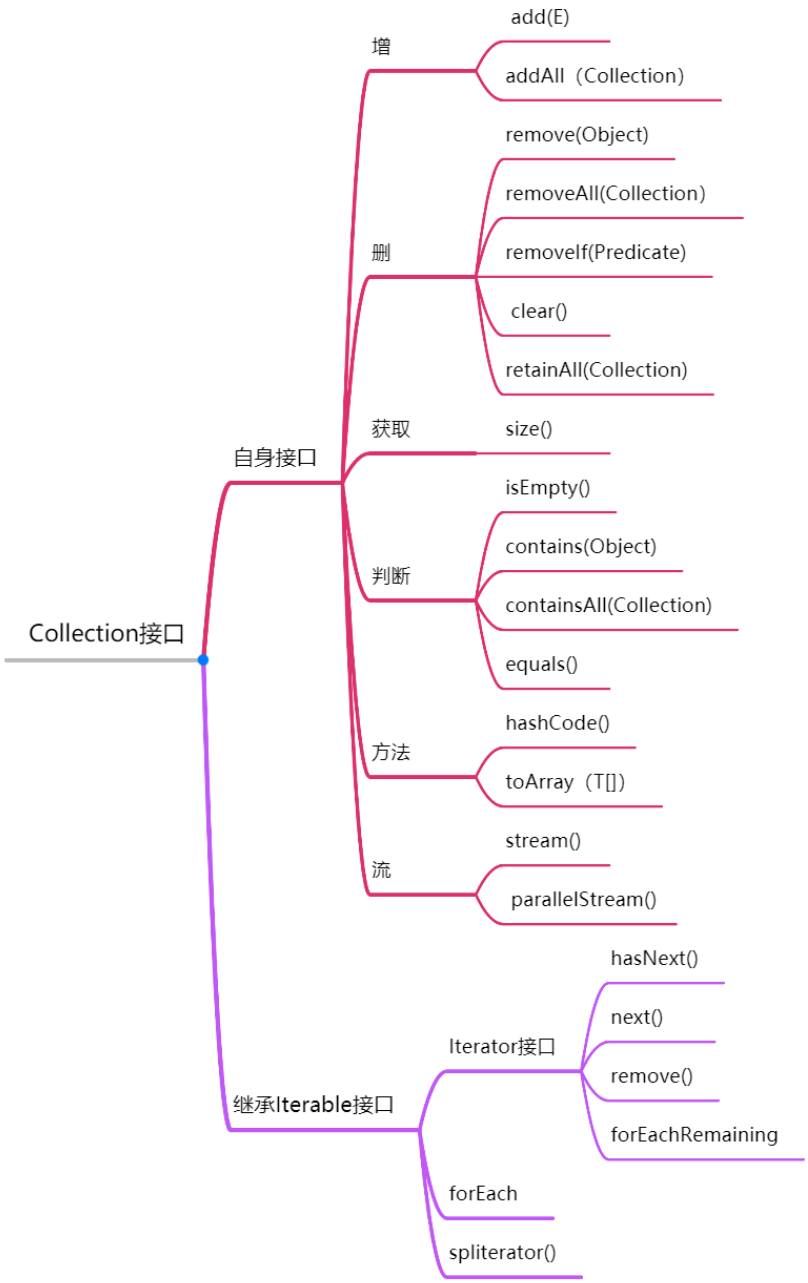


双列集合继承图：



Collection接口

思维导图：



上面的这些方法在实现类中都必须提供，而每一个类都要提供如此多的例行方法将是一件很烦人的事情，为了能够让实现者更容易地实现这个接口，Java 类库提供了一个类 AbstractCollection，它将基础方法 size 和 iterator 抽象化，但是实现了其他的方法。参照上面单列集合继承关系图，Collection下的所有实现类都是继承自AbstractCollection。AbstractCollection下又派生出AbstractList、AbstractSet、AbstractQueue、ArrayQueue。

List下的全部继承自AbstractList，而Set下的继承自AbstractSet。List 是一个有序集合，元素会加到容器的特定位置，并且允许重复元素。而Set是无序集合，并且不允许元素重复。Set要保证没有重复，就需要equals()和hashCode的正确重写。

Java中经常使用的语法糖"for each"循环可以与任何实现了Iterable接口的对象一起工作。Iterable包含了Iterator对象，可以调用iterator()方法，返回一个Iterator对象。这就是常用的迭代器，其中包含hasNext()、next()、remove()、forEachRemaining(Consumer)4个方法

Iterator()的功能简单，只能单向移动：

1、调用iterator()方法返回一个Iterator对象。

2、第一次调用Iterator的next()方法，返回序列的第一个元素。此后每调用一个next(),会返回序列的下一个元素。调用next()方法前最好先调用hasNext()方法，判断序列后面是否还有元素。

3、remove()删除上次next()返回的对象，remove()只能在next()之后使用，并且一个remove()匹配一个next()，不能重复调用.

4、使用hasNext()判断序列中是否还有元素

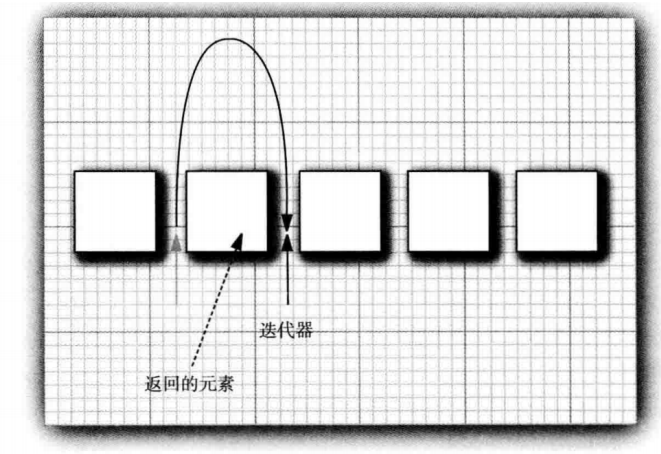
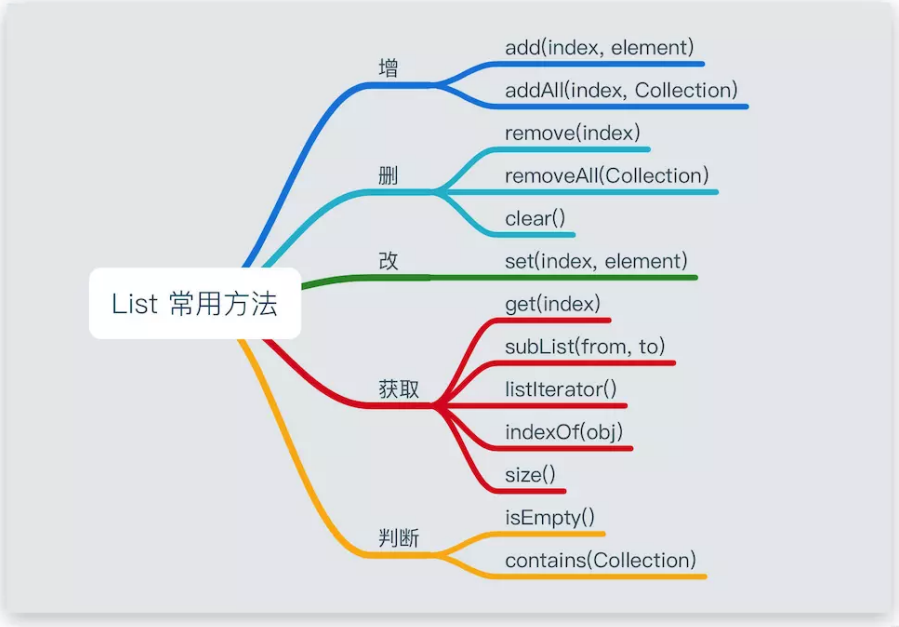


图 9-3　向前移动迭代器

List接口

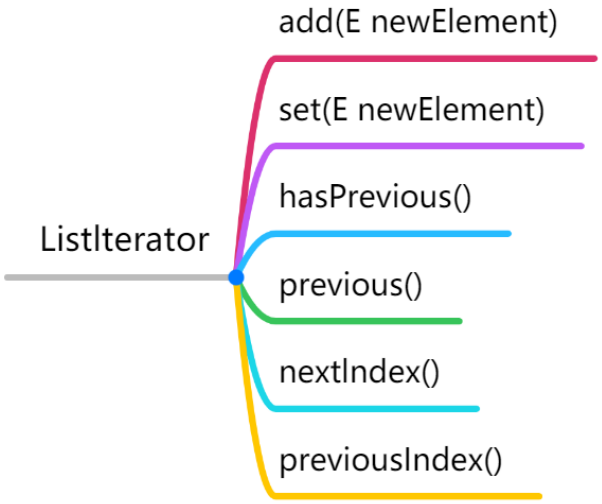
思维导图：



List接口常用的实现类有ArrayList、LinkedList

List接口继承自Collection。List中的方法和Collection中的大致相同，多出sort()、get()、listIterator()等方法，listIterator返回一个链表迭代器。

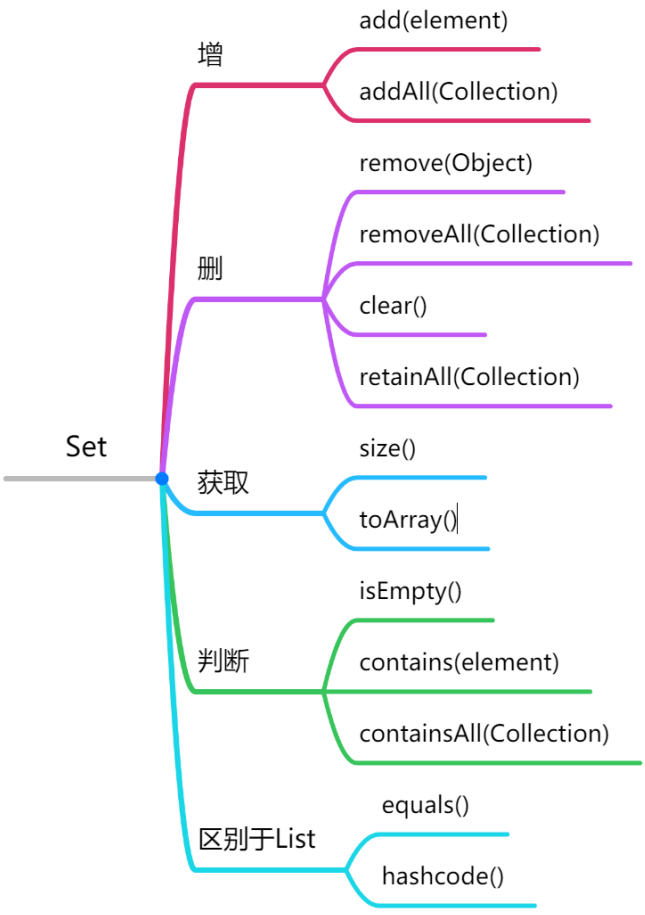
List有着特殊的迭代器接口ListIterator，继承Iterator接口。Iterator是单向的，但是ListIterator是双向的，增加了previous()和hasPreviousO方法用来反向遍历，ListIterator是一个功能更加强大的迭代器。注意只能List使用，其他接口无法使用。



Set接口

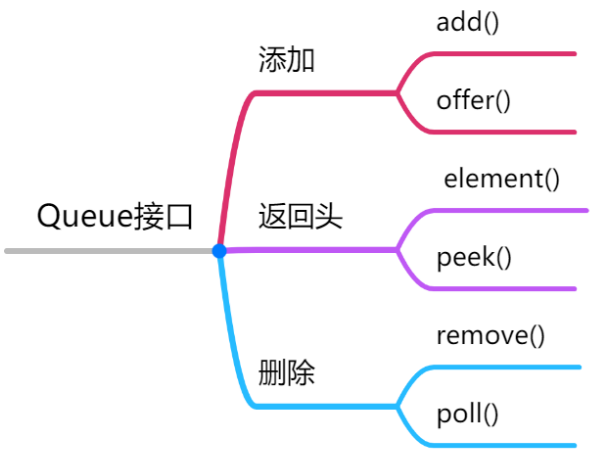
Set继承Collection,其方法的行为有更严谨的定义。Set不允许添加重复元素，为了实现这一点就要适当的定义equals方法和hashcode()方法。

Set常用的实现类有：HashSet、TreeSet、LinkedHashSet

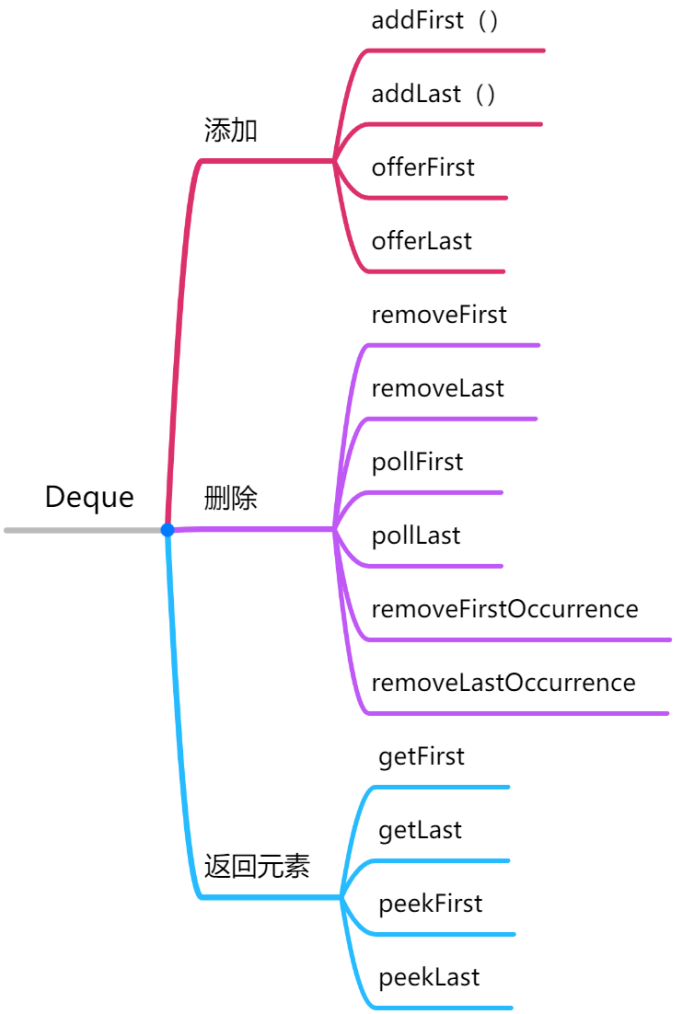


Queue接口

Queue接口也继承自Collenction。很多实现类既实现类List接口，又实现类Queue接口，如LinkedList,也有只实现了一个多，比如ArrayList就没有实现Queue接口，而PriorityQueue也没有实现List接口。这些交叉实现的类最容易混淆。

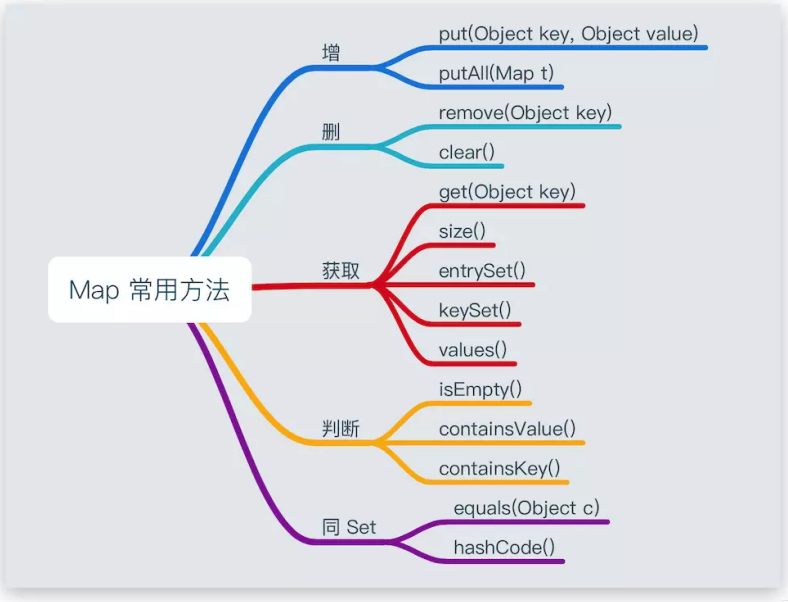


Deque接口继承自Queue,在Deque的基础上更加细化，可以选择从头部或者尾部操作。另外如果Deque中使用Queue的方法，一定要清楚实现类是从头部还是尾部操作的。



Map接口

思维导图：



Map是一组键值对，它是独立于Collection接口的。常用的实现类有：HashMap、TreeMap、HashTable、LinkedHashMap、ConcurrentHashMap等

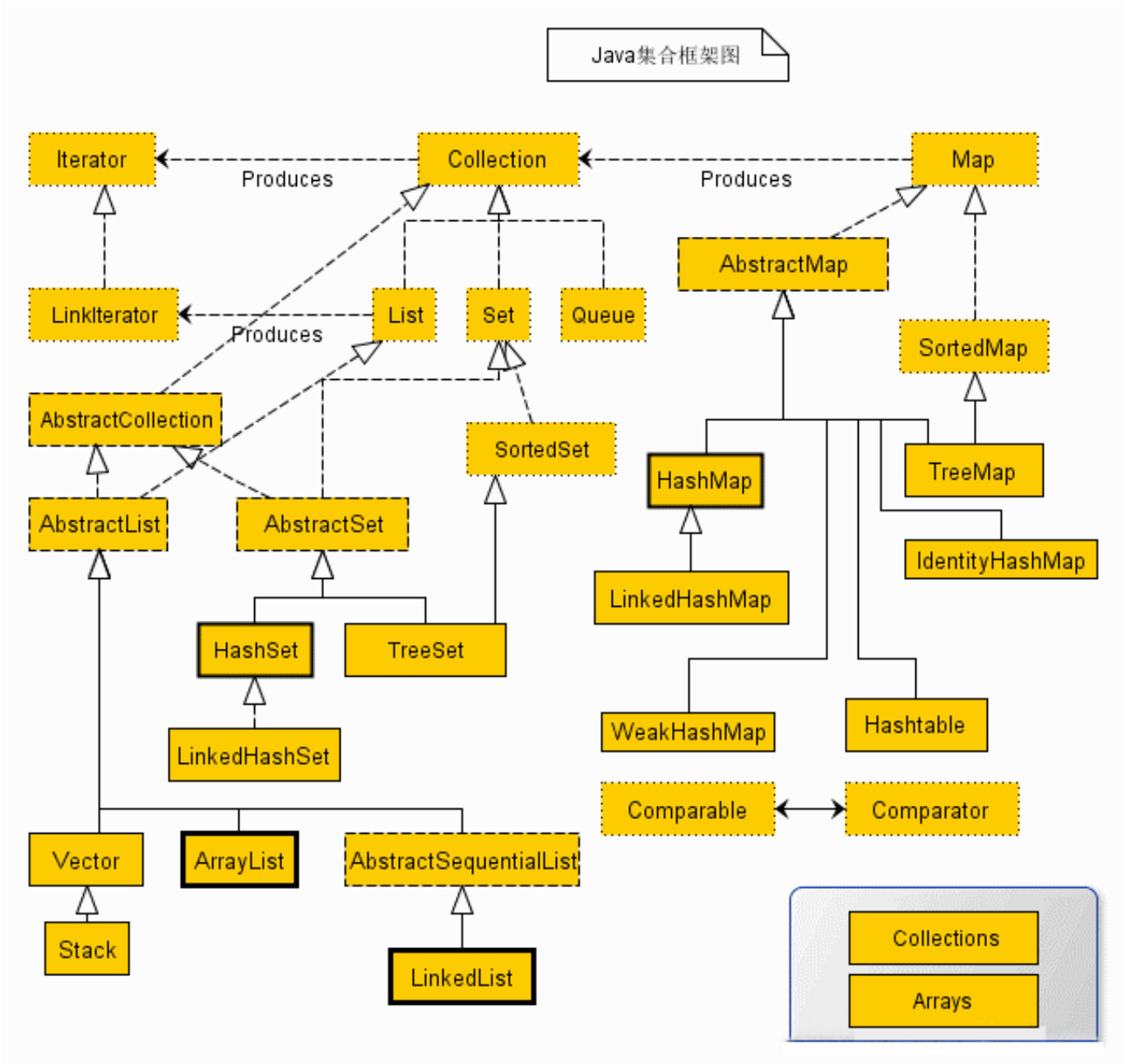
由双列集合继承关系图可知，Map是顶层接口，sortedMap接口提供排序选项。所有实现类都派生于AbstractMap抽象类。

集合基础

Java集合类有哪些

Java中的集合类主要由Collection和Map这两个接口派生而出，其中Collection接口又派生出三个子接口，分别是Set、List、Queue。所有的Java集合类，都是Set、List、Queue、Map这四个接口的实现类，这四个接口将集合分成了四大类。

- **Collection接口：** 是所有集合框架的根接口，包含了对集合进行基本操作的方法。
  - **List接口：** 有序集合，允许重复元素。常见的实现类有`ArrayList`、`LinkedList`等。
  - **Set接口：** 不允许重复元素的集合。常见的实现类有`HashSet`、`LinkedHashSet`、`TreeSet`等。
  - **Queue接口：** 用于表示队列的数据结构。常见的实现类有`LinkedList`、`PriorityQueue`等。
- **Map接口：** 表示键值对的集合。常见的实现类有`HashMap`、`LinkedHashMap`、`TreeMap`等。



集合框架底层使用了什么数据结构

1. List接口的实现

- **ArrayList**： 基于动态数组实现。底层使用数组作为存储结构。
- **LinkedList**： 基于双向链表实现。底层使用节点（Node）连接形成链表结构。
- **Vector**： 类似于 ArrayList，但是是线程安全的。底层也是使用数组实现。

2. Set接口

- **HashSet**： 基于哈希表实现。底层使用一个数组和链表/红黑树的结构来存储元素。
- **LinkedHashSet**： 在 HashSet 的基础上加入了链表，使得迭代顺序可预测。
- **TreeSet**： 基于红黑树实现。底层使用自平衡的二叉搜索树存储元素，以保持有序性。

3. Queue接口

- **LinkedList**： 同时实现了 List、Queue 和 Deque 接口。底层是基于双向链表的。
- **ArrayDeque**： 基于动态数组的双端队列。底层使用循环数组实现。
- **PriorityQueue**： 基于优先级堆实现的队列。底层使用数组表示的二叉堆。

4. Map接口

- **HashMap**： 基于哈希表实现。底层使用一个数组和链表/红黑树的结构来存储键值对。
- **LinkedHashMap**： 在 HashMap 的基础上加入了链表，使得迭代顺序可预测。
- **TreeMap**： 基于红黑树实现。底层使用自平衡的二叉搜索树存储键值对，以保持有序性。
- **Hashtable**： 类似于 HashMap，但是是线程安全的。底层也是使用哈希表。

有哪些集合是线程安全和线程不安全的

线程安全的集合类可以在多线程环境中安全地使用，而线程不安全的集合类在多线程环境中可能会导致不确定的行为。

线程安全

1. **`Vector`**： `Vector` 是一个古老的集合类，它的方法都是同步的，因此是线程安全的。然而，它相对较重，不够灵活，现在通常建议使用 `ArrayList`。
2. **`HashTable`**： `HashTable` 是一个古老的哈希表实现，其方法都是同步的，因此是线程安全的。但它的使用已经不太推荐，通常建议使用 `HashMap`。
3. **`Collections.synchronizedList`**、**`Collections.synchronizedSet`**、**`Collections.synchronizedMap`**： 这些方法可以将非线程安全的集合包装成线程安全的集合。

线程不安全

1. **`ArrayList`**、**`LinkedList`**、**`HashSet`**、**`HashMap`**： 这些集合类是非线程安全的。在多线程环境中，如果没有适当的同步措施，对这些集合的并发操作可能导致不确定的结果。
2. **`TreeMap`**、**`TreeSet`**： 虽然 `TreeMap` 和 `TreeSet` 是有序的集合，但它们也是非线程安全的。

List

ArrayList和Array有什么区别?

1. 大小和自动扩容

- 数组在创建时必须指定大小，且大小是固定的。一旦数组被创建，其大小不能更改。
- ArrayList是动态数组实现的，它的大小可以动态增长或缩小。在不断添加元素时，ArrayList 会自动进行扩容。

2. 支持泛型

- 数组可以存储任何类型的元素，但不支持泛型。
- ArrayList：支持泛型，可以指定存储的元素类型

3. 存储对象

- `Array` 可以直接存储基本类型数据，也可以存储对象。
- `ArrayList` 中只能存储对象。对于基本类型数据，需要使用其对应的包装类（如 Integer、Double 等）

4. 集合功能

- 数组： 是一个简单的数据结构，不提供额外的方法来进行元素的增删改查操作。
- ArrayList是集合框架的一部分，提供了丰富的方法，如添加、删除、查找等。

ArrayList 与 LinkedList 的区别?

1. 内部数据结构

- **ArrayList**： 基于动态数组实现。
- **LinkedList**： 基于双向链表实现。它使用节点来存储元素，每个节点包含一个数据元素和两个指针，分别指向前一个节点和后一个节点。

2. 遍历性能

- **ArrayList**： 支持快速的随机访问和遍历，因为可以直接通过索引访问元素。
- **LinkedList**： 随机访问性能较差，因为必须从链表的头部或尾部开始遍历，直到达到目标索引。

3. 插入和删除

- **ArrayList**： 在末尾进行插入和删除操作是高效的，但在中间或开头插入和删除需要移动元素，性能较差。
- **LinkedList**： 插入和删除元素的性能相对较好，特别是在链表中间或头尾插入和删除元素时。

4. 使用场景

- **ArrayList**： 适用于需要频繁随机访问元素，而对插入和删除操作要求不那么严格的场景。
- **LinkedList**： 适用于需要频繁插入和删除操作，而对随机访问的需求较少的场景。

ArrayList的扩容机制

ArrayList扩容的本质就是计算出新的扩容数组的size后实例化，并将原有数组内容复制到新数组中去。（不是原数组，而是新数组然后给予数组对象地址）。

默认情况下，新的容量会是原容量的1.5倍。 新容量=旧容量右移一位（相当于除于2）在加上旧容量

ArrayList 的底层是用动态数组来实现的。我们初始化一个ArrayList 集合还没有添加元素时，其实它是个空数组，只有当我们添加第一个元素时，内部会调用扩容方法并返回最小容量10，也就是说ArrayList 初始化容量为10。 当前数组长度小于最小容量的长度时（前期容量是10，当添加第11个元素时就就扩容），便开始可以扩容了，ArrayList 扩容的真正计算是在一个grow()里面，新数组大小是旧数组的1.5倍，如果扩容后的新数组大小还是小于最小容量，那新数组的大小就是最小容量的大小，后面会调用一个Arrays.copyof方法，这个方法是真正实现扩容的步骤。

Set

HashSet、LinkedHashSet 和 TreeSet 有什么区别

1. HashSet

- `HashSet` 是基于哈希表实现的，使用了散列函数来存储元素。
- 不保证元素的顺序，因此遍历时的顺序可能是不确定的。
- 允许存储一个 `null` 元素。
- 查找、插入、删除等操作的平均时间复杂度是 O(1)。

2. LinkedHashSet

- `LinkedHashSet` 继承自 `HashSet`，内部使用链表维护元素的插入顺序。
- 保留了元素的插入顺序，遍历时按照插入顺序输出。
- 允许存储一个 `null` 元素。
- 查找、插入、删除等操作的平均时间复杂度是 O(1)。

3. Treeset

- `TreeSet` 是基于红黑树实现的，存储元素时按照元素的自然排序或者指定的比较器排序。
- 元素是有序的，遍历时按照排序顺序输出。
- 不允许存储 `null` 元素。
- 查找、插入、删除等操作的平均时间复杂度是 O(log n)。

4. 总结

- `HashSet` 使用哈希表，不保证元素顺序。
- `LinkedHashSet` 继承自 `HashSet`，使用链表维护插入顺序。
- `TreeSet` 使用红黑树，保证元素有序。

Map和Set有什么区别?

Set代表无序的，元素不可重复的集合；

Map代表具有映射关系（key-value）的集合，其所有的key是一个Set集合，即key无序且不能重复。

List和Map有什么区别

Set代表无序的，元素不可重复的集合；

List代表有序的，元素可以重复的集合。

Queue

ArrayDeque 与 LinkedList 的区别

1. **内部实现：**
   - `ArrayDeque` 使用数组作为底层数据结构来实现双端队列。它的数组会动态地调整大小以适应元素的添加和删除。
   - `LinkedList` 使用双向链表作为底层数据结构。每个节点包含了对前一个节点和后一个节点的引用。
2. **随机访问性能：**
   - `ArrayDeque` 支持高效的随机访问，因为它使用数组，可以通过索引直接访问元素。
   - `LinkedList` 的随机访问性能较差，因为要从头或尾开始遍历链表才能找到指定位置的元素。
3. **空间复杂度：**
   - `ArrayDeque` 在存储元素时通常比 `LinkedList` 更节省空间，因为它不需要存储每个节点的前后引用。
   - `LinkedList` 中的每个节点都需要额外的空间来存储前一个节点和后一个节点的引用。
4. **操作复杂度：**
   - `ArrayDeque` 在头部和尾部进行插入和删除操作的平均时间复杂度是 O(1)。在中间插入或删除元素的操作相对较慢。
   - `LinkedList` 在头部和尾部进行插入和删除操作的平均时间复杂度也是 O(1)。但是由于涉及到节点的指针操作，性能可能相对较低。
5. **迭代性能：**
   - `ArrayDeque` 在迭代元素时通常比 `LinkedList` 更快，因为数组的元素在内存中是连续存储的，对缓存更友好。
   - `LinkedList` 的迭代性能较差，因为节点在内存中是分散存储的。
6. **线程安全：**
   - `ArrayDeque` 和 `LinkedList` 都不是线程安全的。如果在多线程环境中使用，需要进行外部同步。
7. 总结

如果需要高效的随机访问和更好的空间利用，可以选择 `ArrayDeque`。如果需要高效的插入和删除操作，可以选择 `LinkedList`。

说一说你对BlockingQueue的了解

`BlockingQueue` 主要用于在多线程之间安全地传递数据，并提供了阻塞操作，以便在队列为空或队列已满时进行等待或阻塞。

1. **阻塞操作：**
   - `BlockingQueue` 提供了阻塞操作，包括 `put` 和 `take` 方法。
   - `put` 方法用于向队列中添加元素，如果队列已满，则会阻塞直到有空间可用。
   - `take` 方法用于从队列中获取元素，如果队列为空，则会阻塞直到有元素可用。
2. **超时操作：**
   - `offer` 和 `poll` 方法允许设置超时参数，以便在超时后返回而不是无限期等待。
   - `offer` 方法用于向队列中添加元素，如果队列已满，则会等待指定的超时时间，超时后返回 `false`。
   - `poll` 方法用于从队列中获取元素，如果队列为空，则会等待指定的超时时间，超时后返回 `null`。
3. **容量限制：**

- `BlockingQueue` 可以具有有界或无界的容量。有界队列的容量是固定的，而无界队列可以无限制地添加元素。

4. **支持公平性：**
   - 一些 `BlockingQueue` 的实现支持公平性，即在多个等待线程中，按照等待时间的顺序选择下一个执行的线程。
5. **常见实现类：**
   - `ArrayBlockingQueue`：基于数组实现的有界队列。
   - `LinkedBlockingQueue`：基于链表实现的有界或无界队列。
   - `PriorityBlockingQueue`：基于优先级的无界队列。
   - `DelayQueue`：用于实现延迟任务的无界队列。

Map 【常考】

Map接口有那些实现类

Map接口有很多实现类，其中比较常用的有HashMap、LinkedHashMap、TreeMap、ConcurrentHashMap。

对于不需要排序的场景，优先考虑使用HashMap，因为它是性能最好的Map实现。如果需要保证线程安全，则可以使用ConcurrentHashMap。它的性能好于Hashtable，因为它在put时采用分段锁/CAS的加锁机制，而不是像Hashtable那样，无论是put还是get都做同步处理。

对于需要排序的场景，如果需要按插入顺序排序则可以使用LinkedHashMap，如果需要将key按自然顺序排列甚至是自定义顺序排列，则可以选择TreeMap。如果需要保证线程安全，则可以使用Collections工具类将上述实现类包装成线程安全的Map。

HashMap和Hashtabe的区别

> 如果不需要线程安全性，并且更关注性能，通常会选择使用 `HashMap`。如果需要线程安全性，可以考虑使用 `Hashtable`，或者使用 `ConcurrentHashMap`，

1. 同步

`Hashtable` 是同步的，即它的方法是线程安全的。这是通过在每个方法上添加同步关键字来实现的，但这也可能导致性能下降。

`HashMap` 不是同步的，因此它不保证在多线程环境中的线程安全性。如果需要同步，可以使用 `Collections.synchronizedMap()` 方法来创建一个同步的 `HashMap`。

2. 性能

- 由于 `Hashtable` 是同步的，它在多线程环境中的性能可能较差。
- `HashMap` 在单线程环境中可能比 `Hashtable` 更快，因为它没有同步开销。

3. 空值

- `Hashtable` 不允许键或值为 null。
- `HashMap` 允许键和值都为 null。

4. 继承关系

`Hashtable` 是 `Dictionary` 类的子类，而 `HashMap` 是 `AbstractMap` 类的子类，实现了 `Map` 接口。

5. 迭代器

- `Hashtable` 的迭代器是通过 `Enumerator` 实现的。
- `HashMap` 的迭代器是通过 `Iterator` 实现的。

6. 初始容量和加载因子

- `Hashtable` 的初始容量和加载因子是固定的。
- `HashMap` 允许通过构造方法设置初始容量和加载因子，以便更好地调整性能。

HashMap和HashSet的区别

`HashMap` 适用于需要存储键值对的情况，而 `HashSet` 适用于只关心元素唯一性的情况。在某些情况下，可以使用 `HashMap` 来模拟 `HashSet` 的行为，只使用键而将值设为固定的常量。

1. 使用
    - `HashMap` 用于存储键值对，其中每个键都唯一，每个键关联一个值。
    - `HashSet` 用于存储唯一的元素，不允许重复。
2. **内部实现：**
    - `HashMap` 使用键值对的方式存储数据，通过哈希表实现。
    - `HashSet` 实际上是基于 `HashMap` 实现的，它只使用了 `HashMap` 的键部分，将值部分设置为一个固定的常量。
3. **元素类型：**
    - `HashMap` 存储键值对，可以通过键获取对应的值。
    - `HashSet` 存储单一元素，只能通过元素本身进行操作。
4. **允许 null：**
    - `HashMap` 允许键和值都为 null。
    - `HashSet` 允许存储一个 null 元素。
5. **迭代方式：**
    - `HashMap` 的迭代是通过迭代器或增强型 for 循环遍历键值对。
    - `HashSet` 的迭代是通过迭代器或增强型 for 循环遍历元素。
6. **关联关系：**
    - `HashMap` 中的键与值是一一对应的关系。
    - `HashSet` 中的元素没有关联的值，只有元素本身。
7. **性能影响：**
    - `HashMap` 的性能受到键的哈希分布和哈希冲突的影响。
    - `HashSet` 的性能也受到元素的哈希分布和哈希冲突的影响，但由于它只存储键，通常比 `HashMap` 的性能稍好。

HashMap和ConcurrentHashMap的区别

1. **线程安全性：**
    - `HashMap` 不是线程安全的。在多线程环境中，如果同时进行读写操作，可能会导致数据不一致或抛出异常。
    - `ConcurrentHashMap` 是线程安全的，它使用了分段锁（Segment Locking）的机制，将整个数据结构分成多个段（Segment），每个段都有自己的锁。这样，不同的线程可以同时访问不同的段，提高并发

性能。

2. **同步机制：**
   - `HashMap` 在实现上没有明确的同步机制，需要在外部进行同步，例如通过使用 `Collections.synchronizedMap()` 方法。
   - `ConcurrentHashMap` 内部使用了一种更细粒度的锁机制，因此在多线程环境中具有更好的性能。
3. **迭代时是否需要加锁：**
   - 在 `HashMap` 中，如果在迭代过程中有其他线程对其进行修改，可能抛出 `ConcurrentModificationException` 异常。
   - `ConcurrentHashMap` 允许在迭代时进行并发的插入和删除操作，而不会抛出异常。但是，它并不保证迭代器的顺序，因为不同的段可能会以不同的顺序完成操作。
4. **初始化容量和负载因子：**
   - `HashMap` 可以通过构造方法设置初始容量和负载因子。
   - `ConcurrentHashMap` 在Java 8及之后版本中引入了`ConcurrentHashMap(int initialCapacity, float loadFactor, int concurrencyLevel)`构造方法，允许设置初始容量、负载因子和并发级别。
5. **性能：**
   - 在低并发情况下，`HashMap` 的性能可能会比 `ConcurrentHashMap` 稍好，因为 `ConcurrentHashMap` 需要维护额外的并发控制。
   - 在高并发情况下，`ConcurrentHashMap` 的性能通常更好，因为它能够更有效地支持并发访问。

总的来说，如果需要在多线程环境中使用哈希表，而且需要高性能的并发访问，通常会选择使用 `ConcurrentHashMap`。如果在单线程环境中使用，或者能够手动进行外部同步管理，那么 `HashMap` 可能是更简单的选择。

JDK7和JDK8中的HashMap有什么区别?

1. **底层数据结构：**
   - JDK 7 中的 `HashMap` 使用的是数组 + 链表的实现方式，即拉链法。
   - JDK 8 引入了红黑树（Red-Black Tree）来替代链表，以提高在链表长度较长时的查找性能。这种结构被称为树化桶（Tree Bins）。
2. **树化阈值：**
   - JDK 7 中，链表长度超过8时，会将链表转换为红黑树。
   - JDK 8 中，红黑树的树化阈值默认为8，但可以通过 `TREEIFY_THRESHOLD` 进行调整。
3. **扩容机制：**
   - JDK 7 中，`HashMap` 的扩容机制是当元素个数超过容量的 75% 时触发扩容。
   - JDK 8 中，扩容的触发条件变为元素个数超过容量的 75%，并且需要进行树化操作的桶的个数超过容量的 6.25%。
4. **并发性能改进：**
   - JDK 8 对 `HashMap` 进行了一些并发性能的优化，减小了锁粒度，提高了并发访问的效率。
5. **新的方法和接口：**
   - JDK 8 引入了新的方法和接口，如 `forEach` 方法，支持更方便的遍历操作。
   - 接口 `Map.Entry` 中引入了一些新的方法，如 `getKey` 和 `getValue` 的默认方法。
6. **`compute` 方法：**
   - JDK 8 引入了 `compute` 系列的方法，允许在并发环境中原子地计算键的值。

HashMap如何解决哈希冲突的

`HashMap` 主要使用拉链法来解决哈希冲突

HashMap如何解决哈希冲突

HashMap为什么线程不安全？如何实现线程安全

主要原因是它的操作不是原子的，即在多个线程同时进行读写操作时，可能会导致数据不一致性或抛出异常.

1. **并发修改：**当一个线程进行写操作（插入、删除等）时，另一个线程进行读操作，可能会导致读取到不一致的数据，甚至抛出 `ConcurrentModificationException` 异常。
2. **非原子性操作：** `HashMap` 的一些操作不是原子的，例如，检查是否存在某个键、获取某个键对应的值等，这样在多线程环境中可能发生竞态条件。

为了实现线程安全的 `HashMap`，有以下几种方式：

- **使用`Collections.synchronizedMap()`方法：**可以通过 `Collections.synchronizedMap()` 方法创建一个线程安全的 `HashMap`，该方法返回一个同步的 `Map` 包装器，使得所有对 `Map` 的操作都是同步的。

```
Map<String, String> synchronizedMap = Collections.synchronizedMap(new HashMap<>());
```

- **使用`ConcurrentHashMap`：** `ConcurrentHashMap` 是专门设计用于多线程环境的哈希表实现。它使用分段锁机制，允许多个线程同时进行读操作，提高并发性能。

```
Map<String, String> concurrentHashMap = new ConcurrentHashMap<>();
```

- **使用锁机制：**可以在自定义的 `HashMap` 操作中使用显式的锁（例如 `ReentrantLock`）来保证线程安全。

```
Map<String, String> customMap = new HashMap<>();
ReentrantLock lock = new ReentrantLock();

// 在需要线程安全的操作中使用锁
lock.lock();
try {
    // 执行操作
} finally {
    lock.unlock();
}
```

HashMap中的循环链表是如何产生的?

循环链表是由哈希冲突引起的，当多个键映射到同一个桶时，它们会按照插入的顺序形成一个链表。如果链表长度过长，查找特定键的效率就会降低。为了解决这个问题，`HashMap` 在一定条件下选择将链表转换为红黑树，提高查找性能。

JDK 1.7 hashmap循环链表的产生

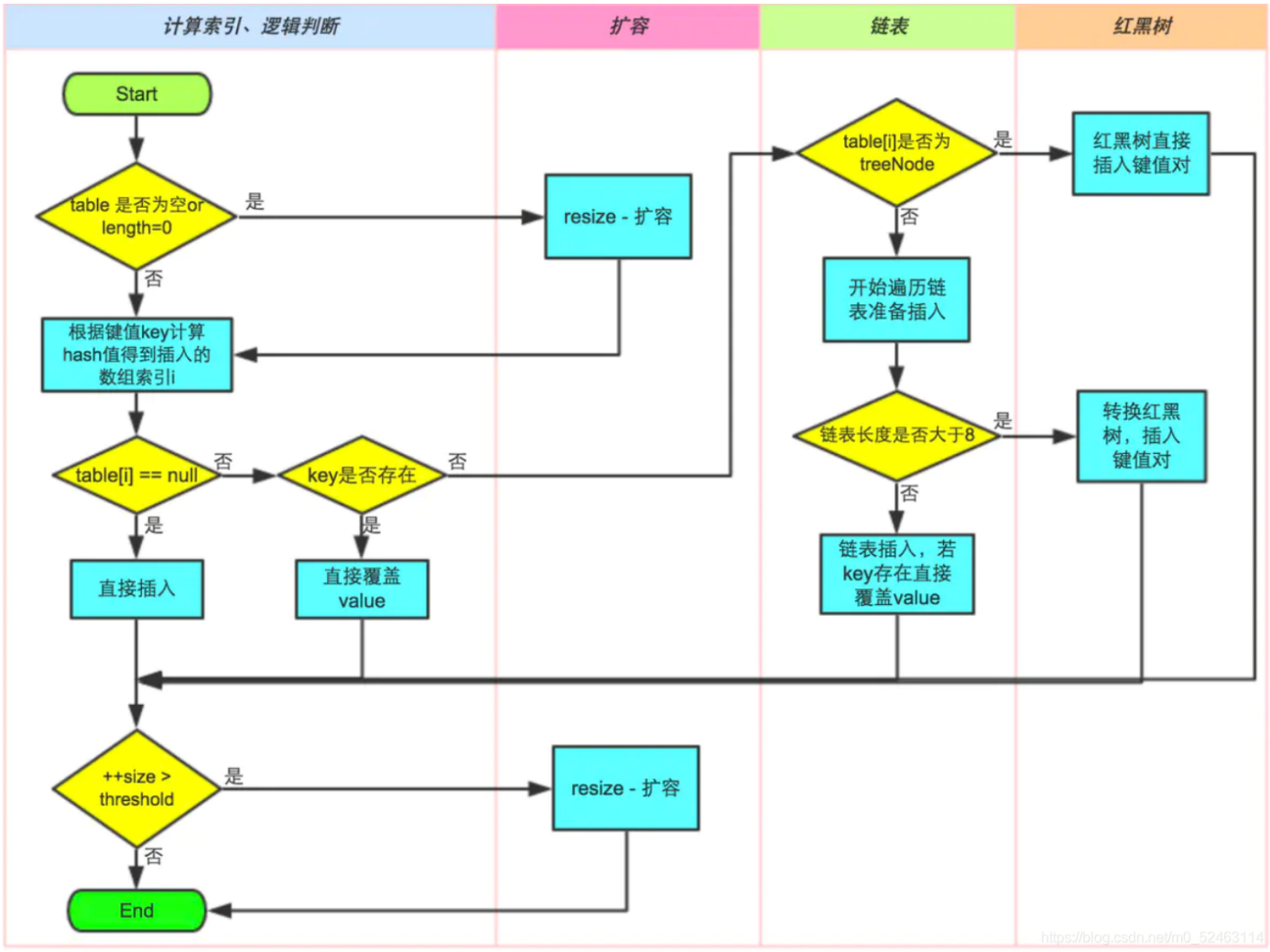
HashMap底层的实现原理和扩容机制

Hashmap实现原理及扩容机制详解

ConcurrentHashMap是怎么实现的?

深入浅出ConcurrentHashMap详解

Map put的过程



1. 首次扩容：

先判断数组是否为空，若数组为空则进行第一次扩容（resize）；

2. 计算索引：

通过hash算法，计算键值对在数组中的索引；

3. 插入数据：

如果当前位置元素为空，则直接插入数据；
如果当前位置元素非空，且key已存在，则直接覆盖其value；
如果当前位置元素非空，且key不存在，则将数据链到链表末端；
若链表长度达到8，则将链表转换成红黑树，并将数据插入树中；
再次扩容

4. 如果数组中元素个数（size）超过threshold，则再次进行扩容操作。

Map的遍历方式

Map的6种遍历方式

IO

IO流基本认识

Java中的I/O（Input/Output）流是用于处理输入和输出的机制。I/O 流以字节（byte）为基本单位，提供了一种灵活的方式来读取和写入数据。I/O 流分为输入流和输出流，根据数据的流向分为输入和输出。

Java的I/O流主要分为两大类：字节流和字符流。字节流用于处理原始的二进制数据，而字符流用于处理文本数据。

1. 字节流

- **InputStream 和 OutputStream**： 是所有字节输入流和输出流的抽象基类。它们分别用于读取和写入字节。
- **FileInputStream 和 FileOutputStream**： 用于从文件中读取字节和向文件中写入字节。
- **ByteArrayInputStream 和 ByteArrayOutputStream**： 分别用于从字节数组中读取数据和将数据写入字节数组。

```
InputStream in = new FileInputStream("input.txt");
OutputStream out = new FileOutputStream("output.txt");

FileInputStream fis = new FileInputStream("file.txt");
FileOutputStream fos = new FileOutputStream("output.txt");

ByteArrayInputStream bais = new ByteArrayInputStream(byteArray);
ByteArrayOutputStream baos = new ByteArrayOutputStream();
```

2. 字符流

- **Reader 和 Writer**： 是所有字符输入流和输出流的抽象基类。它们分别用于读取和写入字符。
- **FileReader 和 FileWriter**： 用于从文件中读取字符和向文件中写入字符。
- **BufferedReader 和 BufferedWriter**： 用于提供缓冲区，提高读取和写入的效率。

```
Reader reader = new FileReader("input.txt");
Writer writer = new FileWriter("output.txt");

FileReader fr = new FileReader("file.txt");
FileWriter fw = new FileWriter("output.txt");

BufferedReader br = new BufferedReader(new FileReader("file.txt"));
BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));
```

3. 高级流

- **ObjectInputStream 和 ObjectOutputStream**： 用于读取和写入对象。可以序列化和反序列化对象。

- **DataInputStream 和 DataOutputStream**： 用于读取和写入基本数据类型。

```
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("object.dat"));
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("object.dat"));
```

谈一下你对IO流的基本理解

> 从字符流和字节流两方面回答包括输入输出以及两者对比

数据流的基本概念

几乎所有的程序都离不开信息的输入和输出，比如从键盘读取数据，从文件中获取或者向文件中存入数 据，在显示器上显示数据。这些情况下都会涉及有关输入/输出的处理。

在Java中，把这些不同类型的输入、输出源抽象为流(Stream)，其中输入或输出的数据称为数据流 (Data Stream)，用统一的接口来表示。

IO流的分类

数据流是指一组有顺序的、有起点和终点的字节集合。 按照流的流向分，可以分为输入流和输出流。注意:这里的输入、输出是针对程序来说的。

输出:把程序(内存)中的内容输出到磁盘、光盘等存储设备中。
输入:读取外部数据(磁盘、光盘等存储设备的数据)到程序(内存)中。

按处理数据单位不同分为字节流和字符流。字节流:每次读取(写出)一个字节，当传输的资源文件有中 文时，就会出现乱码。

字符流:每次读取(写出)两个字节，有中文时，使用该流就可以正确传输显示中文。 1字符 = 2字节; 1字节(byte) = 8位(bit); 一个汉字占两个字节长度。

按照流的角色划分为节点流和处理流。节点流:从或向一个特定的地方(节点)读写数据。如 FileInputStream。

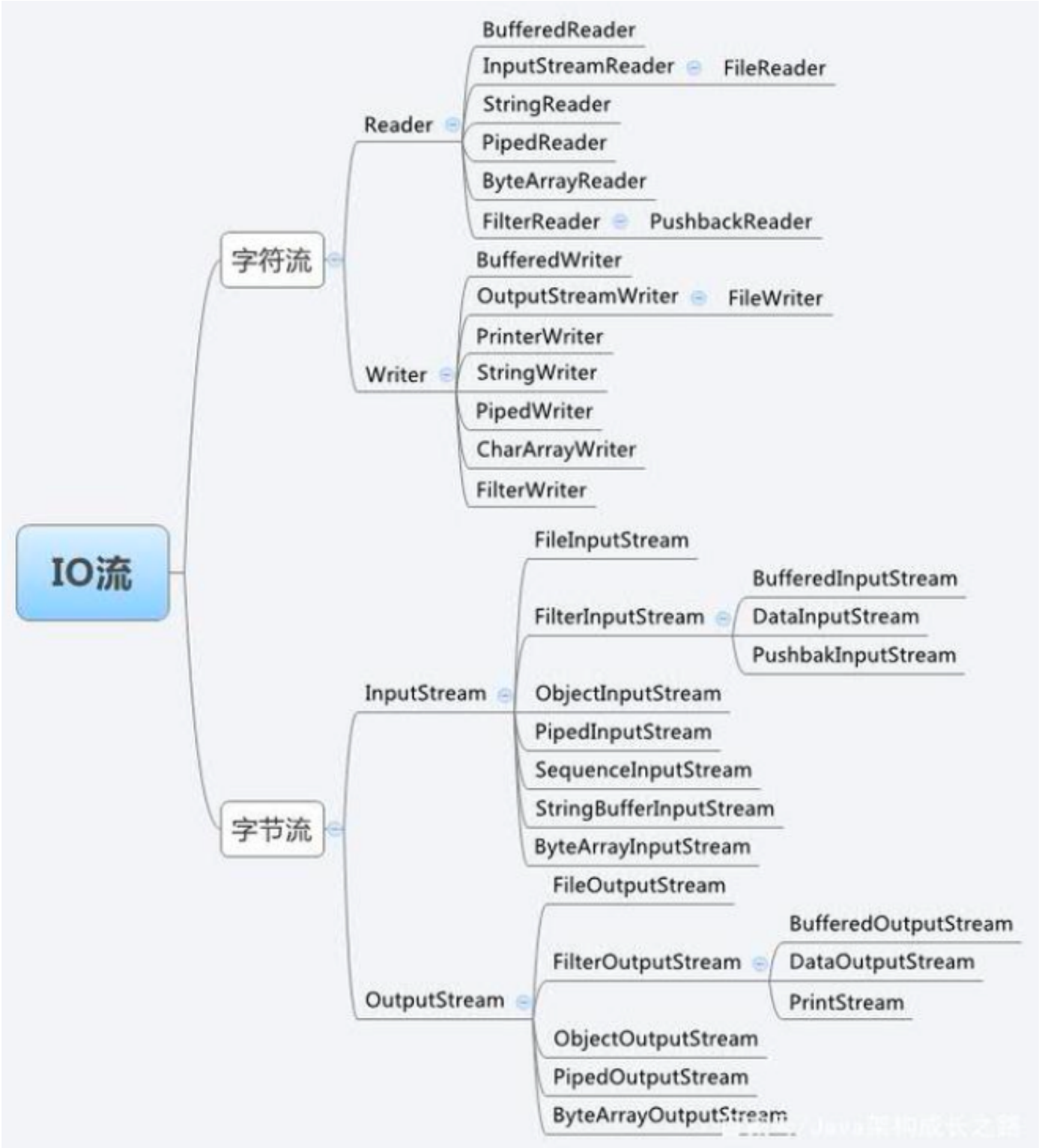
处理流(包装流):是对一个已存在的流的连接和封装，通过所封装的流的功能调用实现数据读写。如 BufferedReader。处理流的构造方法总是要带一个其他的流对象做参数。一个流对象经过其他流的多 次包装，称为流的链接。

注意:一个IO流可以既是输入流又是字节流又或是以其他方式分类的流类型，是不冲突的。比如 FileInputStream，它既是输入流又是字节流还是文件节点流。

Java IO 流有4个抽象基类:

- 输入字节流： `InputStream`
- 输入字符流： `Reader`
- 输出字节流： `OutputStream`
- 输出字符流： `Writer`

其他流都是继承于这四大基类的。下图是Java IO 流的整体架构图:



知道了 IO 流有这么多分类，那我们在使用的时候应该怎么选择呢?比如什么时候用输出流?什么时候 用字节流?可以根据下面三步选择适合自己的流:

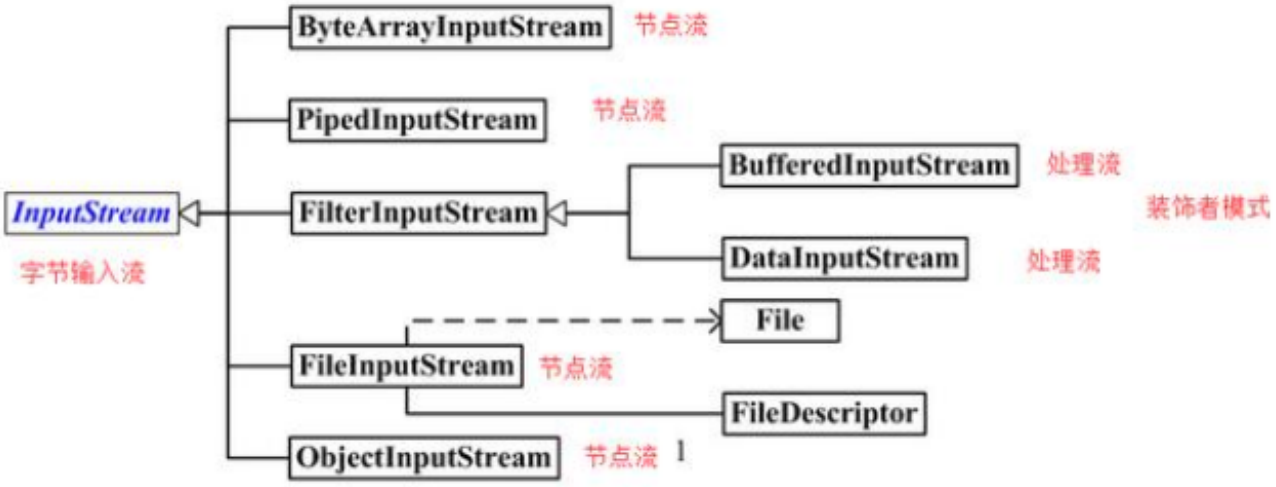
首先自己要知道是选择输入流还是输出流。这就要根据自己的情况决定，如果想从程序写东西到别的地 方，那么就选择输入流，反之就选输出流;然后考虑你传输数据时，是每次传一个字节还是两个字节， 每次传输一个字节就选字节流，如果存在中文，那肯定就要选字符流了。通过前面两步就可以选出一个 合适的节点流了，比如字节输入流 InputStream，如果要在此基础上增强功能，那么就在处理流中选择 一个合适的即可。

字节输入流

java.io 包下所有的字节输入流都继承自 InputStream，并且实现了其中的方法。InputStream 中提供的主要数据操作方法如下:

- int read():从输入流中读取一个字节的二进制数据。
- int read(byte[] b):将多个字节读到数组中，填满 整个数组
- int read(byte[] b, int off, int len):从输入流中读取长度为 len 的数据，从数组 b 中下标为 off 的位置开始放置读入的数据，读完返回读取的字节数。v
- oid close():关闭数据流。
- int available(): 返回目前可以从数据流中读取的字节数(但实际的读操作所读得的字节数可能大于该返回值)
- long skip(long l):跳过数据流中指定数量的字节不读取，返回值表示实际跳过的字节数。对数据流中字节的 读取通常是按从头到尾顺序进行的，如果需要以反方向读取，则需要使用回推(Push Back)操作。在 支持回推操作的数据流中经常用到如下几个方法:
    - boolean markSupported():用于测试数据流是否支持回推操作，当一个数据流支持 mark() 和 reset() 方法时，返回 true，否则返回 false。
    - void mark(int readlimit):用于标记数据流的当前位置，并划出 一个缓冲区，其大小至少为指定参数的大小。
    - void reset():将输入流重新定位到对此流最后调用 mark() 方法时的位置。

字节输入流 InputStream 有很多子类，日常开发中，经常使用的一些类见下 图:



ByteArrayInputStream:字节数组输入流，该类的功能就是从字节数组 byte[] 中进行以字节为单位的 读取，也就是将资源文件都以字节形式存入到该类中的字节数组中去，我们拿数据也是从这个字节数组 中拿。

PipedInputStream:管道字节输入流，它和 PipedOutputStream 一起使用，能实现多线程间的 管道通信。

FilterInputStream:装饰者模式中充当装饰者的角色，具体的装饰者都要继承它，所以在该 类的子类下都是用来装饰别的流的，也就是处理类。

BufferedInputStream:缓冲流，对处理流进行装 饰、增强，内部会有一个缓冲区，用来存放字节，每次都是将缓冲区存满然后发送，而不是一个字节或 两个字节这样发送，效率更高。

DataInputStream:数据输入流，用来装饰其他输入流，它允许通过数 据流来读写Java基本类型。

FileInputStream:文件输入流，通常用于对文件进行读取操作。File:对指 定目录的文件进行操作。

ObjectInputStream:对象输入流，用来提供对“基本数据或对象”的持久存 储。通俗点讲，就是能直接传输Java对象(序列化、反序列化用)。

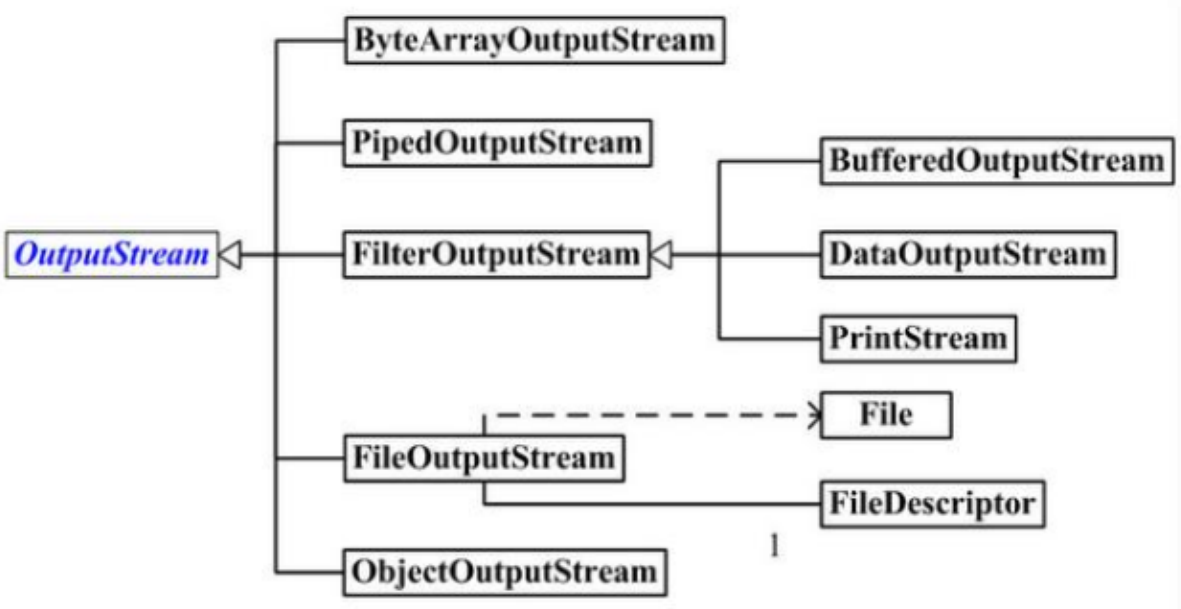
字节输出流

与字节输入流类似，java.io 包下所有字节输出流大多是从抽象类 OutputStream 继承而来的。

OutputStream 提供的主要数据操作方法:

- void write(int i):将字节 i 写入到数据流中，它只输出所读入参数的最低 8 位，该方法是抽象方法，需 要在其输出流子类中加以实现，然后才能使用。
- void write(byte[] b):将数组 b 中的全部 b.length 个字 节写入数据流。
- void write(byte[] b, int off, int len):将数组 b 中从下标 off 开始的 len 个字节写入数据 流。元素 b[off] 是此操作写入的第一个字节，b[off + len - 1] 是此操作写入的最后一个字节。
- void close():关闭输出流。
- void flush():刷新此输出流并强制写出所有缓冲的输出字节。为了加快数据传输 速度，提高数据输出效率，又是输出数据流会在提交数据之前把所要输出的数据先暂时保存在内存缓冲 区中，然后成批进行输出，每次传输过程都以某特定数据长度为单位进行传输，在这种方式下，数据的 末尾一般都会有一部分数据由于数量不够一个批次，而存留在缓冲区里，调用 flush() 方法可以将这部 分数据强制提交。

IO 中输出字节流的继承图可见下图:



作用可以参考上面字节输入流中的各个子类的介绍。

字符流

字符流

从JDK1.1开始，java.io 包中加入了专门用于字符流处理的类，它们是以Reader和Writer为基础派生的 一系列类。

同其他程序设计语言使用ASCII字符集不同，Java使用Unicode字符集来表示字符串和字符。ASCII字符 集以一个字节(8bit)表示一个字符，可以认为一个字符就是一个字节(byte)。但Java使用的 Unicode是一种大字符集，用两个字节(16bit)来表示一个字符，这时字节与字符就不再相同。为了实 现与其他程序语言及不同平台的交互，Java提供一种新的数据流处理方案，称作读者(Reader)和写者 (Writer)。

字符输入流

Reader是所有的输入字符流的父类，它是一个抽象类。

Reader及其一些常用子类:

CharReader和SringReader是两种基本的介质流，它们分别将Char数组、String中读取数据。

PipedReader 是从与其它线程共用的管道中读取数据。

BufferedReader很明显是一个装饰器，它和其他子类负责装饰其他Reader对象。

FilterReader是所有自定义具体装饰流的父类，其子类PushBackReader对Reader对象进行装饰，会增 加一个行号。

InputStreamReader是其中最重要的一个，用来在字节输入流和字符输入流之间作为中介，可以将字节 输入流转换为字符输入流。

FileReader 可以说是一个达到此功能、常用的工具类，在其源代码中明显使用了将FileInputStream 转 变为 Reader 的方法。

Reader 中各个类的用途和使用方法基本和InputStream 中的类使用一致。

字符输出流

Writer是所有的输出字符流的父类，它是一个抽象类。

Writer及其一些常用子类:

CharWriter、StringWriter 是两种基本的介质流，它们分别向Char 数组、String 中写入数据。

PipedWriter 是向与其它线程共用的管道中写入数据。BufferedWriter 是一个装饰器为Writer 提供缓冲 功能。

PrintWriter 和PrintStream 极其类似，功能和使用也非常相似。OutputStreamWriter是其中最重要的 一个，用来在字节输出流和字符输出流之间作为中介，可以将字节输出流转换为字符输出流。

FileWriter 可以说是一个达到此功能、常用的工具类，在其源代码中明显使用了将OutputStream转变 为Writer 的方法。

Writer 中各个类的用途和使用方法基本和OutputStream 中的类使用一致。

字节流和字符流的区别

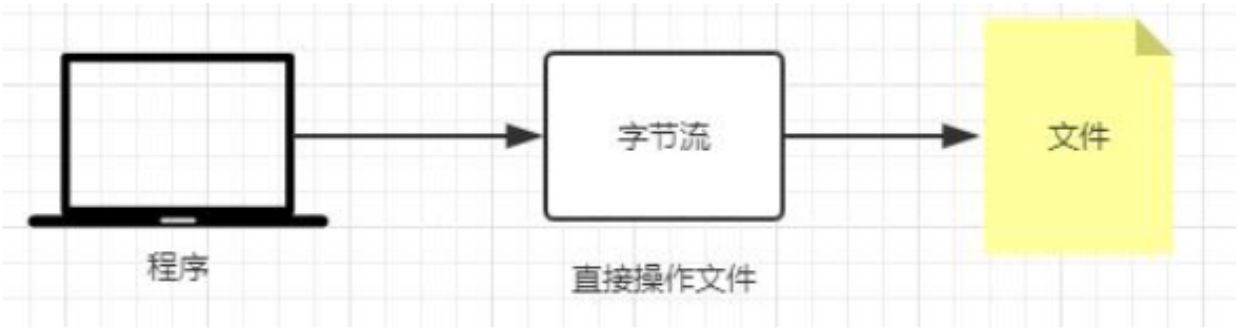
1、要把一片二进制数据数据逐一输出到某个设备中，或者从某个设备中逐一读取一片二进制数据，不 管输入输出设备是什么，我们要用统一的方式来完成这些操作，用一种抽象的方式进行描述，这个抽象 描述方式起名为IO流，对应的抽象类为OutputStream和InputStream ，不同的实现类就代表不同的输 入和输出设备，它们都是针对字节进行操作的。

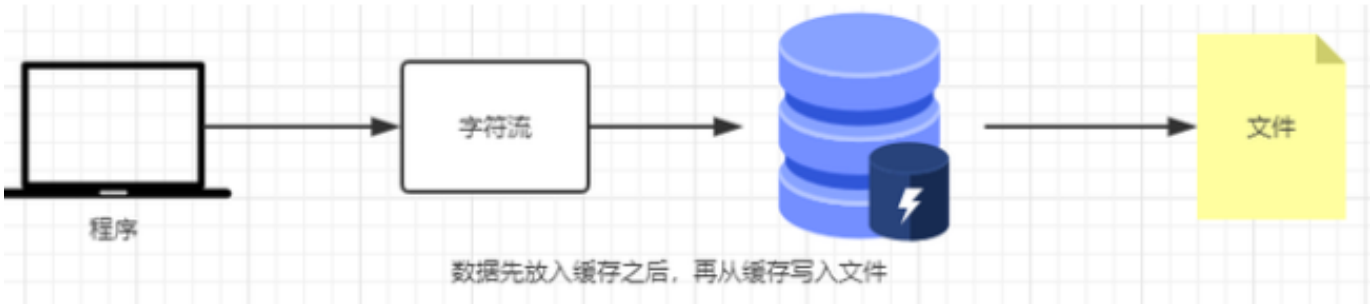
2、在应用中，经常要完全是字符的一段文本输出去或读进来，用字节流可以吗?计算机中的一切最终 都是二进制的字节形式存在。对于“中国”这些字符，首先要得到其对应的字节，然后将字节写入到输出 流。读取时，首先读到的是字节，可是我们要把它显示为字符，我们需要将字节转换成字符。由于这样 的需求很广泛，人家专门提供了字符流的包装类。

3、底层设备永远只接受字节数据，有时候要写字符串到底层设备，需要将字符串转成字节再进行写 入。字符流是字节流的包装，字符流则是直接接受字符串，它内部将串转成字节，再写入底层设备，这 为我们向IO设别写入或读取字符串提供了一点点方便。

4、字符向字节转换时，要注意编码的问题，因为字符串转成字节数组，其实是转成该字符的某种编码 的字节形式，读取也是反之的道理。

字节流在操作时本身不会用到缓冲区(内存)，是文件本身直接操作的;而字符流在操作时使用了缓冲区，通过缓冲区再操作文件。





使用场景

字节流一般用来处理图像，视频，以及PPT，Word类型的文件。字符流一般用于处理纯文本类型的文 件，如TXT文件等。字节流可以用来处理纯文本文件，但是字符流不能用于处理图像视频等非文本类型 的文件。

I/O模型

1. BIO(Blocking I/O阻塞I/O模型):

BIO 属于同步阻塞 IO 模型，读取或写入数据时，线程将一直等待，直到数据准备就绪或者写入操作完成， 但在高并发环境下可能导致性能问题，因为线程在等待 I/O 操作完成时被阻塞，无法执行其他任务。

2. NIO(**Non-blocking I/O**):

在非阻塞 I/O 模型中，线程执行一个 I/O 操作时不会等待，而是继续执行其他任务, 这需要通过轮询（polling）或者使用回调函数等机制来检查 I/O 操作是否完成。

这种模型相对于阻塞 I/O 可以更好地支持并发，但轮询的方式可能会导致 CPU 资源的浪费。

3. IO多路复用

I/O 多路复用模型使用了操作系统提供的选择器（Selector）机制，例如 Java 中的 `Selector` 类。通过选择器，一个线程可以监听多个通道上的 I/O 事件，从而在单线程中处理多个连接。

4. AIO（Asynchronous I/O）

异步I/O允许程序在执行I/O操作时继续执行其他任务，而不需要等待I/O操作完成。在Java中，AIO主要是通过Java NIO.2中的`AsynchronousChannel`和`CompletionHandler`接口来实现的。

AIO 允许程序发起一个I/O操作，并在操作完成时得到通知。在这个过程中，程序可以继续执行其他任务而无需等待I/O操作完成，当操作完成之后，进行回调。

BIO/NI0/AIO的区别

1. BIO

- **工作原理：** 阻塞 I/O 模型中，当一个线程执行 I/O 操作时，它会被阻塞，直到 I/O 操作完成。这会导致线程无法执行其他任务。
- **适用场景：** 适用于连接数较少、并发不高的场景，例如传统的 Socket 编程。

2. NIO

- **工作原理：** 非阻塞 I/O 模型中，一个线程可以处理多个连接，而不需要等待每个连接的 I/O 操作完成。但线程需要通过轮询（polling）或者选择器（Selector）来检查哪些连接已经准备好进行 I/O 操作。
- **适用场景：** 适用于高并发、连接数较多的场景。Java NIO 提供了 `Selector`、`Channel` 等组件，可以更好地支持多连接的管理。

3. AIO

- **工作原理：** 异步 I/O 模型中，程序发起 I/O 操作后，可以继续执行其他任务。当 I/O 操作完成时，系统会通知程序，并调用相应的回调函数。
- **适用场景：** 适用于需要处理大量并发连接的场景，并且希望充分利用系统资源。Java NIO.2 提供了 AIO 支持，通过 `AsynchronousChannel` 和 `CompletionHandler` 实现异步 I/O 操作。

总结：

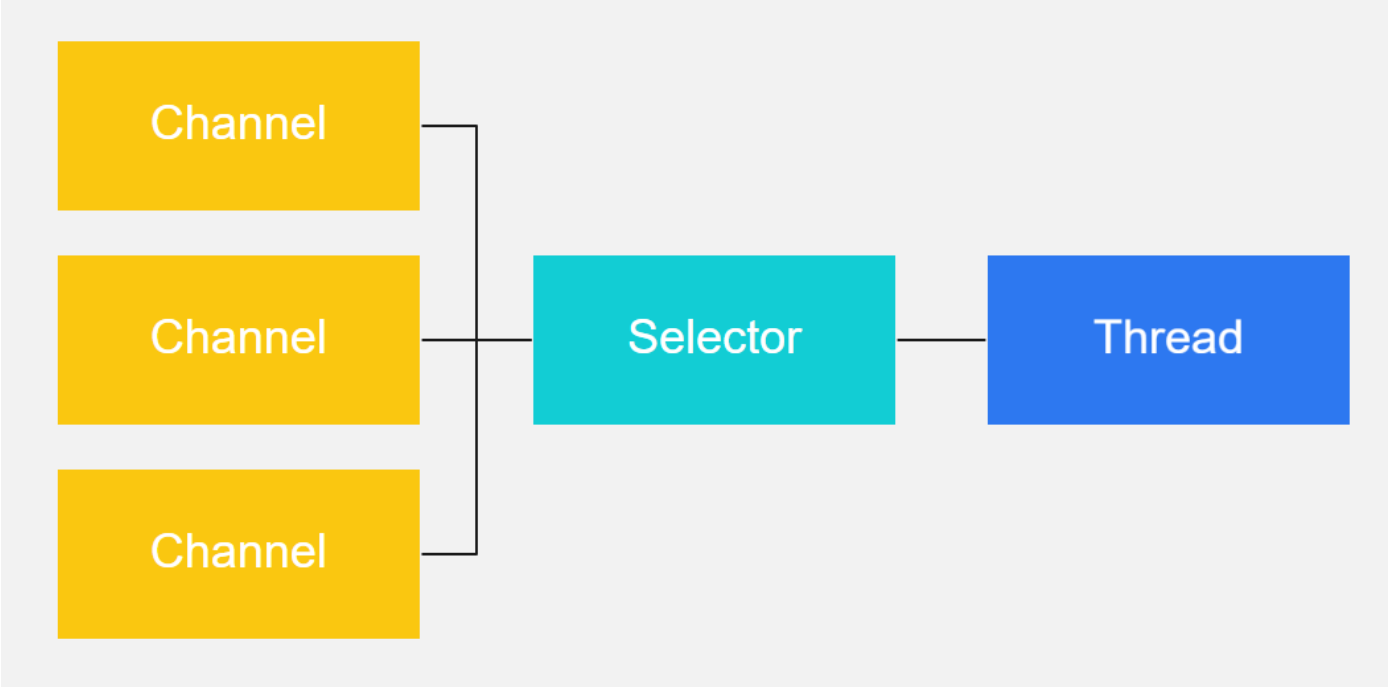
- **BIO：** 适用于连接数较少、并发不高的情况，简单易用。
- **NIO：** 适用于高并发、连接数较多的网络应用，通过选择器实现非阻塞 I/O。
- **AIO：** 适用于需要处理大量并发连接、希望充分利用系统资源的情况，通过异步操作实现。

NIO的实现原理

Java NIO的实现原理主要涉及到以下几个核心概念和组件：

- **Channel（通道）：** `Channel` 是 NIO 中的一个抽象概念，它类似于传统的流，但更加灵活。`Channel` 可以是读、写或者读写的，并且可以异步地进行 I/O 操作。Channel有好几种类型，其中比较常用的有FileChannel、DatagramChannel、SocketChannel、ServerSocketChannel等，这些通道涵盖了UDP和TCP网络IO以及文件IO。
- Buffer（缓冲区）：`Buffer` 是 NIO 中用于存储数据的容器。`Channel` 从 `Buffer` 中读取数据，将数据写入到 `Buffer` 中。`Buffer` 提供了对数据的结构化访问，使得读写操作更为灵活和高效。Java NIO里关键的Buffer实现有CharBuffer、ByteBuffer、ShortBuffer、IntBuffer、LongBuffer、FloatBuffer、DoubleBuffer。这些Buffer覆盖了你能通过IO发送的基本数据类型，即byte、short、int、long、float、double、char。Buffer对象包含三个重要的属性，分别是capacity、position、limit。

- **Selector（选择器）：** `Selector` 是 NIO 的关键组件之一。它允许一个线程同时监控多个 `Channel`，当其中的某个 `Channel` 发生读或写事件时，可以通过 `Selector` 得到通知。这样一个线程可以有效地管理多个网络连接。



工作原理可以分为以下几个步骤：

- **打开 Channel：** 通过 `FileChannel`、`SocketChannel`、`ServerSocketChannel` 等类的静态方法 `open()` 打开一个通道。
- **创建 Buffer：** 创建一个或多个 `Buffer`，用于读取或写入数据。
- **将数据写入 Channel：** 将数据写入 `Buffer`，然后将 `Buffer` 中的数据写入 `Channel`。
- **从 Channel 读取数据：** 将 `Channel` 中的数据读取到 `Buffer` 中。
- **注册 Channel 到 Selector：** 通过 `Selector` 监听一个或多个 `Channel`，当 `Channel` 上发生感兴趣的事件时，`Selector` 将通知程序。
- **处理事件：** 在一个循环中调用 `Selector` 的 `select()` 方法，该方法会阻塞直到至少一个注册的 `Channel` 发生了感兴趣的事件。然后通过迭代 `selectedKeys()` 获取 `SelectionKey`，从而得知哪个 `Channel` 上发生了事件。

介绍一下Java的序列化与反序列化

序列化机制可以将对象转换成字节序列，这些字节序列可以保存在磁盘上，也可以在网络中传输，并允许程序将这些字节序列再次恢复成原来的对象。其中，对象的序列化（Serialize），是指将一个Java对象写入IO流中，对象的反序列化（Deserialize），则是指从IO流中恢复该Java对象。

若对象要支持序列化机制，则它的类需要实现Serializable接口，该接口是一个标记接口，它没有提供任何方法，只是标明该类是可以序列化的，Java的很多类已经实现了Serializable接口，如包装类、String、Date等。

若要实现序列化，则需要使用对象流ObjectInputStream和ObjectOutputStream。其中，在序列化时需要调用ObjectOutputStream对象的writeObject()方法，以输出对象序列。在反序列化时需要调用ObjectInputStream对象的readObject()方法，将对象序列恢复为对象。

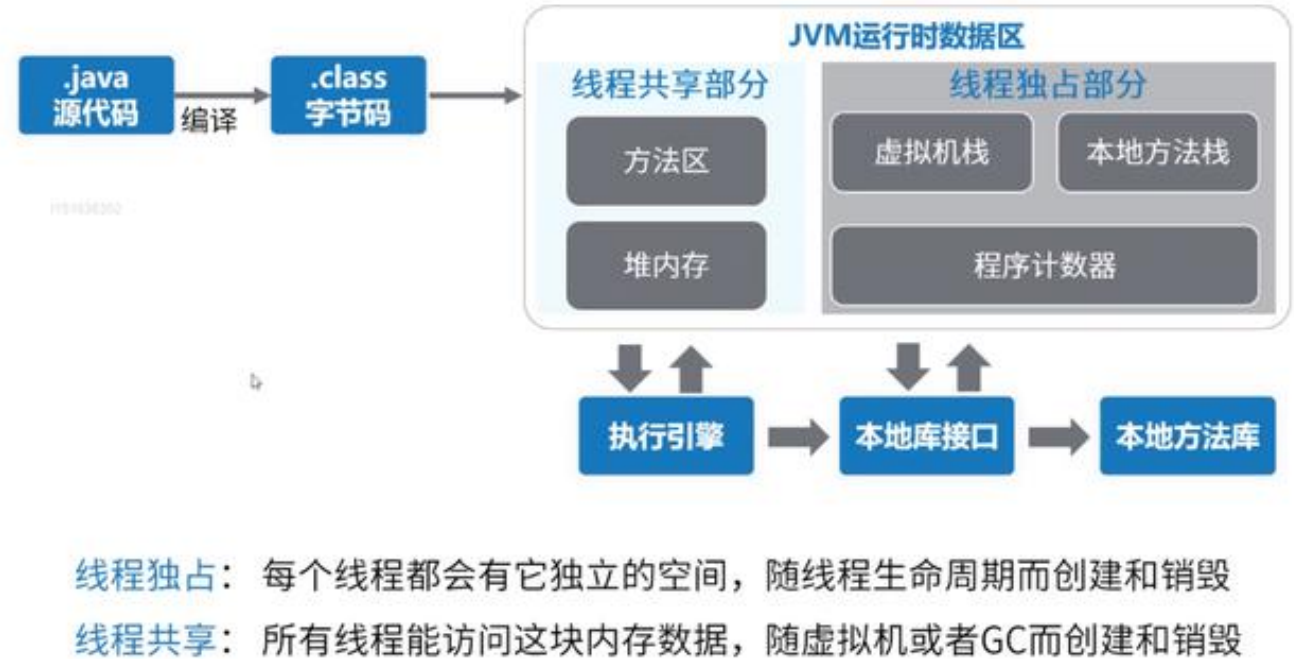
多线程

进程线程基础

什么是进程和线程

进程是系统运行程序的基本单位，在 Java 中，当我们启动 main 函数时其实就是启动了一个 JVM 的进程，而 main 函数所在的线程就是这个进程中的一个线程，也称主线程。

线程是进程中的一个执行单元。一个进程可以包含多个线程，这些线程共享进程的内存空间和系统资源。线程是操作系统调度的最小单位，它负责执行进程中的任务，但是线程的并发执行也可能导致一些问题，如竞态条件、死锁等。



Java创建线程有几种方式

1. 继承自`Thread`类

通过继承 `Thread` 类，可以创建一个新的线程。为了实现线程的执行逻辑，需要重写 `run()` 方法。

```
class MyThread extends Thread {
    public void run() {
        // 线程执行的逻辑
        System.out.println("MyThread is running...");
    }
}

// 创建并启动线程
MyThread myThread = new MyThread();
myThread.start();
```

2. 实现`Runnable`接口

```
class MyRunnable implements Runnable {
    public void run() {
        // 线程执行的逻辑
        System.out.println("MyRunnable is running...");
    }
}

// 创建并启动线程
Thread myThread = new Thread(new MyRunnable());
myThread.start();
```

3. 使用 Executor 框架

`Executor` 框架是 Java 并发编程中的高级工具，它提供了一种更为灵活的方式来管理和执行线程。通过 `Executor`，可以将任务提交给线程池，由线程池来管理线程的生命周期和执行。

```
import java.util.concurrent.Executor;
import java.util.concurrent.Executors;

class MyTask implements Runnable {
    public void run() {
        // 线程执行的逻辑
        System.out.println("MyTask is running...");
    }
}

// 创建线程池并提交任务
Executor executor = Executors.newFixedThreadPool(3);
executor.execute(new MyTask());
```

线程的生命周期

1. 新建（`New`）

线程对象被创建，但尚未启动。使用 `new Thread()` 创建线程对象后，线程处于新建状态。

2. 就绪（`Runnable`）

线程已经被启动，等待系统资源（如 CPU 时间）以便运行, 调用 `start()` 方法后，线程进入就绪状态。

3. 运行（`Running`）

就绪状态的线程获得 CPU 时间，开始执行 `run()` 方法中的代码。正在执行的线程处于运行状态。

4. **阻塞（Blocked）：**

线程因为某些原因放弃了 CPU 使用权，暂时停止运行。可能是等待某个资源、等待 I/O 操作完成、或者调用 `sleep()` 方法等。

5. **等待（Waiting）：**

线程进入等待状态，等待其他线程的通知唤醒。可以通过 `Object.wait()`、`Thread.join()`、`LockSupport.park()` 等方式进入等待状态。

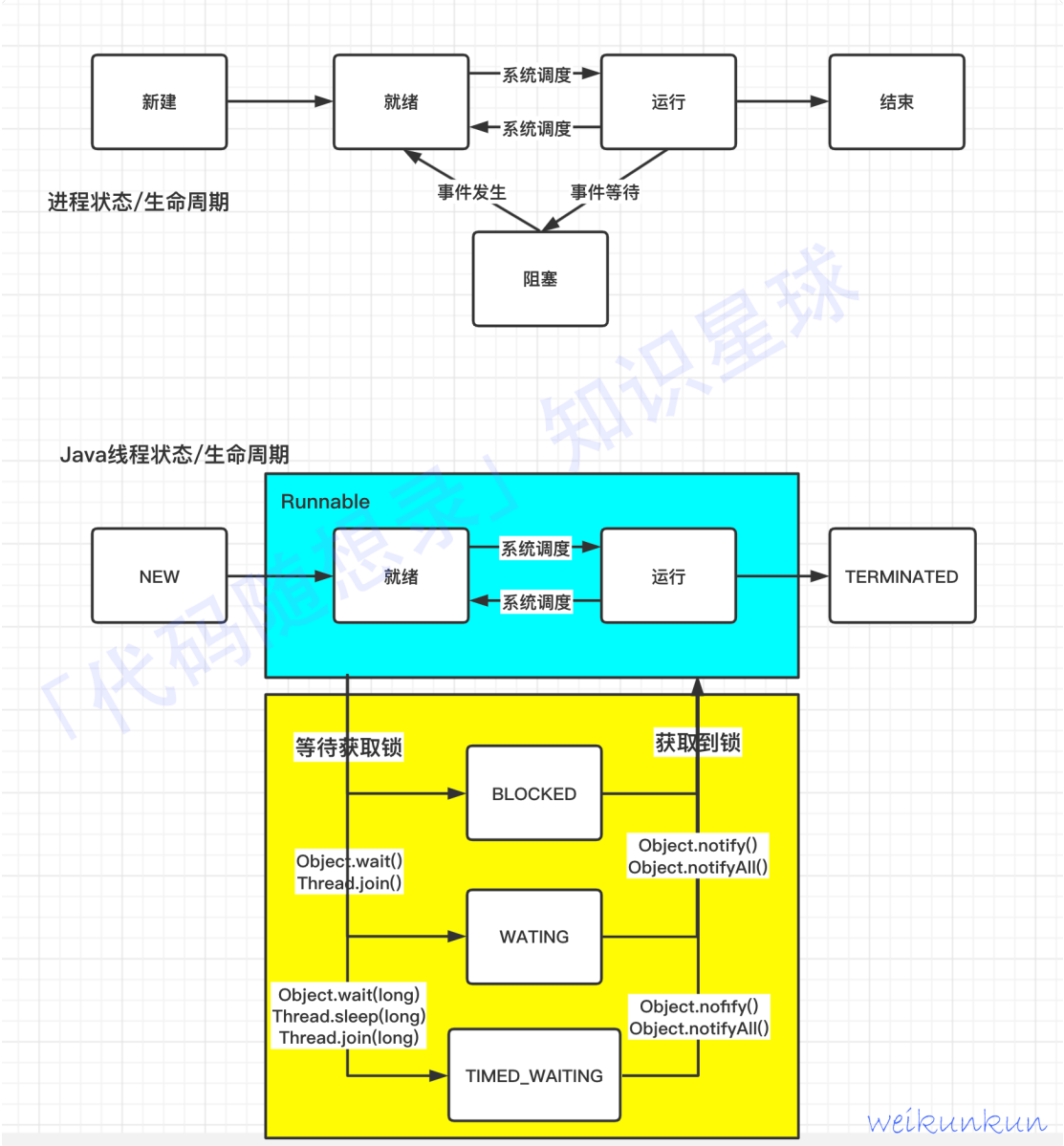
6. **超时等待（Timed Waiting）：**

线程等待一段时间，当时间到达或者其他条件满足时，线程会重新进入就绪状态。通过 `Thread.sleep()`、`Object.wait(timeout)`、`Thread.join(timeout)`、`LockSupport.parkNanos()` 等方式可进入超时等待状态。

7. **终止（Terminated）：**

线程执行完 `run()` 方法或者因异常退出后，进入终止状态。一个终止的线程不能再进入任何状态。

Java线程的状态转换和操作系统中进程的状态转换关系



Java中将操作系统中的

- 就绪状态/运行状态转化为一个状态Runnable
- 阻塞状态 细分为了三种
    - BlOCKED

- WAITING
- TIMED_WAITING

run方法和start方法的区别？可以直接调用run方法吗

在Java多线程中，`run` 方法和 `start` 方法的区别在于：

1. `run` 方法是线程的执行体，包含线程要执行的代码，当直接调用 `run` 方法时，它会在当前线程的上下文中执行，而不会创建新的线程。
2. `start` 方法用于启动一个新的线程，并在新线程中执行 `run` 方法的代码。调用 `start` 方法会为线程分配系统资源，并将线程置于就绪状态，当调度器选择该线程时，会执行 `run` 方法中的代码。

因此，虽然可以直接调用 `run` 方法，但这并不会创建一个新的线程，而是在当前线程中执行 `run` 方法的代码。如果需要实现多线程执行，则应该调用 `start` 方法来启动新线程。

Java 内存模型

先强调，Java内存模型（JMM）和Java运行时数据区的别搞混了。

Java内存模型不是介绍内存是怎么分配的，Java的多线程并发依赖Java内存模型。Java运行时数据区才是将内存分成了哪些部分，各部分分别放的是什么。JMM大概点可以分为基本的模型、3个同步原语的内存语句、happens-before等几块，理解了他们也就理解就JMM。

内存模型：为在特定的操作协议下，对特定的内存或高速缓存进行读写访问的过程抽象。

不同的硬件之间的内存模型有些许差异，Java内存模型屏蔽各种硬件和操作系统的内存访问差异，以实现让Java程序在各种平台下都能达到一致的内存访问效果。

通信和同步：

在并发编程中，线程间的通信有两种模型：共享内存和消息传递。

共享内存是指线程间共享，通过读写内存中的共功状态来实现隐式通信，消息传递线程间没有共享状态，线程间必须通过发送接受消息来进行显示通信。共享内存的同步是显示进行的，必须显示的指定某个方法或者某段代码在程序之间互斥执行，

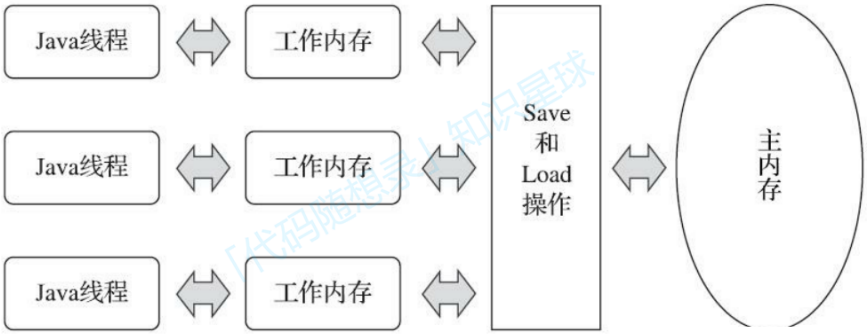
而消息传递的同步是隐式执行的，消息的发送一定在消息的接受之前嘛。

Java采用的是共享内存模型，隐式通信，显示同步。

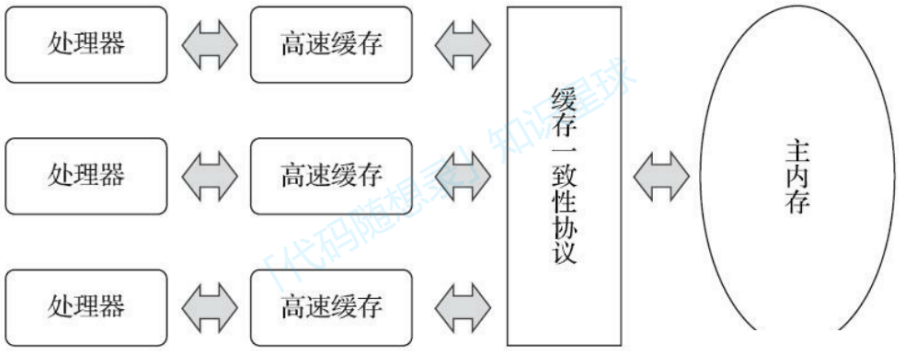
共享模型：

简单理解就是有一个主内存，每个线程有自己的工作内存。

线程工作时先从主内存中把需要的数据拷贝到工作内存中，然后线程从工作内存中读取相关数据进行处理。随后将处理完成的数据从工作内存写回到主内存中。这样当另一个线程从内存中读取数据时，得到的就是之前线程处理过的，两个线程之间完成了通信（隐式通信）。但如何确保线程从主内存中读取的先后顺序，比如后面的线程一定是等之前的线程将数据处理完之后再从主内存读取，这需要在程序中显示的指定互斥执行（显示同步）。



其实工作内存并不是一块真正的内存，它是缓存、写缓冲区、寄存器以及其他的硬件和编译器优化等等一系列的抽象。在硬件层面，真正流程是下图：



主内存中存放的数据应该是线程间共享的。

这里的主内存对应于Java堆中的对象实例数据部分，在Java中，所有实例域、静态域和数组元素都存放堆内存中，堆内存在线程间共享。而局部变量表和方法参数等是线程私有的，并不会被共享，局部变量表和方法参数是分配在虚拟机栈上的，并不是堆。

高速缓存：

处理器首先需要先读取数据，然后才能进行各种运算。但是处理器的速度要比内存的速度高出几个数量级，要是从内存读一个数然后处理器运算一个指令，处理器的时间都被浪费在等待内存上面了。

所以现代计算机系统都不得不加入一层或多层读写速度尽可能接近处理器运算速度的高速缓存（Cache）来作为内存与处理器之间的缓冲：将运算需要使用的数据复制到缓存中，让运算能快速进行，当运算结束后再从缓存同步回内存之中，这样处理器就无须等待缓慢的内存读写了。

工作内存概念不仅有高速缓存，还包括寄存器、硬件和编译器优化等待。

可见性在并发编程中，会出现各种各样的可见性问题。简单的说就是程序员认为的逻辑，和代码真正执行的逻辑并不一致，就会造成可见性问题。程序员不断的检查自己程序逻辑，程序逻辑正确但是运行结果却错误，造成各种认为的“玄学”问题。而只有真正了解计算机是如何处理代码的，才能破解“玄学”。

可见性其实就是一个线程执行的结果，其他的线程是否可以正确的访问到。假设线程A处理完后，将结果放入自己的工作内存，但并没有从工作内存刷回到主内存，其他线程是看不到这个结果的。或者在线程 A 执行之前，其他线程就去主内存中查看，这种肯定也看不到。JMM 通过控制主内存与每个线程的本地内存之间的交互，来为 Java 程序员提供内存可见性保证。

重排序：

在执行程序时，为了提高性能，编译器和处理器会对指令做重排序。分为三类：

1、编译器优化的重排序：

编译器在不改变单线程语义的前提下，可以重新安排语句执行顺序。

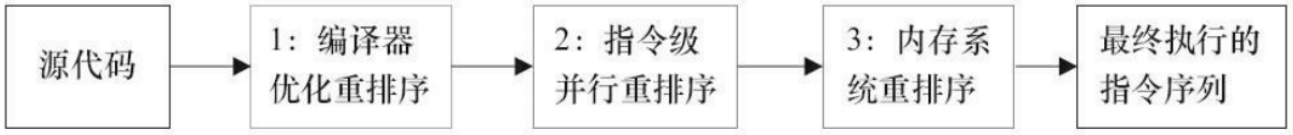
2、指令级别重排序：

如果不存在数据依赖，处理器可以改变语句对机器执行指令的顺序。

3、内存系统的重排序：

由于处理器使用缓存和读写缓冲区，这使的加载和存储操作看上去是乱序执行。

从源代码到最后执行的指令需要分别经历三次排序



数据依赖性：

如果两个操作访问同一个变量，且这两个操作中有一个为写操作，此时这两个操作之间就存在数据依赖性。

数据依赖分为 写后读、写后写、读后写三种情况（只要前后任意操作涉及到写，就存在数据依赖性），只要重排序两个操作的执行顺序，程序的执行结果就会被改变。编译器和处理器在重排序时，会遵守数据依赖性，编译器和处理器不会改变存在数据依赖关系的两个操作的执行顺序。

但此处的数据依赖性只是单个处理器和单个线程中的操作，不同处理器和不同线程之间的数据依赖不被编译器和处理器考虑。

as-if-serial 语义：

不管怎么重排序，单线程程序的执行结果不能被改变。各种重排序必须遵守as-if-serial 语义。比如上面的数据依赖性，并不会重排存在数据依赖性的操作，因为这样会改变结果。单线程程序无论怎样重拍，最后的结果一定是正确的。as-if-serial很好理解，单线程如果都不能保证，那整个程序就乱套啦。它使单线程程序员无需担心重排序会干扰程序，也无需担心内存可见性问题。

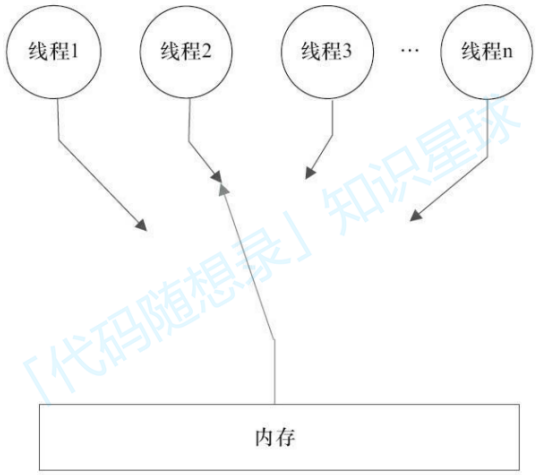
但as-if-serial 隐含的意思时，只要重排序不影响最后的结果，那么你怎样重排都是可以的。单线程不会有影响，但多线程就会出现可见性问题。此时就需要正确的同步。

顺序一致性模型：

顺序一致性模型是一种理想化理论参考模型

1、一个线程中的所有操作必须按照程序的顺序来执行。

2、（不管程序是否同步）所有线程都只能看到一个单一的操作执行顺序。在顺序一致性内存模型中，每个操作都必须原子执行且立刻对所有线程可见。



JMM并没有实现顺序一致模型，因为那样的成本太大了，各种优化措施都不能用。

JMM只保证：如果程序是正确同步的，程序的执行将具有顺序一致性-----即程序的执行结果与该程序在顺序一致性内存模型中的执行结果相同。

注意是执行结果相同，并不是按照一致性执行，JMM允许重排序且只有在正确同步下才能保证可见性。对于未同步或未正确同步的多线程程序，JMM 只提供最小安全性。

顺序一致性模型中，所有操作完全按程序的顺序串行执行。

JMM中，临界区内的代码可以重排序（但是不允许临界区内的代码“逃逸”，那样会破坏互斥性，并不是正确的同步）。

JMM 会在退出临界区和进入临界区这两个关键时间点做一些特别处理，使得线程在这两个时间点具有与顺序一致性模型相同的内存视图。

因为同步的互斥性，使得其他线程根本看不都临界区内重排序，线程只能在公共节点出看到相同的内存视图。

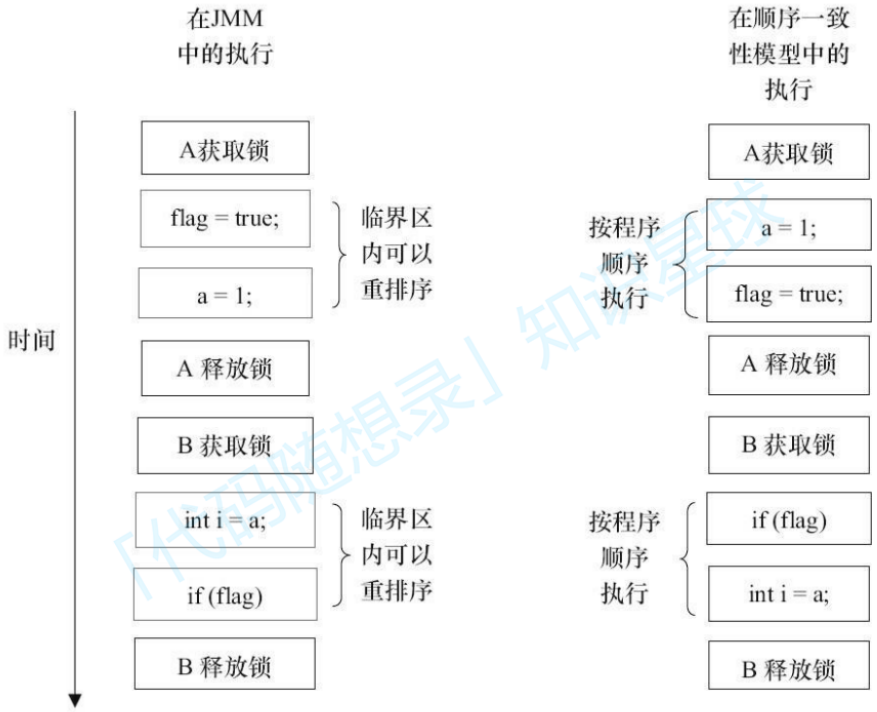


图 3-13　两个内存模型中的执行时序对比图

JMM的基本方针：在不改变正确同步的程序执行结果前提下，尽可能地为编译器和处理器的优化打开方便之门。

未正确同步的程序，JMM提供最小安全性：线程读取到的值，要么是默认值（0，null等），要么是其他线程写入的值。

注意，保证其他是其他线程写入的值并不是说这个值就是正确的，最小安全性只是确保读到的值不是胡乱产生的而已。假设64位数据，先写入低32位后，此时其他线程来读取此值，此种情况虽然读取到的64位数不正确（一半写入一半没写），但是依然符合最小安全性。因为即使这个64位数是拼凑起来的，但它也是之前的值（以前线程写入）和后32位（现在线程写入），都是由线程写入的，并不是胡乱产生的。

JMM 不保证未同步程序的执行结果与该程序在顺序一致性模型中的执行结果一致。不同步JMM认为这两个线程间没有关系，各自执行优化。

volatile 的内存语义

volatile 变量自身具有下列特性：

1、可见性：

对一个 volatile 变量的读，总是能看到（任意线程）对这个 volatile 变量最后的写入。

2、原子性：

对任意单个 volatile 变量的读/写具有原子性，但类似于 volatile++这种复合操作不具有原子性。

3、volatile 写的内存语义：

当写一个 volatile 变量时，JMM 会把该线程对应的本地内存中的共享变量值刷新到主内存。

4、volatile 读的内存语义：

当读一个 volatile 变量时，JMM 会把该线程对应的本地内存置为无效。线程接下来将从主内存中读取共享变量。

从内存语义的角度讲，volatile写和释放锁具有相同的内存效果，把本线程处理的共享变量的刷回到主内存，其他线程便可看见。volatile读和获取锁具有相同的内存效果，从主内存中读取最新的共享变量。

线程 A 写一个 volatile 变量，实质上是线程 A 向接下来将要读这个 volatile 变量的某个线程发出了（其对共享变量所做修改的）消息。

线程 B 读一个 volatile 变量，实质上是线程 B 接收了之前某个线程发出的（在写这个 volatile 变量之前对共享变量所做修改的）消息。

线程 A 写一个 volatile 变量，随后线程 B 读这个 volatile 变量，这个过程实质上是线程 A 通过主内存向线程 B 发送消息。

volatile 内存语义的实现

对于主内存和工作内存之间的交互，JMM规定了8种操作：

lock（锁定）：作用于主内存的变量，它把一个变量标识为一条线程独占的状态。

unlock（解锁）：作用于主内存的变量，它把一个处于锁定状态的变量释放出来，释放后的变量 才可以被其他线程锁定。

read（读取）：作用于主内存的变量，它把一个变量的值从主内存传输到线程的工作内存中，以 便随后的load动作使用。

load（载入）：作用于工作内存的变量，它把read操作从主内存中得到的变量值放入工作内存的 变量副本中。

use（使用）：作用于工作内存的变量，它把工作内存中一个变量的值传递给执行引擎，每当虚 拟机遇到一个需要使用变量的值的字节码指令时将会执行这个操作。

assign（赋值）：作用于工作内存的变量，它把一个从执行引擎接收的值赋给工作内存的变量， 每当虚拟机遇到一个给变量赋值的字节码指令时执行这个操作。

store（存储）：作用于工作内存的变量，它把工作内存中一个变量的值传送到主内存中，以便随 后的write操作使用。

write（写入）：作用于主内存的变量，它把store操作从工作内存中得到的变量的值放入主内存的 变量中。

同时还有许多规则，比如unlock必须在lock之后，read和load、store和write两两成对，不允许单独出现等等。和volatile相关的只需要记住两个，load和store。标注volatile变量会限制一部分重排序：

是否能重排序	第二个操作		
第一个操作	普通读 / 写	volatile 读	volatile 写
普通读 / 写			NO
volatile 读	NO	NO	NO
volatile 写		NO	NO

表 3-5　volatile 重排序规则表

1. 第二个操作是volatile写，不管第一个操作是啥都不允许重排序。确保 volatile 写之前的操作不会被编译器重排序到 volatile 写之后
2. 第一个操作时volatile读，不管第二个操作是啥都不允许重排序。确保 volatile 读之后的操作不会被编译器重排序到 volatile 读之前。
3. 第一个是volatile写第二个是volatile读不允许重排序。

为了实现volatile限制重排序的功能，编译器在生成字节码时会插入内存屏障来静止特定类型的处理器重排序。而因为每个处理器重排序规则都不一样，JMM采取了保守策略插入内存屏障，保证JMM在不同处理器上最后重拍限制都相同。

在每个 volatile 写操作的前面插入一个 StoreStore 屏障
在每个 volatile 写操作的后面插入一个 StoreLoad 屏障

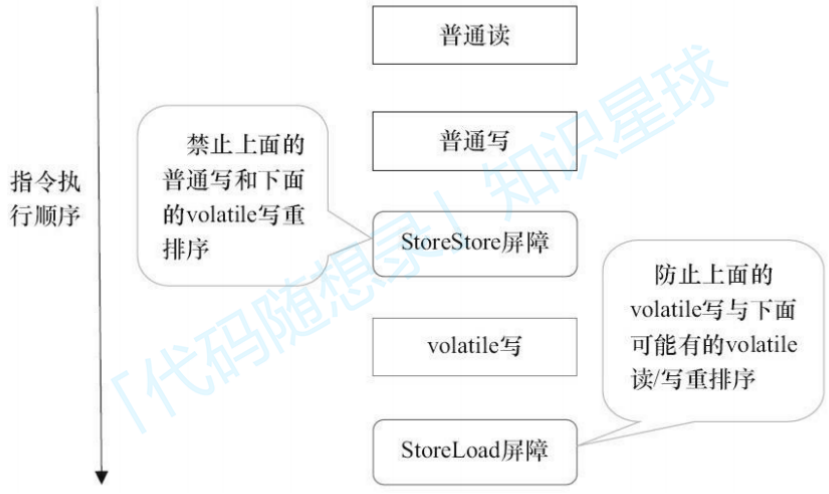


图 3-19　指令序列示意图

在每个 volatile 读操作的后面插入一个 LoadLoad 屏障
在每个 volatile 读操作的后面插入一个 LoadStore 屏障

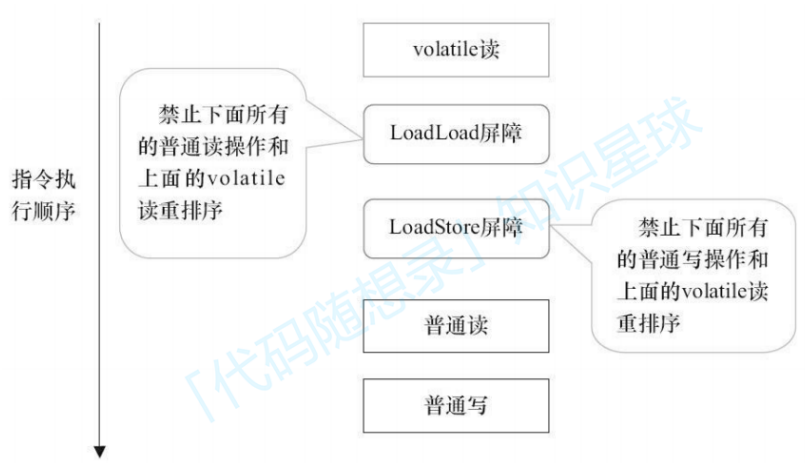


图 3-20　指令序列示意图

并不是严格的要求必须插入这些内存屏障，上述只是最保守的策略。在实际执行时，只要不该改变volatile写-读的内存语义，编译器可以优化掉不必要的内存屏障。

上面提到，store和load分别是存储到主内存和从主内存读取。在内存屏障的两侧，各有一些数据，内存屏障会保证在执行后面的执行之前，之前的相关指令已经全部完成。比如storestore屏障，在后面数据刷回到主内存之前，前面的数据已经全部刷回主内存。LodaStore屏障保证在后面数据刷回到主内存之前，已经从主内存中载入数据

表 3-3　内存屏障类型表

屏障类型	指令示例	说　明
LoadLoad Barriers	Load1; LoadLoad; Load2	确保 Load1 数据的装载先于 Load2 及所有后续装载指令的装载
StoreStore Barriers	Store1; StoreStore; Store2	确保 Store1 数据对其他处理器可见（刷新到内存）先于 Store2 及所有后续存储指令的存储
LoadStore Barriers	Load1; LoadStore; Store2	确保 Load1 数据装载先于 Store2 及所有后续的存储指令刷新到内存
StoreLoad Barriers	Store1; StoreLoad; Load2	确保 Store1 数据对其他处理器变得可见（指刷新到内存）先于 Load2 及所有后续装载指令的装载。StoreLoad Barriers 会使该屏障之前的所有内存访问指令（存储和装载指令）完成之后，才执行该屏障之后的内存访问指令

锁的内存语义

当线程释放锁时，JMM 会把该线程对应的本地内存中的共享变量刷新到主内存中。当线程获取锁时，JMM 会把该线程对应的本地内存置为无效。从而使得被监视器保护的临界区代码必须从主内存中读取共享变量。锁的实现也采用了 volatile内存语义。

在ReentrantLock 中，依赖于AQS框架实现锁，分为公平锁和非公平锁。

公平锁语义的实现：加锁方法首先读 volatile 变量 state(AQS的state)，在释放锁的最后写 volatile 变量 state。

非公平锁的实现：在非公平锁时，会采用CAS设置state.CAS具有 volatile 读和写的内存语义。

final内存语义

对于 final 域，编译器和处理器要遵守两个重排序规则。

1. 在构造函数内对一个 final 域的写入，与随后把这个被构造对象的引用赋值给一个引用变量，这两个操作之间不能重排序。
2. 初次读一个包含 final 域的对象的引用，与随后初次读这个 final 域，这两个操作之间不能重排序。

final内存语义实现：

写final:

JMM 禁止编译器把 final 域的写重排序到构造函数之外

编译器会在 final 域的写之后，构造函数 return 之前，插入一个 StoreStore 屏障。这个屏障禁止处理器把 final 域的写重排序到构造函数之外。

读final:

在一个线程中，初次读对象引用与初次读该对象包含的final 域，JMM 禁止处理器重排序这两个操作。编译

器会在读 final 域操作的前面插入一个 LoadLoad 屏障

final为引用类型(前提为final引用不能从构造函数溢出)：

在构造函数内对一个 final 引用的对象的成员域的写入，与随后在构造函数外把这个被构造对象的引用赋值给一个引用变量，这两个操作之间不能重排序。

happens-before

happens-before 是 JMM 最核心的概念

对于程序员：JMM 向程序员提供的 happens-before 规则能满足程序员的需求，也向程序员提供了足够强的内存可见性 保证。

对于处理器和编译器：

只要不改变程序的执行结果（指的是单线程程序和正确同步的多线程程序），编译器和处理器怎么优化都行。例如锁消除和volatile消除。

定义： 此处的定义分别呼应上面的两点

1. 如果一个操作 happens-before 另一个操作，那么第一个操作的执行结果将对第二个操作可见，而且第一个操作的执行顺序排在第二个操作之前。
2. 两个操作之间存在 happens-before 关系，并不意味着 Java 平台的具体实现必须要按照 happens-before 关系指定的顺序来执行。如果重排序之后的执行结果，与按happens-before 关系来执行的结果一致，那么这种重排序并不非法。

happens-before 规则

1. 程序顺序规则：一个线程中的每个操作，happens-before 于该线程中的任意后续操作。
2. 监视器锁规则：对一个锁的解锁，happens-before 于随后对这个锁的加锁。
3. volatile 变量规则：对一个 volatile 域的写，happens-before 于任意后续对这个volatile 域的读。
4. 传递性：如果 A happens-before B，且 B happens-before C，那么 A happens-before C。
5. start()规则：如果线程 A 执行操作 ThreadB.start()（启动线程 B），那么 A 线程的ThreadB.start()操作 happens-before 于线程 B 中的任意操作。
6. join()规则：如果线程 A 执行操作 ThreadB.join()并成功返回，那么线程 B 中的任意操作 happens-before 于线程 A 从 ThreadB.join()操作成功返回。

线程同步和锁

什么是线程死锁

在典型的线程死锁情况下，每个线程都持有一些资源，并且正在等待获取其他线程持有的资源。由于每个线程都在等待，没有一个线程能够继续执行，这样整个程序就被阻塞了。

线程死锁的产生通常需要满足以下四个条件，称为死锁的必要条件：

1. **互斥条件（Mutual Exclusion）：** 至少有一个资源是不可共享的，即一次只能被一个线程使用。
2. **占有且等待条件（Hold and Wait）：** 一个线程持有至少一个资源，并等待获取其他线程持有的资源。
3. **非抢占条件（No Preemption）：** 资源只能在被线程显式释放之后才能被其他线程获取，不能被强制抢占。
4. **循环等待条件（Circular Wait）：** 一组线程形成一个循环，每个线程都在等待下一个线程所持有的资源。

```
class DeadlockExample {
    private final Object resource1 = new Object();
    private final Object resource2 = new Object();

    public void execute() {
        Thread thread1 = new Thread(() -> {
            synchronized (resource1) {
                System.out.println("Thread 1: Locked resource 1");
                try {
                    Thread.sleep(100); // Introducing delay to make deadlock more
likely
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                synchronized (resource2) {
                    System.out.println("Thread 1: Locked resource 2");
                }
            }
        });

        Thread thread2 = new Thread(() -> {
            synchronized (resource2) {
                System.out.println("Thread 2: Locked resource 2");
                synchronized (resource1) {
                    System.out.println("Thread 2: Locked resource 1");
                }
            }
        });

        thread1.start();
        thread2.start();
    }

    public static void main(String[] args) {
        DeadlockExample example = new DeadlockExample();
        example.execute();
```

```
    }
}
```

如何预防和避免线程死锁

如何预防死锁? 破坏死锁的产生的必要条件即可：

- **破坏请求与保持条件**：一次性申请所有的资源。
- **破坏不剥夺条件**：占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源。
- **破坏循环等待条件**：靠按序申请资源来预防。按某一顺序申请资源，释放资源则反序释放。破坏循环等待条件。

你知道Java中有哪些锁吗

各种锁

公平锁/非公平锁
可重入锁
独享锁/共享锁 互斥锁/读写锁 乐观锁/悲观锁
分段锁 偏向锁/轻量级锁/重量级锁 自旋锁

上面是很多锁的名词，这些分类并不是全是指锁的状态，有的指锁的特性，有的指锁的设计，下面总结的内容是对每个锁的名词进行一定的解释。

公平锁/非公平锁

公平锁是指多个线程按照申请锁的顺序来获取锁。 非公平锁是指多个线程获取锁的顺序并不是按照申请锁的顺序，有可能后申请的线程比先申请的线程优 先获取锁。有可能，会造成优先级反转或者饥饿现象。
对于Java ReentrantLock 而言，通过构造函数指定该锁是否是公平锁，默认是非公平锁。非公平锁的 优点在于吞吐量比公平锁大。
对于 Synchronized 而言，也是一种非公平锁。由于其并不像 ReentrantLock 是通过AQS的来实现线 程调度，所以并没有任何办法使其变成公平锁。

可重入锁

可重入锁又名递归锁，是指在同一个线程在外层方法获取锁的时候，在进入内层方法会自动获取锁。说 的有点抽象，下面会有一个代码的示例。
对于Java ReentrantLock 而言, 他的名字就可以看出是一个可重入锁，其名字是 Re entrant Lock 重 新进入锁。

对于 Synchronized 而言,也是一个可重入锁。可重入锁的一个好处是可一定程度避免死锁。

```
synchronized void setA() throws Exception{
    Thread.sleep(1000);
    setB();
}
synchronized void setB() throws Exception{
    Thread.sleep(1000);
}
```

上面的代码就是一个可重入锁的一个特点，如果不是可重入锁的话，setB可能不会被当前线程执行，可 能造成死锁。

独享锁/共享锁

独享锁是指该锁一次只能被一个线程所持有。共享锁是指该锁可被多个线程所持有。

对于Java ReentrantLock 而言，其是独享锁。但是对于Lock的另一个实现类 ReadWriteLock ，其读锁 是共享锁，其写锁是独享锁。
读锁的共享锁可保证并发读是非常高效的，读写，写读 ，写写的过程是互斥的。 独享锁与共享锁也是通过AQS来实现的，通过实现不同的方法，来实现独享或者共享。

对于 Synchronized 而言，当然是独享锁。

互斥锁/读写锁

上面讲的独享锁/共享锁就是一种广义的说法，互斥锁/读写锁就是具体的实现。

互斥锁在Java中的具体实现就是 ReentrantLock

读写锁在Java中的具体实现就是 ReadWriteLock

乐观锁/悲观锁

乐观锁与悲观锁不是指具体的什么类型的锁，而是指看待并发同步的角度。
悲观锁认为对于同一个数据的并发操作，一定是会发生修改的，哪怕没有修改，也会认为修改。因此对于同一个数据的并发操作，悲观锁采取加锁的形式。悲观的认为，不加锁的并发操作一定会出问题。
乐观锁则认为对于同一个数据的并发操作，是不会发生修改的。在更新数据的时候，会采用尝试更新，不断重新的方式更新数据。乐观的认为，不加锁的并发操作是没有事情的。

从上面的描述我们可以看出，悲观锁适合写操作非常多的场景，乐观锁适合读操作非常多的场景，不加 锁会带来大量的性能提升。
悲观锁在Java中的使用，就是利用各种锁。

乐观锁在Java中的使用，是无锁编程，常常采用的是CAS算法，典型的例子就是原子类，通过CAS自旋 实现原子操作的更新。

分段锁

分段锁其实是一种锁的设计，并不是具体的一种锁，对于 ConcurrentHashMap 而言，其并发的实现就 是通过分段锁的形式来实现高效的并发操作。
我们以 ConcurrentHashMap 来说一下分段锁的含义以及设计思想， ConcurrentHashMap 中的分段锁 称为 Segment，它即类似于HashMap(JDK7与JDK8中HashMap的实现)的结构，即内部拥有一个 Entry数组，数组中的每个元素又是一个链表;同时又是一个ReentrantLock(Segment继承了 ReentrantLock)。

当需要put元素的时候，并不是对整个hashmap进行加锁，而是先通过hashcode来知道他要放在那一 个分段中，然后对这个分段进行加锁，所以当多线程put的时候，只要不是放在一个分段中，就实现了真正的并行的插入。

但是，在统计size的时候，可就是获取hashmap全局信息的时候，就需要获取所有的分段锁才能统计。 分段锁的设计目的是细化锁的粒度，当操作不需要更新整个数组的时候，就仅仅针对数组中的一项进行 加锁操作。

偏向锁/轻量级锁/重量级锁

这三种锁是指锁的状态，并且是针对 Synchronized 。在Java 5通过引入锁升级的机制来实现高效 Synchronized 。这三种锁的状态是通过对象监视器在对象头中的字段来表明的。

偏向锁是指一段同步代码一直被一个线程所访问，那么该线程会自动获取锁。降低获取锁的代价。
轻量级锁是指当锁是偏向锁的时候，被另一个线程所访问，偏向锁就会升级为轻量级锁，其他线程会通过自旋的形式尝试获取锁，不会阻塞，提高性能。
重量级锁是指当锁为轻量级锁的时候，另一个线程虽然是自旋，但自旋不会一直持续下去，当自旋一定次数的时候，还没有获取到锁，就会进入阻塞，该锁膨胀为重量级锁。重量级锁会让其他申请的线程进入阻塞，性能降低。

自旋锁

在Java中，自旋锁是指尝试获取锁的线程不会立即阻塞，而是采用循环的方式去尝试获取锁，这样的好 处是减少线程上下文切换的消耗，缺点是循环会消耗CPU。

谈谈你对volatile关键字的理解

- volatile 通常被比喻成"轻量级的 synchronized "，也是Java并发编程中比较重要的一个关键 字。
- 和 synchronized 不同， volatile 是一个变量修饰符，只能用来修饰变量。无法修饰方法及代码 块等。
- 被volatile修饰的共享变量，就具有了以下两点特性:
    - 保证了不同线程对该变量操作的内存**可见性**
    - 禁止指令重排序

volatile是java虚拟机提供的轻量级的同步机制具有以下特点:

- 保证可见性
- 不保证原子性
- 禁止指令重排

1. 保证可见性

> 可见性是指当多个线程访问同一个变量时，一个线程修改了这个变量的值，其他线程能够立即看得到 修改的值。

如果我们将变量声明为 `volatile` ，这就指示 JVM，这个变量是共享且不稳定的，每次使用它都到主存中进行读取。

2. 禁止指令重排序

线程安全

Java 线程安全的实现

Java中线程安全的实现方式---战略性观望

1、阻塞同步：

说白了也就是使用锁实现，具体采用什么锁，有两种选择：内置锁也就是synchronized关键字，JUC下具体锁的实现

2、非阻塞同步：

使用锁带来的主要问题，是频繁的线程阻塞、唤醒操作以及用户态内核态的切换带来的性能问题。

可能这些额外的操作带来的时间消耗远大于线程自身的业务执行执行时间。所以引入非阻塞同步，也就是基于CAS操作。

最直接的实现就是JUC下的各种原子类的实现。虽然CAS避免了锁带来的性能开销，不过其仅适用于少部分同步场景，没有阻塞同步更加具有普适性。

缺点：

1. 未获取同步资源的线程陷入自旋状态，所以对于CPU的消耗很高
2. 仅能操作单个共享资源，对于组合类型还是需要加锁处理，或者重新组合为一个共享资源
3. ABA问题

3、无同步方案:

线程的本地存储，主要是用于对于一个共享资源都尽可能的在同一个线程中执行，使用场景：

1. 譬如生产者-消费者模型中，消费者消费消费一个内容
2. web服务中，一个请求对用于一个服务线程（也属于生产者-消费者模型）
3. 链路跟踪中动态采集方法的执行信息

Java线程锁的特点/性能和使用场景

多线程的理由

在出现了进程之后，操作系统的性能得到了大大的提升。虽然进程的出现解决了操作系统的并发问题，

但是人们仍然不满足，人们逐渐对实时性有了要求。 使用多线程的理由之一是和进程相比，它是一种非常花销小，切换快，更”节俭”的多任务操作方式。

在Linux系统下，启动一个新的进程必须分配给它独立的地址空间，建立众多的数据表来维护它的代码 段、堆栈段和数据段，这是一种”昂贵”的多任务工作方式。而在进程中的同时运行多个线程，它们彼此 之间使用相同的地址空间，共享大部分数据，启动一个线程所花费的空间远远小于启动一个进程所花费 的空间，而且，线程间彼此切换所需的时间也远远小于进程间切换所需要的时间。

多线程的问题

由于多个线程是共同占有所属进程的资源和地址空间的，那么就会存在一个问题: 如果多个线程要同时访问某个资源，怎么处理?

在Java并发编程中，经常遇到多个线程访问同一个 共享资源 ，这时候作为开发者必须考虑如何维护数据 一致性，这就是Java锁机制(同步问题)的来源。

Java提供了多种多线程锁机制的实现方式，常见的有:

1. synchronized
2. ReentrantLock
3. 3. Semaphore
4. AtomicInteger等

每种机制都有优缺点与各自的适用场景，必须熟练掌握他们的特点才能在Java多线程应用开发时得心应 手。

4种线程锁

1. **.synchronized**

在Java中synchronized关键字被常用于维护数据一致性。

synchronized机制是给共享资源上锁，只有拿到锁的线程才可以访问共享资源，这样就可以强制使得对 共享资源的访问都是顺序的。

Java开发人员都认识synchronized，使用它来实现多线程的同步操作是非常简单的，只要在需要同步的 对方的方法、类或代码块中加入该关键字，它能够保证在同一个时刻最多只有一个线程执行同一个对象 的同步代码，可保证修饰的代码在执行过程中不会被其他线程干扰。使用synchronized修饰的代码具有 原子性和可见性，在需要进程同步的程序中使用的频率非常高，可以满足一般的进程同步要求。

```
synchronized (obj) { //方法
 .......
 }
```

synchronized实现的机理依赖于软件层面上的JVM，因此其性能会随着Java版本的不断升级而提高。 到了Java1.6，synchronized进行了很多的优化，有适应自旋、锁消除、锁粗化、轻量级锁及偏向锁等，

效率有了本质上的提高。在之后推出的Java1.7与1.8中，均对该关键字的实现机理做了优化。 需要说明的是，当线程通过synchronized等待锁时是不能被Thread.interrupt()中断的，因此程序设计时必须检查确保合理，否则可能会造成线程死锁的尴尬境地。

最后，尽管Java实现的锁机制有很多种，并且有些锁机制性能也比synchronized高，但还是强烈推荐在 多线程应用程序中使用该关键字，因为实现方便，后续工作由JVM来完成，可靠性高。只有在确定锁机 制是当前多线程程序的性能瓶颈时，才考虑使用其他机制，如ReentrantLock等。

2.ReentrantLock

可重入锁，顾名思义，这个锁可以被线程多次重复进入进行获取操作。

ReentantLock继承接口Lock并实现了接口中定义的方法，除了能完成synchronized所能完成的所有工作 外，还提供了诸如可响应中断锁、可轮询锁请求、定时锁等避免多线程死锁的方法。

Lock实现的机理依赖于特殊的**CPU**指定，可以认为不受**JVM**的约束，并可以通过其他语言平台来完成底 层的实现。在并发量较小的多线程应用程序中，ReentrantLock与synchronized性能相差无几，但在高 并发量的条件下，synchronized性能会迅速下降几十倍，而ReentrantLock的性能却能依然维持一个水 准。

因此我们建议在高并发量情况下使用**ReentrantLock**。

ReentrantLock引入两个概念:公平锁与非公平锁。

公平锁指的是锁的分配机制是公平的，通常先对锁提出获取请求的线程会先被分配到锁。反之，JVM按 随机、就近原则分配锁的机制则称为不公平锁。

ReentrantLock在构造函数中提供了是否公平锁的初始化方式，默认为非公平锁。这是因为，非公平锁 实际执行的效率要远远超出公平锁，除非程序有特殊需要，否则最常用非公平锁的分配机制。

ReentrantLock通过方法lock()与unlock()来进行加锁与解锁操作，与synchronized会被JVM自动解锁机 制不同，ReentrantLock加锁后需要手动进行解锁。为了避免程序出现异常而无法正常解锁的情况，使 用**ReentrantLock**必须在**finally**控制块中进行解锁操作。通常使用方式如下所示:

```
Lock lock = new ReentrantLock();
try {
  lock.lock();
  //...进行任务操作5
}
finally {
    lock.unlock();
}
```

3. Semaphore

上述两种锁机制类型都是“互斥锁”，学过操作系统的都知道，互斥是进程同步关系的一种特殊情况，相 当于只存在一个临界资源，因此同时最多只能给一个线程提供服务。但是，在实际复杂的多线程应用程 序中，可能存在多个临界资源，这时候我们可以借助Semaphore信号量来完成多个临界资源的访问。

Semaphore基本能完成ReentrantLock的所有工作，使用方法也与之类似，通过acquire()与release()方 法来获得和释放临界资源。

经实测，Semaphone.acquire()方法默认为可响应中断锁，与ReentrantLock.lockInterruptibly()作用效 果一致，也就是说在等待临界资源的过程中可以被Thread.interrupt()方法中断。

此外，Semaphore也实现了可轮询的锁请求与定时锁的功能，除了方法名tryAcquire与tryLock不同，其 使用方法与ReentrantLock几乎一致。Semaphore也提供了公平与非公平锁的机制，也可在构造函数中 进行设定。

Semaphore的锁释放操作也由手动进行，因此与ReentrantLock一样，为避免线程因抛出异常而无法正 常释放锁的情况发生，释放锁的操作也必须在**finally**代码块中完成。

4. AtomicInteger

首先说明，此处AtomicInteger是一系列相同类的代表之一，常见的还有AtomicLong、AtomicLong等， 他们的实现原理相同，区别在与运算对象类型的不同。

我们知道，在多线程程序中，诸如++i
 或 i++等运算不具有原子性，是不安全的线程操作之一。通常我们会使用synchronized将该操作变成一个原 子操作，但JVM为此类操作特意提供了一些同步类，使得使用更方便，且使程序运行效率变得更高。通 过相关资料显示，通常AtomicInteger的性能是ReentantLock的好几倍。

线程锁总结

1.synchronized:

在资源竞争不是很激烈的情况下，偶尔会有同步的情形下，synchronized是很合适的。原因在于，编译 程序通常会尽可能的进行优化synchronize，另外可读性非常好。

2.ReentrantLock:

在资源竞争不激烈的情形下，性能稍微比synchronized差点点。但是当同步非常激烈的时候， synchronized的性能一下子能下降好几十倍，而ReentrantLock确还能维持常态。

高并发量情况下使用**ReentrantLock**。

3.Atomic:

和上面的类似，不激烈情况下，性能比synchronized略逊，而激烈的时候，也能维持常态。激烈的时 候，Atomic的性能会优于ReentrantLock一倍左右。但是其有一个缺点，就是只能同步一个值，一段代 码中只能出现一个Atomic的变量，多于一个同步无效。因为他不能在多个Atomic之间同步。

所以，我们写同步的时候，优先考虑synchronized，如果有特殊需要，再进一步优化。ReentrantLock 和Atomic如果用的不好，不仅不能提高性能，还可能带来灾难。

以上就是Java线程锁的详解，除了从编程的角度应对高并发，更多还需要从架构设计的层面来应对高并 发场景，例如:Redis缓存、CDN、异步消息等，详细的内容如下。

并发容器的原理

在Java并发编程中，经常听到Java集合类，同步容器、并发容器，那么他们有哪些具体分类，以及各自

之间的区别和优劣呢?

只有把这些梳理清楚了，你才能真正掌握在高并发的环境下，正确使用好并发容器，我们先从**Java**集合 类，同步容器谈起。

1. 什么是并发容器

Java的集合容器框架中，主要有四大类别:List、Set、Queue、Map，大家熟知的这些集合类

ArrayList、LinkedList、HashMap这些容器都是非线程安全的。

如果有多个线程并发地访问这些容器时，就会出现问题。因此，在编写程序时，在多线程环境下必须要求程序员手动地在任何访问到这些容器的地方进行同步处理，这样导致在使用这些容器的时候非常地不方便。

所以，Java先提供了同步容器供用户使用。

同步容器可以简单地理解为通过**synchronized**来实现同步的容器，比如Vector、Hashtable以及SynchronizedList等容器。

2. 同步容器分类

- Vector
- Stack
- HashTable
- Collections.synchronized方法生成

同步容器面临的问题

可以通过查看Vector，Hashtable等这些同步容器的实现代码，可以看到这些容器实现线程安全的方式 就是将它们的状态封装起来，并在需要同步的方法上加上关键字**synchronized**。

这样做的代价是削弱了并发性，当多个线程共同竞争容器级的锁时，吞吐量就会降低。

例如: HashTable只要有一条线程获取了容器的锁之后，其他所有的线程访问同步函数都会被阻塞，因 此同一时刻只能有一条线程访问同步函数。

因此为了解决同步容器的性能问题，所以才有了并发容器。

3. 什么是并发容器

java.util.concurrent包中提供了多种并发类容器。

并发类容器是专门针对多线程并发设计的，使用了锁分段技术，只对操作的位置进行同步操作，但是其 他没有操作的位置其他线程仍然可以访问，提高了程序的吞吐量。

采用了**CAS**算法和部分代码使用**synchronized**锁保证线程安全。

4. 并发容器分类

1.ConcurrentHashMap

对应的非并发容器:HashMap
目标:代替Hashtable、synchronizedMap，支持复合操作

原理:JDK6中采用一种更加细粒度的加锁机制Segment"分段锁"，JDK8中采用CAS无锁算法。
2.CopyOnWriteArrayList
对应的非并发容器:ArrayList
目标:代替Vector、synchronizedList

原理:利用高并发往往是读多写少的特性，对读操作不加锁，对写操作，先复制一份新的集合，在新的 集合上面修改，然后将新集合赋值给旧的引用，并通过volatile 保证其可见性，当然写操作的锁是必不可 少的了。

3.CopyOnWriteArraySet

对应的非并发容器:HashSet

目标:代替synchronizedSet

原理:基于CopyOnWriteArrayList实现，其唯一的不同是在add时调用的是CopyOnWriteArrayList的 addIfAbsent方法，其遍历当前Object数组，如Object数组中已有了当前元素，则直接返回，如果没有 则放入Object数组的尾部，并返回。

4.ConcurrentSkipListMap

对应的非并发容器:TreeMap
目标:代替synchronizedSortedMap(TreeMap)
原理:Skip list(跳表)是一种可以代替平衡树的数据结构，默认是按照Key值升序的。

5.ConcurrentSkipListSet
对应的非并发容器:TreeSet
目标:代替synchronizedSortedSet
原理:内部基于ConcurrentSkipListMap实现
6.ConcurrentLinkedQueue

不会阻塞的队列
对应的非并发容器:Queue
原理:基于链表实现的FIFO队列(LinkedList的并发版本)
7.LinkedBlockingQueue、**ArrayBlockingQueue**、**PriorityBlockingQueue**

对应的非并发容器:BlockingQueue

特点:拓展了Queue，增加了可阻塞的插入和获取等操作

原理:通过ReentrantLock实现线程安全，通过Condition实现阻塞和唤醒

实现类:

LinkedBlockingQueue:基于链表实现的可阻塞的FIFO队列

ArrayBlockingQueue:基于数组实现的可阻塞的FIFO队列

PriorityBlockingQueue:按优先级排序的队列

线程池

什么是线程池

线程池是一种多线程处理形式，处理过程中将任务添加到队列，然后在创建线程后自动启动这些任务。线程池线程都是后台线程。每个线程都使用默认的堆栈大小，以默认的优先级运行，并处于多线程单元中。如果某个线程在托管代码中空闲(如正在等待某个事件)，则线程池将插入另一个辅助线程来使所有处理器保持繁忙。如果所有线程池线程都始终保持繁忙，但队列中包含挂起的工作，则线程池将在一段时间后创建另一个辅助线程但线程的数目永远不会超过最大值。超过最大值的线程可以排队，但他们要等到其他线程完成后才启动。

为什么需要线程池

我们有两种常见的创建线程的方法，一种是继承Thread类，一种是实现Runnable的接口，Thread类其 实也是实现了Runnable接口。但是我们创建这两种线程在运行结束后都会被虚拟机销毁，如果线程数 量多的话，频繁的创建和销毁线程会大大浪费时间和效率，更重要的是浪费内存。那么有没有一种方法 能让线程运行完后不立即销毁，而是让线程重复使用，继续执行其他的任务哪?

这就是线程池的由来，很好的解决线程的重复利用，避免重复开销

线程池的优点?

1)重用存在的线程，减少对象创建销毁的开销。

2)可有效的控制最大并发线程数，提高系统资源的使用率，同时避免过多资源竞争，避免堵塞。

3)提供定时执行、定期执行、单线程、并发数控制等功能。

Java 提供了哪几种线程池?

Java 主要提供了下面4种线程池

- FixedThreadPool: 该方法返回一个固定线程数量的线程池。该线程池中的线程数量始终不变。 当有一个新的任务提交时，线程池中若有空闲线程，则立即执行。若没有，则新的任务会被暂存在 一个任务队列中，待有线程空闲时，便处理在任务队列中的任务。
- SingleThreadExecutor: 方法返回一个只有一个线程的线程池。若多余一个任务被提交到该线程 池，任务会被保存在一个任务队列中，待线程空闲，按先入先出的顺序执行队列中的任务。
- CachedThreadPool: 该方法返回一个可根据实际情况调整线程数量的线程池。线程池的线程数 量不确定，但若有空闲线程可以复用，则会优先使用可复用的线程。若所有线程均在工作，又有新的任务提交，则会创建新的线程处理任务。所有线程在当前任务执行完毕后，将返回线程池进行复 用。
- ScheduledThreadPoolExecutor: 主要用来在给定的延迟后运行任务，或者定期执行任务。ScheduledThreadPoolExecutor又分为:ScheduledThreadPoolExecutor(包含多个线程)和SingleThreadScheduledExecutor (只包含一个线程)两种。

4种线程池各自的使用场景是什么?

- FixedThreadPool: 适用于为了满足资源管理需求，而需要限制当前线程数量的应用场景。它适 用于负载比较重的服务器;
- SingleThreadExecutor: 适用于需要保证顺序地执行各个任务并且在任意时间点，不会有多个线 程是活动的应用场景;
- CachedThreadPool: 适用于执行很多的短期异步任务的小程序，或者是负载较轻的服务器;
- ScheduledThreadPoolExecutor: 适用于需要多个后台执行周期任务，同时为了满足资源管理需 求而需要限制后台线程的数量的应用场景。
- SingleThreadScheduledExecutor: 适用于需要单个后台线程执行周期任务，同时保证顺序地执 行各个任务的应用场景。

创建线程池的方式

（1） 使用 Executors 创建

我们上面刚刚提到了 Java 提供的几种线程池，通过 Executors 工具类我们可以很轻松的创建我们上面 说的几种线程池。但是实际上我们一般都不是直接使用Java提供好的线程池，另外在《阿里巴巴Java开 发手册》中强制线程池不允许使用 Executors 去创建，而是通过 ThreadPoolExecutor 构造函数 的方 式，这样的处理方式让写的同学更加明确线程池的运行规则，规避资源耗尽的风险。

（2） ThreadPoolExecutor的构造函数创建

我们可以自己直接调用 ThreadPoolExecutor 的构造函数来自己创建线程池。在创建的同时，给BlockQueue 指定容量就可以了。

这种情况下，一旦提交的线程数超过当前可用线程数时，就会抛出 java.util.concurrent.RejectedExecutionException，这是因为当前线程池使用的队列是有边界队列，队 列已经满了便无法继续处理新的请求。但是异常(Exception)总比发生错误(Error)要好。

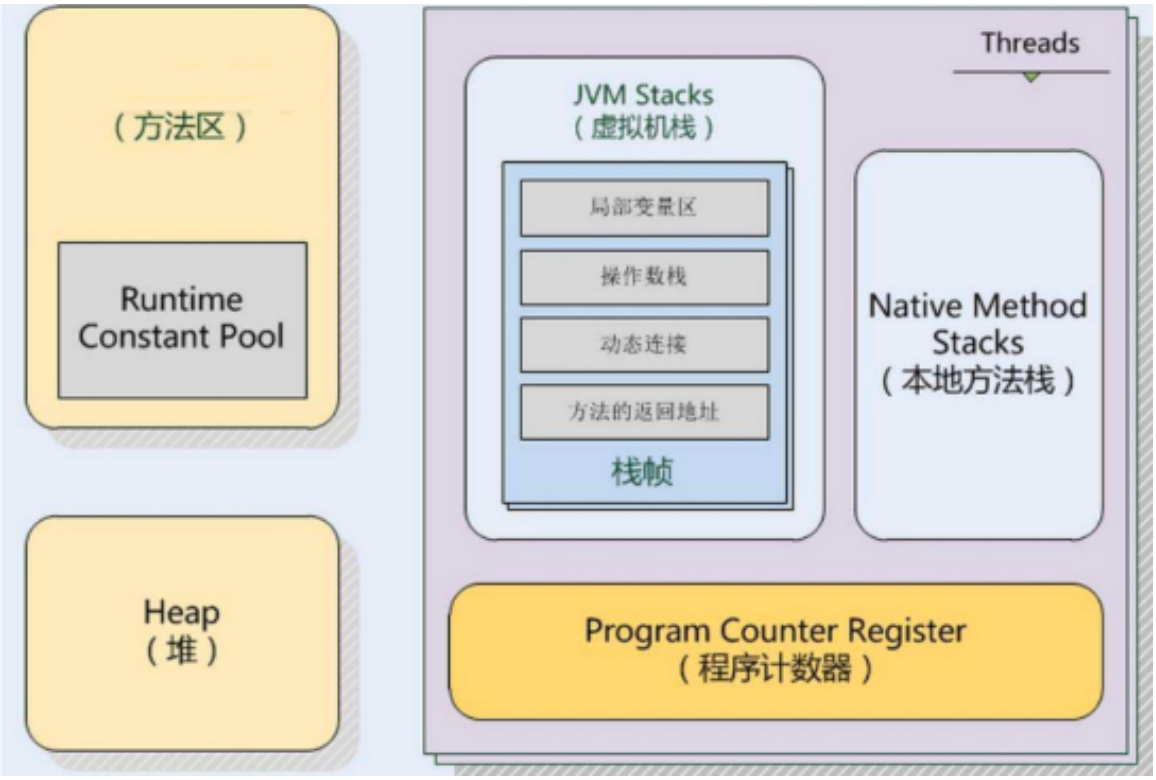
（3） 使用开源类库

Hollis 大佬之前在他的文章中也提到了:“除了自己定义ThreadPoolExecutor外。还有其他方法。这个 时候第一时间就应该想到开源类库，如apache和guava等。”他推荐使用guava提供的 ThreadFactoryBuilder来创建线程池。

JVM

请介绍一下JVM的内存结构

根据 JVM 规范，JVM 内存共分为虚拟机栈、堆、方法区、程序计数器、本地方法栈五个部分。



名称	特征	作用	配置参数	异常
程序计数器	占用内存小，线程私有，生命周期与线程相同	大致为字节码行号指示器	无	无
虚拟机栈	线程私有，生命周期与线程相同，使用连续的内存空间	Java 方法执行的内存模型，存储局部变量表、操作栈、动态链接、方法出口等信息	-Xss	StackOverflowError/ OutOfMemoryError
堆	线程共享，生命周期与虚拟机相同，可以不使用连续的内存地址	保存对象实例，所有对象实例（包括数组）都要在堆上分配	-Xms -Xsx -Xmn	OutOfMemoryError
方法区	线程共享，生命周期与虚拟机相同，可以不使用连续的内存地址	存储已被虚拟机加载的类信息、常量、静态变量、即时编译器编译后的代码等数据	-XX:PermSize:16M-XX:MaxPermSize64M/-XX:MetaspaceSize=16M-XX:MaxMetaspaceSize=64M	OutOfMemoryError
本地方法栈	线程私有	为虚拟机使用到的 Native 方法服务	无	StackOverflowError/ OutOfMemoryError

堆和栈的区别

- 堆是运行时确定内存大小，而栈在编译时即可确定内存大小
- 堆内存由用户管理(Java中由JVM管理)，栈内存会被自动释放
- 栈实现方式采用数据结构中的栈实现，具有先进后出的顺序特点，堆为一块一块的内存
- 栈由于其实现方式，在分配速度上比堆快的多。分配一块栈内存不过是简单的移动一个指针
- 栈为线程私有, 而堆为线程共享

什么是OOM

OOM，全称“Out Of Memory”，官方说明:当JVM因为没有足够的内存来为对象分配空间并且垃圾回 收器也已经没有空间可回收时，就会抛出这个error。

常见的内存溢出现象

- 堆内存溢出
    - 在JVM可使用的最大堆内存可以在启动的时候通过-Xmx参数指定。堆内存溢出是最为常见的 内存溢出问题，发生堆内存溢出时，JVM会报告如下错误:java.lang.OutOfMemoryError : java heap space。
    - 分析: 内存溢出顾名思义就是，堆内存不够用了，如果程序设计的最大对内存已经耗尽，那说 明程序设计存在问题，不该申请很多内存的逻辑申请了很多的内存，该释放的对象没有释放
    - 解决方式:
        - 检查程序，看是否有死循环或不必要地重复创建大量对象。找到原因后，修改程序和算 法
        - 增加Java虚拟机中Xms(初始堆大小)和Xmx(最大堆大小)参数的大小。如:set JAVA_OPTS= -Xms256m -Xmx1024m

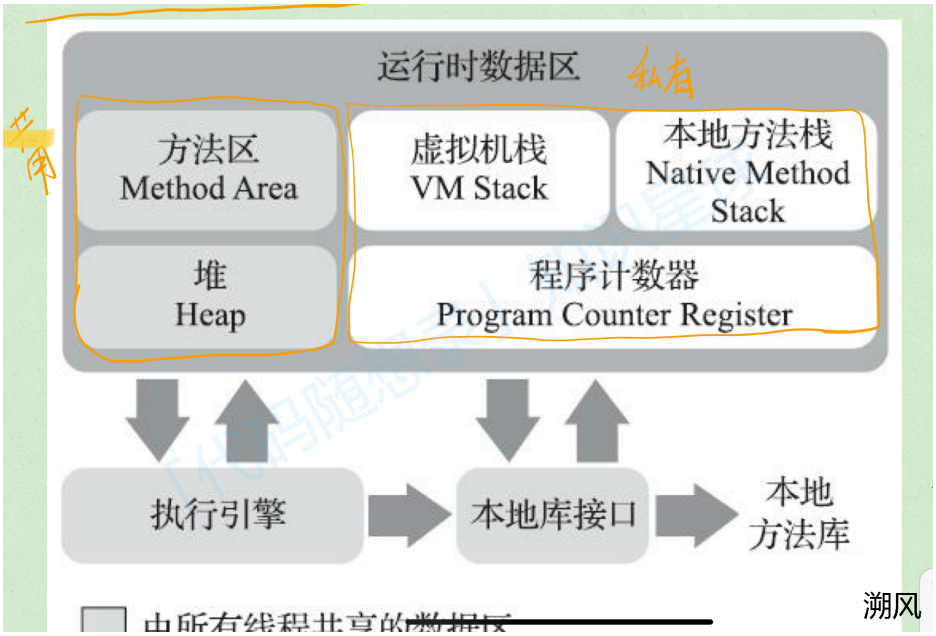
JVM内存管理

Java 运行时数据区组成部分？哪些是共享的，哪些是私有的?

有以下部分：

Java堆，方法区，Java 虚拟机栈，本地方法栈，程序计数器。

公有私有：



私有数据区

1、程序计数器

程序计数器，记录的是正的是正在执行的虚拟机字节码指令的地址。字节码解释器工作时，就是通过改变这个计数器的值来选取下一条需要执行的字节码指令，完成程序的流程控制。

在多线程情况下，每个线程都需要有一个独立的程序计数器，以便于切换回来后可以恢复到正确的执行位置。

2、Java 虚拟机栈

Java 虚拟机栈由一个个栈帧组成，每个方法被执行时，Java 虚拟机都会同步创建一个栈帧。

一个方法从开始被调用到执行完毕，对应的就是一个栈帧在虚拟机中入栈到出栈的过程。

栈帧 Stack Frame,用于存储 局部变量表，操作数栈，动态连接，方法出口等信息。

3、局部变量表

存放编译期间可知的 各种 Java 虚拟机基本数据类型，对象引用和 returnAddress 类型（指向了一条字节码指令的地址）。

这些数据类型在局部变量表中以局部变量槽来表示，局部变量表所需的内存空间在编译期间完成分配，方法运行期间，不会改变槽的数量，具体的内存空间还是由虚拟机来决定。

4、本地方法栈

为虚拟机使用到的本地方法 native 方法服务，其余的同 Java 虚拟机栈。

共享数据区

1、Java堆

Java 堆 在虚拟机启动时创建，是垃圾收集器管理的内存区域。这里暂时就这么多，再往下细分永久代啥的，以后再来

2、方法区

方法区用于存储已经被虚拟机加载的类型信息，常量，静态变量，即时编译器编译后的代码缓存等数据。

方法区与永久代，可以说 方法区是接口，是规范，永久代是 HotSpot 虚拟机给出的实现，

3、运行时常量池

运行时常量池是方法区的一部分，包括类的描述信息和常量池表，

其中 类的描述信息包括类的版本，字段，方法，接口。

常量池表用于存储 编译期生成的各种字面量与符号引用。在类加载后，常量池表存放到方法区的运行时常量池中。

运行时常量池具有动态性，运行期间也可以将新的常量放入。

对象的创建

五步

1、类加载检查

当 Java 虚拟机遇到一条字节码 new 指令时，首先将去检查这个指令的参数是否能够在常量池中定位到一个类的符号引用，并且检查这个符号引用代表的类是否已经被加载，解析和初始化过，如果没有，那么就要先执行相应的类加载过程。

2、分配内存

在类加载完成后，对象所需要的内存大小完全确定，可以对新生对象分配内存。

分配内存有两种方式：

（1）内存规整

如果内存规整，就是指针碰撞，将指针向空闲空间方向移动与对象大小相等的距离

（2）内存不规整

如果内存不规整，那就是空闲列表，维护一个列表，记录哪些内存块可用，在分配时，从列表中找到一块足够大的空间划分给对象实例，并更新列表上的记录。

划分空间时的并发问题的解决方法:

CAS + 失败重试，另一种是 TLAB :先预先分配一块内存，称为本地内存缓冲 Thread Local Allocation Buffer，线程先在自己的 TLAB 中分配，用完了之后，分配新的缓冲区时，进行同步锁定。

3、初始化内存空间

分配到的内存空间（对象头除外）全部分配为 0 值，这步保证了对象的实例字段在 Java 代码中可以不赋值就直接使用，使得程序可以访问到这些字段的数据类型所对应的零值。

4、进行对象头的设置

在对象的对象头中保存一些必要的信息：这个对象是哪个类的实例，如何才能找到类的元数据信息， 对象的哈希码，对象的GC 分代年龄等信息。

5、执行构造函数

按照程序员的意愿初始化对象

对象的结构

一个对象分为以下几个部分： 对象头，实例数据，对齐填充

1、对象头

对象头包括两类数据：对象自身的运行时数据和类型指针。

（1）对象自身的运行时数据

对象自身的运行时数据叫做 Mark Word, 包括哈希码， GC 分代年龄，锁状态标志，线程持有的锁，偏向线程ID,偏向时间戳等。

Mark Word 是一个动态的数据结构，为的就是在极小的空间内存储尽量多的数据。

（2）类型指针

类型指针，对象指向元数据的指针，Java 通过这个来确定对象是哪个类的实例。

如果对象是数组类型，那么在对象头中还必须有一块用于记录数组长度的数据，因为虚拟机可以通过普通 Java 对象的元数据信息确定 Java 对象的大小，但是如果数据的长度不确定，将无法通过元数据中的信息判断出数组的大小。

2、实例数据

实例数据是对象真正存储的有效信息，也就是我们在程序代码中所定义的各种类型的字段内容，包括从父类继承的。 这部分的存储顺序会受到 HotSpot 虚拟机默认的分配策略参数 和 字段在 Java 源码中定义顺序的影响。

3、对齐填充

对齐填充其实不是必然的，作用就是在占位符，保证对象的起始地址是 8 字节的整数倍。

也就是说，任何对象的大小都必须是 8 字节的整数倍，这个是 HotSpot 虚拟机的自动内存管理系统的要求。

对象访问定位

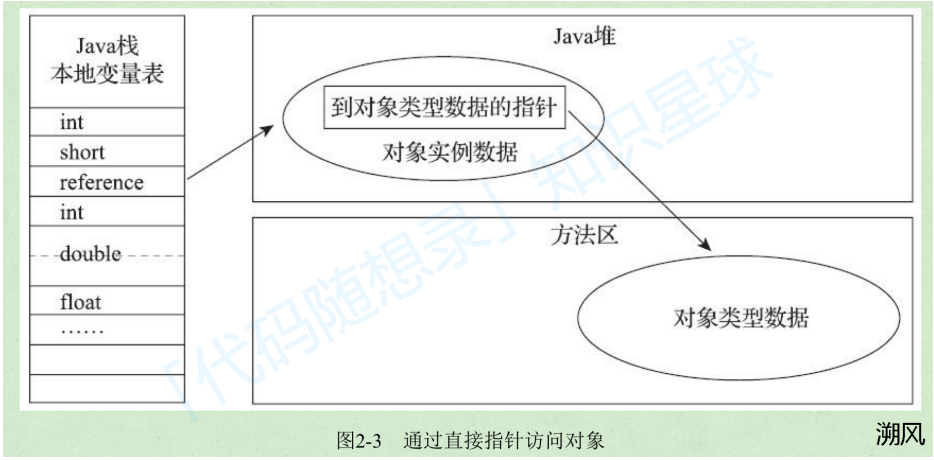
对象的访问的两种方式？区别和优缺点

Java 通过栈上的 reference 数据来操作堆上的具体对象。

对象访问的两种方式为句柄和直接指针

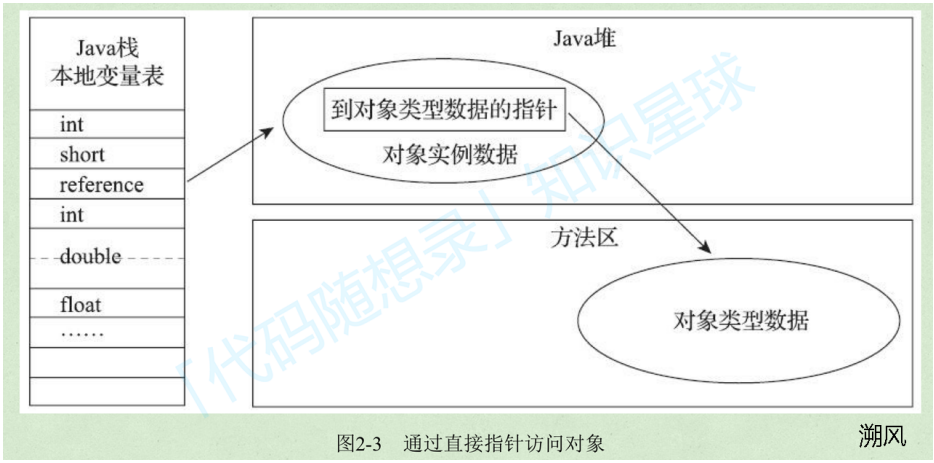
1、句柄方式

Java 堆中将可能会划分除一块内存来作为 句柄池，然后 reference 储存句柄池的地址，句柄中包含了指向对象实例数据的指针，和对象类型数据的指针。



2、直接指针

reference 中放的是对象地址



3、优缺点

句柄方式比较稳定：对象被移动时，只会改变句柄中的实例数据指针，不需要修改 reference 本身。

直接指针的话，少了一次指针定位的开销，所以好处就是访问速度更快。

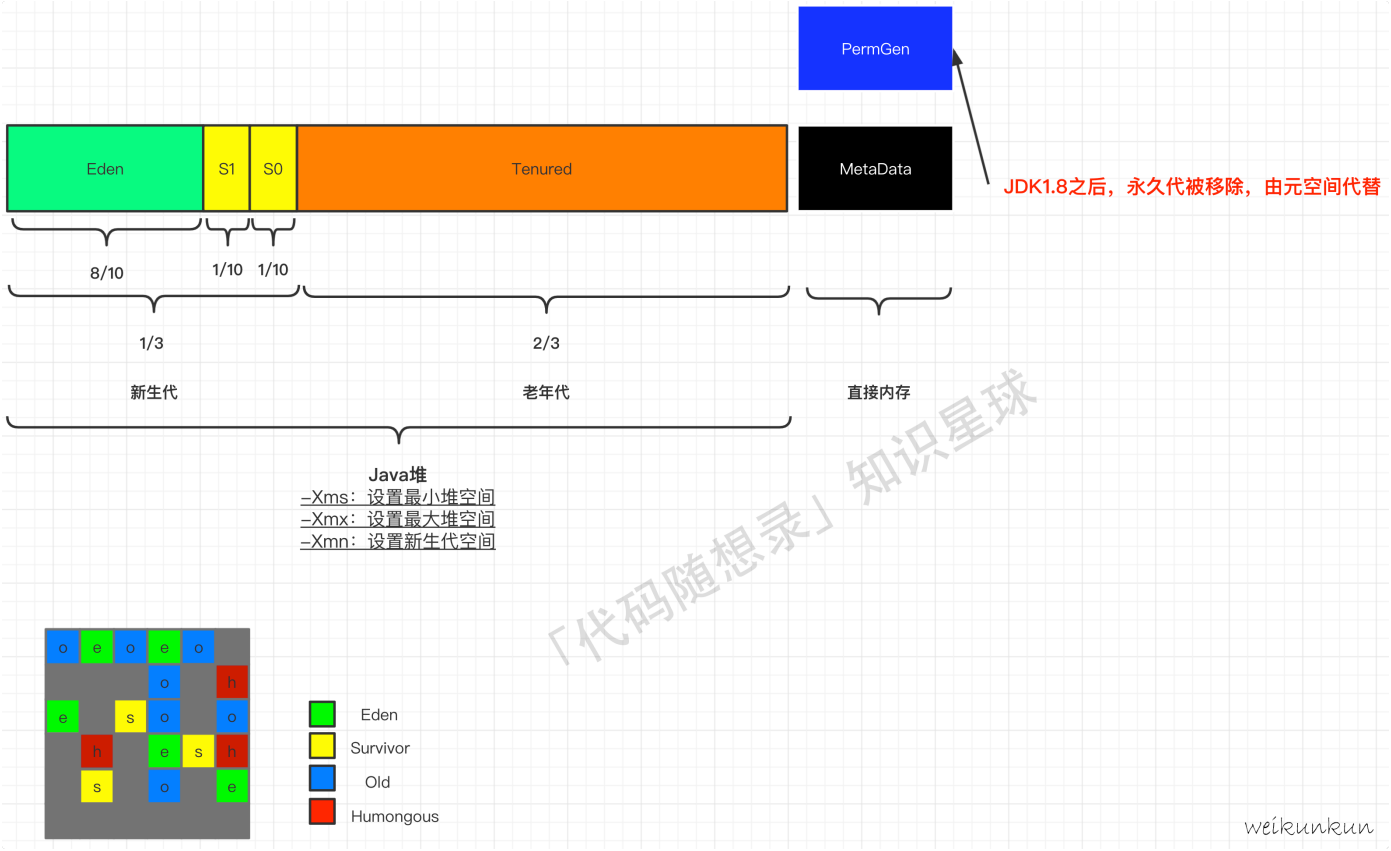
JVM之经典收集器---G1

全称Garbage-First，之前的收集器因为是直接将堆内存物理划分为新生代老年代

所以需要来两种收集器配合使用才能实现全代的回收，G1将整个堆内存划分为多个等大的region（内部连续）

然后每个region不同时间代表角色不固定，不过整体分为四种：Eden、Survioor、Old、Humongous（存储大对象）

对于G1来说整体采用的是标记-整理算法，然后从region的角度来看，采用的是复制算法



G1 垃圾收集过程主要分为4个阶段：

> 这里不同的地方，总结不同，最后还是选择以深入理解Java虚拟机为标准。

1、初始标记

1. 联GC Roots直接关联的对象
2. 触发STW

2、并发标记

和用户程序并发执行，标记出所有回收对象

3、最终标记

1. 处理并发标记后，新产生的对象
2. 触发STW

4、筛选回收

1. 先对Region的回收价值进行排序，然后根据期望暂停时间，选择性回收Region
2. 回收时采用标记复制，多条收集器线程并发执行
3. 不追求一次全部清理完
4. 触发STW

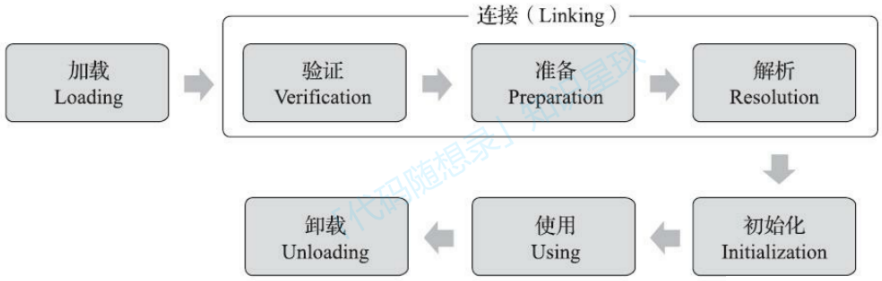
类加载机制

类加载机制：

类的数据从Class文件加载到内存，并对数据进行校验、转换解析和初始化，最 终形成可以被虚拟机直接使用的Java类型。

类的生命周期：

一个类型从被加载到虚拟机内存中开始，到卸载出内存为止，它的整个生命周期将会经历加载 （Loading）、验证（Verification）、准备（Preparation）、解析（Resolution）、初始化 （Initialization）、使用（Using）和卸载（Unloading）七个阶段，其中验证、准备、解析三个部分统称 为连接（Linking）。



其中解析也可以在初始化之后再开始，位置并不是唯一的。

加载

在加载阶段，虚拟机要完成三件事情

1、通过类的全限定名来获取定义此类的二进制字节流

2、将这个字节流所代表的静态存储结构转化为方法区的运行时数据结构

3、在内存中生成一个代表这个类的java.lang.class对象，作为方法区这个类的各种数据访问的入口

这个阶段需要用到加载器，“双亲委派模型”与此有关。

数组类对象是虚拟机直接在动态内存中构造出来，但这个数组的组件类型由加载器加载。比如`int[]`，`char[]`组件类型不是引用类型，由引导类加载器加载，而类似Integer[]这种组件为引用类型的，就按照本文说的步骤加载。

而关于第一点，获取类的二进制字节流。并没有规定必须从哪里获取，从ZIP压缩包读取，从网络中获取（例如Web Applet），运算时计算生成（如动态代理），其他文件生成（如JSP），从加密文件中获取等等。

连接

验证

Class文件可以由Java源代码编译而来，但它也可以由其他途径产生。毕竟Class文件只是一种格式，最底层的还是0/1，它可以使用包括靠键盘0和1直接在二进制编辑器中敲出 Class 文件在内的任何途径产。

Java虚拟机如果不检查输入的字节流，对其完全信任的话，很可能会因为载入了有错误或有恶意企图的字节码流而导致整个系统受攻击甚至崩溃。

1、文件格式验证：

检查是否符合Class文件的格式，如Class文件开头的魔数，主次版本号是否可以被虚拟机接受、常量池是否正确等等

2、元数据验证：

对类的元数据信息进行语义校验。比如是否有父类，是否继承了不被允许的类、是否实现了接口或者抽象类中的方法等等

3、字节码验证：

通过数据流分析和控制流分析，确定 程序语义是合法的、符合逻辑的。就是对将要执行的程序字节码验证（即Class中的Code属性），是否出现了逻辑性的错误。**字节码验证有问题，则程序一定不对，但字节码验证没问题，程序不一定能正确执行。**通过程序去校验程序逻辑是无法做到绝对准确的，不可能用程序来准确判定一段程序是否存在Bug。这一阶段很费时间，所以在编译器生成Class文件时就会生成StackMapTable，这样虚拟机直接检查StackMapTable记录是否合法就行。

4、符号引用验证：

验证将符号引用转化为直接引用时候合法，判断该类是否缺少或者被禁止访问它依赖的某些外部类、方法、字段等资源。这个动作实在解析阶段进行的，所以这7个步骤并不是严格的顺序，很多时候是交替进行的。

验证阶段有点类似查杀病毒，确保虚拟机执行的 Class 文件都是正确的。它很重要但并不是必须的，如果使用的 Class 类都是经过反复使用和验证的，那就可以关闭类验证措施，缩小类加载的时间。就像不开启防火墙，不下载一些恶意软件也能正常使用计算机。

准备

准备阶段是为类中静态变量分配内存并设置类变量初始值的阶段。

此处仅包括静态变量（static修饰），而不包括实例变量，实例变量实在对象实例化的时候随着对象一起分配在堆中。而且此处的初始值指的数据类型默认的零指。

比如`public static int a = 123`, 准备阶段会给 a 赋值为 0，而不是 123。a 变成 123 是类构造器完成的，所以是在初始化阶段才会执行。

也有例外，如果此处类字段属性存在 ConstantValue 属性，那么在准备阶段就会赋值。

例如`public static final int value = 123`，添加 final 关键字后类会有 ConstantValue 属性，此时在准备阶段 a 就会被赋值为 123。

解析

解析阶段是Java虚拟机将常量池内的符号引用替换为直接引用的过程。

符号引用是以一组符号来描述所引用的目标，它与虚拟机实现的内存布局无关，引用的目标并不一定是已经加载到虚拟机内存当中的内容。而直接引用是可以直接指向目标的指针、相对偏移量或者是一个能间接定位到目标的句柄。如果有了直接引用，那引用的目标必定已经在虚拟机的内存中存在。

初始化

类加载过程最后一步。进行准备阶段时，静态变量已经赋过一次系统要求的初始零值，而在初始化阶段，则会根据程序员通过程序编码制定的主观计划去初始化类变量和其他资源。

通俗的说初始化就是执行类构造器()方法的过程。()方法是由编译器自动收集类中的所有类变量的赋值动作和静态语句块（static{}块）中的语句合并产生的，编译器收集的顺序是由语句在源文件中出现的顺序决定的，静态语句块中只能访问到定义在静态语句块之前的变量，定义在它之后的变量，在前面的静态语句块可以赋值，但是不能访问。

```
public class Test {
    static  i = 0; // 给变量复制可以正常编译通过 可以给后面定义都赋值
    System.out.print(i); // 这句编译器会提示“非法向前引用” static int i=1在此之后才定义 无法访问}
    static int i = 1; //此处才定义
}
```

父类中的()方法在子类之前执行，保证在子类执行前父类的()已经执行完毕。()方法对于类或接口来说并不是必需的，如果一个类中没有静态语句块，也没有对变量的 赋值操作，那么编译器可以不为这个类生成()方法。

类加载器及加载机制

类加载器

在类加载流程第一步“加载”阶段，通过一个类的全限定名来获取描述该类的二进制字节流，实现这一功能的代码叫做类加载器。

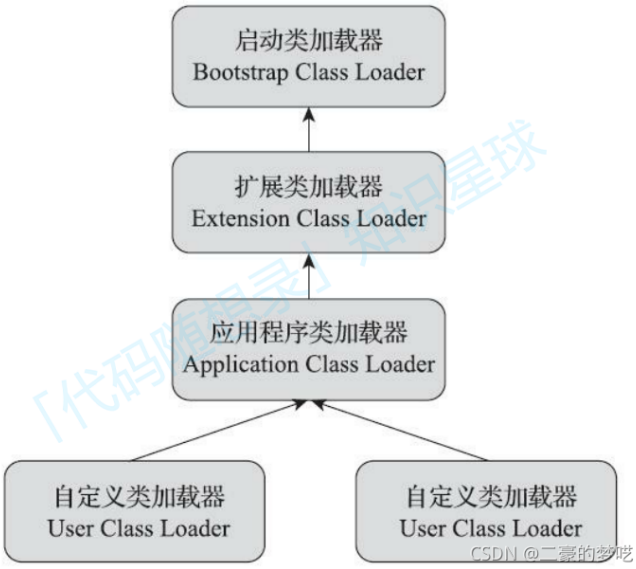
Java中的任意一个类，都必须由加载它的类加载器和这个类本身一起共同确立其在Java虚拟机中的唯一性，每一个类加载器，都拥有一个独立的类名称空间。

判断两个类是否相等，必须在这两个类是被同一个加载器加载的前提下才有意义。否则即使Class文件相同，被同一个虚拟机加载，但是使用不同的类加载器，那么两个类也是不同的。

双亲委派模型

如果一个类被不同的加载器加载，那虚拟机会认定是不同的类，这样Java体系最基础的行为也无从保证，应用程序会变得混乱。

所以有了双亲委派模型，双亲委派一共分了四层的加载器。求除了顶层的启动类加载器外，其余的类加载器都应有自己的父类加载器，子类使用“组合”关系来复用父类加载器的代码，而不是继承。



双亲委派模型一共分四层：

1、启动类加载器：

负责加载存放在 <JAVA_HOME>\lib目录，或者被-Xbootclasspath参数所指定的路径中存放的类

2、扩展类加载器：

它负责加载<JAVA_HOME>\lib\ext目录中，或者被java.ext.dirs系统变量所 指定的路径中所有的类库

3、应用类加载器：

负责加载用户类路径 （ClassPath）上所有的类库，开发者同样可以直接在代码中使用这个类加载器。如果应用程序中没有 自定义过自己的类加载器，一般情况下这个就是程序中默认的类加载器。

4、自定义加载器：

用户自行扩展，典型的如增加除了磁盘位置之外的Class文件来源，或者通过类 加载器实现类的隔离、重载等功能。

如果一个类加载器收到了类加载的请求，它首先不会自己去尝试加载这个类，而是把这个请求委派给父类加载器去完成，每一个层次的类加载器都是如此，因此所有的加载请求最终都应该传送到最顶层的启动类加载器中，只有当父加载器反馈自己无法完成这个加载请 求（它的搜索范围中没有找到所需的类）时，子加载器才会尝试自己去完成加载。

破坏双亲委派

亲委派很好地解决了各个类加载器协作时基础类型的一致性问题（越基础的类由越上层的加载器进行加载），但如果有基础类型又要调用回用户的代码就不得不破坏双亲委派模型，也就是父类加载器需要去请求子类加载器去完成需要的类。

典型例子如JNDI，JNDI的代码由启动类加载器完成加载，但是这个类需要应用程序ClassPath下的JNDI服务提供者接口，启动类加载器无法加载这些代码。为解决这个问题使用线程上下文类加载器，JNDI服务使用这个线程上下文类加载器去加载所需的SPI服务代码，这是一种父类加载器去请求子类加载器完成类加载的行为，这种行为实际上是打通了双亲委派模型的层次结构来逆向使用类加载器。

OSGi实现模块化热部署：在OSGi环境下，类加载器不再双亲委派模型推荐的树状结构，而是进一步发展为更加复杂的网状结构。

内部类

使用原因：

1. 内部类可以对**同一个包中的其他类隐藏**
2. 内部类方法可以访问**定义这个类的**作用域中的数据，包括**私有数据**

1. 使用内部类访问对象状态

一个内部类方法可以访问自身的数据字段，也可以访问创建它的外围对象的数据字段，所以，内部类的对象总有一个 隐式引用，指向创建它的外部类对象

外围类的引用在构造器中设置，编译器会修改所有内部类的构造器，添加提个对应的外围类引用的参数

只有内部类是可以是 private，而常规类只可以有 包可见性(default、protected) 或 公共可见性(public)

2. 内部类的特殊语法规则

1、外围类引用`OuterClass.this`

如：Person.this

2、内部类对象的构造器

```
outerObject.new InnerClass(construction param)
```

3、外围类的作用域之外引用内部类

```
OuterClass.InnerClass
```

4、内部类中**声明的所有静态字段都必须是 final**，并初始化为一个编译时常量

5、内部类不能有static方法，除了**访问外围类的静态字段和方法**的静态方法

3. 内部类与编译器

内部类是一个编译器现象，与虚拟机无关，编译器会讲内部类转换为常规的类文件，用 `$` 分隔外部类名和内部类名

4. 局部内部类

当这个类的对象只被一个方法创建一次时，在一个方法中的局部地定义这个类

1、声明局部类时

不能有访问说明符(private，public) 局部类的作用域被限定在声明这个局部类的块中

2、局部类的优势

对外部世界完全隐藏，**连外围类中的其他代码都不能访问它**

5. 由外部方法访问变量

与其他内部类相比较，局部类可以访问 **方法中的局部变量**
不过，这些变量**必须是 final 变量**

6. 匿名内部类

只创建这个类的一个对象，不需要为类指定名字

1、语法： `new superType(construcrion param) {...}`

superType可以是接口，内部类要实现这个接口，也可以是一个类，内部类就要扩展这个类

2、构造器名字必须与类名相同

匿名内部类没有类名，所以匿名内部类不能有构造器，实际上，构造参数要传递给超类的构造器。但，只要内部类实现一个接口，就不能有任何构造参数，不过仍然要小括号

尽管匿名内部类不能有构造器，但可以提供一个对象初始化块

```
new Person("asd") {   {init}   ....}
```

3、如果构造参数列表的结束小括号后面跟一个开始大括号，就是在定义匿名内部类

4、**静态方法中要得到当前类名**，不能使用 this，静态方法没有this，可以使用如下技巧

```
new Object(){}.getClass().getEnclosingClass()
```
上式，创建了一个匿名内部类，getEnclosingClass得到他的外围类

7. 双括号初始化

利用了内部类语法，使用匿名列表

```
ArrayList<Integer> array = new ArrayList<>(){{
    add(1);
    add(2);
    add(3);
}};
System.out.println(array.toString());
```

外层括号建立了ArrayList的一个**匿名子类**，内层括号则是一个**对象初始化块**

适用于这些数组列表不需要再使用的情况

Map，Deque，Set等集合也有类似技巧

8. 静态内部类

static声明的类

1. 静态内部类类似于其他内部类，不过静态内部类的独享没有生成它外围类对象的引用
2. 只要内部类不需要访问外围类对象，就应该使用静态内部类
3. 与常规内部类不同，静态内部类可以由静态字段和方法
4. 在接口中声明的内部类自动是 public 和 static

代理类

1. 创建代理对象

使用Proxy类的newProxyInstance方法，有三个参数

1. 一个类加载器
2. 一个Class对象数组
3. 一个调用处理器

```
Student s1 = new Student();
//动态代理增强s1对象
/*
三个参数:
 类加载器：真实对象.getClass().getClassloader()
 接口数组：真实对象.getClass().getInterfaces()
 处理器：new InvocationHandler()
*/
Student proxy_s1 = (Student)Proxy.newProxyInstance(s1.getClass().getClassLoader(),
s1.getClass().getInterfaces(), new InvocationHandler() {
   //代理逻辑编写的方法，代理对象调用的所有方法都会触发该方法的执行
   /*
    参数：proxy：代理对象
      method：代理对象调用的方法，被封装为对象
      args：代理对象调用实际方法时，传递的实际参数
   */
            @Override
```

```
            public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
             System.out.println("每次都会调用我");

    if(method.getName().equals("setName")){//如果调用setName()方法就增强，否则不增强
     String name = args[0];
     name = "李四";
     String obj = (String) method.invoke(s1,name);//使用真实对象调用该方法，改变参数调用，同时
改变返回值为String
                return obj+"和王五";
    }else{
     Object obj = method.invoke(s1,args);//使用真实对象调用该方法,原样调用
                return obj;
    }
            }
        });
      System.out.println(proxy_s1.setName("张三");)//输出“每次都会调用我”，“李四和王五”，
      //证明每次调用代理对象的真实对象的方法都会执行invoke()
      //并且method对象是代理对象调用的方法的被封装后的对象，args是传递的参数
```

2. 代理类的特性

代理类是在程序运行过程中动态创建的，一旦被创建，就成为了常规类

所有代理类都扩展Proxy类，一个代理类只有一个实例字段，调用处理器，在Proxy超类中定义，完成代理对象任务所需要的任何额外数据都必须存储在调用处理器中

代理类总是 public 或 final，如果代理类实现的所有接口都是 public 这个代理类就不输入任何特定的包，斗则所有非公共的接口都必须属于同一个包，代理类也属于这个包

新特性

Lambda 表达式

将代码块传递到某个对象，这个对象将会在某个时间调用（**延迟执行**）

1. 语法

1、即使lambda表达式没有参数，仍然要提供空括号，类似于无参方法

`() -> {....}`

2、如果可以推导出一个lambda表的参数类型，则可以忽略其类型

`(s1, s2) -> {...}`

3、如果方法只有一个参数，而且这个参数类型可以推导出，那么可以省略小括号

`a -> {....}`

4、如果一个lambda表达式只在一些分支返回值，这是不合法的

如`(int x) -> {if (x > 0) return 1;}`

2. 函数式接口

1、只有一个抽象方法的接口，可以提供一个lambda表达式

2、在Java中，对lambda表达式所能做的也就是转换为 函数式接口

3、ArrayList类有一个 removeIf 方法，它的参数就是一个 **Predicate，这个接口用来传递 lambda 表达式**

如，下面的语句将从一个数组列表删除所有null值

`list.removeIf(e -> e == null)`

4、**供应者（supplier）**没有参数，调用时会生成一个T类型的值，用于实现懒计算

```
LocalDate hire = Objects.requireNunNullOrElseGet(day,
new LocalDate(1970,1,1));
```

这种情况下，每次都会构建默认的LocalDate，而day 为 null 的情况很少，因此，可用通过supplier实现延迟这个计算

```
LocalDate hire = Objects.requireNunNullOrElseGet(day, ()->new LocalDate(1970,1,1));
```

此时只有在需要值时，才会调用供应者

3. 方法引用

```
System.out::println;
```

1、指示编译器生成一个函数式接口的实例

覆盖这个接口的抽象方法来调用给定的方法

2、类似于lambda表达式，方法引用也不是一个对象

3、要用 `::`运算符分隔方法名 与 对象 或 类名

主要有三种情况：

（1）`对象::实例方法` lambda参数作为方法的显示参数传入

（2）`类::实例方法` String::trim ，lambda表达式会成为隐式对象

（4）`类::静态方法` Integer::valueOf，lambda表达式会传递到这个静态方法

只有当那个lambda表达式的体只调用一个方法而不做其他操作时，才能把lambda表达式重写成方法引用

4、方法引用不能独立存在，总是会转换为函数式接口的实例

5、包含对象的方法引用 与 等价的lambda表达式还有一个细微差别

如果对象为空，方法引用会直接抛出异常，而lambda表达式只有在调用时才会抛出异常

6、可以在方法引用中使用 this 、super参数

4. 构造器引用

1、与方法引用类似，不过方法名为 new

`Person::new`这就是Person构造器的一个引用，具体哪一个构造器呢，取决于上下文

2、数组的构造器引用`Integer[]::new`

5. 变量作用域

1、lambda表达式可以“捕获”外围作用域中变量的值，只要确保所捕获的值时明确定义的

（1）**lambda表达式中，只能引用值不会改变的外部变量**
（2）lambda表达式中捕获的变量必须是 final 变量

2、**lambda表达式的与 嵌套块 有相同的作用域**，在lambda表达式中声明一个与局部变量同名的参数或局部变量是不合法的

3、lambda表达式中的 this 含义与外面一致

Spring

SpringBoot、SpringMVC、Spring有什么区别

1、Spring

SpringBoot、SpringMVC、Spring有什么区别

Spring是一个全面的Java企业级应用程序开发框架，提供了广泛的功能，包括依赖注入、AOP（面向切面编程）、事务管理等。

2、SpringMVC

Spring MVC 是 Spring 中的一个很重要的模块，主要赋予 Spring 快速构建 MVC 架构的 Web 程序的能力。MVC 是模型(Model)、视图(View)、控制器(Controller)的简写，其核心思想是通过将业务逻辑、数据、显示分离来组织代码。 主要关注于处理Web请求、管理用户会话、控制应用程序流程等。

3、SpringBoot

Springboot是一个微服务框架，延续了spring框架的核心思想IOC和AOP，简化了应用的开发和部署。Spring Boot是为了简化Spring应用的创建、运行、调试、部署等而出现的，提供约定大于配置的方式, 使用它可以做到专注于Spring应用的开发，而无需过多关注XML的配置。使得构建独立的、自包含的Spring应用程序变得更加容易。

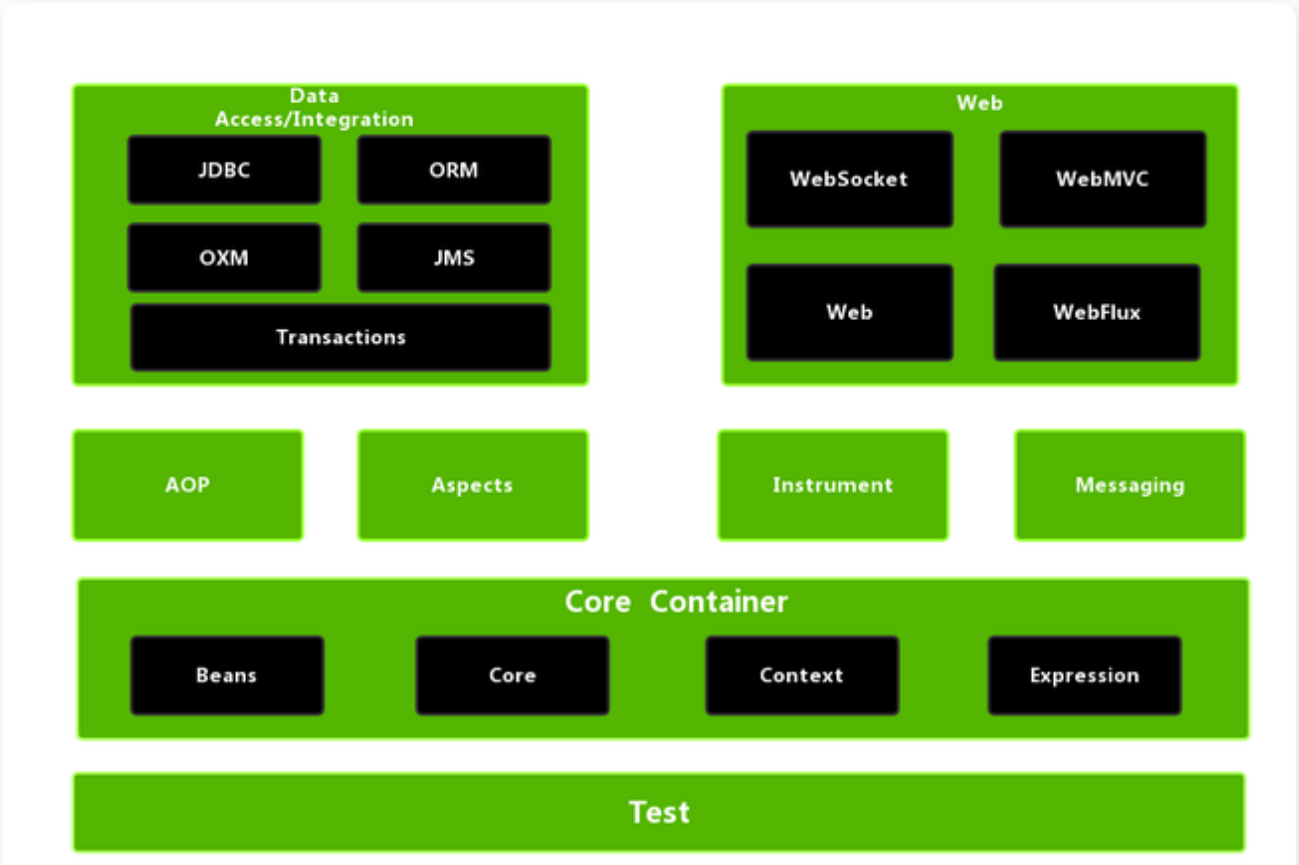
简而言之，Spring是一个全面的框架，Spring MVC是Spring框架的一部分，专注于Web应用程序开发，而Spring Boot是基于Spring的微服务框架，旨在简化和加速Spring应用程序的开发。

简单介绍一下Spring的框架体系

Spring框架至今已集成了20多个模块，这些模块分布在以下模块中:

- 核心容器(Core Container)
- 数据访问/集成(Data Access/Integration)层
- Web层
- AOP(Aspect Oriented Programming)模块

- 植入(Instrumentation)模块
- 消息传输(Messaging)
- 测试(Test)模块



1. 核心容器

Spring的核心容器是其他模块建立的基础，有Spring-core、Spring-beans、Spring-context、Spring-context-support和Spring-expression(String表达式语言)等模块组成。

- Spring-core模块**:提供了框架的基本组成部分，包括控制反转(Inversion of Control，IOC)和 依赖注入(Dependency Injection，DI)功能。**
- Spring-beans模块**: 提供了BeanFactory，是工厂模式的一个经典实现，Spring将管理对象称为 Bean。**
- Spring-context模块**:建立在Core和Beans模块的基础之上，提供一个框架式的对象访问方式， 是访问定义和配置的任何对象的媒介。ApplicationContext接口是Context模块的焦点。**
- Spring-context-support模块**:支持整合第三方库到Spring应用程序上下文，特别是用于高速缓 存(EhCache、JCache)和任务调度(CommonJ、Quartz)的支持。**
- Spring-expression模块:提供了强大的表达式语言去支持运行时查询和操作对象图。这是对 JSP2.1规范中规定的统一表达式语言(Unified EL)的扩展。该语言支持设置和获取属性值、属性 分配、方法调用、访问数组、集合和索引器的内容、逻辑和算术运算、变量命名以及从Spring的 IOC容器中以名称检索对象。它还支持列表投影、选择以及常用的列表聚合。

2. AOP和**Instrumentation**

- Spring-aop模块**:提供了一个符合AOP要求的面向切面的编程实现，允许定义方法拦截器和切入 点，将代码按照功能进行分离，以便干净地解耦。**
- Spring-aspects模块**:提供了与AspectJ的集成功能，AspectJ是一个功能强大且成熟的AOP框架。**
- Spring-instrument模块: 提供了类植入(Instrumentation)支持和类加载器的实现，可以在特定的应用服务器中使用。

3. 消息

- Spring4.0以后新增了消息(Spring-messaging)模块，该模块提供了对消息传递体系结构和协议的支持。

4. 数据访问/集成

数据访问/集成层由JDBC、ORM、OXM、JMS和事务模块组成。

- Spring-jdbc模块:提供了一个JDBC的抽象层，消除了烦琐的JDBC编码和数据库厂商特有的错误 代码解析
- Spring-orm模块:为流行的对象关系映射(Object-Relational Mapping)API提供集成层，包括 JPA和Hibernate。使用Spring-orm模块可以将这些O/R映射框架与Spring提供的所有其他功能结 合使用，例如声明式事务管理功能。
- Spring-oxm模块:提供了一个支持对象/XML映射的抽象层实现，例如JAXB、Castor、JiBX和 XStream
- Spring-jms模块(Java Messaging Service):指Java消息传递服务，包含用于生产和使用消息 的功能。自Spring4.1以后，提供了与Spring-messaging模块的集成。
- Spring-tx模块(事务模块): 支持用于实现特殊接口和所有POJO(普通Java对象)类的编程和声 明式事务管理。

5. Web

Web层由Spring-web、Spring-webmvc、Spring-websocket和Portlet模块组成。

- Spring-web模块:提供了基本的Web开发集成功能，例如多文件上传功能、使用Servlet监听器初 始化一个IOC容器以及Web应用上下文
- Spring-webmvc模块:也称为Web-Servlet模块，包含用于web应用程序的Spring MVC和REST Web Services实现。Spring MVC框架提供了领域模型代码和Web表单之间的清晰分离，并与 Spring Framework的所有其他功能集成。
- Spring-websocket模块 :Spring4.0以后新增的模块，它提供了WebSocket和SocketJS的实现。

6. 测试

- Spring-test模块支持使用JUnit或TestNG对Spring组件进行单元测试和集成测试。

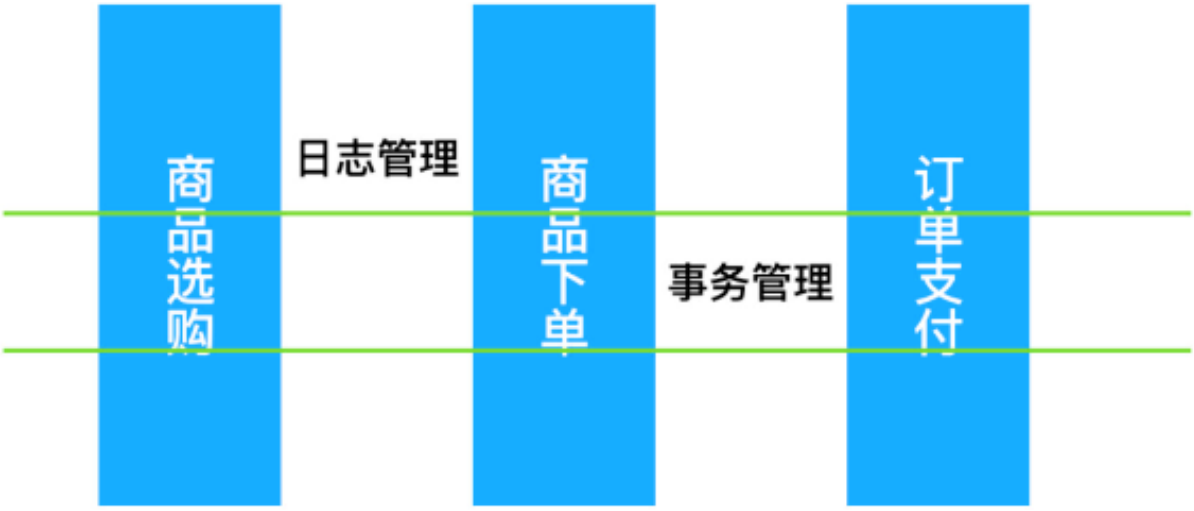
介绍一下Spring中的AOP

1. 什么是AOP

AOP (Aspect Orient Programming),直译过来就是 面向切面编程。AOP 是一种编程思想，是面 向对象编程的一种补充。面向对象编程将程序抽象成各个层次的对象，而面向切面编程是将程序抽 象成各个切面。

2. 为什么需要AOP

AOP采用的是横向切面的方式，注入与主业务流程无关的功能，例如事务管理和日志管理。

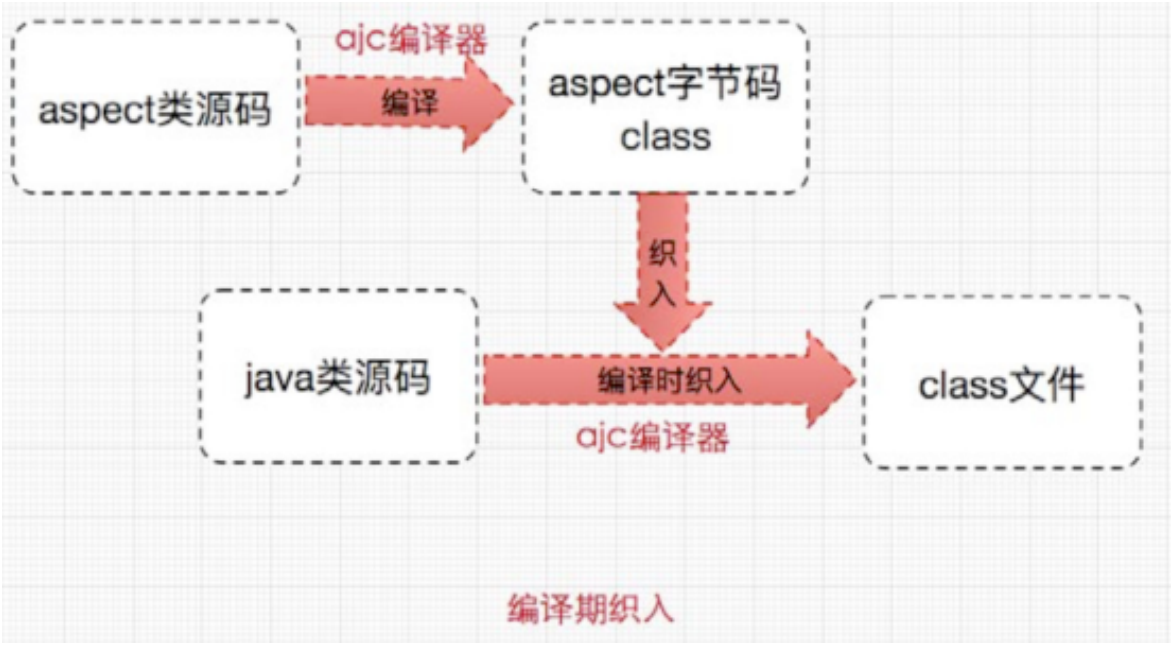


3. AOP实现分类

AOP 要达到的效果是，保证开发者不修改源代码的前提下，去为系统中的业务组件添加某种通用 功能。

按照 AOP 框架修改源代码的时机，可以将其分为两类:

- **AspectJ** 静态 AOP 实现， AOP 框架在编译阶段对程序源代码进行修改，生成了静态的 AOP 代理类(生成的 *.class 文件已经被改掉了，需要使用特定的编译器)。



- 动态 AOP 实现， AOP 框架在运行阶段对动态生成代理对象(在内存中以 JDK 动态代理，或 CGlib 动态地生成 AOP 代理类)，如 SpringAOP。

JDK动态代理:使用JDK动态代理，目标类必须实现的某个接口，如果某个类没有实现接口则不能 生成代理对象。

CGLIB代理:Cglib原理是针对目标类生成一个子类，覆盖其中的所有方法，所以目标类和方法不 能声明为final类型。

谈一谈你对Spring AOP的理解

> 将AOP的几个核心概念说出来，将AOP的实现方式说出来

AOP

AOP(Aspect Oriented Programming)，即面向切面编程，可以说是OOP(Object Oriented Programming，面向对象编程)的补充和完善。OOP引入封装、继承、多态等概念来建立一种对象层 次结构，用于模拟公共行为的一个集合。不过OOP允许开发者定义纵向的关系，但并不适合定义横向的 关系，例如日志功能。日志代码往往横向地散布在所有对象层次中，而与它对应的对象的核心功能毫无 关系对于其他类型的代码，如安全性、异常处理和透明的持续性也都是如此，这种散布在各处的无关的 代码被称为横切(cross cutting)，在OOP设计中，它导致了大量代码的重复，而不利于各个模块的重 用。

AOP技术恰恰相反，它利用一种称为"横切"的技术，剖解开封装的对象内部，并将那些影响了多个类的 公共行为封装到一个可重用模块，并将其命名为"Aspect"，即切面。所谓"切面"，简单说就是那些与业 务无关，却为业务模块所共同调用的逻辑或责任封装起来，便于减少系统的重复代码，降低模块之间的 耦合度，并有利于未来的可操作性和可维护性。

使用"横切"技术，AOP把软件系统分为两个部分:**核心关注点**和**横切关注点**。业务处理的主要流程是核 心关注点，与之关系不大的部分是横切关注点。横切关注点的一个特点是，他们经常发生在核心关注点 的多处，而各处基本相似，比如权限认证、日志、事物。AOP的作用在于分离系统中的各种关注点，将 核心关注点和横切关注点分离开来。

AOP核心概念

1、横切关注点 对哪些方法进行拦截，拦截后怎么处理，这些关注点称之为横切关注点 2、切面(aspect) 类是对物体特征的抽象，切面就是对横切关注点的抽象 3、连接点(joinpoint)

被拦截到的点，因为Spring只支持方法类型的连接点，所以在Spring中连接点指的就是被拦截到的方 法，实际上连接点还可以是字段或者构造器

4、切入点(pointcut)

对连接点进行拦截的定义

5、通知(advice)

所谓通知指的就是指拦截到连接点之后要执行的代码，通知分为前置、后置、异常、最终、环绕通知五
类

6、目标对象
代理的目标对象
7、织入(weave)
将切面应用到目标对象并导致代理对象创建的过程 8、引入(introduction) 在不修改代码的前提下，引入可以在**运行期**为类动态地添加一些方法或字段

Spring对AOP的支持

Spring中AOP代理由Spring的IOC容器负责生成、管理，其依赖关系也由IOC容器负责管理。因此， AOP代理可以直接使用容器中的其它bean实例作为目标，这种关系可由IOC容器的依赖注入提供。 Spring创建代理的规则为:

1、默认使用Java动态代理来创建AOP代理，这样就可以为任何接口实例创建代理了

2、当需要代理的类不是代理接口的时候，Spring会切换为使用CGLIB代理，也可强制使用CGLIB AOP编程其实是很简单的事情，纵观AOP编程，程序员只需要参与三个部分:

1、定义普通业务组件

2、定义切入点，一个切入点可能横切多个业务组件

3、定义增强处理，增强处理就是在AOP框架为普通业务组件织入的处理动作

所以进行AOP编程的关键就是定义切入点和定义增强处理，一旦定义了合适的切入点和增强处理，AOP 框架将自动生成AOP代理，即:代理对象的方法=增强处理+被代理对象的方法。

动态代理的方式

Spring AOP中的动态代理主要有两种方式，JDK动态代理和CGLIB动 态代理:

- JDK动态代理只提供接口的代理，不支持类的代理。核心InvocationHandler接口和Proxy类，InvocationHandler 通过invoke()方法反射来调用目标类中的代码，动态地将横切逻辑和业务编织在一 起;接着，Proxy利用 InvocationHandler动态创建一个符合某一接口的的实例, 生成目标类的代理对 象。
- 如果代理类没有实现 InvocationHandler 接口，那么Spring AOP会选择使用CGLIB来动态代理目标 类。CGLIB(Code Generation Library)，是一个代码生成的类库，可以在运行时动态的生成指定类的 一个子类对象，并覆盖其中特定方法并添加增强代码，从而实现AOP。CGLIB是通过继承的方式做的动 态代理，因此如果某个类被标记为final，那么它是无法使用CGLIB做动态代理的。

静态代理与动态代理区别在于生成AOP代理对象的时机不同，相对来说AspectJ的静态代理方式具 有更好的性能，但是AspectJ需要特定的编译器进行处理，而Spring AOP则无需特定的编译器处理。

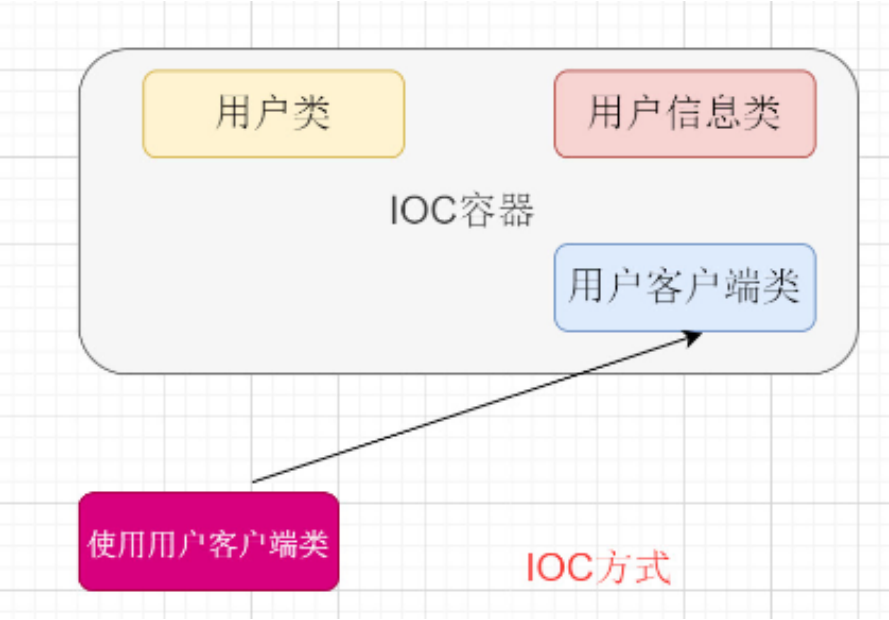
介绍一下Spring中的IOC和DI

1. 什么是IOC

Ioc—Inversion of Control，即控制反转，不是什么技术，而是一种设计思想。在Java开发 中，Ioc意味着将你设计好的对象交给容器控制，而不是传统的在你的对象内部直接控制。

2. IOC的好处

使用IOC后，我们不需要自己去创建某个类的实例，而由IOC容器去创建，当我们需要使用某个对 象时，直接到容器中去获取就可以了



3. 什么是DI

创建被调用对象由Spring来完成，在容器实例化对象的时候主动的将被调用者注入给调用者，这样

就产生为了“依赖注入”的概念，也就是"DI"。

4. Ioc和DI的关系

其实它们是同一个概念的不同角度描述，由于控制反转概念比较含糊(可能只是理解为容器控制对 象这一个层面，很难让人想到谁来维护对象关系)，所以2004年大师级人物Martin Fowler又给出 了一个新的名字:“依赖注入”，相对IoC 而言，“依赖注入”明确描述了“被注入对象依赖IoC容器配 置依赖对象”。

IoC

IoC: 控制反转，意思就是将创建对象的控制权从自己硬编码new的一个对象反转到了第三方身上。

IoC的主要实现方式是依赖注入，Spring中的依赖注入方式有：构造方法注入、settter注入、接口注入。

目的：

帮助我们接耦各种有依赖关系的业务对象之间的绑定关系

IoC-Provider

虽然不需要我们自己来做绑定关系，但是这部分的工作还是需要有人来实现的，所以IoC Provider就担任了这个角色，同时IoC Provider的职责也不仅仅这些，其基础职责如下：

1、业务对象的构建管理：

IoC中，业务对象不需要关心所依赖的对象如何构建如何获取，这部分任务交由IoC Provider

2、业务对象之间的依赖绑定：

通过结合之前构建和管理的所有业务对象，以及各个业务对象之间可识别的依赖关系，将这些对象所依赖的对象注入绑定。从而保证每个业务对象在使用的时候，可以出狱就绪状态。

Spring的IoC容器

担任了IoC Provider的职责，同时在此基础上，还增加了对Bean生命周期的管理、AOP支持内容。

从整体来看Spring的IoC容器的作用，共分为两部分：

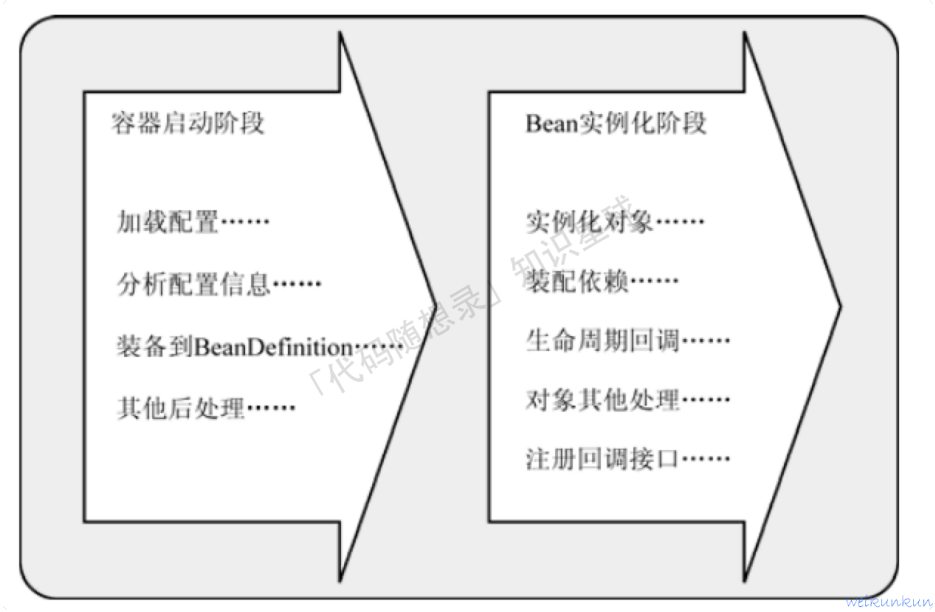
1、容器启动阶段：

以某种方式将配置的Bean信息（XML、注解、Java编码）加载如整个Spring应用

2、Bean实例化阶段：

将加载的Bean配置信息组装成应用需要的业务对象

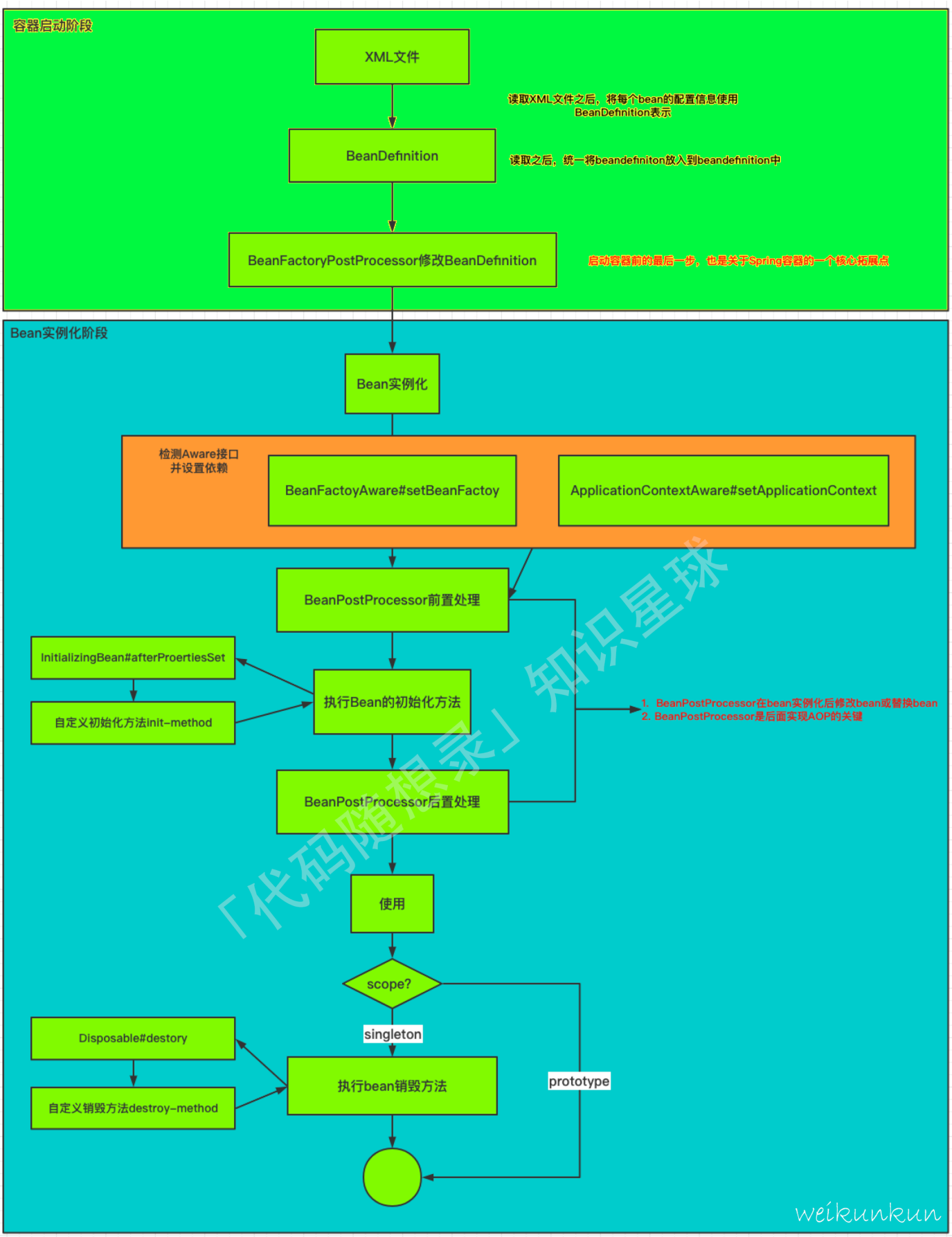
在此基础上，还充分运用了这两个阶段不同的特点，都预留了拓展钩子，供我们根据业务场景进行自定义拓展。



一些核心的接口、类介绍：

1. Resource: 用于解决 IoC 容器中的内容从哪里来的问题，也就是 配置文件从哪里读取、配置文件如何读取 的问题
2. BeanDefinition: 用于解决 Bean 的具体定义问题，包括 Bean 的名字是什么、它的类型是什么，它的属性赋予了哪些值或者引用，也就是 如何在 IoC 容器中定义一个 Bean，使得 IoC 容器可以根据这个定义来生成实例的问题。
3. BeanFactoy: 用于解决 IoC 容器在 已经获取 Bean 的定义的情况下，如何装配、获取 Bean 实例 的问题。
4. ApplcationContext: 对上述的内容进行了功能的封装，解决 根据地址获取 IoC 容器并使用 的问题。

从Bean的角度来看，其整个生命周期如下：



Spring只帮我们管理单例Bean的生命周期，对于prototype类型的bean，Spring在创建好交给使用者使用之后，就不在管理其后续的生命周期了。

粗略涉及具体源码的整体流程：

1. BeanDefinitionReader读取Bean的配置信息（XML等），将读取到的每个Bean的配置信息使用BeanDefinition表示，同时注册到相应的BeanDefinitionRegistry（一个map）中 。
2. 通过实现了BeanFactoryPostProcessor的类，自定义修改BeanDefinition中的信息（如果有的话）
3. Bean的实例化:
   (1) 采用策略化bean的实例， 两种方式：cglib、反射
   (2) 获取Bean的实例之后，根据BeanDefinition中信息，填充Bean的属性、依赖
4. 检测各种Aware接口，BeanFactory的、ApplicationContext的等
5. 调用BeanPostProcessor接口的前置处理方法，处理符合要求的Bean实例
6. 如果实现了InitializingBean接口，执行对应的afterPropertiesSet()方法
7. 如果定义了init-method，执行对应的自定义初始化方法
8. 调用BeanPostProcessor接口的前置处理方法，处理符合要求的Bean实例
9. 使用
10. 判断Bean的Scope，如果是prototype类型，不再管理
11. 如果是单例类型，如果实现了DisposableBean接口，执行对应的destoy方法
12. 如果定义了destory-method，执行对应的自定义销毁方法

两个拓展点：

1、BeanFactoryPostProcess

Spring提供的容器扩展机制，允许我们在bean实例化之前修改bean的定义信息即BeanDefinition的信息

2、BeanPostProcessor

也是Spring提供的容器扩展机制，不同于BeanFactoryPostProcessor的是，BeanPostProcessor在bean实例化后修改bean或替换bean。BeanPostProcessor是后面实现AOP的关键。

Bean的作用域

> 将五种作用域的详细的介绍一下，其次要将它从创建到销毁的过程说一遍。

在 Spring 中，那些组成应用程序的主体及由 Spring IOC 容器所管理的对象，被称之为 bean。简单地 讲，bean 就是由 IOC 容器初始化、装配及管理的对象，除此之外，bean 就与应用程序中的其他对象 没有什么区别了。而 bean 的定义以及 bean 相互间的依赖关系将通过配置元数据来描述。

Spring中的bean默认都是单例的，这些单例Bean在多线程程序下如何保证线程安全呢? 例如对于 Web应用来说，Web容器对于每个用户请求都创建一个单独的Sevlet线程来处理请求，引入Spring框架 之后，每个Action都是单例的，那么对于Spring托管的单例Service Bean，如何保证其安全呢? Spring的单例是基于BeanFactory也就是Spring容器的，单例Bean在此容器内只有一个，Java的单例是基于 JVM，每个JVM内只有一个实例。

作用域

创建一个bean定义，其实质是用该bean定义对应的类来创建真正实例的“配方”。把bean定义看成一个 配方很有意义，它与class很类似，只根据一张“处方”就可以创建多个实例。不仅可以控制注入到对象中 的各种依赖和配置值，还可以控制该对象的作用域。这样可以灵活选择所建对象的作用域，而不必在 Java Class级定义作用域。Spring Framework支持五种作用域，分别阐述如下表。

类别	说明
singleton	在Spring IoC容器中仅存在一个Bean实例，Bean以单例方式存在，默认值
prototype	每次从容器中调用Bean时，都返回一个新的实例，即每次调用getBean()时，相当于执行new XxxBean()
request	每次HTTP请求都会创建一个新的Bean，该作用域仅适用于WebApplicationContext环境
session	同一个HTTP Session 共享一个Bean，不同Session使用不同Bean，仅适用于WebApplicationContext 环境
globalSession	一般用于Portlet应用环境，该作用域仅适用于WebApplicationContext 环境

五种作用域中，**request**、session 和 **global session** 三种作用域仅在基于web的应用中使用(不必关 心你所采用的是什么web应用框架)，只能用在基于 web 的 Spring ApplicationContext 环境。

Signleton-唯一bean实例

当一个 bean 的作用域为 singleton，那么Spring IoC容器中只会存在一个共享的bean实例，并且所 有对bean 的请求，只要 id 与该bean定义相匹配，则只会返回bean的同一实例。 singleton 是单例 类型(对应于单例模式)，就是在创建起容器时就同时自动创建了一个bean的对象，不管你是否使用，他 都存在了，每次获取到的对象都是同一个对象。注意，singleton 作用域是Spring中的缺省作用域。要 在XML中将 bean 定义成 singleton ，可以这样配置:

```
<bean id="ServiceImpl" class="cn.csdn.service.ServiceImpl" scope="singleton">
```

也可以通过 @Scope 注解(它可以显示指定bean的作用范围。)的方式

```
@Service
@Scope("singleton")
public class ServiceImpl{
}
```

prototyoe

> 每次请求都会创建一个新的 bean 实例

当一个bean的作用域为 prototype，表示一个 bean 定义对应多个对象实例。prototype 作用域的 bean 会导致在每次对该 bean 请求**(将其注入到另一个 bean 中，或者以程序的方式调用容器的 getBean() 方法**)时都会创建一个新的 bean 实例。prototype 是原型类型，它在我们创建容器的时候 并没有实例化，而是当我们获取bean的时候才会去创建一个对象，而且我们每次获取到的对象都不是 同一个对象。根据经验，对有状态的 bean 应该使用 prototype 作用域，而对无状态的 bean 则应该使 用 singleton 作用域。 在 XML 中将 bean 定义成 prototype ，可以这样配置:

```
<bean id="account" class="com.foo.DefaultAccount" scope="prototype"/>
或者
<bean id="account" class="com.foo.DefaultAccount" singleton="false"/>
```

request

> 每一次**HTTP请求都会产生一个新的bean，该bean仅在当前**HTTP request内有效

request只适用于Web程序，每一次HTTP 请求都会产生一个新的**bean，同时该bean仅在当前HTTP request内有效，当请求结束后，该对象的生命周期即告结束。** 在 XML 中将 bean 定义成 prototype ，可以这样配置:

session

> 每一次HTTP请求都会产生一个新的 bean，该bean 仅在当前 HTTP session 内有效

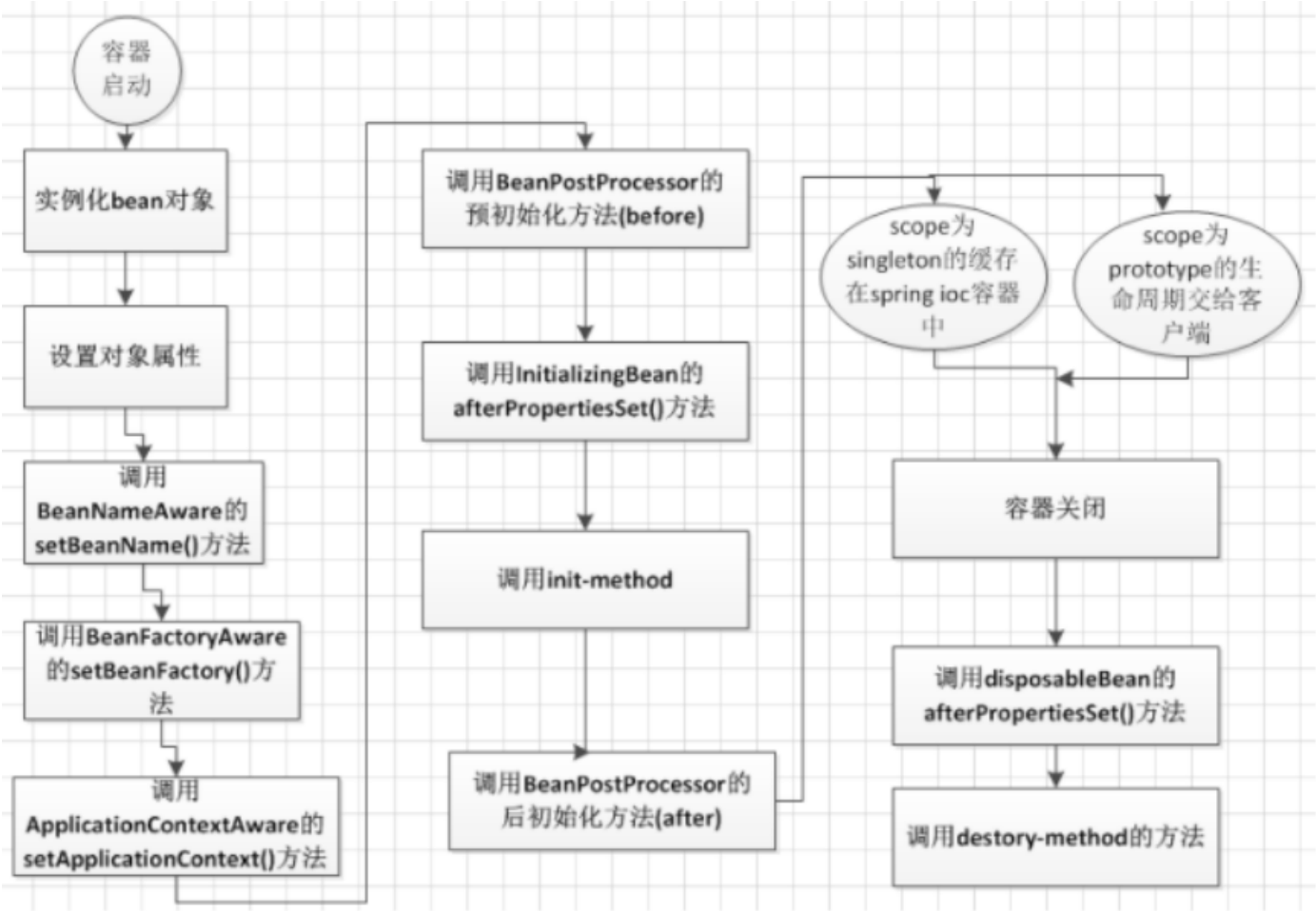
session只适用于Web程序，session 作用域表示该针对每一次 HTTP 请求都会产生一个新的 bean， 同时该 bean 仅在当前 HTTP session 内有效.与request作用域一样，可以根据需要放心的更改所创建 实例的内部状态，而别的 HTTP session 中根据 userPreferences 创建的实例，将不会看到这些特定 于某个 HTTP session 的状态变化。当 HTTP session最终被废弃的时候，在该HTTP session作用域内的bean也会被废弃掉。

globalSession

global session 作用域类似于标准的 HTTP session 作用域，不过仅仅在基于 portlet 的 web 应用中才 有意义。Portlet 规范定义了全局 Session 的概念，它被所有构成某个 portlet web 应用的各种不同的 portle t所共享。在 global session 作用域中定义的 bean 被限定于全局portlet Session的生命周期范围 内。

Bean的生命周期

ApplicationContext Bean生命周期



ApplicationContext容器中，Bean的生命周期流程如上图所示，流程大致如下:

1.首先容器启动后，会对scope为singleton且非懒加载的bean进行实例化，

2.按照Bean定义信息配置信息，注入所有的属性，

3.如果Bean实现了BeanNameAware接口，会回调该接口的setBeanName()方法，传入该Bean的id， 此时该Bean就获得了自己在配置文件中的id，

4.如果Bean实现了BeanFactoryAware接口,会回调该接口的setBeanFactory()方法，传入该Bean的BeanFactory，这样该Bean就获得了自己所在的BeanFactory，

5.如果Bean实现了ApplicationContextAware接口,会回调该接口的setApplicationContext()方法，传入 该Bean的ApplicationContext，这样该Bean就获得了自己所在的ApplicationContext，

6.如果有Bean实现了BeanPostProcessor接口，则会回调该接口的postProcessBeforeInitialzation()方 法，

7.如果Bean实现了InitializingBean接口，则会回调该接口的afterPropertiesSet()方法， 8.如果Bean配置了init-method方法，则会执行init-method配置的方法，

9.如果有Bean实现了BeanPostProcessor接口，则会回调该接口的postProcessAfterInitialization()方 法，

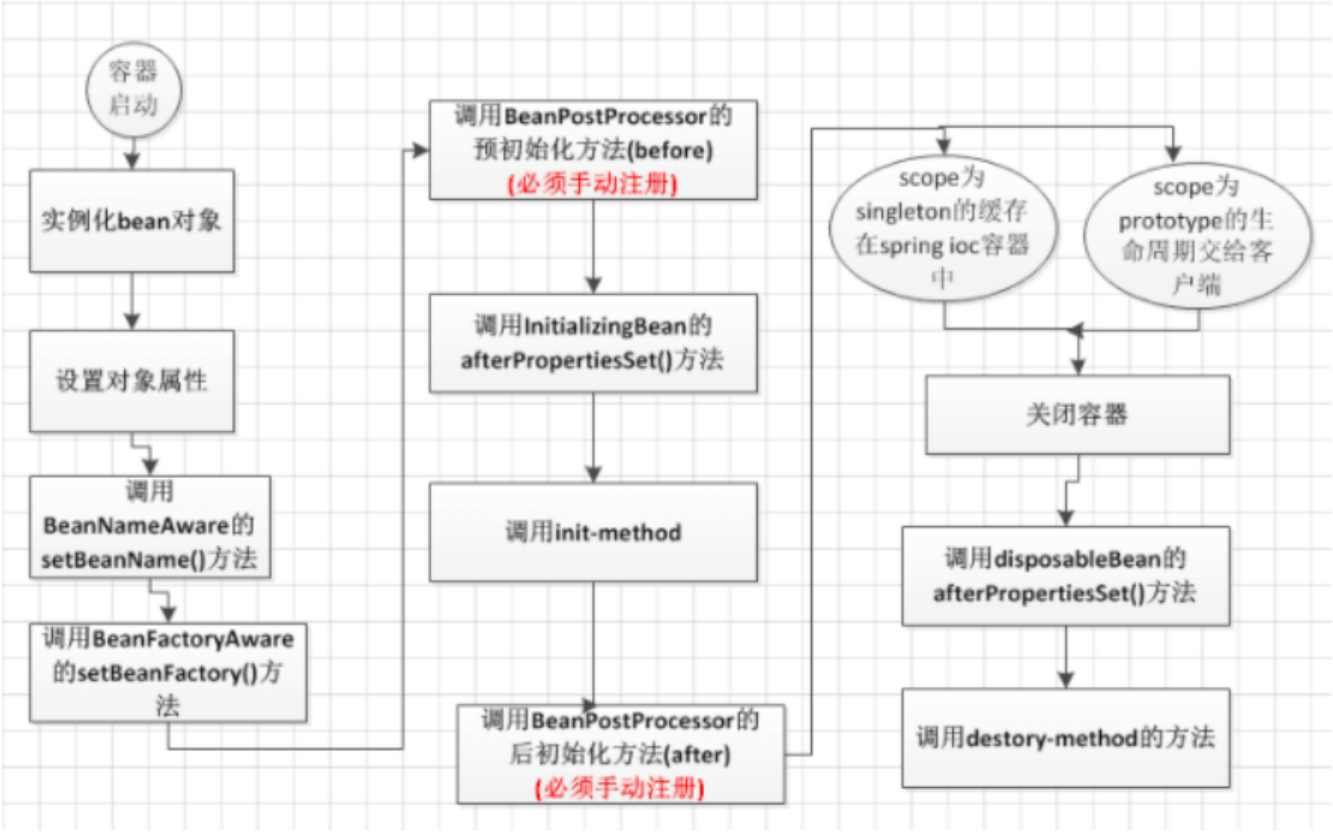
10.经过流程9之后，就可以正式使用该Bean了,对于scope为singleton的Bean,Spring的ioc容器中会缓 存一份该bean的实例，而对于scope为prototype的Bean,每次被调用都会new一个新的对象，期生命周 期就交给调用方管理了，不再是Spring容器进行管理了

11.容器关闭后，如果Bean实现了DisposableBean接口，则会回调该接口的destroy()方法， 12.如果Bean配置了destroy-method方法，则会执行destroy-method配置的方法，至此，整个Bean的

生命周期结束

BeanFactory Bean生命周期

ApplicationContext会利用Java反射机制自动识别出配置文件中定义的BeanPostProcesser,InstantiationAwareBeanPostProcessor和BeanFactoryPostProcessor，并自动 将它们注册到应用上下文中。而BeanFactory需要手动addBeanPostProcessor()去进行注册。



init-method 属性指定一个方法，在实例化 bean 时，立即调用该方法。同样，destroy-method 指定 一个方法，只有从容器中移除 bean 之后，才能调用该方法。

@Autowired和@Resource注解有什么区别?

1. @Autowired是Spring提供的注解，@Resource是JDK提供的注解。
2. @Autowired是只能按类型注入，@Resource默认按名称注入，也支持按类型注入。
3. @Autowired按类型装配依赖对象，默认情况下它要求依赖对象必须存在，如果允许null值，可以设置它required属性为false，如果我们想使用按名称装配，可以结合@Qualifier注解一起使用。@Resource有两个中重要的属性：name和type。name属性指定byName，如果没有指定name属性，当注解标注在字段上，即默认取字段的名称作为bean名称寻找依赖对象，当注解标注在属性的setter方法上，即默认取属性名作为bean名称寻找依赖对象。需要注意的是，@Resource如果没有指定name属性，并且按照默认的名称仍然找不到依赖对象时， @Resource注解会回退到按类型装配。但一旦指定了name属性，就只能按名称装配了。

说说你知道的关于BeanFactory和FactoryBean的区别

1、区别

BeanFactory是Spring容器的顶级接口，给具体的IOC容器的实现提供了规范。

FactoryBean也是接口，为IOC容器中Bean的实现提供了更加灵活的方式，FactoryBean在IOC容器的基 础上给Bean的实现加上了一个简单工厂模式和装饰模式(如果想了解装饰模式参考:修饰者模式(装饰者 模式，Decoration)我们可以在getObject()方法中灵活配置。其实在Spring源码中有很多FactoryBean 的实现类.

区别:BeanFactory是个Factory，也就是IOC容器或对象工厂，FactoryBean是个Bean。在Spring中， 所有的**Bean**都是由**BeanFactory(**也就是**IOC**容器**)**来进行管理的。

但对**FactoryBean**而言，这个**Bean**不是简单的**Bean**，而是一个能生产或者修饰对象生成的工厂**Bean,** 它的实现与设计模式中的工厂模式和修饰器模式类似。

2、BeanFactory

BeanFactory，以Factory结尾，表示它是一个工厂类(接口)， 它负责生产和管理bean的一个工厂 。在 **Spring** 中，**BeanFactory**是**IOC**容器的核心接口，它的职责包括:******实例化、定位、配置应用程序中的 对象及建立这些对象间的依赖。

BeanFactory只是个接口，并不是IOC容器的具体实现，但是Spring容器给出了很多种实现，如 DefaultListableBeanFactory、XmlBeanFactory、ApplicationContext等，其中XmlBeanFactory就是 常用的一个，该实现将以XML方式描述组成应用的对象及对象间的依赖关系。XmlBeanFactory类将持 有此XML配置元数据，并用它来构建一个完全可配置的系统或应用。

都是附加了某种功能的实现。它为其他具体的IOC容器提供了最基本的规范，例如 DefaultListableBeanFactory,XmlBeanFactory,ApplicationContext 等具体的容器都是实现了 BeanFactory，再在其基础之上附加了其他的功能。

BeanFactory和**ApplicationContext**就是**spring**框架的两个**IOC**容器，现在一般使用 **ApplicationnContext**，其不但包含了**BeanFactory**的作用，同时还进行更多的扩展。

BeanFacotry是spring中比较原始的Factory。如XMLBeanFactory就是一种典型的BeanFactory。 原始的BeanFactory无法支持spring的许多插件，如AOP功能、Web应用等。ApplicationContext接口,

它由BeanFactory接口派生而来， ApplicationContext包含BeanFactory的所有功能，通常建议比BeanFactory优先

ApplicationContext以一种更向面向框架的方式工作以及对上下文进行分层和实现继承， ApplicationContext包还提供了以下的功能:

MessageSource, 提供国际化的消息访问
资源访问，如URL和文件
事件传播
载入多个(有继承关系)上下文 ，使得每一个上下文都专注于一个特定的层次，比如应用的web 层;

BeanFactory提供的方法及其简单，仅提供了六种方法供客户调用:

boolean containsBean(String beanName) 判断工厂中是否包含给定名称的bean定义，若有 则返回true
Object getBean(String) 返回给定名称注册的bean实例。根据bean的配置情况，如果是 singleton模式将返回一个共享实例，否则将返回一个新建的实例，如果没有找到指定bean,该方法 可能会抛出异常

Object getBean(String, Class) 返回以给定名称注册的bean实例，并转换为给定class类型
Class getType(String name) 返回给定名称的bean的Class,如果没有找到指定的bean实例，则排 除NoSuchBeanDefinitionException异常
boolean isSingleton(String) 判断给定名称的bean定义是否为单例模式
String[] getAliases(String name) 返回给定bean名称的所有别名

3、FactoryBean

一般情况下，**Spring**通过反射机制利用 <bean> 的**class**属性指定实现类实例化**Bean**，在某些情况下， 实例化**Bean**过程比较复杂，如果按照传统的方式，则需要在 <bean> 中提供大量的配置信息。配置方 式的灵活性是受限的，这时采用编码的方式可能会得到一个简单的方案。

Spring为此提供了一个org.springframework.bean.factory.FactoryBean的工厂类接口，用户可以通过 实现该接口定制实例化Bean的逻辑。FactoryBean接口对于Spring框架来说占用重要的地位，Spring自 身就提供了70多个FactoryBean的实现。它们隐藏了实例化一些复杂**Bean**的细节，给上层应用带来了 便利。从Spring3.0开始，FactoryBean开始支持泛型，即接口声明改为 FactoryBean<T> 的形式

以Bean结尾，表示它是一个Bean，不同于普通Bean的是:它是实现了 FactoryBean<T> 接口的 Bean，根据该Bean的ID从BeanFactory中获取的实际上是FactoryBean的getObject()返回的对象，而 不是FactoryBean本身，如果要获取FactoryBean对象，请在id前面加一个&符号来获取。

例如自己实现一个FactoryBean，功能:用来代理一个对象，对该对象的所有方法做一个拦截，在调用 前后都输出一行LOG，模仿ProxyFactoryBean的功能。

FactoryBean是一个接口，当在IOC容器中的Bean实现了FactoryBean后，通过getBean(String BeanName)获取到的Bean对象并不是FactoryBean的实现类对象，而是这个实现类中的getObject()方 法返回的对象。要想获取FactoryBean的实现类，就要getBean(&BeanName)，在BeanName之前加上 &。

```
package org.springframework.beans.factory;
public interface FactoryBean<T> {
    T getObject() throws Exception;
    Class<?> getObjectType();
    boolean isSingleton();
}
```

在该接口中还定义了以下3个方法:

- **TgetObject()**:返回由FactoryBean创建的Bean实例，如果isSingleton()返回true，则该实例会放 到Spring容器中单实例缓存池中;
- **booleanisSingleton()**:返回由FactoryBean创建的Bean实例的作用域是singleton还是 prototype;
- **ClassgetObjectType()**:返回FactoryBean创建的Bean类型。

当配置文件中 <bean> 的class属性配置的实现类是FactoryBean时，通过getBean()方法返回的不是 FactoryBean本身，而是FactoryBean#getObject()方法所返回的对象，相当于 FactoryBean#getObject()代理了getBean()方法。

3. 总结

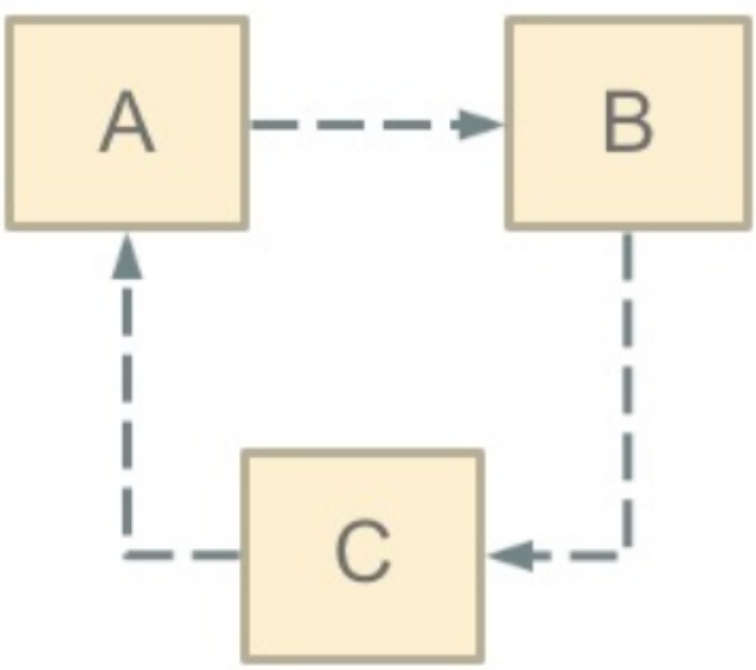
BeanFactory是个Factory，也就是IOC容器或对象工厂，FactoryBean是个Bean。在Spring中，所有的 Bean都是由BeanFactory(也就是IOC容器)来进行管理的。但对FactoryBean而言，这个Bean不是简单 的Bean，而是一个能生产或者修饰对象生成的工厂Bean,它的实现与设计模式中的工厂模式和修饰器模 式类似

Spring循环依赖及解决方式

1. 什么是循环依赖

循环依赖其实就是循环引用，也就是两个或者两个以上的bean互相持有对方，最终形成闭环。比如A依

赖于B，B依赖于C，C又依赖于A。如下图:



注意，这里不是函数的循环调用，是对象的相互依赖关系。循环调用其实就是一个死循环，除非有终结条件。

Spring中循环依赖场景有:

(1)构造器的循环依赖

(2)field属性的循环依赖

其中，构造器的循环依赖问题无法解决，只能抛出BeanCurrentlyInCreationException异常，在解决属 性循环依赖时，spring采用的是提前暴露对象的方法。

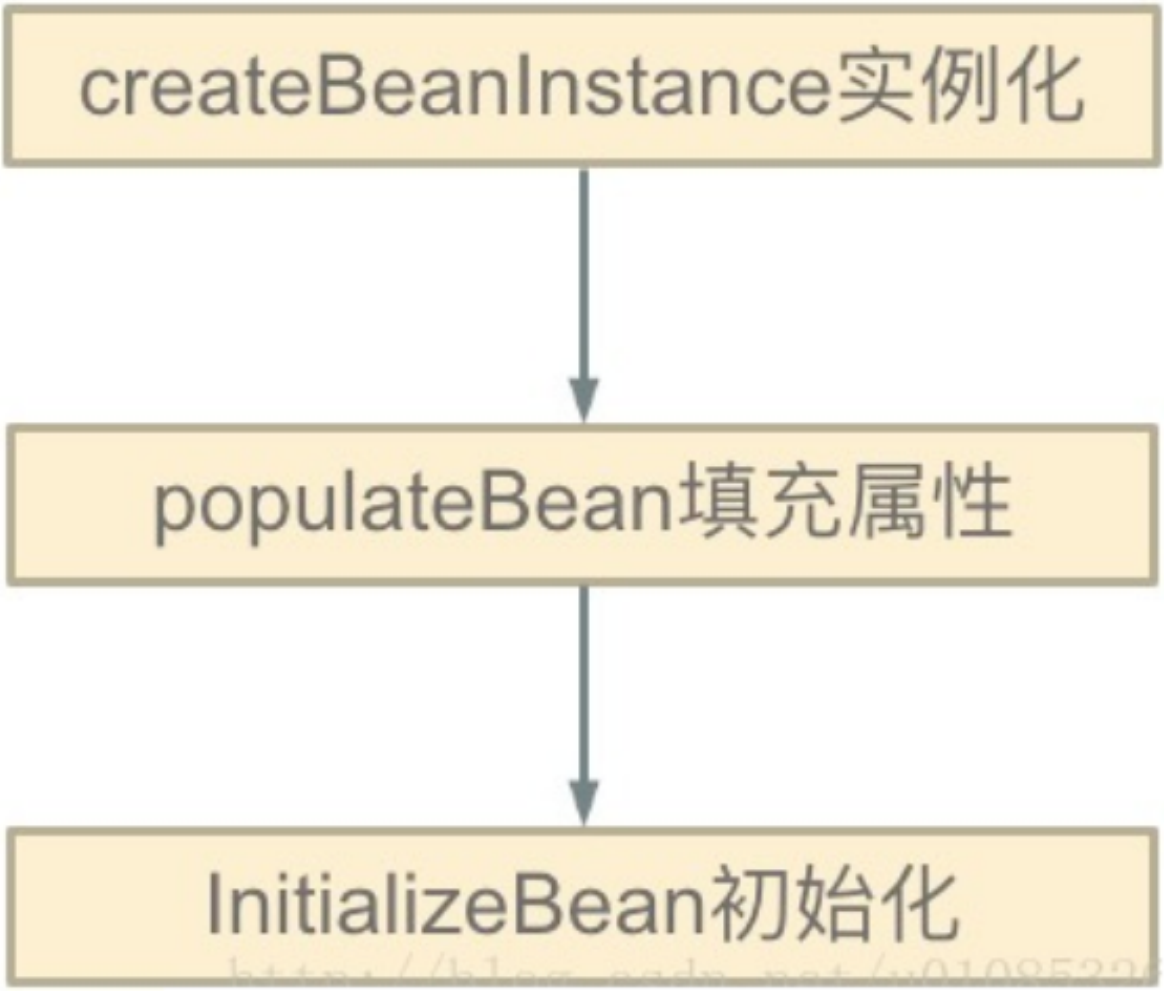
2. 怎么检测是否存在循环依赖

检测循环依赖相对比较容易，Bean在创建的时候可以给该Bean打标，如果递归调用回来发现正在创建中的话，即说明了循环依赖了。

3. 如何解决

Spring的循环依赖的理论依据基于Java的引用传递，当获得对象的引用时，对象的属性是可以延后设置 的。(但是构造器必须是在获取引用之前)

Spring的单例对象的初始化主要分为三步:



(1)createBeanInstance:实例化，其实也就是调用对象的构造方法实例化对象

(2)populateBean:填充属性，这一步主要是多bean的依赖属性进行填充

(3)initializeBean:调用spring xml中的init 方法。

从上面单例bean的初始化可以知道:循环依赖主要发生在第一、二步，也就是构造器循环依赖和field 循环依赖。那么我们要解决循环引用也应该从初始化过程着手，对于单例来说，在Spring容器整个生命 周期内，有且只有一个对象，所以很容易想到这个对象应该存在Cache中，Spring为了解决单例的循环 依赖问题，使用了**三级缓存**。

这三级缓存分别指:
singletonFactories : 单例对象工厂的cache

earlySingletonObjects :提前曝光的单例对象的Cache

singletonObjects:单例对象的cache

在创建bean的时候，首先想到的是从cache中获取这个单例的bean，这个缓存就是singletonObjects。 如果获取不到，并且对象正在创建中，就再从二级缓存earlySingletonObjects中获取。如果还是获取不 到且允许singletonFactories通过getObject()获取，就从三级缓存singletonFactory.getObject()(三级缓 存)获取，如果获取到了则:从singletonFactories中移除，并放入earlySingletonObjects中。其实也就 是从三级缓存移动到了二级缓存。

从上面三级缓存的分析，我们可以知道，Spring解决循环依赖的诀窍就在于singletonFactories这个三 级cache。这个cache的类型是ObjectFactory。这里就是解决循环依赖的关键，发生在 createBeanInstance之后，也就是说单例对象此时已经被创建出来(调用了构造器)。这个对象已经被生 产出来了，虽然还不完美(还没有进行初始化的第二步和第三步)，但是已经能被人认出来了(根据对 象引用能定位到堆中的对象)，所以Spring此时将这个对象提前曝光出来让大家认识，让大家使用。

这样做有什么好处呢?让我们来分析一下“A的某个field或者setter依赖了B的实例对象，同时B的某个 field或者setter依赖了A的实例对象”这种循环依赖的情况。A首先完成了初始化的第一步，并且将自己 提前曝光到singletonFactories中，此时进行初始化的第二步，发现自己依赖对象B，此时就尝试去 get(B)，发现B还没有被create，所以走create流程，B在初始化第一步的时候发现自己依赖了对象A，

于是尝试get(A)，尝试一级缓存singletonObjects(肯定没有，因为A还没初始化完全)，尝试二级缓存earlySingletonObjects(也没有)，尝试三级缓存singletonFactories，由于A通过ObjectFactory将自 己提前曝光了，所以B能够通过ObjectFactory.getObject拿到A对象(虽然A还没有初始化完全，但是总 比没有好呀)，B拿到A对象后顺利完成了初始化阶段1、2、3，完全初始化之后将自己放入到一级缓存 singletonObjects中。此时返回A中，A此时能拿到B的对象顺利完成自己的初始化阶段2、3，最终A也 完成了初始化，进去了一级缓存singletonObjects中，而且更加幸运的是，由于B拿到了A的对象引用， 所以B现在hold住的A对象完成了初始化。

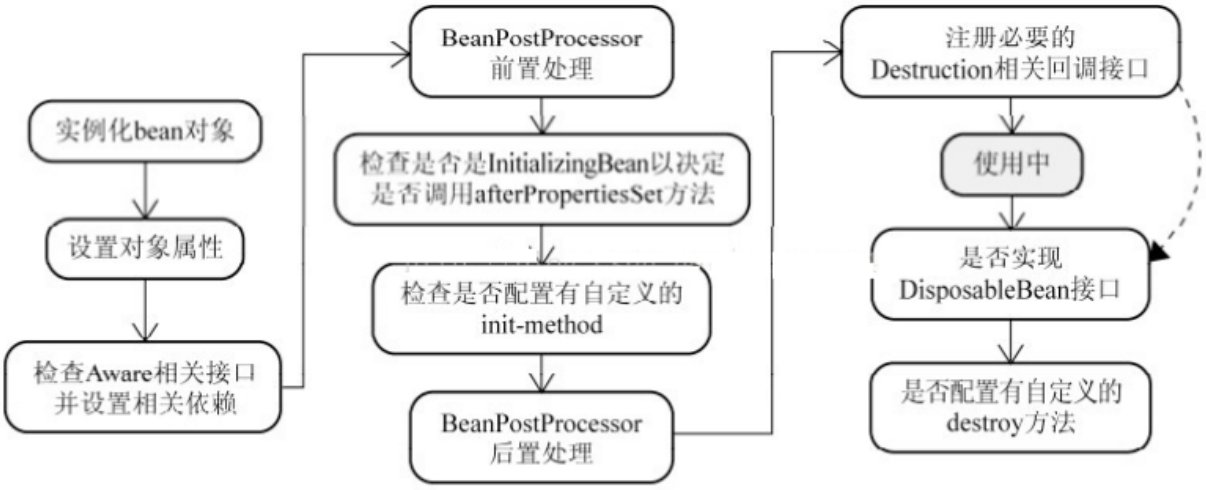
知道了这个原理时候，肯定就知道为啥Spring不能解决“A的构造方法中依赖了B的实例对象，同时B的构 造方法中依赖了A的实例对象”这类问题了!因为加入singletonFactories三级缓存的前提是执行了构造 器，所以构造器的循环依赖没法解决。

4. 基于构造器的循环依赖

Spring容器会将每一个正在创建的Bean 标识符放在一个“当前创建Bean池”中，Bean标识符在创建过程 中将一直保持在这个池中，因此如果在创建Bean过程中发现自己已经在“当前创建Bean池”里时将抛出BeanCurrentlyInCreationException异常表示循环依赖;而对于创建完毕的Bean将从“当前创建Bean 池”中清除掉。

Spring容器先创建单例A，A依赖B，然后将A放在“当前创建Bean池”中，此时创建B,B依赖C ,然后将B放 在“当前创建Bean池”中,此时创建C，C又依赖A， 但是，此时A已经在池中，所以会报错，，因为在池中 的Bean都是未初始化完的，所以会依赖错误，(初始化完的Bean会从池中移除)

5. 基于setter属性的循环依赖



我们结合上面那张图看，Spring先是用构造实例化Bean对象 ，创建成功后，Spring会通过以下代码提 前将对象暴露出来，此时的对象A还没有完成属性注入，属于早期对象，此时Spring会将这个实例化结 束的对象放到一个Map中，并且Spring提供了获取这个未设置属性的实例化对象引用的方法。 结合我 们的实例来看，当Spring实例化了A、B、C后，紧接着会去设置对象的属性，此时A依赖B，就会去Map 中取出存在里面的单例B对象，以此类推，不

会出来循环的问题喽

Spring可以配置多例对象吗

在Spring中，bean可以被定义为两种模式:prototype(多例)和singleton(单例)。所谓单例，就 是所有的请求都由一个对象来进行处理，比如我们常用的service和dao层的对象通常都是单例的，而多 例则指每个请求用一个新的对象来处理。

- singleton单例):只有一个共享的实例存在，所有对这个bean的请求都会返回这个唯一的实 例。不管new多少次，只生成一个对象。
- prototype(多例):对这个bean的每次请求都会创建一个新的bean实例，类似于new。

什么时候用单例？什么时候用多例?

当对象含有可改变的状态时(更精确的说就是在实际应用中该状态会改变)，则多例，否则单例。就 如dao和service这两层的数据一般不会有相应的属性修改，所以可以考虑用单例，而Controller层会存 储很多需要操作的vo类(value object值对象)，此时这个对象的状态就会被改变，就要使用多例。

如何配置多例

- 单例模式:每个bean定义只生成一个对象实例,每次getBean请求获得的都是此实例
- 单例模式分为饿汉模式和懒汉模式

饿汉模式	spring singleton的缺省是饿汉模式:启动容器时(即实例化容器时), 为所有spring配置文件中定义的bean都生成一个实例
懒汉模式	在第一个请求时才生成一个实例,以后的请求都调用这个实例 spring singleton设置为懒汉模式:

spring bean 默认是单例默认，在对应.xml文件中的配置是:

singleton就是配置这个bean是否是单例的，如果不写，就是默认值true。

```
<bean id="user" class="..." scope="singleton"/>
```

- 多例模式:任何一个实例都是新的实例，调用getBean时,就new一个新实例

```
<bean id="user" class="..." scope="prototype"/>
```

Spring中用到了哪些设计模式

- 工厂设计模式 : Spring使用工厂模式通过 BeanFactory、ApplicationContext 创建 bean 对象。
- 代理设计模式 : Spring AOP 功能的实现。
- 单例设计模式 : Spring 中的 Bean 默认都是单例的。
- 模板方法模式 : Spring 中 jdbcTemplate、hibernateTemplate 等以 Template 结尾的对数据库操 作的类，它们就使用到了模板模式。
- 包装器设计模式 : 我们的项目需要连接多个数据库，而且不同的客户在每次访问中根据需要会去访 问不同的数据库。这种模式让我们可以根据客户的需求能够动态切换不同的数据源。

- 观察者模式: Spring 事件驱动模型就是观察者模式很经典的一个应用。
- 适配器模式 :Spring AOP 的增强或通知(Advice)使用到了适配器模式、spring MVC 中也是用到了 适配器模式适配Controller。

Spring事务的传播机制和隔离级别

Spring事务 事务是逻辑处理原子性的保证手段，通过使用事务控制，可以极大的避免出现逻辑处理失败导致的脏数据等问题。
事务最重要的两个特性，是事务的传播级别和数据隔离级别。传播级别定义的是事务的控制范围，事务隔离级别定义的是事务在数据库读写方面的控制范围。

事务传播机制

1)**PROPAGATION_REQUIRED** ，Spring默认的事务传播级别，使用该级别的特点是，如果上下文中 已经存在事务，那么就加入到事务中执行，如果当前上下文中不存在事务，则新建事务执行。所以这个 级别通常能满足处理大多数的业务场景。

2)**PROPAGATION_SUPPORTS** ，从字面意思就知道，supports，支持，该传播级别的特点是，如果 上下文存在事务，则支持事务加入事务，如果没有事务，则使用非事务的方式执行。所以说，并非所有 的包在transactionTemplate.execute中的代码都会有事务支持。这个通常是用来处理那些并非原子性 的非核心业务逻辑操作。应用场景较少。

3)**PROPAGATION_MANDATORY** ， 该级别的事务要求上下文中必须要存在事务，否则就会抛出异 常!配置该方式的传播级别是有效的控制上下文调用代码遗漏添加事务控制的保证手段。比如一段代码 不能单独被调用执行，但是一旦被调用，就必须有事务包含的情况，就可以使用这个传播级别。

4)**PROPAGATION_REQUIRES_NEW** ，从字面即可知道，new，每次都要一个新事务，该传播级别的 特点是，每次都会新建一个事务，并且同时将上下文中的事务挂起，执行当前新建事务完成以后，上下 文事务恢复再执行。

这是一个很有用的传播级别，举一个应用场景:现在有一个发送100个红包的操作，在发送之前，要做 一些系统的初始化、验证、数据记录操作，然后发送100封红包，然后再记录发送日志，发送日志要求 100%的准确，如果日志不准确，那么整个父事务逻辑需要回滚。 怎么处理整个业务需求呢?就是通过这个PROPAGATION_REQUIRES_NEW 级别的事务传播控制就可以 完成。发送红包的子事务不会直接影响到父事务的提交和回滚。

5)**PROPAGATION_NOT_SUPPORTED** ，这个也可以从字面得知，not supported ，不支持，当前级 别的特点就是上下文中存在事务，则挂起事务，执行当前逻辑，结束后恢复上下文的事务。

这个级别有什么好处?可以帮助你将事务极可能的缩小。我们知道一个事务越大，它存在的风险也就越 多。所以在处理事务的过程中，要保证尽可能的缩小范围。比如一段代码，是每次逻辑操作都必须调用 的，比如循环1000次的某个非核心业务逻辑操作。这样的代码如果包在事务中，势必造成事务太大，导 致出现一些难以考虑周全的异常情况。所以这个事务这个级别的传播级别就派上用场了。用当前级别的 事务模板抱起来就可以了。

6)**PROPAGATION_NEVER** ，该事务更严格，上面一个事务传播级别只是不支持而已，有事务就挂 起，而PROPAGATION_NEVER传播级别要求上下文中不能存在事务，一旦有事务，就抛出runtime异 常，强制停止执行!这个级别上辈子跟事务有仇。

7)**PROPAGATION_NESTED** ，字面也可知道，nested，嵌套级别事务。该传播级别特征是，如果上 下文中存在事务，则嵌套事务执行，如果不存在事务，则新建事务

事务隔离级别

1、**Serializable** :最严格的级别，事务串行执行，资源消耗最大;

2、**REPEATABLE READ** :保证了一个事务不会修改已经由另一个事务读取但未提交(回滚)的数据。 避免了“脏读取”和“不可重复读取”的情况，但是带来了更多的性能损失。

3、**READ COMMITTED** :大多数主流数据库的默认事务等级，保证了一个事务不会读到另一个并行事务 已修改但未提交的数据，避免了“脏读取”。该级别适用于大多数系统。

4、**Read Uncommitted** :保证了读取过程中不会读取到非法数据。 上面的解释其实每个定义都有一些拗口，其中涉及到几个术语:脏读、不可重复读、幻读。

这里解释一下:
脏读 :所谓的脏读，其实就是读到了别的事务回滚前的脏数据。比如事务B执行过程中修改了数据X，在

未提交前，事务A读取了X，而事务B却回滚了，这样事务A就形成了脏读。

不可重复读 :不可重复读字面含义已经很明了了，比如事务A首先读取了一条数据，然后执行逻辑的时 候，事务B将这条数据改变了，然后事务A再次读取的时候，发现数据不匹配了，就是所谓的不可重复读 了。

幻读 :小的时候数手指，第一次数十10个，第二次数是11个，怎么回事?产生幻觉了? 幻读也是这样子，事务A首先根据条件索引得到10条数据，然后事务B改变了数据库一条数据，导致也 符合事务A当时的搜索条件，这样事务A再次搜索发现有11条数据了，就产生了幻读。

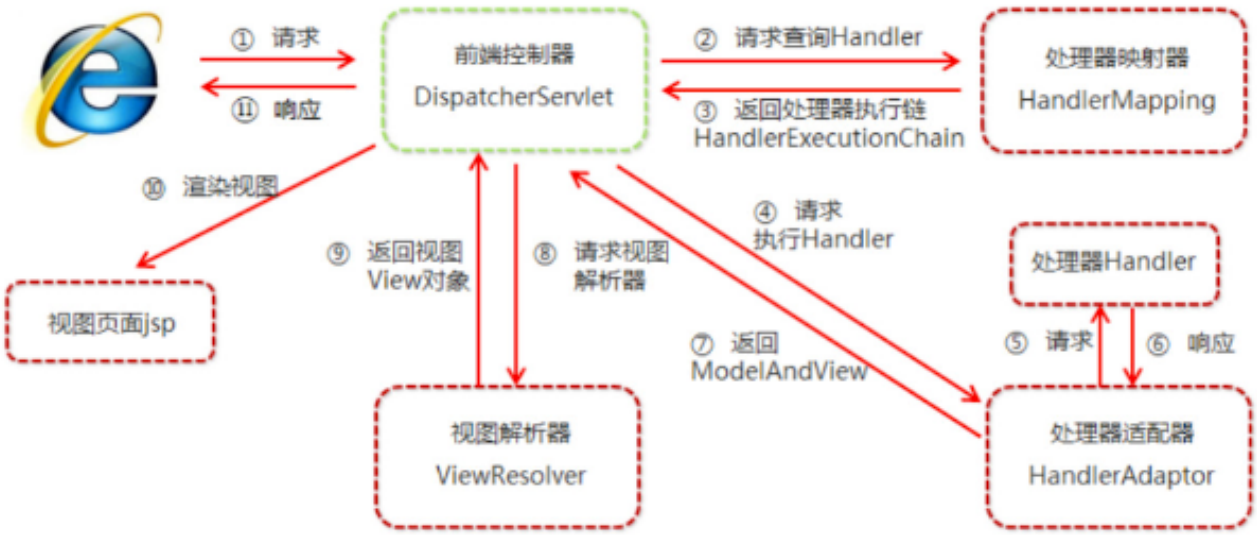
分布式事务

在分布式的高并发环境下，对于核心业务逻辑的检查，要采用加锁机制。 比如事务开启需要读取一条数据进行验证，然后逻辑操作中需要对这条数据进行修改，最后提交。 这样的一个过程，如果读取并验证的代码放到事务之外，那么读取的数据极有可能已经被其他的事务修 改，当前事务一旦提交，又会重新覆盖掉其他事务的数据，导致数据异常。 所以在进入当前事务的时候，必须要将这条数据锁住，使用for update就是一个很好的在分布式环境下的控制手段。

一种好的实践方式是使用编程式事务而非生命式，尤其是在较为规模的项目中。对于事务的配置，在代 码量非常大的情况下，将是一种折磨，而且人肉的方式，绝对不能避免这种问题。 将DAO保持针对一张表的最基本操作，然后业务逻辑的处理放入manager和service中进行，同时使用 编程式事务更精确的控制事务范围。 特别注意的，对于事务内部一些可能抛出异常的情况，捕获要谨慎，不能随便的catch Exception 导致 事务的异常被吃掉而不能正常回滚。

Spring MVC

描述一下SpringMVC的执行流程



SpringMVC流程

```
1.用户发送请求至前端控制器DispatcherServlet。
2.DispatcherServlet收到请求调用HandlerMapping处理器映射器。
3.处理器映射器找到具体的处理器(可以根据xml配置、注解进行查找)，生成处理器对象及处理器拦截 器(如果有则生成)一并返回给DispatcherServlet。
4.DispatcherServlet调用HandlerAdapter处理器适配器。
5.HandlerAdapter经过适配调用具体的处理器(Controller，也叫后端控制器)。
6.Controller执行完成返回ModelAndView。
7.HandlerAdapter将controller执行结果ModelAndView返回给DispatcherServlet。
8.DispatcherServlet将ModelAndView传给ViewReslover视图解析器。
9.ViewReslover解析后返回具体View。
10.DispatcherServlet根据View进行渲染视图(即将模型数据填充至视图中)
11.DispatcherServlet响应用户。
```

SpringMVC中常用的注解有哪些，作用是什么

1. `@Controller`

@Controller注解在类上，表明这个类是Spring MVC里的Controller，将其声明为Spring的一个 Bean，Dispatch Servlet会自动扫描注解了此注解的类，并将Web请求映射到注解了 @RequestMapping的方法上，需要注意的是，在Spring MVC声明控制器Bean的时候，只能使用 @Controller。

2. `@RequestMapping`

@RequestMapping注解是用来映射Web请求(访问路径和参数)、处理类和方法的。它可以注解 在类和方法上。注解在方法上的@RequestMapping路径会继承注解在类上的路径， @RequestMapping支持Servlet的request和response作为参数，也支持对它们的媒体类型进行配 置。

```
@Controller //声明此类是一个控制器
@RequestMapping("/anno") //映射此类的访问路径是 /anno
public class DemoAnnoController {}
```

3. `@ResponseBody`

@ResponseBody支持将返回值放在response体内，而不是返回一个页面。Ajax的异步访问程序，可以以此注解返回的是数据而不是返回页面;此注解可以放在返回值或者方法上。

```
@RequestMapping("/index")
@ResponseBody
public String index(HttpServletRequest request, HttpServletResponse
response){
    return "url: " + request.getRequestURL() +" can access";
}
```

4. `@RequestBody`

@RequestBody允许request的参数在request体中，而不是在直接链接在地址后面。此注解放在

参数前。

@RequestBody的作用其实是将json格式的数据转为java对象

```
//前端
$.ajax({
  data:{"username":"tom","address":"北京"}
})
@RequestMapping("/insert")
public void insert(@RequestBody User user){}
```

5. `@PathVariable`

@PathVariable 用来接收路径参数，如/news/001,可接收001作为参数，此注解放置在参数前。

```
@RequestMapping("/show/{id}/{name}")
public String show(@PathVariable("id") Long ids,@PathVariable("name") String
names){
    return "hello";
}
```

6. `@RestController`

@RestController是一个组合注解，组合了@Controller和@ResponseBody，意味着当只开发一个 和页面交互数据的控制的时候，需要使用此注解。 若没有此注解，要想实现上述功能，则需自己 在代码中加@Controller和@ResponseBody两个注解。

SpringMVC中的组件有哪些

1. 前端控制器:`DispatcherServlet`

用户请求到达前端控制器，它就相当于 MVC 模式中的 C，DispatcherServlet 是整个流程控制的

中心，由它调用其它组件处理用户的请求，DispatcherServlet 的存在降低了组件之间的耦合性。

2. 处理器映射器:`HandlerMapping`

HandlerMapping 负责根据用户请求找到 Handler 即处理器，SpringMVC 提供了不同的映射器实现不同的映射方式，例如:配置文件方式，实现接口方式，注解方式等。

3. 处理器适配器: `HandlerAdapter`

通过 HandlerAdapter 对处理器进行执行，这是适配器模式的应用，通过扩展适配器可以对更多类型

的处理器进行执行。

4. 处理器:`Handler` **【开发者编写】**

它就是我们开发中要编写的具体业务控制器。由 DispatcherServlet 把用户请求转发到Handler。由Handler 对具体的用户请求进行处理。

5. 视图解析器: `ViewResolver`

View Resolver 负责将处理结果生成 View 视图，View Resolver 首先根据逻辑视图名解析成物理视图名，即具体的页面地址，再生成 View 视图对象，最后对 View 进行渲染将处理结果通过页面展示给 用户。

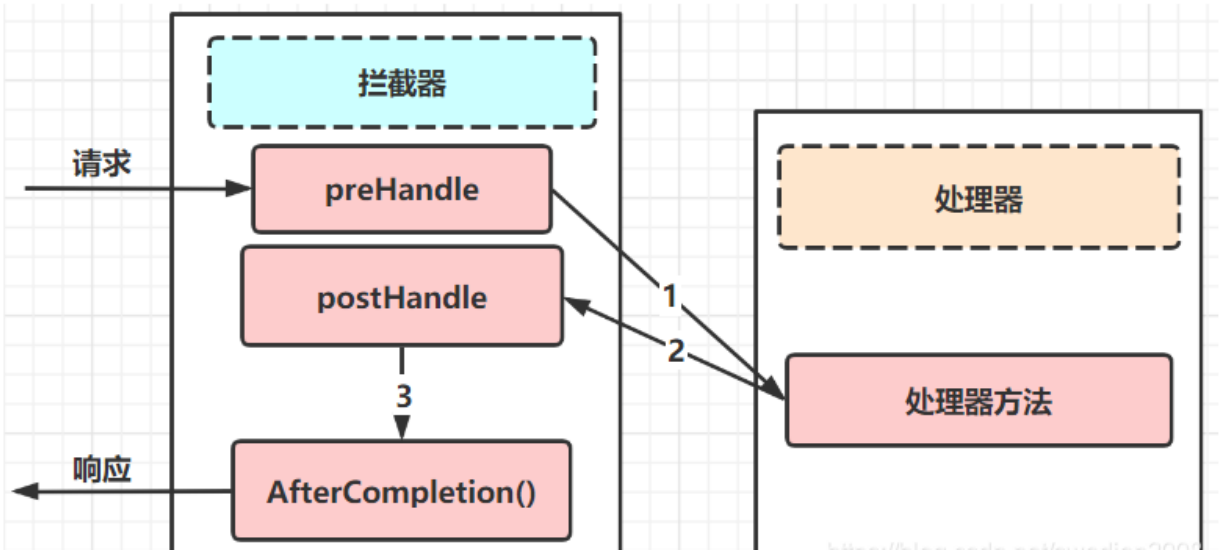
6. 视图:`View` **【开发者编写】**

SpringMVC 框架提供了很多的 View 视图类型的支持，包括:jstlView、freemarkerView、pdfView等。最常用的视图就是 jsp。一般情况下需要通过页面标签或页面模版技术将模型数据通过页面展 示给用户，需要由程序员根据业务需求开发具体的页面。

什么是SpringMVC 拦截器

拦截器会对处理器进行拦截，这样通过拦截器就可以增强处理器的功能。Spring MVC中，所有的拦截器都需要实现HandlerInterceptor接口，该接口包含如下三个方法：preHandle()、postHandle()、afterCompletion()。

通过上图可以看出，Spring MVC拦截器的执行流程如下：



- 执行preHandle方法，它会返回一个布尔值。如果为false，则结束所有流程，如果为true，则执行下一步。
- 执行处理器逻辑，它包含控制器的功能。
- 执行postHandle方法。
- 执行视图解析和视图渲染。

- 执行afterCompletion方法。

Spring MVC拦截器的开发步骤如下：

1. 开发拦截器：

   实现handlerInterceptor接口，从三个方法中选择合适的方法，实现拦截时要执行的具体业务逻辑。

2. 注册拦截器：

   定义配置类，并让它实现WebMvcConfigurer接口，在接口的addInterceptors方法中，注册拦截器，并定义该拦截器匹配哪些请求路径。

MyBatis

JDBC编程有哪些不足之处，MyBatis是如何解决这些问题的?

1. 数据库连接创建、释放频繁造成系统资源浪费从而影响系统性能，如果使用数据库连接池可解决此 问题。

解决:在`mybatis-config.xml`中配置数据连接池，使用连接池管理数据库连接。

2. Sql语句写在代码中造成代码不易维护，实际应用sql变化的可能较大，sql变动需要改变java代码。

解决:将Sql语句配置在`XXXXmapper.xml`文件中与java代码分离。

3. 向sql语句传参数麻烦，因为sql语句的where条件不一定，可能多也可能少，占位符需要和参数一

一对应。

解决:Mybatis自动将java对象映射至sql语句。

4. 对结果集解析麻烦，sql变化导致解析代码变化，且解析前需要遍历，如果能将数据库记录封装成

pojo对象解析比较方便。

解决:Mybatis自动将sql执行结果映射至java对象。

说一说你对Mybatis Plugin的了解

> 可以讲出Mybatis插件的原理，第二要讲出它所涉及的JDK动态代理和责任链设计模式

Mybatis作为一个应用广泛的优秀的ORM框架，已经成了JavaWeb世界近乎标配的部分，这个框架具有 强大的灵活性，在四大组件(Executor、StatementHandler、ParameterHandler、 ResultSetHandler)处提供了简单易用的插件扩展机制。Mybatis对持久层的操作就是借助于四大核心 对象。MyBatis支持用插件对四大核心对象进行拦截，对mybatis来说插件就是拦截器，用来增强核心 对象的功能，增强功能本质上是借助于底层的动态代理实现的，换句话说，MyBatis中的四大对象都是 代理对象。

1. 四大核心对象

ParameterHandler:处理SQL的参数对象

ResultSetHandler:处理SQL的返回结果集

StatementHandler:数据库的处理对象，用于执行SQL语句

Executor:MyBatis的执行器，用于执行增删改查操作

2. 插件原理

- Mybatis的插件借助于JDK动态代理和责任链设计模式进行对拦截的处理
- 使用动态代理对目标对象进行包装，达到拦截的目的
- 作用于Mybatis的作用域对象之上

3. 插件接口

- Intercept方法，插件的核心方法
- plugin方法，生成target的代理对象
- setProperties方法，传递插件所需参数

Mybatis中Dao接口和XML文件的SQL如何建立关联

1. Mybatis会把每个SQL标签封装成SqlSource对象,XML文件中的每一个SQL标签就对应一个**MappedStatement**对象，这里面有两个属性很重要。

- id:全限定类名+方法名组成的ID。
- sqlSource:当前SQL标签对应的SqlSource对象。

2. Dao接口的工作原理是JDK动态代理，Mybatis运行时会使用JDK动态代理为Dao接口生成代理proxy对象，代理对象proxy会拦截接口方法，转而执行`MappedStatement`所代表的sql，然后将 sql执行结果返回。

Mapper接口是没有实现类的，当调用接口方法时，接口全限名+方法名拼接字符串作为key 值，可唯一定位一个`MappedStatement`

```
举例:com.mybatis3.mappers.StudentDao.findStudentById,
找到 namespace为 com.mybatis3.mappers.StudentDao下面的 id = findStudentById的
MappedStatement。
```

Mybatis是如何将sql执行结果封装为目标对象并返回的?都有哪些映射形式?

1. 第一种是使用标签，逐一定义列名和对象属性名之间的映射关系

```
<resultMap type="com.pojo.User" id="myResult">
    <id property="uid" column="uid"/>
    <result property="uname" column="uname"/>
</resultMap>
<select id="selectUser" resultMap="myResult">
    select
  * from user
</select>
```

2. 第二种是使用sql列的别名功能，将列别名书写为对象属性名

```
<select id="selectUser" resultType="com.pojo.User">
    select
    uid AS id,
            uname AS name
        from user
</select>
```

3. resultmap是手动提交，人为提交，resulttype是自动提交

MyBatis中在查询进行select映射的时候，返回类型可以用resultType，也可以用resultMap， resultType是直接表示返回类型的，而resultMap则是对外部ResultMap的引用，但是resultType跟 resultMap不能同时存在

Mybatis动态sql是做什么的?都有哪些动态sql?

Mybatis动态sql可以让我们在Xml映射文件内，以标签的形式编写动态sql，完成逻辑判断和动态 拼接sql的功能

- if

```
<if test="name!=null and name!=''">
   e.emp_name=#{name,jdbcType=VARCHAR}
</if>
```

- where：只要将sql放入where动态标签内，至少有一个条件符合的时候，才会插入where语句并且会将条件语句前的 and 去掉。

```
<where>
    <if test="name!=null and name!=''">
     and  e.emp_name=#{name,jdbcType=VARCHAR}
    </if>
    <if test="dep!=null">
     and e.emp_dep=#{dep.id,jdbcType=INTEGER}
    </if>
</where>
```

- set: set 元素可以用于动态包含需要更新的列，而舍去其它的

```
<update id="updateAuthorIfNecessary">
  update Author
    <set>
      <if test="username != null">username=#{username},</if>
      <if test="password != null">password=#{password},</if>
</set>
  where id=#{id}
</update>
```

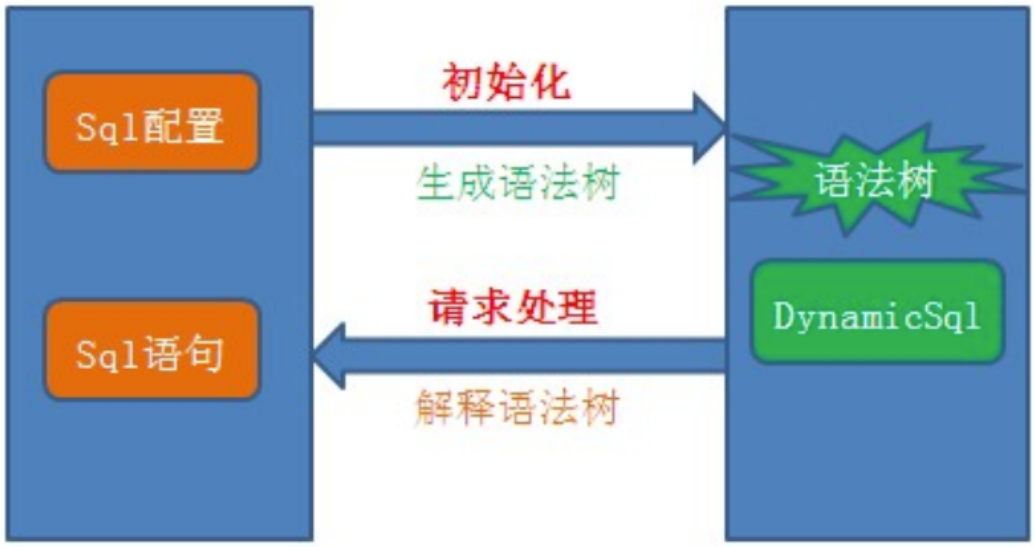
- foreach

```
<select id="countByUserList" parameterType="list">
  SELECT COUNT(*) FROM users
  WHERE user_id IN
  <foreach item="userList" collection="list" separator="," open="("
close=")" index="">
    #{item.userId}
  </foreach>
</select>
```

从设计上看，dynamic sql的实现主要涉及三个模式:
解释器模式: 初始化过程中构建出抽象语法树，请求处理时根据参数对象解释语法树，生成sql语句。

工厂模式: 为动态标签的处理方式创建工厂类(SqlTagHandlerFactory)，根据标签名称获取对应的 处理方式。
策略模式: 将动态标签处理方式抽象为接口，针对不同标签有相应的实现类。解释抽象语法树时， 定义统一的解释流程，再调用标签对应的处理方式完成解释中的各个子环节。

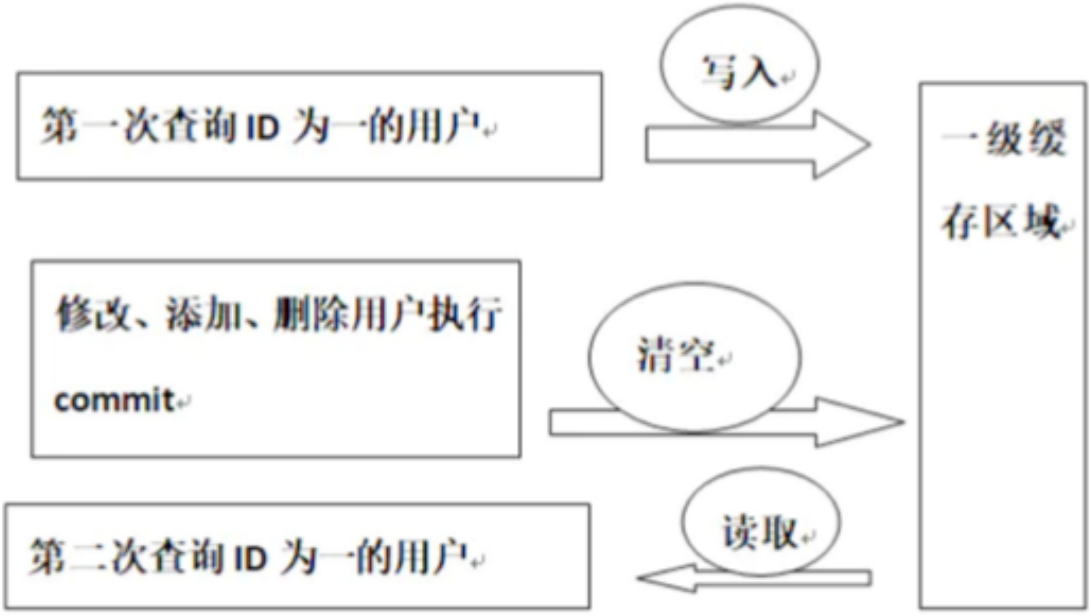
最后，以一张图总结动态sql的实现原理:



Mybatis中一级缓存和二级缓存的区别

- 一级缓存 是 SqlSession级别的缓存。在操作数据库时需要构造sqlSession对象，在对象中有一个数据结 构用于存储缓存数据。

  不同的sqlSession之间的缓存数据区域是互相不影响的。也就是他只能作用在同一个sqlSession中，不 同的sqlSession中的缓存是互相不能读取的。



用户发起查询请求，查找某条数据，sqlSession 先去缓存中查找，是否有该数据，如果有，读取; 如果没有，从数据库中查询，并将查询到的数据放入一级缓存区域，供下次查找使用。

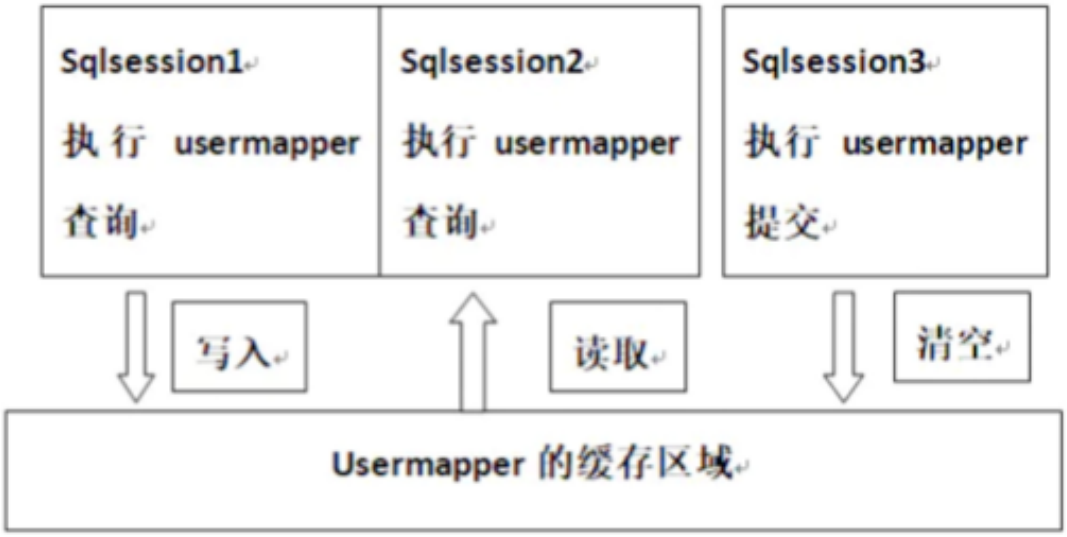
但 sqlSession 执行commit，即增删改操作时会清空缓存。这么做的目的是避免脏读。

既然有了一级缓存，那么为什么要提供二级缓存呢?

1. 二级缓存是 mapper 级别的缓存，多个SqlSession去操作同一个Mapper的sql语句，多个 SqlSession可以共用二级缓存，二级缓存是跨SqlSession的。二级缓存的作用范围更大。
2. 还有一个原因，实际开发中，MyBatis 通常和 Spring 进行整合开发。

   Spring 将事务放到 Service 中管理，对于每一个 service 中的 sqlsession 是不同的，
   这是通过 mybatis-spring 中的 org.mybatis.spring.mapper.MapperScannerConfigurer 创建 sqlsession 自动注入到 service 中的。 每次查询之后都要进行关闭 sqlSession ，关闭之后数据被清空。 所以 spring 整合之后，如果没有事务，**一级缓存** 是没有意义的。

```
<span style="font-size:18px;"><settings> <!--开启二级缓存-->
    <setting name="cacheEnabled" value="true"/>
</settings> </span>
```



二级缓存是 mapper 级别的缓存，多个 SqlSession 去操作同一个Mapper的sql语句，多个 SqlSession 可以共用二级缓存，二级缓存是跨 SqlSession 的。

UserMapper有一个二级缓存区域(按 namespace 划分)，每个 mapper 也有自己的二级缓存区域 (按namespace 分)。

每一个 namespace 的 mapper 都有一个二级缓存区域，如果相同两个 mapper 的 namespace ，这两 个mapper 执行sql查询到数据将存在相同的二级缓存区域中。

MyBatis中接口绑定有几种实现方式,是怎么实现的?

- 通过注解绑定，在接口的方法上面加上 @Select@Update等注解里面包含Sql语句来绑定(Sql语句比较简单的时候，推荐注解绑定)
- 通过xml里面写SQL来绑定, 指定xml映射文件里面的namespace必须为接口的全路径名(SQL语句比较复杂的时候，推荐xml绑定)

MyBatis和Hibernate有哪些不同

- Mybatis和hibernate不同，它不完全是一个ORM框架，因为MyBatis需要程序员自己编写Sql语 句。
- Mybatis直接编写原生态sql，可以严格控制sql执行性能，灵活度高，非常适合对关系数据模型要 求不高的软件开发，因为这类软件需求变化频繁，一但需求变化要求迅速输出成果。但是灵活的前 提是mybatis无法做到数据库无关性，如果需要实现支持多种数据库的软件，则需要自定义多套 sql映射文件，工作量大。
- Hibernate对象/关系映射能力强，数据库无关性好，对于关系模型要求高的软件，如果用 hibernate开发可以节省很多代码，提高效率。

MyBatis初始化过程

在 MyBatis 初始化过程中，会加载 mybatis-config.xml 配置文件、 Mapper.xml 映射配置文件以及Mapper 接口中的注解信息，解析后的配置信息会形成相应的对象并保存到 Configuration 对象中。

初始化过程可以分成三部分:

- 解析mybatis-config.xml 配置文件
  - SqlSessionFactoryBuilder
  - XMLConfigBuilder

    - Configuration
- 解析 Mapper.xml 映射配置文件
    - XMLMapperBuilder::parse()
    - XMLStatementBuilder::parseStatementNode ()
    - XMLLanguageDriver
    - SqlSource
    - MappedStatement
- 解析Mapper接口中的注解
    - MapperRegistry
    - MapperAnnotationBuilder::parse()

总结：在 MyBatis 初始化过程中，会加载 配置文件、 映射配置文件以及 Mapper 接口中的注解信息，解析后的配置信息会形成相应的对象并全部保存到 Configuration 对象 中，并创建 DefaultSqlSessionFactory 供SQl执行过程创建出顶层接口 SqlSession 供给用户进行操 作。

Mybatis延迟加载

1. 什么是延迟加载

MyBatis中的延迟加载，也称为懒加载，是指在进行表的关联查询时，按照设置延迟规则推迟对关联对 象的select查询。例如在进行一对多查询的时候，只查询出一方，当程序中需要多方的数据时， mybatis再发出sql语句进行查询，这样子延迟加载就可以的减少数据库压力。MyBatis 的延迟加载只是 对关联对象的查询有迟延设置，对于主加载对象都是直接执行查询语句的。

注意:延迟加载的应用要求:关联对象的查询与主加载对象的查询必须是分别进行的select语句，不能 是使用多表连接所进行的select查询

2. 加载时机

mybatis对于延迟加载的时机支持三种形式
直接加载:执行完对主加载对象的 select 语句，马上执行对关联对象的 select 查询。
侵入式延迟: 执行对主加载对象的查询时，不会执行对关联对象的查询。但当要访问主加载对象的详情 属性时，就会马上执行关联对象的select查询。
深度延迟: 执行对主加载对象的查询时，不会执行对关联对象的查询。访问主加载对象的详情时也不会 执行关联对象的select查询。只有当真正访问关联对象的详情时，才会执行对关联对象的 select 查询。

SpringBoot

Spring Boot Starter有什么用?

Spring Boot Starter的作用是简化和加速项目的配置和依赖管理。

- Spring Boot Starter可以理解为一种预配置的模块，它封装了特定功能的依赖项和配置, ，开发者只需引入相关的Starter依赖，无需手动配置大量的参数和依赖项。
- Starter还管理了相关功能的依赖项，包括其他Starter和第三方库，确保它们能够良好地协同工作，避免版本冲突和依赖问题。
- Spring Boot Starter的设计使得应用可以通过引入不同的Starter来实现模块化的开发。每个Starter都关注一个特定的功能领域，如数据库访问、消息队列、Web开发等。
- 开发者可以创建自定义的Starter，以便在项目中共享和重用特定功能的配置和依赖项。

SpringBoot的常用注解

SpringBoot常用注解大全

1. @SpringBootApplication:
用于标识主应用程序类，通常位于项目的顶级包中。这个注解包含了@Configuration、@EnableAutoConfiguration和@ComponentScan。

2. @Controller:
用于标识类作为Spring MVC的Controller。

3. @RestController:
类似于@Controller，但它是专门用于RESTful Web服务的。它包含了@Controller和@ResponseBody。

4. @RequestMapping:
用于将HTTP请求映射到Controller的处理方法。可以用在类级别和方法级别。

5. @Autowired:
用于自动注入Spring容器中的Bean，可以用在构造方法、字段、Setter方法上。

6. @Service:
用于标识类作为服务层的Bean。

7. @Repository:
用于标识类作为数据访问层的Bean，通常用于与数据库交互。

8. @Component:
通用的组件注解，用于标识任何Spring托管的Bean。

9. @Configuration:
用于定义配置类，类中可能包含一些@Bean注解用于定义Bean。

10. @EnableAutoConfiguration:
用于启用Spring Boot的自动配置机制，根据项目的依赖和配置自动配置Spring应用程序。

11. @Value:
用于从属性文件或配置中读取值，将值注入到成员变量中。

12.@Qualifier:
与@Autowired一起使用，指定注入时使用的Bean名称。

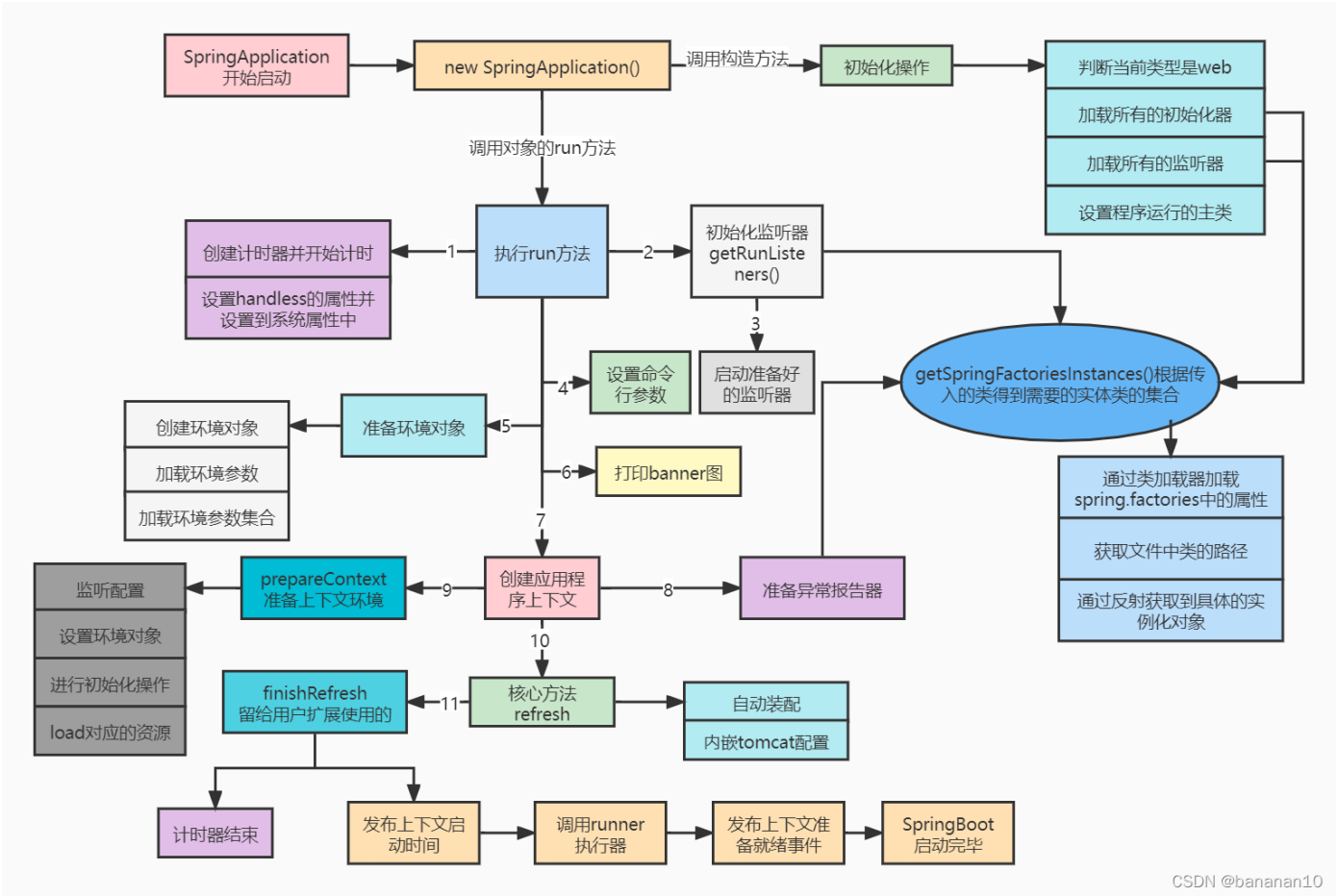
13. @ConfigurationProperties:
用于将配置文件中的属性映射到Java Bean。

14. @Profile:
用于定义不同环境下的配置，可以标识在类或方法上。

15. @Async:
用于将方法标记为异步执行。

SpringBoot的启动流程

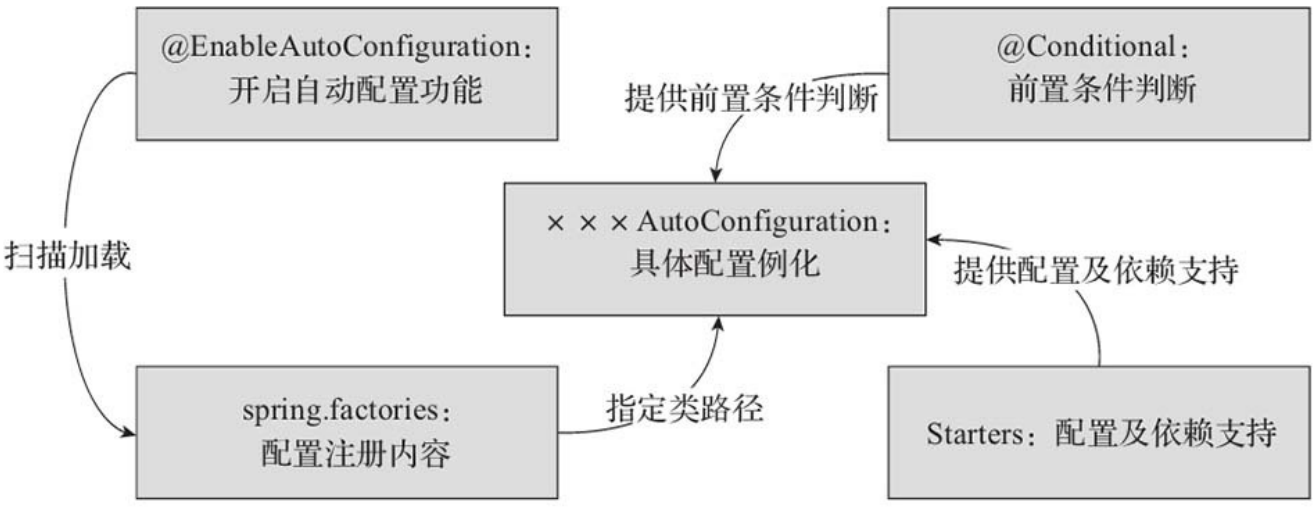
[SpringBoot 应用程序启动过程探秘]



CSDN @bananan10

Spring Boot自动装配的过程

使用Spring Boot时，我们只需引入对应的Starters，Spring Boot启动时便会自动加载相关依赖，配置相应的初始化参数，以最快捷、简单的形式对第三方软件进行集成，这便是Spring Boot的自动配置功能。Spring Boot实现该运作机制锁涉及的核心部分如下图所示：



整个自动装配的过程是：Spring Boot通过@EnableAutoConfiguration注解开启自动配置，加载spring.factories中注册的各种AutoConfiguration类，当某个AutoConfiguration类满足其注解@Conditional指定的生效条件（Starters提供的依赖、配置或Spring容器中是否存在某个Bean等）时，实例化该AutoConfiguration类中定义的Bean（组件等），并注入Spring容器，就可以完成依赖框架的自动配置。

SpringBoot如何在启动时设置初始化数据

在实际开发中，我们一般都会在配置文件(application.properties或者application.yml)中配置各个

项目中集成的属性值，来进行各个组件的设置，比如配置端口:server.port=8080，但是在开发中我们

需要配置业务得属性值来添加到springboot容器中，那么这个时候我们应该怎么办呢?

解决方案:

- **实现ApplicationRunner 接口**
- **实现 CommandLineRunner接口**

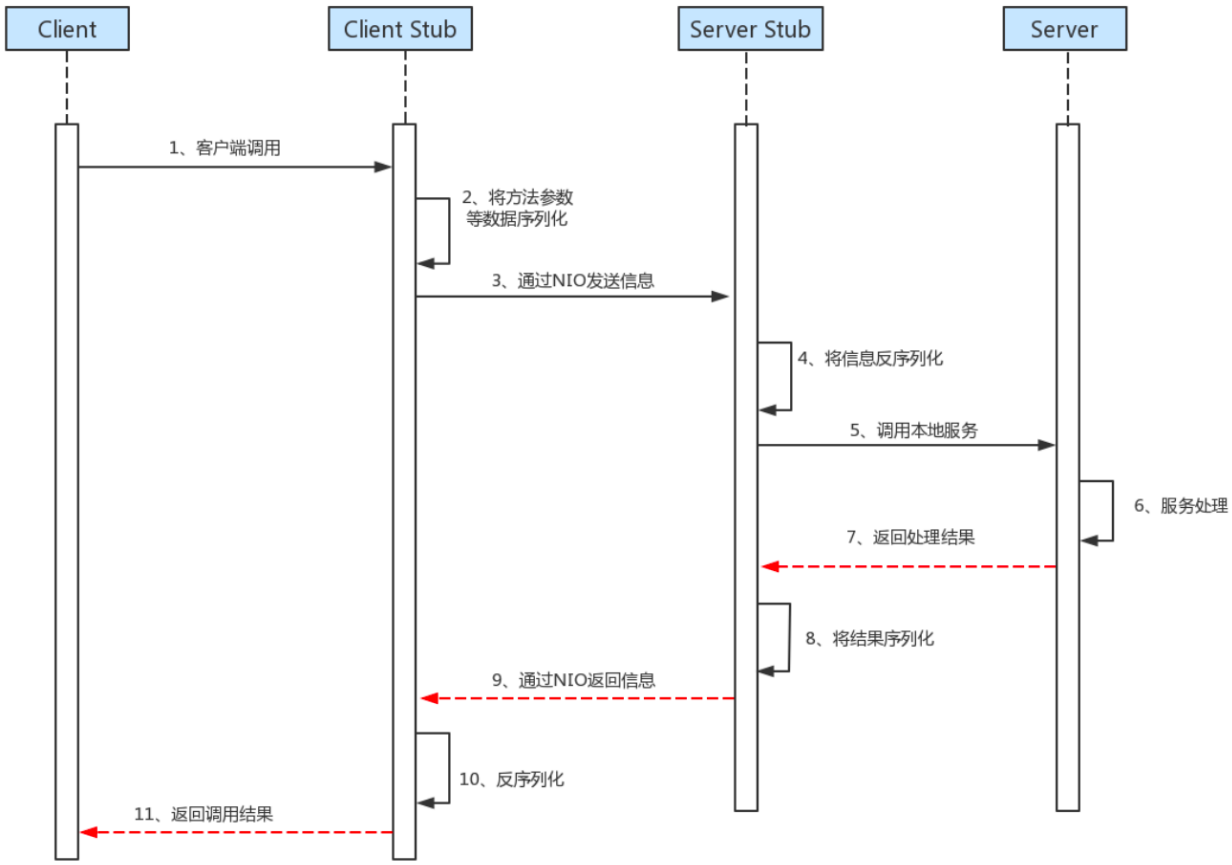
默认情况下ApplicationRunner比CommandLineRunner 先执行

所有CommandLineRunner/ApplicationRunner的执行时点是在SpringBoot应用的 ApplicationContext完全初始化开始工作之后， callRunners() 可以看出是run方法内部最后一 个调用的方法(可以认为是main方法执行完成之前最后一步)

只要存在于当前SpringBoot应用的ApplicationContext中的任何 CommandLineRunner/ApplicationRunner，都会被加载执行(不管你是手动注册还是自动扫描去 Ioc容器)
使用@Order注解可以设置加载的顺序
一般情况我们只需要使用一个接口来加载初始化数据即可

RPC框架

Dubbo的工作流程



具体调用过程:

1. 服务消费者(client客户端)通过调用本地服务的方式调用需要消费的服务;
2. 客户端存根(client stub)接收到调用请求后负责将方法、入参等信息序列化(组装)成能够进行 网络传输的消息体;
3. 客户端存根(client stub)找到远程的服务地址，并且将消息通过网络发送给服务端;
4. 服务端存根(server stub)收到消息后进行解码(反序列化操作);
5. 服务端存根(server stub)根据解码结果调用本地的服务进行相关处理;
6. 本地服务执行具体业务逻辑并将处理结果返回给服务端存根(server stub);
7. 服务端存根(server stub)将返回结果重新打包成消息(序列化)并通过网络发送至消费方;
8. 客户端存根(client stub)接收到消息，并进行解码(反序列化);
9. 服务消费方得到最终结果;
10. 反序列化
11. 返回调用结果

Dubbo支持哪些协议，每种协议的应用场景，优缺点?

- `dubbo`: 单一长连接和NIO异步通讯，适合大并发小数据量的服务调用，以及消费者远大于提供 者。传输协议TCP，异步，Hessian序列化;
- `rmi`: 采用JDK标准的rmi协议实现，传输参数和返回参数对象需要实现Serializable接口，使用 java标准序列化机制，使用阻塞式短连接，传输数据包大小混合，消费者和提供者个数差不多，可传文件，传输协议TCP。

  多个短连接，TCP协议传输，同步传输，适用常规的远程服务调用和rmi互操作。在依赖低版本的 Common-Collections包，java序列化存在安全漏洞;

- `webservice`：基于WebService的远程调用协议，集成CXF实现，提供和原生WebService的互操 作。多个短连接，基于HTTP传输，同步传输，适用系统集成和跨语言调用;
- `http`: 基于Http表单提交的远程调用协议，使用Spring的HttpInvoke实现。多个短连接，传输协 议HTTP，传入参数大小混合，提供者个数多于消费者，需要给应用程序和浏览器JS调用;
- `hessian`：集成Hessian服务，基于HTTP通讯，采用Servlet暴露服务，Dubbo内嵌Jetty作为服务 器时默认实现，提供与Hession服务互操作。多个短连接，同步HTTP传输，Hessian序列化，传入 参数较大，提供者大于消费者，提供者压力较大，可传文件;
- `memcache`: 基于memcached实现的RPC协议
- `redis`: 基于redis实现的RPC协议

RPC使用了那些关键技术

1、动态代理

生成Client Stub(客户端存根)和Server Stub(服务端存根)的时候需要用到Java动态代理技术，可以 使用JDK提供的原生的动态代理机制，也可以使用开源的:CGLib代理，Javassist字节码生成技术。

2序列化和反序列化

在网络中，所有的数据都将会被转化为字节进行传送，所以为了能够使参数对象在网络中进行传输，需

要对这些参数进行序列化和反序列化操作。 序列化:把对象转换为字节序列的过程称为对象的序列化，也就是编码的过程。 反序列化:把字节序列恢复为对象的过程称为对象的反序列化，也就是解码的过程。 目前比较高效的开源序列化框架:如Kryo、FastJson和Protobuf等。

3、NIO通信
出于并发性能的考虑，传统的阻塞式 IO 显然不太合适，因此我们需要异步的 IO，即 NIO。

Java 提供了 NIO 的解决方案，Java 7 也提供了更优秀的 NIO.2 支持。可以选择Netty或者MINA来解决 NIO数据传输的问题。

4、服务注册中心

可选:Redis、Zookeeper、Consul 、Etcd。

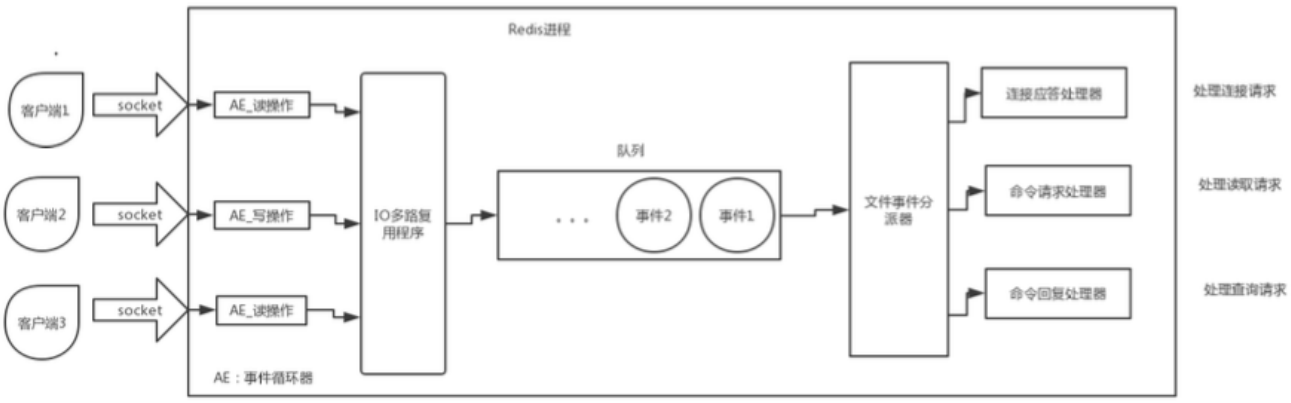
一般使用ZooKeeper提供服务注册与发现功能，解决单点故障以及分布式部署的问题(注册中心)。

分布式存储

redis是单线程的吗？那如何支持高并发

> 官方答案：因为Redis是基于内存的操作，CPU不是Redis的瓶颈，Redis的瓶颈最有可能是机器内存的 大小或者网络带宽。既然单线程容易实现，而且CPU不会成为瓶颈，那就顺理成章地采用单 线程的方案了。

redis 内部使用了一个叫 **文件事件处理器**(file event handler)的东西，这个文件 事件处理器是 **单线程**的，所以才有了 redis 是单线程的这一说法。它采用IO多路复用机制来同时 监听多个Socket，根据Socket上的事件类型来选择对应的事件处理器来处理这个事件。



1. Redis采用了单线程的模型，保证了每个操作的原子性，也避免了不必要的上下文切换和竞争条 件，也不存在多进程或者多线程导致的切换而消耗CPU，不用去考虑各种锁的问题，不存在加锁释 放锁操作，没有因为可能出现死锁而导致的性能消耗
2. 灵活多样的数据结构。redis内部使用一个redisObject对象来表示所有的key和value。redisObject 主要的信息包括数据类型、编码方式、数据指针、虚拟内存等。它包含String，Hash，List， Set，Sorted Set五种数据类型，针对不同的场景使用对应的数据类型，减少内存使用的同时，节 省网络流量传输。
3. Redis是纯内存数据库，一般都是简单的存取操作，线程占用的时间很多，时间的花费主要集中在 IO上，所以读取速度快。
4. 再说一下IO，Redis使用的是非阻塞IO多路复用，使用了单线程来轮询描述，将数据库的开、关、 读、写都转换成了事件，减少了线程切换时上下文的切换和竞争。 非阻塞IO多路复用:https://www.zhihu.com/question/32163005

总结:

起初当看到别人说 redis 是单线程时，很容易想成 redis 服务端只开启了一个线程用于做所有 事情，然而实际上我们所说的 **redis 单线程**只是针对 redis 网络请求模块，即上文提到的**文 件事件处理器**

缓存穿透、缓存雪崩、缓存击穿

缓存雪崩

在高并发下，大量的缓存key在**同一时间**失效，导致大量的请求落到数据库上，如活动系统里面同时进 行着非常多的活动，但是在某个时间点所有的活动缓存全部过期。
解决方案:

1. 缓存数据的过期时间设置随机，防止同一时间大量数据过期现象发生。
2. 如果缓存数据库是分布式部署，将热点数据均匀分布在不同搞得缓存数据库中。 3. 设置热点数据永远不过期。

缓存穿透

访问一个**不存在的key**(查询 userid = -10)，缓存不起作用，请求会穿透到DB，流量大时DB会挂掉 解决方案:

1. 接口层增加校验，如用户鉴权校验，id做基础校验，id<=0的直接拦截;
2. 从缓存取不到的数据，在数据库中也没有取到，这时也可以将key-value对写为key-null，缓存有效

   时间可以设置短点，如30秒(设置太长会导致正常情况也没法使用)。这样可以防止攻击用户反 复用同一个id暴力攻击

缓存击穿

一个存在的key，在缓存过期的那一刻，同时有大量的请求，这些请求都会击穿到DB，造成瞬时DB请 求量大、压力骤增
解决方案:

1. 设置热点数据永远不过期。
2. 加互斥锁:业界比较常用的做法。简单地来说，就是在缓存失效的时候(判断拿出来的值是否为

   空)，不是立即去加载数据库，而是先使用缓存工具的某些带成功操作返回值的操作(比如Redis 的SETNX)去set一个mutex key，当操作返回成功时，再进行加载数据库的操作并回设缓存;否 则，就重试整个get缓存的方法。

```
public String get(key) {
    String value = redis.get(key);
    if (value == null) { //代表缓存值过期
    //设置3分钟的超时，防止删除操作失败的时候，下次缓存过期不能一直加载数据库 if
(redis.setnx(key_mutex,  1,  3 * 60) == 1) { //代表设置成功
        value = db.get(key); // 去加载数据库
        redis.set(key,  value,  expire_secs); // 将数据库的数据放在redis中，并设置过期时间
        redis.del(key_mutex);
    } else { //这个时候代表同时候的其他线程已经load db并回设到缓存了，这时候重试获取缓存值即可
        sleep(50);
        get(key); //重试
    }
    } else {
       return value;
} }
```

消息中间件

MQ消息积压以及处理方案

这种时候只能操作临时扩容，以更快的速度去消费数据了

- 增加多个消费者，加速消费
- 新建topic引流，将消息引导别的程序中，洪水一个道理。

如何保证MQ中的消息不丢失

生产者丢失消息

1. 可以选择使用rabbitmq提供是事务功能，就是生产者在发送数据之前开启事务，然后发送消 息，如果消息没有成功被rabbitmq接收到，那么生产者会受到异常报错，这时就可以回滚事 务，然后尝试重新发送;如果收到了消息，那么就可以提交事务。

   缺点:rabbitmq事务已开启，就会变为同步阻塞操作，生产者会阻塞等待是否发送成功，太 耗性能会造成吞吐量的下降。

2. 可以开启confirm模式。在生产者哪里设置开启了confirm模式之后，每次写的消息都会分配 一个唯一的id，然后如何写入了rabbitmq之中，rabbitmq会给你回传一个ack消息，告诉你 这个消息发送OK了;如果rabbitmq没能处理这个消息，会回调你一个nack接口，告诉你这 个消息失败了，你可以进行重试。而且你可以结合这个机制知道自己在内存里维护每个消息 的id，如果超过一定时间还没接收到这个消息的回调，那么你可以进行重发

二者不同

事务机制是同步的，你提交了一个事务之后会阻塞住，但是confirm机制是异步的，发送消息 之后可以接着发送下一个消息，然后rabbitmq会回调告知成功与否。

一般在生产者这块避免丢失，都是用confirm机制。

rabbitmq自己弄丢了数据

- 设置消息持久化到磁盘。设置持久化有两个步骤:
    - 创建queue的时候将其设置为持久化的，这样就可以保证rabbitmq持久化queue的元数 据，但是不会持久化queue里面的数据。
    - 发送消息的时候讲消息的deliveryMode设置为2，这样消息就会被设为持久化方式，此时 rabbitmq就会将消息持久化到磁盘上。

      必须要同时开启这两个才可以。
- 而且持久化可以跟生产的confirm机制配合起来，只有消息持久化到了磁盘之后，才会通知生 产者ack，这样就算是在持久化之前rabbitmq挂了，数据丢了，生产者收不到ack回调也会进 行消息重发。

消费者弄丢了数据

使用rabbitmq提供的ack机制，首先关闭rabbitmq的自动ack，然后每次在确保处理完这个

消息之后，在代码里手动调用ack。这样就可以避免消息还没有处理完就ack

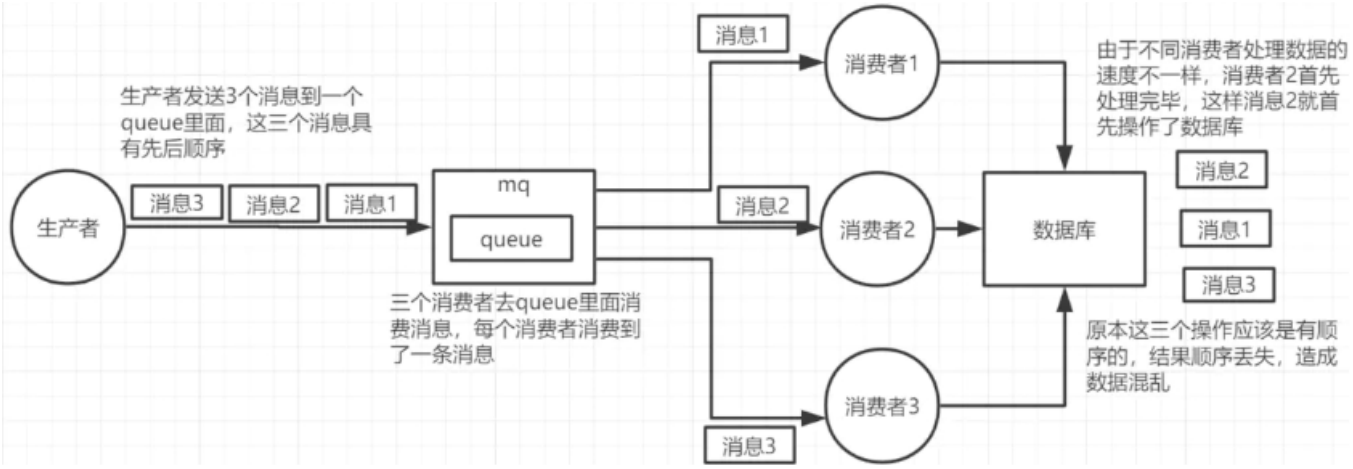
如何保证MQ中消息的顺序性

- **为什么要保证顺序**

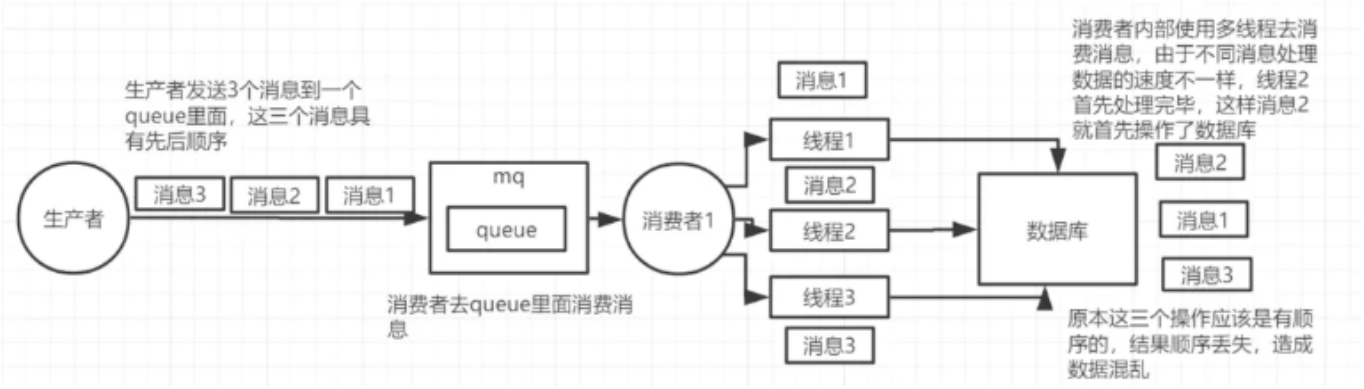
消息队列中的若干消息如果是对同一个数据进行操作，这些操作具有前后的关系，必须要按 前后的顺序执行，否则就会造成数据异常。
举例:比如通过mysql binlog进行两个数据库的数据同步，由于对数据库的数据操作是具有 顺序性的，如果操作顺序搞反，就会造成不可估量的错误。比如数据库对一条数据依次进行 了 插入->更新->删除操作，这个顺序必须是这样，如果在同步过程中，消息的顺序变成了 删 除->插入->更新，那么原本应该被删除的数据，就没有被删除，造成数据的不一致问题。

- 出现顺序错乱的场景

1. 一个queue => 多个consumer去消费，这样就会造成顺序的错误，consumer从MQ里面读取 数据是有序的，但是每个consumer的执行时间是不固定的，无法保证先读到消息的 consumer一定先完成操作，这样就会出现消息并没有按照顺序执行，造成数据顺序错误。

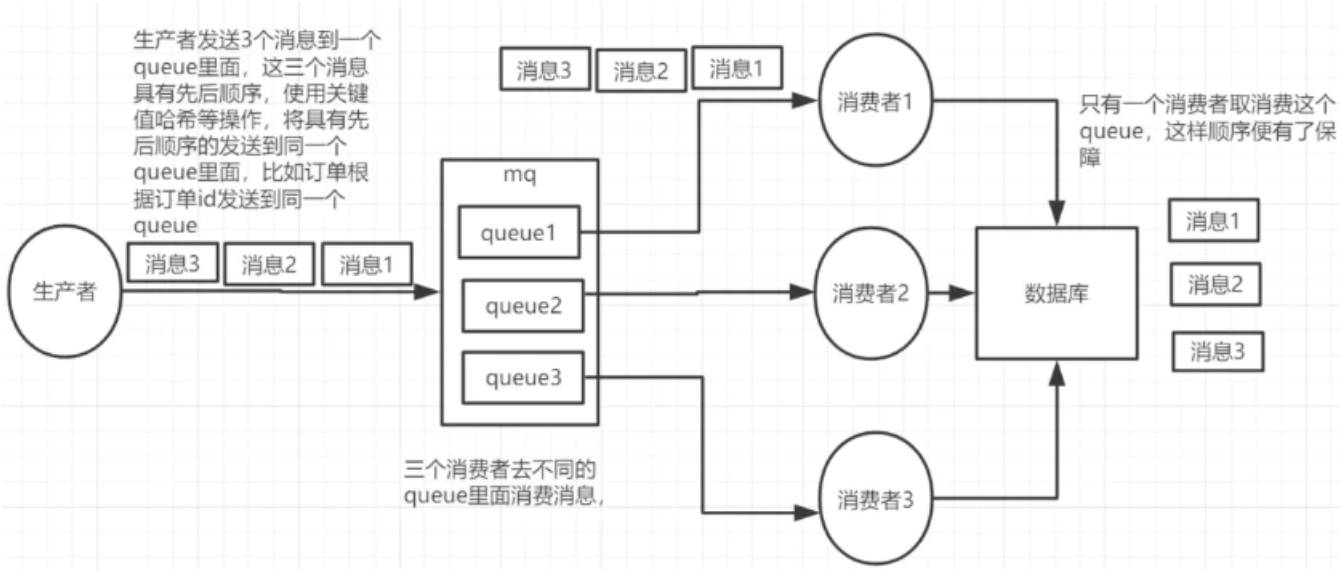


2. 一个queue => 一个consumer，但是consumer里面进行了多线程消费，这样也会造成消息 消费顺序错误。

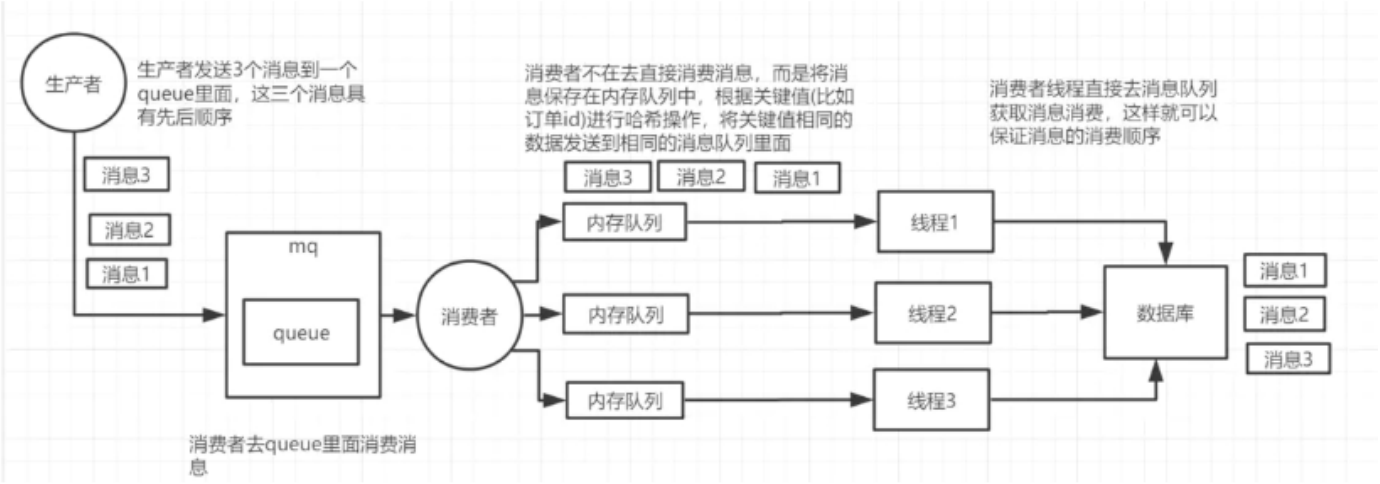


- **保证消息的消费顺序**

1. 拆分多个queue，每个queue一个consumer，就是多一些queue而已，确实是麻烦点;这样 也会造成吞吐量下降，可以在消费者内部采用多线程的方式去消费。



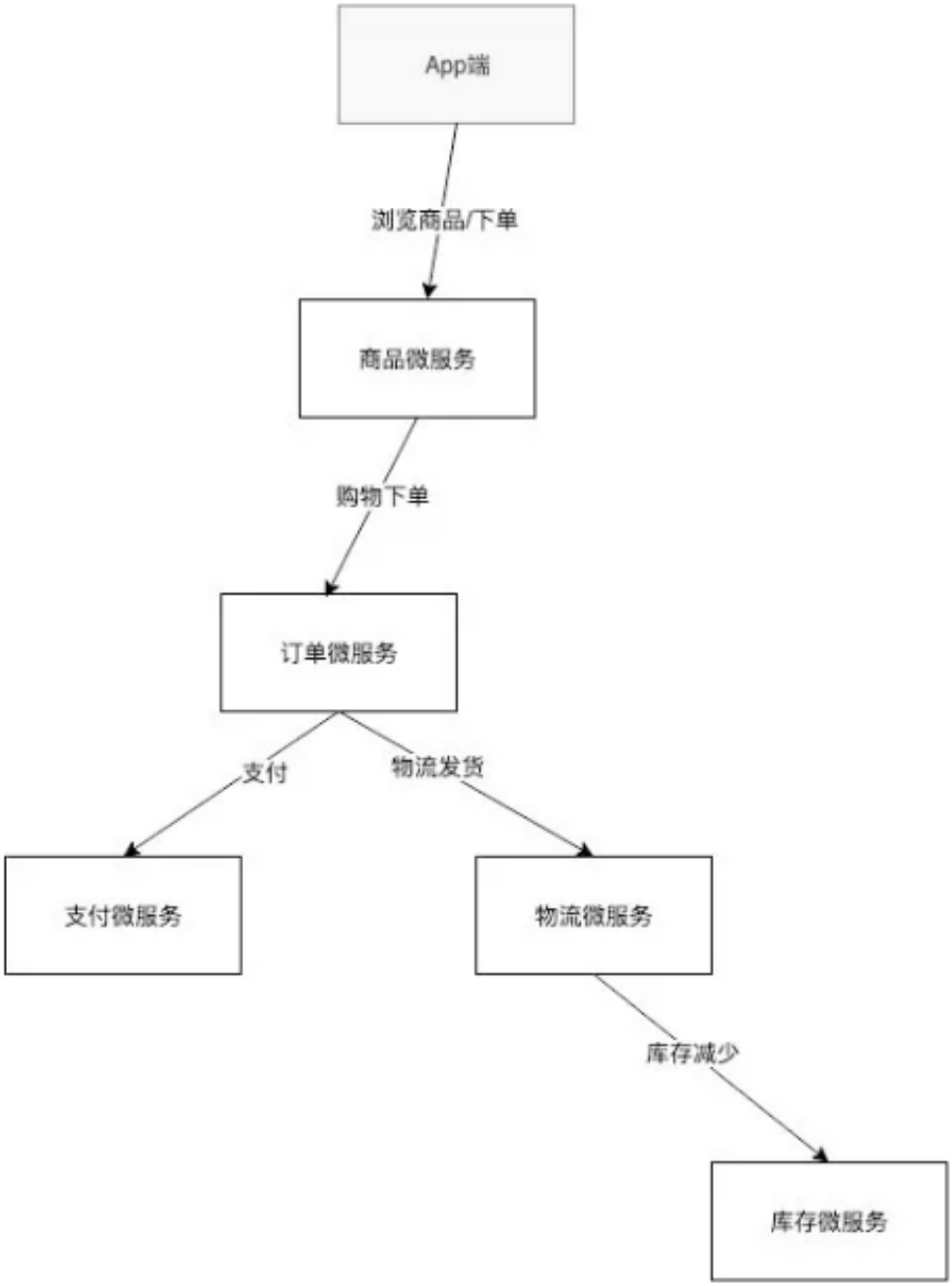
2. 一个queue对应一个consumer，然后这个consumer内部用内存队列做排队，然后分发给底 层不同的worker来处理



微服务

什么是服务熔断、服务降级

在复杂的分布式系统中，微服务之间的相互调用，有可能出现各种各样的原因导致服务的阻塞，在高并发场景下，服务的阻塞意味着线程的阻塞，导致当前线程不可用，服务器的线程全部阻塞，导致服务器崩溃，由于服务之间的调用关系是同步的，会对整个微服务系统造成服务雪崩



为了解决某个微服务的调用响应时间过长或者不可用进而占用越来越多的系统资源引起雪崩效应就需要进行服务熔断和服务降级处理。

所谓的服务熔断指的是某个服务故障或异常一起类似现实世界中的“保险丝"当某个异常条件被触发 就直接熔断整个服务，而不是一直等到此服务超时。

服务熔断就是相当于我们电闸的保险丝，一旦发生服务雪崩的，就会熔断整个服务，通过维护一个 自己的线程池，当线程达到阈值的时候就启动服务降级，如果其他请求继续访问就直接返回 fallback的默认值

Eureka和ZooKeeper都可以提供服务注册与发现的功 能，请说说两个的区别

- ZooKeeper保证的是CP，Eureka保证的是AP
- ZooKeeper在选举期间注册服务瘫痪，虽然服务最终会恢复，但是选举期间不可用的
- Eureka各个节点是平等关系，只要有一台Eureka就可以保证服务可用，而查询到的数据并不是最 新的
- 自我保护机制会导致Eureka不再从注册列表移除因长时间没收到心跳而应该过期的服务
- Eureka仍然能够接受新服务的注册和查询请求，但是不会被同步到其他节点(高可用)
- 当网络稳定时，当前实例新的注册信息会被同步到其他节点中(最终一致性)
- Eureka可以很好的应对因网络故障导致部分节点失去联系的情况，而不会像ZooKeeper一样使得 整个注册系统瘫痪
- ZooKeeper有Leader和Follower角色，Eureka各个节点平等
- ZooKeeper采用过半数存活原则，Eureka采用自我保护机制解决分区问题
- Eureka本质上是一个工程，而ZooKeeper只是一个进程

负载均衡策略

- 使用负载均衡带来的好处很明显:
    - 当集群里的1台或者多台服务器down的时候，剩余的没有down的服务器可以保证服务的继 续使用
    - 使用了更多的机器保证了机器的良性使用，不会由于某一高峰时刻导致系统cpu急剧上升

负载均衡有好几种实现策略，常见的有:

- 随机 (Random)
- 轮询 (RoundRobin)
- 哈希 (Hash)
    - ip信息%机器数量
- 一致性哈希 (ConsistentHash)
    - 服务器ip信息 % 2的32次方-1 的结果落在哈希环上，顺时针取最近的

Ribbon内置的负载均衡策略

负载均衡策略	描述
RoundRobinRule：轮询策略	默认超过10次获取到的server都不可用，会返回一个空的server
RandomRule：随机策略	如果随机到的server为null或者不可用的话，会while不停的循环选取
RetryRule：重试策略	一定时限内循环重试。默认继承RoundRobinRule，也支持自定义注入，RetryRule会在每次选取之后，对选举的server进行判断，是否为null，是否alive，并且在500ms内会不停的选取判断。而RoundRobinRule失效的策略是超过10次，RandomRule是没有失效时间的概念，只要serverList没都挂。
BestAvailableRule：最小连接数策略	遍历serverList，选取出可用的且连接数最小的一个server。该算法里面有一个LoadBalancerStats的成员变量，会存储所有server的运行状况和连接数。如果选取到的server为null，那么会调用RoundRobinRule重新选取。
AvailabilityFilteringRule：可用过滤策略	扩展了轮询策略，会先通过默认的轮询选取一个server，再去判断该server是否超时可用，当前连接数是否超限，都成功再返回。
ZoneAvoidanceRule：区域权衡策略（**默认策略**）	扩展了轮询策略，继承了2个过滤器：ZoneAvoidancePredicate和AvailabilityPredicate，除了过滤超时和链接数过多的server，还会过滤掉不符合要求的zone区域里面的所有节点，在一个区域/机房内的服务实例中轮询。**先过滤再轮询**

Java 源码学习

Linklist

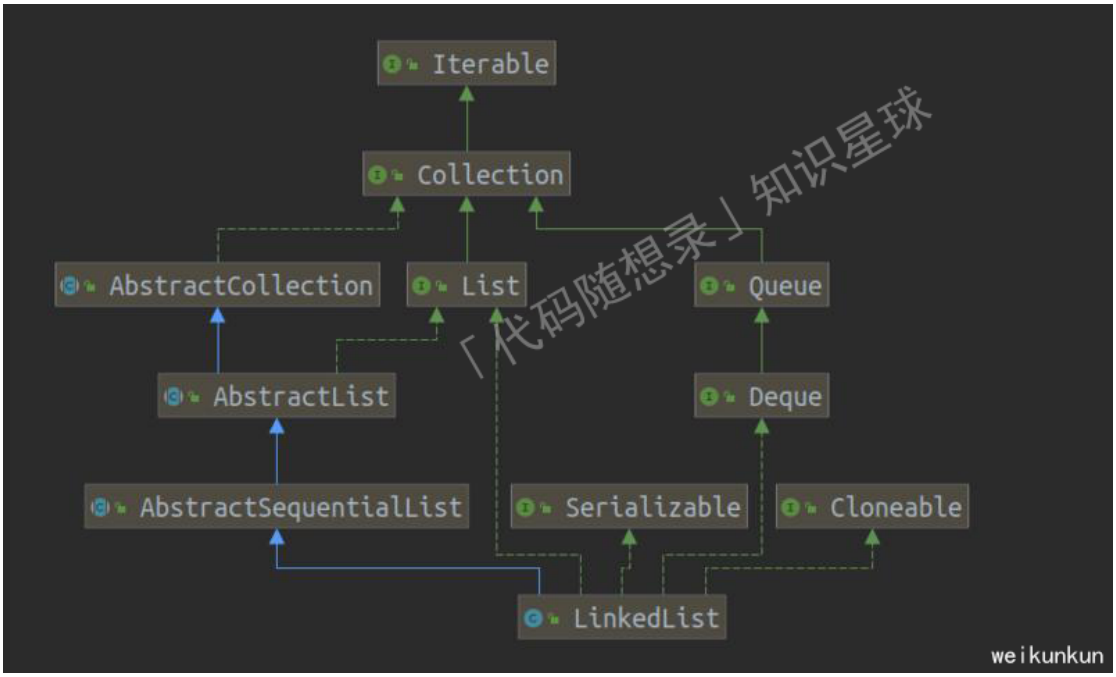
1. 概述

LinkedList底层数据结构是链表，是实现了List接口和Deque接口的双端链表;
其能够高效的实现插入删除操作, 而且也拥有了队列（简单的队列、双端队列）所拥有的特性。

和ArrayList相比，因为其没有实现RandomAccess，是以下标进行访问元素，
所以对于元素访问不及ArrayList，随机访问元素慢。

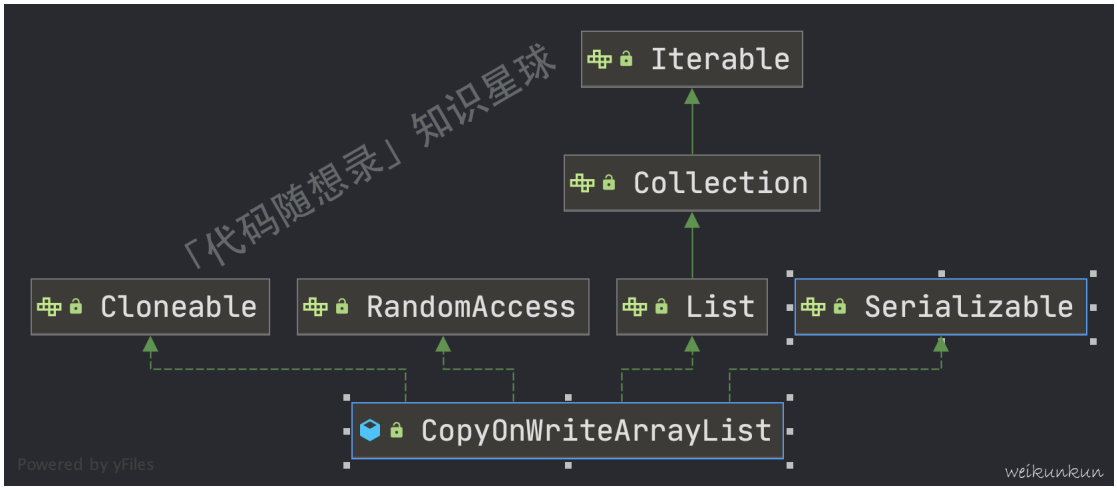
2. 细节

1. 对于普通的单向链表而言，按照下标查找查找的时间复杂度为O(n)，但是对于双向链表按照下标查找，可以利用其特性，选择从头部查找还是从尾部查找，所以更精确来讲，双向链表按照下标查找是O(n/2)，这应该也是LinkedList使用双向链表实现的一个原因吧，在源码里面也能够发现绝大多数的CRUD的API都是先按照下标找到对节点，再进行后续操作。
2. 默认的插入方式是尾部插入，不过也可以明确指定插入方式（头部插入、中间插入、尾部插入），remove方法同理，有这么多插入、删除的形式，为了就是能够在其基础上，实现栈、队列等数据结构。
3. 具有fail-fast机制。



CopyOnWriteArrayList

1. 概述



CopyOnWriteArrayList可以理解成是ArrayList的线程安全的版本，内部也是使用数组实现；

每次对数组的修改都完全拷贝一份新的数组来修改，修改后再替换原来的老数组，这样子只阻塞的了写操作，不阻塞读操作，实现读写分离。

有一问题是，其无法保证实时的一致性，只能保证最终的一致性，所以适用于对实时性要求不高，读多写少的场景，譬如黑白名单。

2. 部分细节

1. 每次对原数组的修改操作，都先加锁，然后copy一份新的数组，在新数组上做修改
2. 锁使用的是ReentrantLock，独占的不公平锁
3. 整个数组使用volatile修饰保证了可见性，结合锁之后，也就确保了单个api操作的线程安全
4. 内部无size属性，直接通过获取当前数组大小得到对应的元素的个数
5. 需要注意的是，因为写时会复制一个近乎等大小的数组，所以需要考虑内存空间和集合使用的业务场景
6. 看完源码之后，加深了System.arraycopy、Arrays.copyOf的理解

3. 相似的思想

使用fork创建子进程，避免了内核直接将整个父进程的虚拟内存的数据（堆、栈、数据段、执行代码段）内容整个复制并分配给子进程。

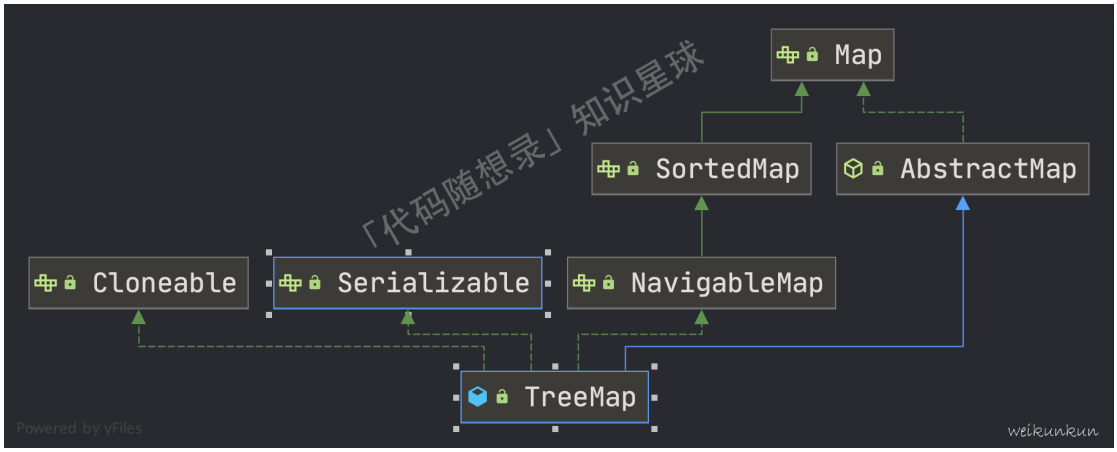
刚开始是让父子进程共享同一个副本，只有在需要写入的时候，数据才会被复制，从而父子进程才拥有各自的副本，在此之前都属于共享的读。

这样之后，对父进程实际物理内存中页的复制被推迟到发生写入的时候。

并且有的时候，共享的副本可能都不会被写（fork之后，调用exec）。

通过写时复制之后，fork的实际开销就变成了复制父进程的页表和给子进程创建PCB。

treemap



概述：

TreeMap底层基于红黑树实现，可以保证在log(n)的时间复杂度内完成containsKey、get、put、remove操作。

同时因为其由红黑树构成，也就是说明了能够维持内部元素的有序性，关于支持内部元素有序性的集合还有LinkedHashMap。

并没有对红黑树做很深的了解，只是跟着整体的思路，使用AVL树实现了一遍。

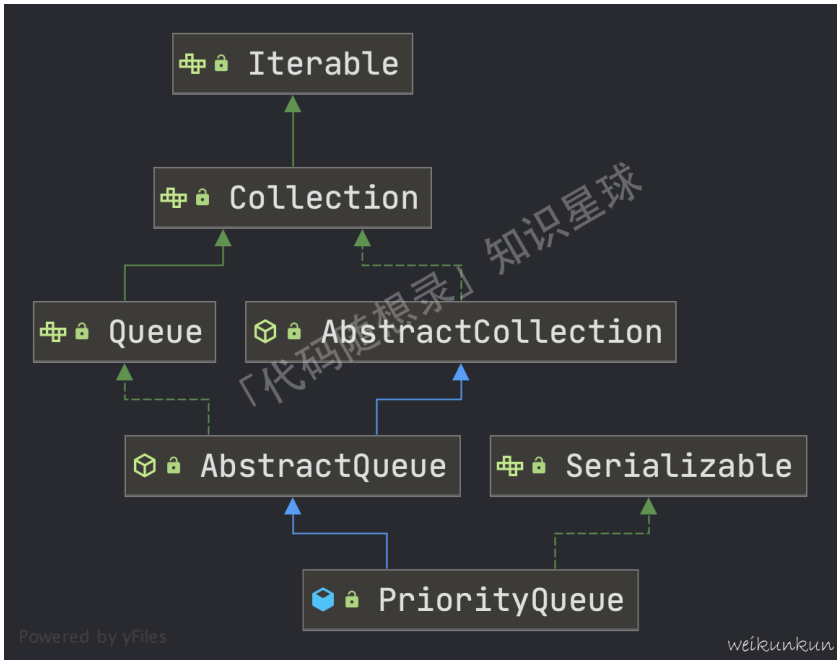
部分细节：

1. 红黑树相比于AVL树，牺牲了部分平衡性，以换取删除/插入操作时少量的旋转次数，整体来说，性能优于AVL树，但是做了性能测试，发现优化了的AVL树和红黑树相比差不了太多。
2. AVL树为了维护严苛的平衡条件，在破坏了平衡之后（插入、删除），需要执行旋转操作。共分为四种：左单旋、先左后右旋、右单旋、先右后左旋。
3. TreeMap内部无扩容的概念，因为使用的是树的链式存储结构
4. 支持范围查找，查找最近的元素
5. 以为内部是按照key进行排序的，所以不支持key为null
6. 排序依据，根据存放的对象是否实现Comprable接口。若实现了，则依据其自定义的compareTo方法，否者需要自定义外部比较器（Comparator），若是都为实现，则报错。

相关练习题：(Leetcode)

285. 二叉搜索树中的中序后继节点(plus会员)
108. 将有序数组转换为二叉搜索树
110. 平衡二叉树
450. 删除二叉搜索树中的节点
270. 最接近的二叉搜索树值(plus会员)

PriorityQueue



概述：

优先队列，是0个或者多个元素构成的集合，集合中按照某种排序方式（元素自身的权重）进行排序，不保证内部元素整体有序，但是每次弹出的元素的优先级最高/低。

内部的数据结构是堆，因为堆底层的结构是完全二叉树，对于树的存储包含有链式存储或者顺序存储，PriorityQueue使用的是顺序存储，所以使用的是Object[]数组，然后利用完全二叉树的性质，解决父子节点关系问题。

默认实现是小根堆

部分细节：

1. 未指定初始化容量大小，默认为11，感觉对于底层使用数组实现的集合，默认大小的规定好要没有啥规律可循
2. 扩容比其他集合多了一步，在数组长度 < 64 时，扩容为原先的两倍+2，超过64时，扩容为原先的1.5倍，同时做了放溢出处理，支持最大元素个数Integer.MAX_VALUE;
3. 删除和插入操作均会破坏当前的堆结果，所以每次都需要调用siftUp、siftDown动态调整
4. 插入操作是插入当前堆的末尾，调用siftUp，自底向上调整
5. 删除操作弹出堆顶元素，然后将堆最后一个元素置于堆顶，调用siftDown，自顶向下调整
6. 同样具有fast-fail机制

相关题目：

题目的话，感觉还是根据具体情况使用，对于子序列DP问题也可以引入优先队列来优化暴力搜索的过程

(1) 频率相关问题，结合map使用
(2) TopK问题（海量数据处理）

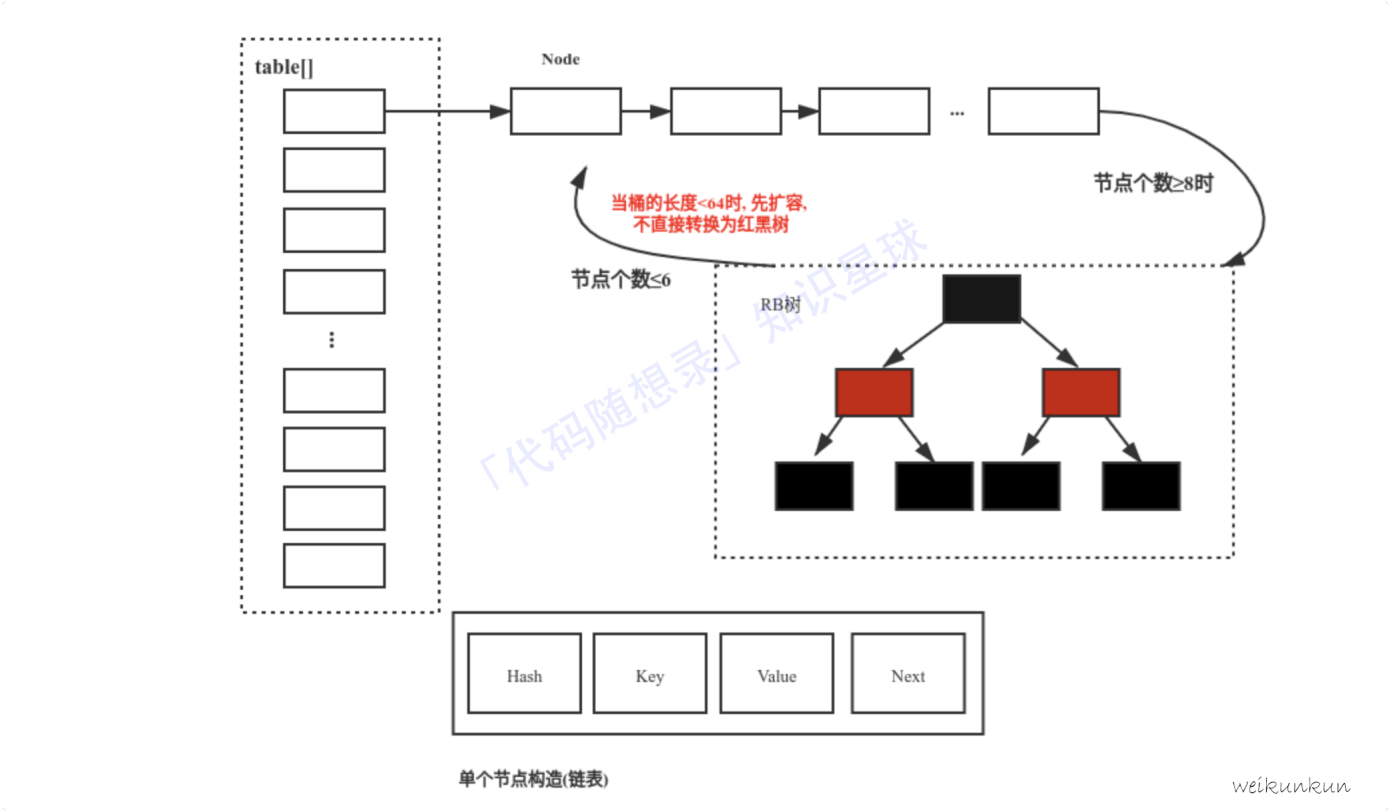
HashMap

概述：

HashMap从宏观来看是基于Hash表实现的，hash冲突的解决方式是拉链法。

具体实现上，在JDK1.8时有做过优化。引入了红黑树，避免因为大量的冲突导致某个位置链表长度过长，使得某些元素的查询效率变低。

同时将链表插入操作的头插法改为了尾插法，保证了插入顺序的同时，也避免了并发操作下的一些异常（并发操作，肯定要想办法解决线程安全嘛，不然出现异常才是正常现象吧）。同时扩容后的rehash的过程也做了优化。



部分细节：

1. 内部采用Node数组来表示Hash表，数组长度为2的幂次，主要是为了能够通过位运算获取key的索引位置，提升计算的效率
2. 链表长度>8时会进行树化，也就是转变为红黑树。大前提是整个Node数组容量>64。同时当树节点数量小于6时又会变为链表
3. 既然是一个集合类，肯定涉及到动态扩容，HashMap内部的扩容机制，默认是扩容为原数组长度的2倍，其次在初始化是若是没有指定容量大小，直接从0扩容为16。如果初始化时指定了容量大小，则整个Hash表的容量大小为最接近指定容量且大于指定容量的2的幂次
4. 扩容过程也伴随着rehash的过程，1.8之前是扩容后，对所有元素rehash，之后映射到对应的位置。1.8时做了优化，巧用的运用了2次幂的扩展(指长度扩为原来2倍)之后，元素的位置要么是在原位置，要么是在原位置再移动2次幂的位置这个特点，避免了rehash的过程，而是直接将key的hash值和oldCap相与，如果为0，则保持原位，如果为1，则放入到原位+oldCap的位置。
5. 内部没有capacity变量，Node数组的容量大小是在扩容时确定的
6. 默认的负载因子为0.75，和Redis中的一样，ThreadLocal的为黄金分割点

7. 对key取hash值，采用的是高地位异或求值，从hash分布的情况来看，离散型更好。
8. Redis中的rehash过程和HashMap1.8之前的实现很像，不过是渐进式的，同时其内部是有新旧两个hash表的，在删除、查找、更新操作时，会操作两个hash表，不过在新增操作时，操作的是新的hash表。

相关题目：

1. 用于dfs、递归中的缓存操作，防止超时
2. 用于查找、频率统计和查重，需要明确key和value的分别代表什么
3. 映射关系，譬如判断给定的字符是否属于ABA这种格式

Java 编译过程

前言

“说下Java代码编译和执行过程？”

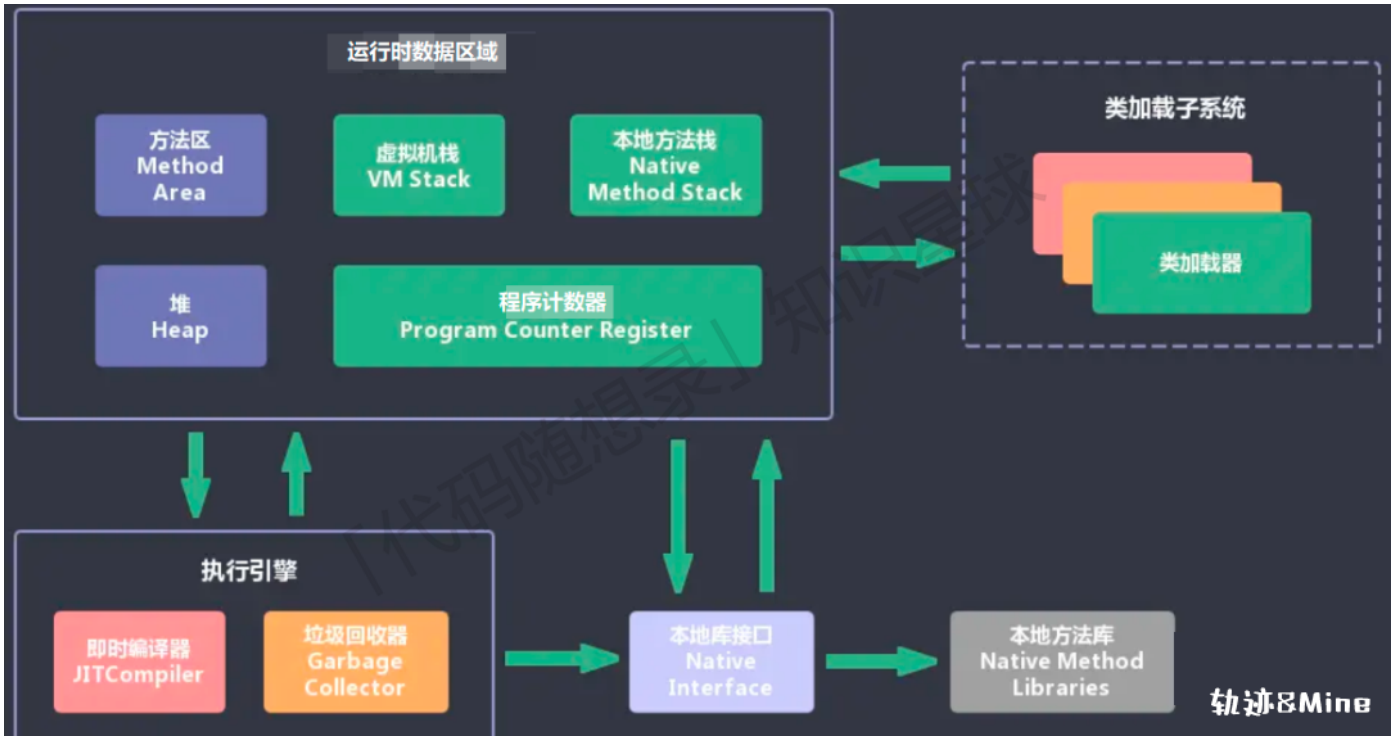
“为什么需要执行引擎，它的工作过程是怎样的？”

“什么是解释器，什么是JIT编译器，它们有什么区别？”

“为什么有了JIT编译器还需要解释器？”

面试过程中，你是否被问到过这些问题，如果不能完全回答出来，那么这篇文章一定要看下去。

执行引擎



从上图可以看出，JVM包含两个子系统和两个组件。

两个子系统：

1. 类加载子系统：负责将字节码文件装载内存成为一个对象。
2. 执行引擎：包含解释器、JIT编译器、垃圾收集器（GC）

两个组件：

1. 运行时数据区
2. 本地接口

执行引擎概述

执行引擎是JVM核心组成部分之一。这里的执行引擎特制虚拟机（VM)中的执行引擎，其实物理机中也存在执行引擎。物理机中执行引擎是直接建立在处理器、缓存、指令集和操作系统层面上的，而虚拟机中的执行引擎是由软件自行实现的。因此，可以不受物理条件制约地定制指令集与执行引擎的结构体系，能够执行那些不被硬件直接支持的指令集格式。那么，什么是不被硬件直接支持的指令集格式呢，别急，我们把这个问题放一放，后面自有解答。

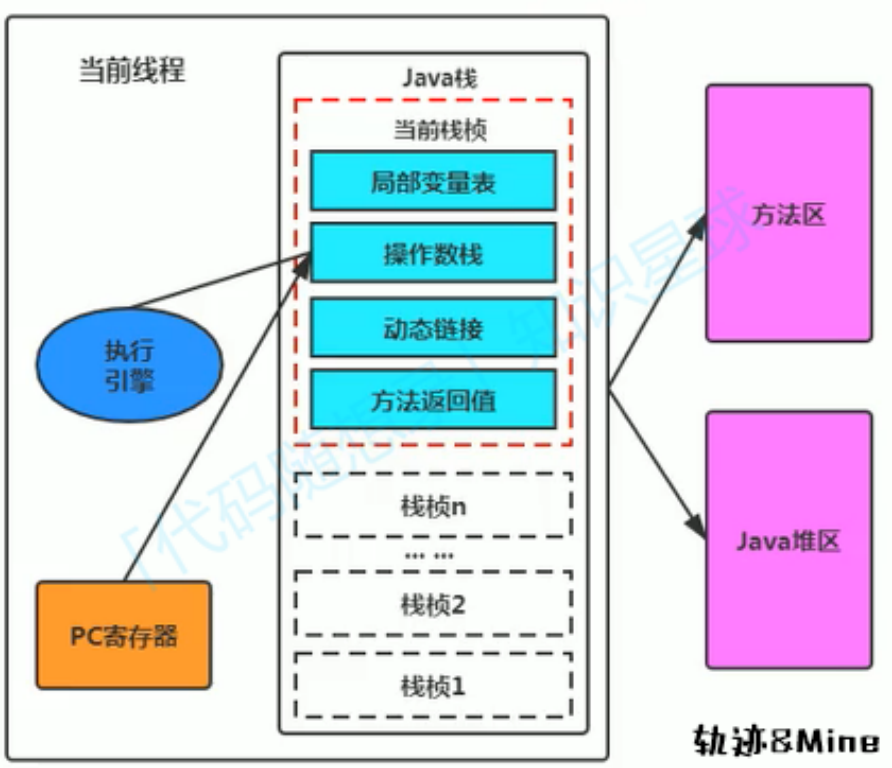
工作流程

为什么需要执行引擎呢？我们知道，jvm主要任务是装载字节码到其内部，但字节码不能直接运行在操作系统上，因为它内部仅仅包含一些能被jvm所识别的字节码指令、符号表以及其他辅助信息。所以，想让java程序执行起来，需要执行引擎将字节码指令解释/编译为对应平台上的本机机器指令。

执行引擎的执行过程：

如下图所示，执行引擎执行过程中执行什么指令依赖于程序计数器（PC寄存器）中的地址，它会根据该地址找到对应的指令。执行完成一条指令操作后，程序计数器会更新下一条需要被执行的指令地址。

在方法执行过程中，执行引擎也可能根据虚拟机栈中局部变量表中存储的对象引用找到堆中对应的对象实例信息，还可能通过对象头中的元数据指针定位到目标对象的类型信息。



指令集

包括：机器码&指令&汇编&字节码

了解代码编译和执行过程之前，先回顾下一些基础知识。

机器码：

各种用二进制编码方式表示的指令

任何用机器码编写的程序，cpu可以直接读取并运行，速度最快，不过可读性差

指令和指令集：

方便人们记忆，将机器码中的0、1序列简化表示为对应的指令。

由于硬件平台不同，执行同一个操作，对应的机器码可能不同，因此不同的硬件平台的同一个指令，对应 的机器码可能不同（如mov）

机器指令与CPU 紧密相关，不同的机器支持不同的指令。

每个平台所支持的指令称为指令集，如x86架构平台的机器支持x86指令集，ARM架构的机器支持ARM指令集

汇编：

由于指令的可读性还是太差，于是人们又发明了汇编语言。

在汇编语言中，用助记符（Mnemonics）代替机器指令的操作码，用地址符号（Symbol）或标号（Label）代替指令或操作数的地址。

在不同的硬件平台，汇编语言对应着不同的机器语言指令集，通过汇编过程转换成机器指令。

由于计算机只认识指令码，所以用汇编语言编写的程序还必须翻译成机器指令码，计算机才能识别和执行。

字节码：

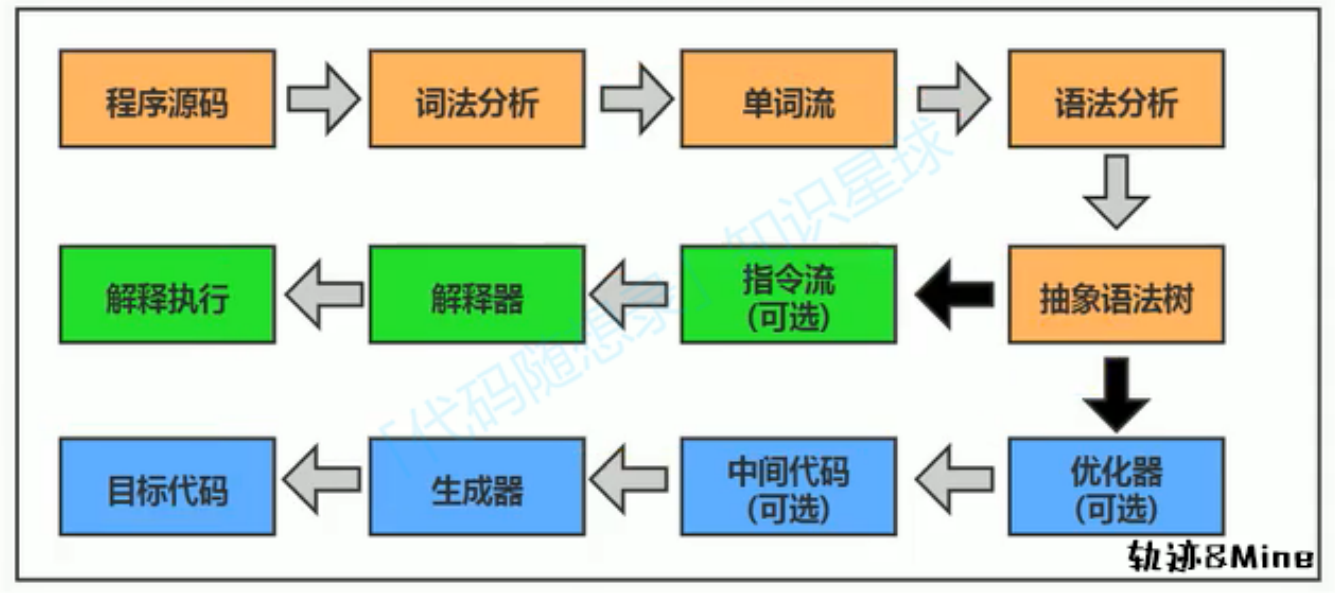
一种中间状态的二进制代码（文件）,它比机器码更加抽象，需要直译器转译后才能转为机器码。

主要是为了实现跨平台的特点，使得特定软件运行和软件环境、与硬件环境无关。

实现方式是通过编译器和虚拟机器。编译器将源码编译为字节码，特定平台上的虚拟机器将字节码转译为自己能识别并执行的指令。

Java代码执行与编译

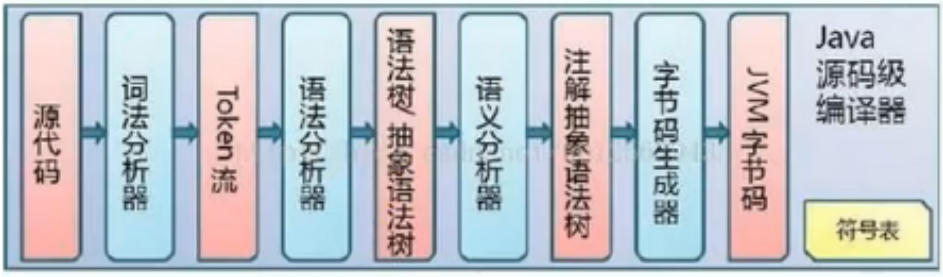
首先我们需要明确的是，大部分程序代码在转换为物理机器能理解的指令集之前，会经过以下步骤：



其中，橙色部分由javac编译器完成，生成线性的字节码指令。通过 javac 编译器，我们可以很方便地将 java 源文件编译成字节码文件。相对于JIT编译器，因为其处于编译的前期，因此又被称为前端编译器。

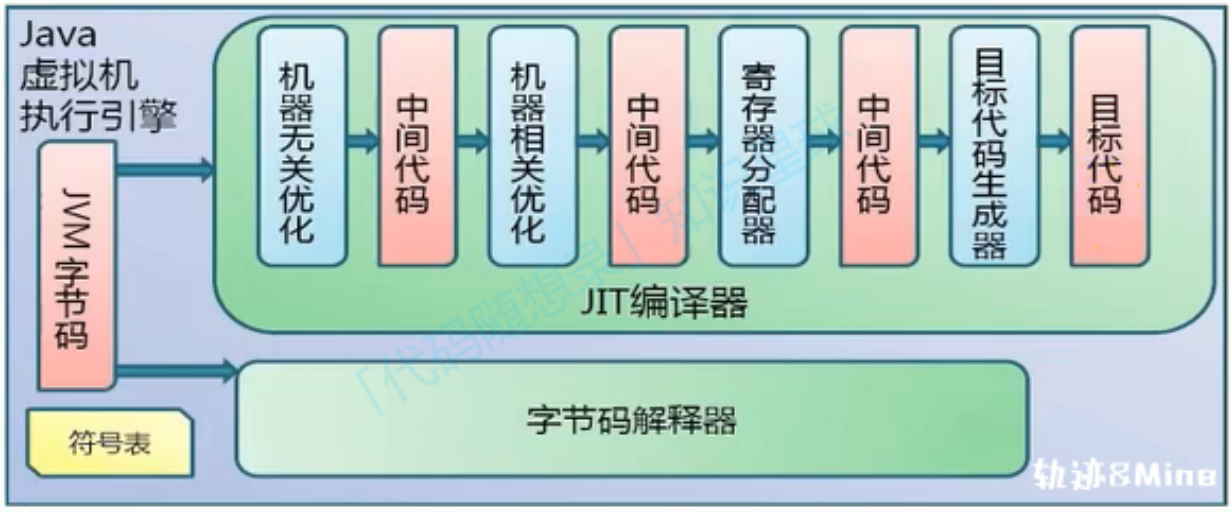
这个过程和JVM无关，因为对于 Java 虚拟机来说，其实际输入的是字节码文件，而不是 Java 文件。

javac编译过程如下所示：



执行步骤中的绿色部分由解释器完成，这也说明了java语言的半解释性。

蓝色对应的是如下的编译过程：



综上，当JVM启动时会根据预定义的规范对字节码进行逐行解释的方式执行，将字节码文件中的内容“翻译”为操作系统能理解的指令。

那么问题来了，为什么说Java是半解释型半编译型语言呢?

实际上，在jdk1.0时，java是解释执行的，后来做了优化。执行引擎在执行字节码指令时既可以选择解释器也可以选择编译器。这样JVM在执行java代码时，可以将解释执行和编译执行结合起来进行。

解释器

问题引出

为什么需要字节码作为中介，不直接将源代码编译为机器能识别的机器指令呢?

这是因为Java设计者的初衷是为了实现跨平台，因此避免采用类似C、C++那种静态编译的方式直接生成机器指令，从而诞生了通过编译器在运行时逐行解释字节码指令从而执行程序的想法。所以，解释器真正意义上所承担的角色就是一个运行时“翻译者”，将字节码文件中的内容“翻译”为对应平台的本地机器指令执行。

解释器的分类

字节码解释器

需要在执行代码时通过纯软件代码模拟字节码的执行，效率十分低

模板解释器

将每一条字节码和一个模板函数相关联，模板函数中直接产生这条字节码执行时的机器码，能提高性能
Hotspot中，解释器主要有Interpreter模块和Code模块组成。

Interpreter模块：

实现了解释器的核心功能

Code模块：

用于管理Hotspot在运行时生成的本地机器指令

解释器的现状

虽然解释器在设计和实现上很简单，但基于它执行已经沦为低效的代名词。针对该问题，jvm平台支持一种称为及时编译的技术。它的目的是避免函数被解释执行，而是将整个函数体编译为机器码，每次函数执行时，只执行编译后的机器码，实现提高程序的执行效率。

JIT编译器

JIT编译器，英文为Just In Time Compiler。根据它，JVM可将源代码直接编译为和本地机器平台相关的机器语言指令。既然根据及时编译的技术，可以提高程序的执行效率，那么为什么不把解释器抛弃掉呢？为什么要将JIT编译器和解释器并存呢？

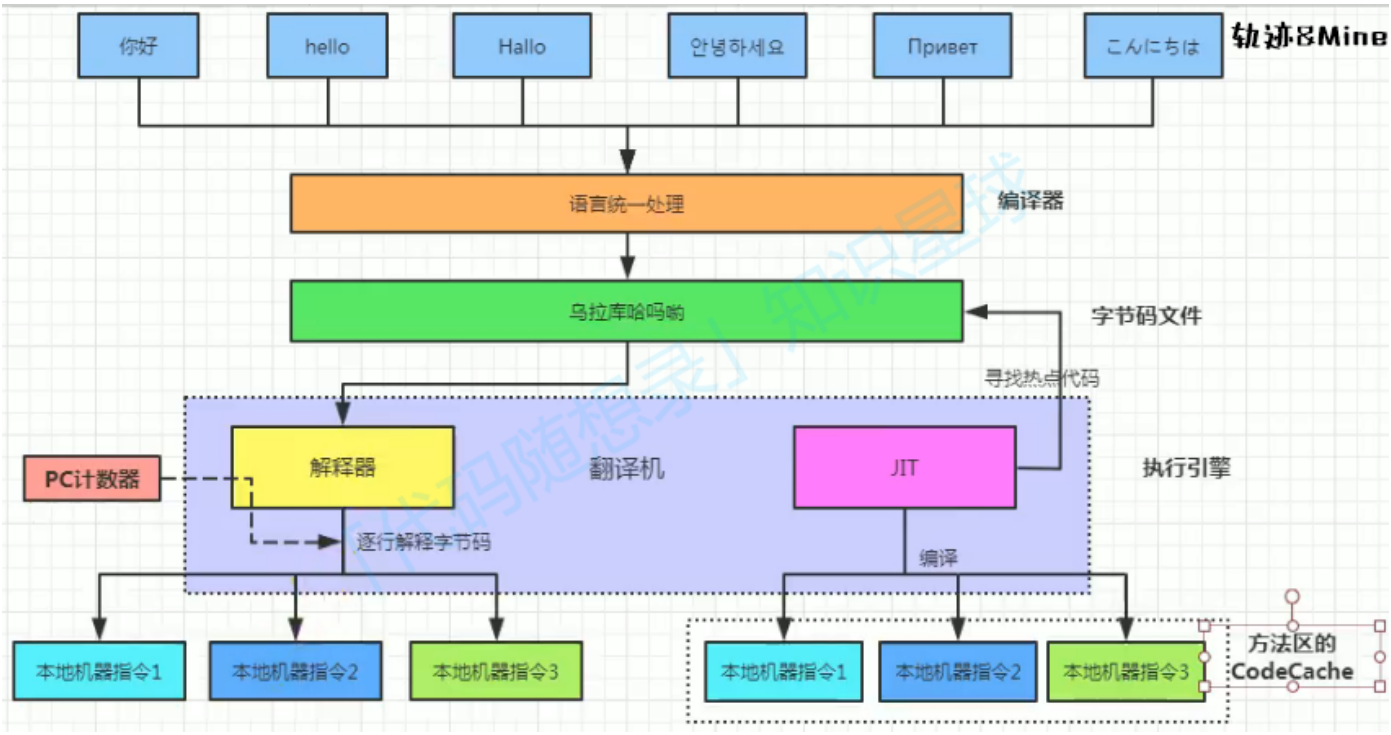
JVM执行代码主要有两种方式：

解释执行：

一段代码，解释一行执行一行。即源程序被编译为字节码文件后，解释器逐行将字节码解释成机器指令并执行。

编译执行：

事先已经被编译成机器码，直接执行，不用解释。源程序被编译为字节码文件后，JIT编译器将字节码编译为机器指令，再由cpu执行。



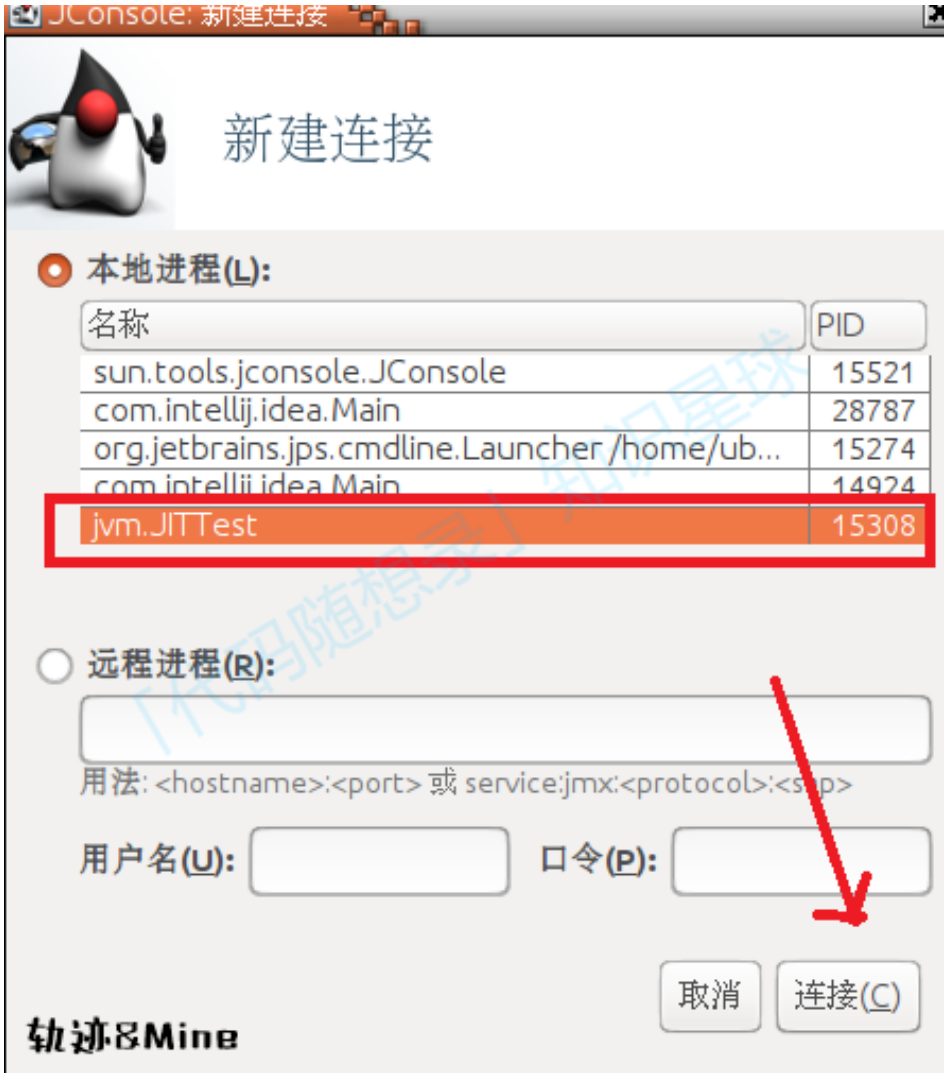
虽然JIT编译器能提高执行效率，但是在执行机器指令前，它需要对字节码进行编译，因此造成响应时间较长，而解释器可以直接对字节码进行解释执行。因此，Hotspot虚拟机采用二者并存的方式，在jvm执行过程中，二者相互协作，各自取长补短，尽全力去权衡编译本地代码的时间和直接解释执行代码的时间，保证执行效率。

在jvm启动时，解释器可以先发挥作用，而不必等待JIT编译器全部编译完成后再去执行，省去不必要的编译的时间。而随着程序运行时间的推移，即时编译器逐渐发挥作用，根据热点探测功能，将有价值的字节码编译为本地机器指令，以换取更高的执行效率。

以下通过小例子查看JIT编译器的存在：

```
package jvm;
/**
 * 运行后，打开jconsole，查看JIT编译器
 */

import java.util.ArrayList;

public class JITTest {
    public static void main(String[] args) {
        ArrayList list = new ArrayList();
        for (int i=0;i<100;i++){
            list.add("Hello!");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

运行以上程序过程中，打开jconsole工具，查看指定的Java进程：



在jconsole的VM概要一栏中，可以看到JIT编译器是确实存在的：

总结

关于执行引擎、解释器、编译器、JIT编译器，值得深入的点还有很多，比如热点探测、如何判断热点代码、热点检测方式，这篇文章只是做一个初步的介绍，待深入学习后，会及时更新相应的内容。

Java 后端编译优化

虚拟机最开始是通过解释器进行解释执行的，解释器是最基础的，但是没有优化，效率和速度都有待提高。但是只依靠解释器不进行优化也是可以的。即时编译器和提前编译器都是采取的优化措施，是可选择的。

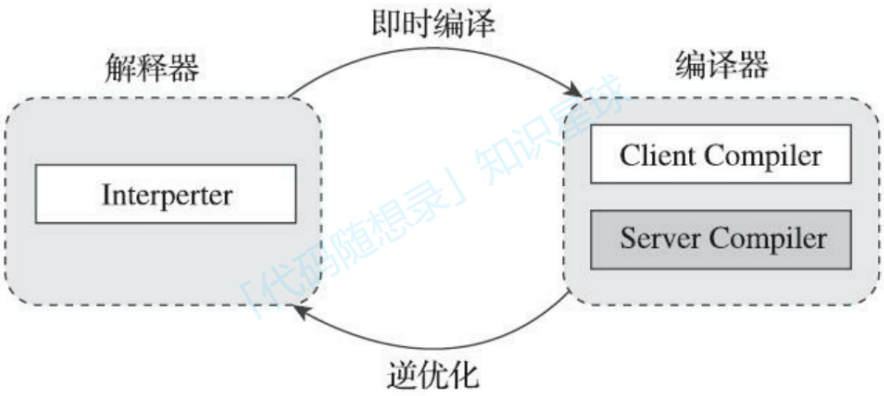
即时编译器

当虚拟机发现某个方法或代码块的运行特别频繁，就会把这些代码认定为“热点代码”（Hot Spot Code），为了提高热点代码的执行效率，在运行时，虚拟机将会把这些代码编译成本地机器码，并以各种手段尽可能地进行代码优化，运行时完成这个任务的后端编译器被称为即时编译器。

解释器与编译器

主流的虚拟机内部都同时包含解释器与编译器，二者各有优势：当程序需要迅速启动和执行的时候，解释器可以首先发挥作用，省去编译的时间，立即运行。当程序 启动后，随着时间的推移，编译器逐渐发挥作用，把越来越多的代码编译成本地代码，这样可以减少 解释器的中间损耗，获得更高的执行效率。

编译器是将代码转换为本机机器码保存下来，需要占用内存。所以内存资源不多的时候用解释器执行，反之可以用编译器来提高程序执行速度和效率。同时，编译器的优化不能保证完全成功，一些激进优化措施是有失败的风险，此时就不能再用编译器，要退回到解释器状态继续执行。



虚拟机有三种编译模式：

混合模式：解释器与编译器搭配使用

解释模式：解释器工作，编译器完全不工作

编译模式：优先采用编译器，但编译器无法进行时采用解释器

HotSpo内置的编译器有：“客户端编译器（C1编译器）”和“服务端编译器（C2编译器）”

分层编译：

第0层：程序纯解释执行，并且解释器不开启性能监控功能（Profiling）。

第1层：使用客户端编译器将字节码编译为本地代码来运行，进行简单可靠的稳定优化，不开启性能监控功能。

第2层：仍然使用客户端编译器执行，仅开启方法及回边次数统计等有限的性能监控功能。

第3层：仍然使用客户端编译器执行，开启全部性能监控，除了第2层的统计信息外，还会收集如分支跳转、虚方法调用版本等全部的统计信息。

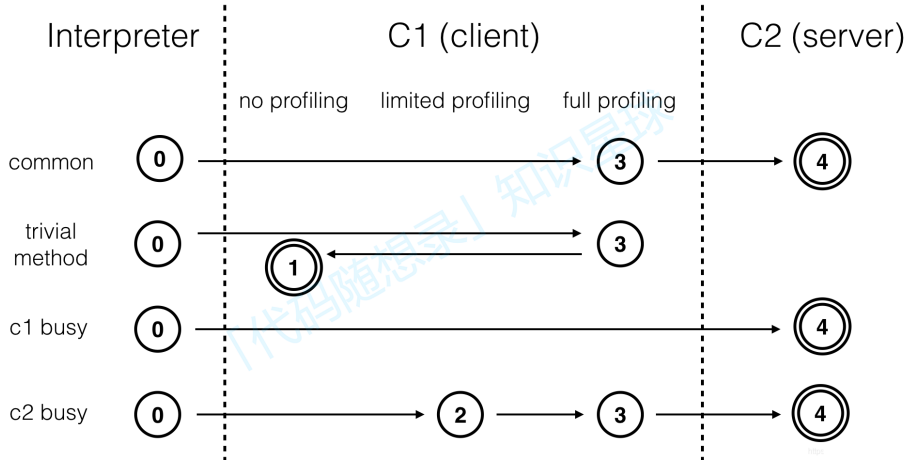
第4层：使用服务端编译器将字节码编译为本地代码，相比起客户端编译器，服务端编译器会启用更多编译耗时更长的优化，还会根据性能监控信息进行一些不可靠的激进优化。

解释器执行的启动速度是比编译器快的，所以程序先再第0层用解释器执行。

而编译器的启动速度也有区别，C1编译器启动要比C2编译器快，这是因为C2会进行更多的优化，所以C2编译出的代码要比C1的代码执行效率好很多。

从启动速度上看 解释器>C1编译器>C2编译器，从程序执行效率上看 4>1>2>3>0, 随着C1性能监控搜集的信息越来越多，性能开销也越大。第二层和第三层基本是为第4层服务的，减少第四层编译执行的时间。

程序最后会在1或者4编译，C2编译出的效果最好，所以最后要进入4，但是如果一段程序本身就简单，可以优化的地方不多，那么就不值得花费时间去搜集程序执行信息，直接去第1层用C1编译就可以，这样编译的效果也很好。



通常情况下，热点代码会被第三层C1编译，然后交给第4层的C2编译。

在字节码较少的情况下，此时性能监控收集的数据很少，就交给第1层的C1进行编译。

在C1比较繁忙时，会在第0层解释执行收集程序执行状态数据，然后直接交给第4层的C2编译。

在C2繁忙的时候，先交给第2层的C1，再交给低3层的C1，最后交给第4层的C2。

热点代码

1. 被多次调用的方法，触发标准编译请求
2. 被多次执行的循环体，触发栈上编译请求

热点代码编译的都是整个方法体。即使是方法中的循环体被认定是热点代码，热点只是整个方法的一部分，但编译器依然必须以整个方法作为编译对象，只是执行入口（从方法第几条指令开始执行）有不同。

编译时会传入入口点字节码序号，这种编译方式因为 编译发生在方法执行的过程中因此称为"栈上替换（OSR）"，即方法还在方法栈上但是具体执行的指令已经被替换了，注意方法栈本身并没有改变也没有移动，只是执行到热点代码时去内存中读取编译后的方法。因为循环体是从某条指令开始执行，一轮完成之后会再次跳转到开始位置，所以只传入一个入口点序号就可以（入口点也是出口）。

热点探测：

判断一个方法是不是热点代码，需不需要出发即时编译。主流有两种方法

1、基于采样的热点探测：

虚拟机会周期性地检查各个线程的调用栈顶，如果发现某个（或某些）方法经常出现在栈顶，那这个方法就是"热点方法"。好处是实现简单，只需要采样当前的栈顶即可。缺点是精度不行，容易因为受到线程阻塞或别的外界因素的影响而扰乱热点探测。

2、基于计数器的热点探测：

虚拟机为每个方法建立计数器，统计执行次数，超过一定阈值则认为是热点代码。好处是结果精确，缺点是比较麻烦，为每个方法建立计数器。

HotSpot采用此种方法，给每个方法准备两类计数器：方法调用计数器和回边计数器。

方法调用计数器：

统计方法被调用的次数。方法调用时判断是否存在编译过的版本，如果存在则执行编译后的本地代码。如果不存在，方法计数器+1，和阈值比较判断是否触发即时编译。一旦超过，申请即时编译。

统计绝对次数：

统计方法所有时间段内执行次数，计数器值不会减少。只要程序执行足够多，一定会触发即时编译。

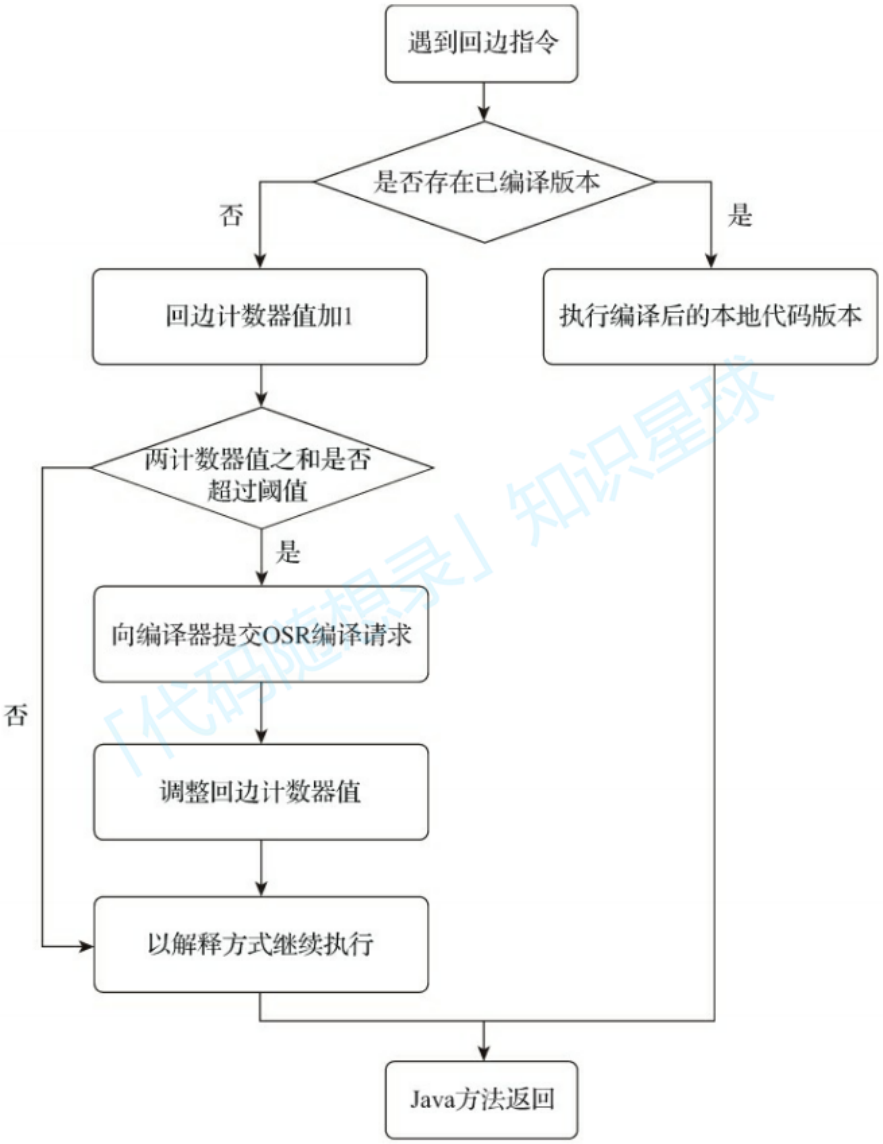
统计相对次数：

统计一段时间执行次数，如果超出时间还未能出发即时编译，统计次数减少一半。这个称为方法调用计数器热度的衰减，这段时间半衰周期。默认统计相对次数。

注意上面这两个方法只有在方法调用器种可以选择，回边调用器只能记录绝对次数。

回边计数器：

统计一个方法中循环体代码执行的次数，在字节码中遇到控制流向后跳转的指令就称为“回边“。当计数达到阈值后会触发栈上的替换操作（OSR），每个循环体都应该有一个回边计数器，所以一个方法内可能存在多个回边计数器。回边调用只记录绝对次数，触发的阈值可以间接设置。



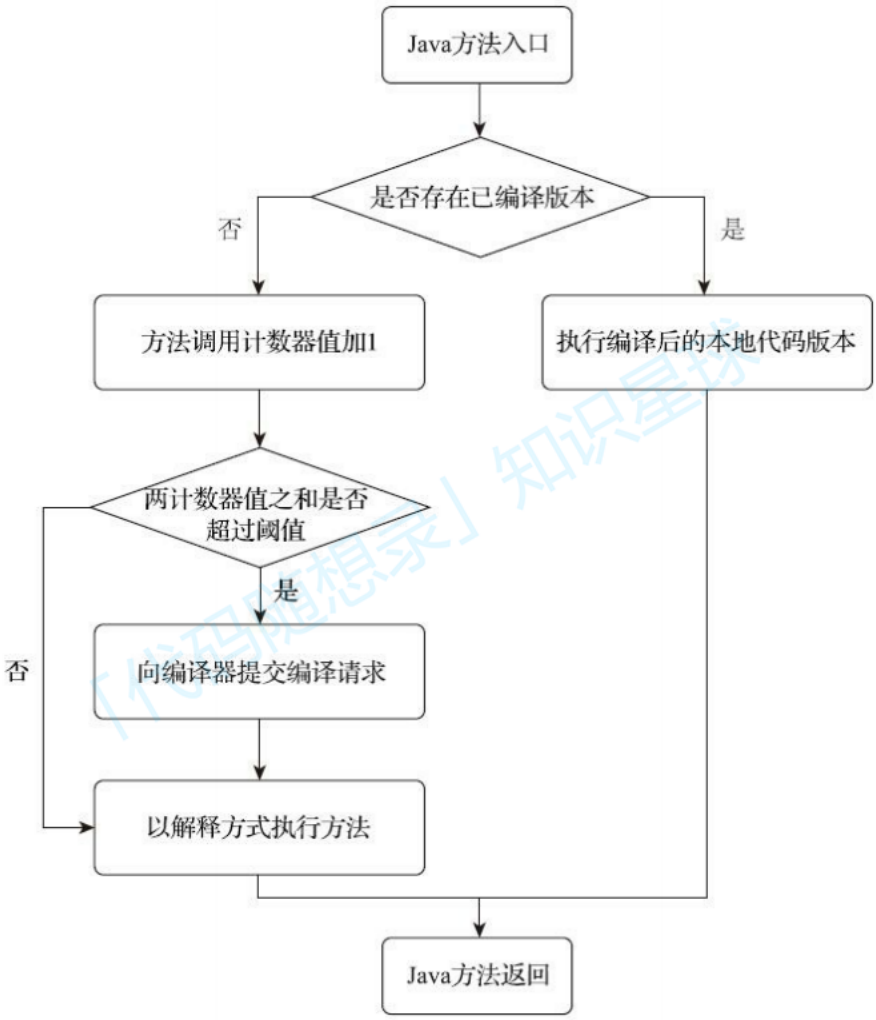


图11-3　方法调用计数器触发即时编译

热点代码经过编译后是放在虚拟机的内存中，这样以后调用时可以直接从内存中读取。

但是如果代码缓存空间用完了，虚拟机遇到热点代码也不会进行即时编译，而是启用解释器执行。

代码缓存占满后并不会抛出OOM异常，只是等待下一次GC清理出空间。当程序较大想要保存更多的编译后本地代码，应该扩大代码缓存空间。

提前编译器

1、在程序运行之前把程序代码编译成机器码的静态翻译工作

2、把原本即时编译器在 运行时要做的编译工作提前做好并保存下来，下次运行到这些代码（譬如公共库代码在被同一台机器 其他Java进程使用）时直接把它加载进来使用。目前主流虚拟机均支持此种方法。

即时编译是在程序运行时发生的，优化需要时间和运算资源，而提前编译是在程序运行之前就编译好，可以减少系统运行时优化消耗。

提前编译和即时编译优劣

提前编译的好处是省资源，避免了在运行时进行优化，节约了运算资源。而即时编译相对提前编译，有以下优点：

1. 性能分析制导优化
2. 激进预测性优化

3. 链接时优化

编译优化技术

方法内联：

把目标方法的代码原封不动地“复制”到发起调用的方法之中，避免发生真实的方法调用。这是编译器最重要的优化手段，被称为优化之母。

除了消除方法调用的成本之外，它更重要的意义是为其他优化手段建立良好的基础，没有内联，多数其他优化都无法有效进。

方法内联需要确定目标方法的具体版本，但是除了在编译期间进行解析的非虚方法和final修饰的方法，其他方法都是虚方法。

虚方法可能会进行重载或者重写，为了确定方法的版本Java虚拟机采用类型继承关系分析技术（CHA），这个可以确定在目前已加载的类中，某个接口是否有多于一种的实现、某个类是否存在子类、某个子类是否覆盖了父类的某个虚方法等信息。

守护内联：

如果内联的是非虚方法，那么直接进行内联就行，这样一定时安全的，要内联的方法版本不会发生改变。

但是如果时虚方法，CHA会查询此方法在当前程序状态下是否真的有多个目标版本可供选择，如果查询到只有一个版本，那就可以假设“应用程序 的全貌就是现在运行的这个样子”来进行内联，这种内联被称为守护内联。

如果后期类的关系没有变化，就一直按照内联优化的程序运行，如果加载了导致继承关系发生变化的新类，那么就必须抛弃已经编译的代码，退回到解释状态进行执行，或者重新进行编译。

内联缓存：

如果向CHA查询出来的结果是该方法确实有多个版本的目标方法可供选择,编译器将采用内联缓存的方式来减少方法调用开销。

在未发生方法调用之前，内联缓存为空。第一次调用方法时，记录下方法接受者的版本信息，随后的每次调用都先比较接收者版本信息，如果一致则认为调用的是同一种方法，就通过缓存来调用，称为单态内联缓存。

如果接收者信息不一样，就会退化为超多态内联缓存。单态内联缓存只是比普通内联调用多一次比较，速度还是很快，但是如果是超多态内联缓存，其速度相当于真正查找虚方法表来进行方法分派。

逃逸分析

分析对象动态作用域，当一个对象在方法里面被定义后，它可能被外部方法所引用，例如作为调用参数传递到其他方法中，这种称为方法逃逸；

甚至还有可能被外部线程访问到，譬如赋值给可以在其他线程中访问的实例变量，这种称为线程逃逸；

从不逃逸、方法逃逸到线程逃逸，称为对象由低到高的不同逃逸程度。

逃逸分析与类型继承关系分析一 样，并不是直接优化代码的手段，而是为其他优化措施提供依据的分析技术。

如果一个对象不会逃到方法和线程之外，即其他方法和线程无法通过任何手段访问到这个对象，或者逃逸程度低（只是方法逃逸）则可以为这个对象实现不同程度的优化：

栈上分配：

如果确定一个对象不会逃逸出线程之外，让这个对象在栈上分配内存，对象所占用的内存 空间就可以随栈帧出栈而销毁。因为不会逃逸出线程，所以其他线程是无法访问到的，那么在堆上创建会浪费GC资源，直接在此线程的栈上创建，随着栈帧出栈而销毁，减少GC压力。栈上分配可以支持方法逃逸，但不能支持线程逃逸

标量替换：

若一个数据已经无法再分解成更小的数据（int char long）来表示就称为标量，如果一个数据可以继续分解(Java对象)，那它就被称为聚合量。

如果把一个Java对象拆散，根据程序访问的情况，将其用到的成员变量恢复为原始类型来访问，这个过程就称为标量替换。

假如逃逸分析能够证明一个对象不会被方法外部访问，并且这个对象可以被拆散，那么程序真正执行的时候将可能不去创建这个对象，而改为直接创建它的若干个被这个方法使用的成员变量来代替。标量替换不允许对象逃逸出方法。

同步消除：

如果对象不会逃逸出线程，就不必做线程间的同步，对这个变量实施的同步措施也就可以安全地消除掉。

公共子表达式消除：

如果一个表达式E之前已经被计算过了，并且从先前的计算到现在E中所有变量的值都没有发生变化，那么E 的这次出现就称为公共子表达式。没有必要花时间再对它重新进行计算，直接用之前的计算结果代表E即可。

数组边界检查消除：

在Java语言中访问数组元素系统将会自动进行上下界的范围检查，对于拥有大量数组访问的程序代码，这必定是一种性能负担。

如果编译器只 要通过数据流分析就可以判定循环变量的取值范围永远在区间[0，length)之内，那么在循环中就可以把整个数组的上下界检查消除掉，这可以节省很多次的条件判断操作。

直白的说就是用过编译器更聪明的判断是否需要进行边界检查，消除不必要的检查节省时间。

6.
7.

Java 并发

Synchronized

1、作用

Java中的锁分为显示锁和隐式锁。隐式锁由synchronized关键字实现，而显示锁是由实现了Lock接口和AQS框架等等类来实现。

Java中的synchronized，通过使用内置锁，来实现对共享变量的同步操作，进而解决了对共享变量操作的原子性、保证了其他线程对共享变量的可见性、有序性，从而确保了并发情况下的线程安全。

同时synchronized是可重入的锁，避免了同一个线程重复请求自身已经获取的锁时出现死锁问题（请求于保持、不可剥夺感觉都有体现）

2、基础用法

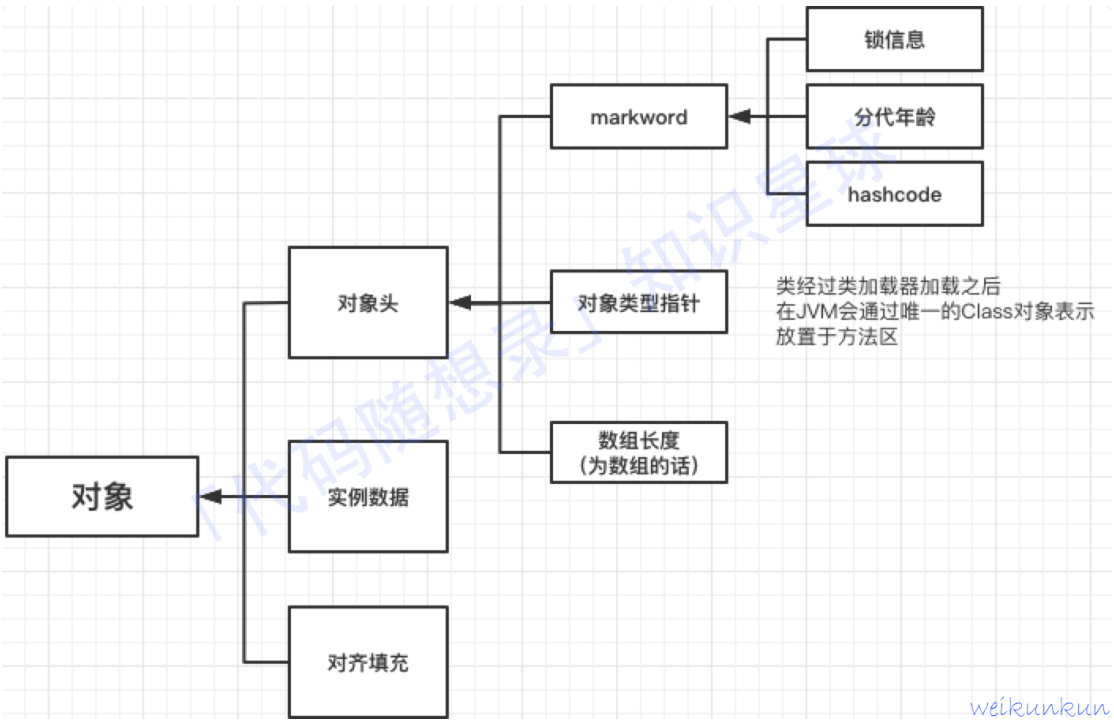
普通同步方法、静态同步方法、同步代码块儿

3、什么是内置锁

在Java中，每个对象都有一把锁，放置于对象头中，用于记录当前对象被哪个线程所持有。

相对于实例数据，对象头属于额外开销，所以被设计的极小来提高效率（一个对象在堆中的分布：对象头、实例数据、对齐填充）。

对象头中的markword更加体现了这一点。markword非结构化的，这样在不同的锁状态下，能够复用相同的bit位。markword中就有存储锁的信息的部分。



4、内置锁又是通过什么实现

synchronized被编译之后，使用的是monitorEnter和monitorExit两个字节码指令(同步代码块儿)，而这两个字节码指令实质上是依赖于操作系统中的mutex lock实现。

因为使用的是互斥锁，需要CPU的参与，涉及到线程的唤醒、同时切换需要记录线程的私有数据、寄存器等不共享的数据，可能这些操作带来的时间开销远大于线程自身执行的时间消耗，所以JDK1.6开始，引入了无锁、偏向锁、轻量级锁、重量级锁的锁优化过程，进而优化synchronized的性能

5、锁的分类

从宏观上看，锁的分类有多种不同划分。可以分为乐观锁和悲观锁，可以分为共享锁和排他锁，还可以分为可重入锁和不可重入等等。

乐观锁：

乐观锁就是对数据冲突保持乐观点态度，认为不会有其他线程同时修改数据。因此乐观锁不会上锁，只是在更新数据都时候判断是否有其他线程更新，如果没有其他线程修改则跟新数据，有其他线程修改则放弃数据，重新读取数据处理。

悲观锁：

悲观锁被数据冲突持悲观的态度，认为总是发生数据冲突。因此它以一种预防的态度，先行把数据锁住，知道操作完成才释放锁，在此期间其他线程无法操作数据。

6、synchronized关键字

Java 中的每一个对象都可以作为锁，有三种加锁的方式：

（1）对于普通同步方法，锁是当前实例对象。

（2）对于静态同步方法，锁是当前类的 Class 对象。

（3）对于同步方法块，锁是 Synchonized 括号里配置的对象。

7、对象头

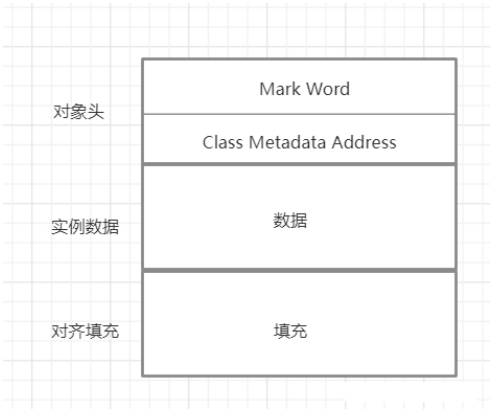
synchronized关键字的实现，依赖于Java的对象头。

一个对象由三部分组成：对象头、实体数据、对齐填充。对象头的长度不是固定的，如果是数据类型则对象头占12个字节，非数组类型对象头占8个字。

非数组类型的对象头分两部分，Mark Word 和对象类型指针，数组类型对象会多一部分来存储数组的长度。而synchronized关键字的实现，就和对象头中的Mark Word密切相关。

表 2-2　Java 对象头的长度

长　　度	内　　容	说　　明
32/64bit	Mark Word	存储对象的 hashCode 或锁信息等
32/64bit	Class Metadata Address	存储到对象类型数据的指针
32/32bit	Array length	数组的长度（如果当前对象是数组）



Java 对象头里的 Mark Word 里默认存储对象的 HashCode、分代年龄、是否偏向锁以及锁标记位。32位 JVM 的 Mark Word 的默认存储结构如下

锁状态	25bit	4bit	1bit 是否是偏向锁	2bit 锁标志位
无锁状态	对象的 hashCode	对象分代年龄	0	01

为了提高虚拟机空间的使用效率，Mark Word被设计成一个非固定的动态数据结构，以便存储更多的信息。不同状态下对象头Mark Word存储的信息如下图

锁状态	32bit				
	25bit		4bit	1bit	2bit
	23bit	2bit		偏向模式	标志位
未锁定	对象哈希码		分代年龄	0	01
轻量级锁定	指向调用栈中锁记录的指针				00
重量级锁定（锁膨胀）	指向重量级锁的指针				10
GC 标记	空				11
可偏向	线程 ID	Epoch	分代年龄	1	01

由图可知分为未锁定、偏向所、轻量级锁、重量级锁、GC，但是在轻量级锁升级为重量级锁时，还可能进行自旋。

synchronized锁的状态被分为4种，级别从低到高依次是：无锁、偏向锁、轻量级锁、重量级锁。

注意在锁标志中并没有出现自旋锁，它仅仅是锁可能存在的一种状态，是暂时性的，并没有官方的标志。32位虚拟机下，Mark Word的最后3位就可以判断锁的状态。

无锁和偏向锁标志位都是01，只是偏向锁时偏向模式会被置为1。锁可以升级但不能降级，意味着偏向锁升级成轻量级锁后不能降级成偏向锁。这种锁升级却不能降级的策略，目的是为了提高获得锁和释放锁的效率。

难道锁的不同仅仅是标志不同吗?

肯定不是，不同的锁各自实现的途径不同，适合的场景也各不相同。理解各种锁是怎样实现的，也就理解了synchronized关键字，理解了锁优化过程。

8、无锁

按最后3位标识位来判断，无锁应该是001状态，偏向标志为0，锁标志为01。

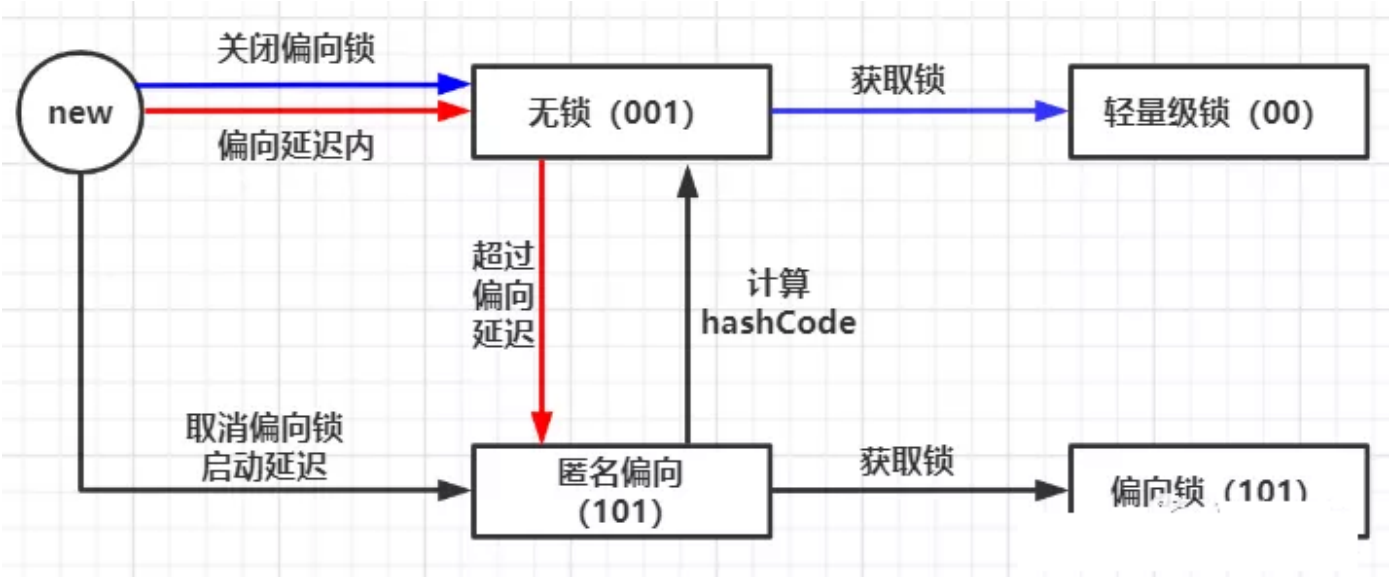
更准确的讲001状态是无锁不可偏，还有一种是101状态，无锁可偏（又叫匿名偏向）。无锁不可偏状态下遇到同步，会直接升级为轻量级锁，而不会变为偏向锁（看名字也知道，不可偏嘛）。只有在无锁可偏的状态下，才可能变成偏向锁。匿名偏向状态下虽然标识码是101，但是线程ID部分全部0，意味着没有线程实际获得偏向锁。

为啥要分无锁可偏和无锁不可偏呢?

因为所有偏向锁的起点就是匿名偏向状态，无锁不可偏状态会直接变为轻量级锁。

首先如果JVM设置取消偏向锁，那么无锁状态只可能是无锁不可偏。JDK8默认启动了偏向锁，但是偏向锁时在JVM启动几秒（默认4秒，但可以设置更改）之后才会启动，此时能设置为匿名偏向的会全部设置为匿名偏向，因此匿名偏向是偏向锁的起点。

而JVM为啥会在几秒之后才会启用偏向锁，这是因为JVM内部的代码会使用synchronized，这些类里有很多激烈线程竞争，如果采用锁升级策略这些锁会浪费很多时间。所以JVM索性先不开启偏向锁，先执行这些库类，等过几秒差不多都执行完了再开启偏向锁。至于JDK8为啥默认4秒，这是个经验值，4秒大多数类都启动完了，此数值可以修改。



9、偏向锁

大多数情况下，锁不仅不存在多线程竞争，而且总是由同一线程多次获得，为了让线程获得锁的代价更低而引入了偏向锁。

这个锁永远会偏向于获得它的线程，如果在获得锁之后并没有其他线程获取，则获得偏向锁的线程永远不需要同步，减少锁带来的时间消耗。

9.1、偏向锁的获取

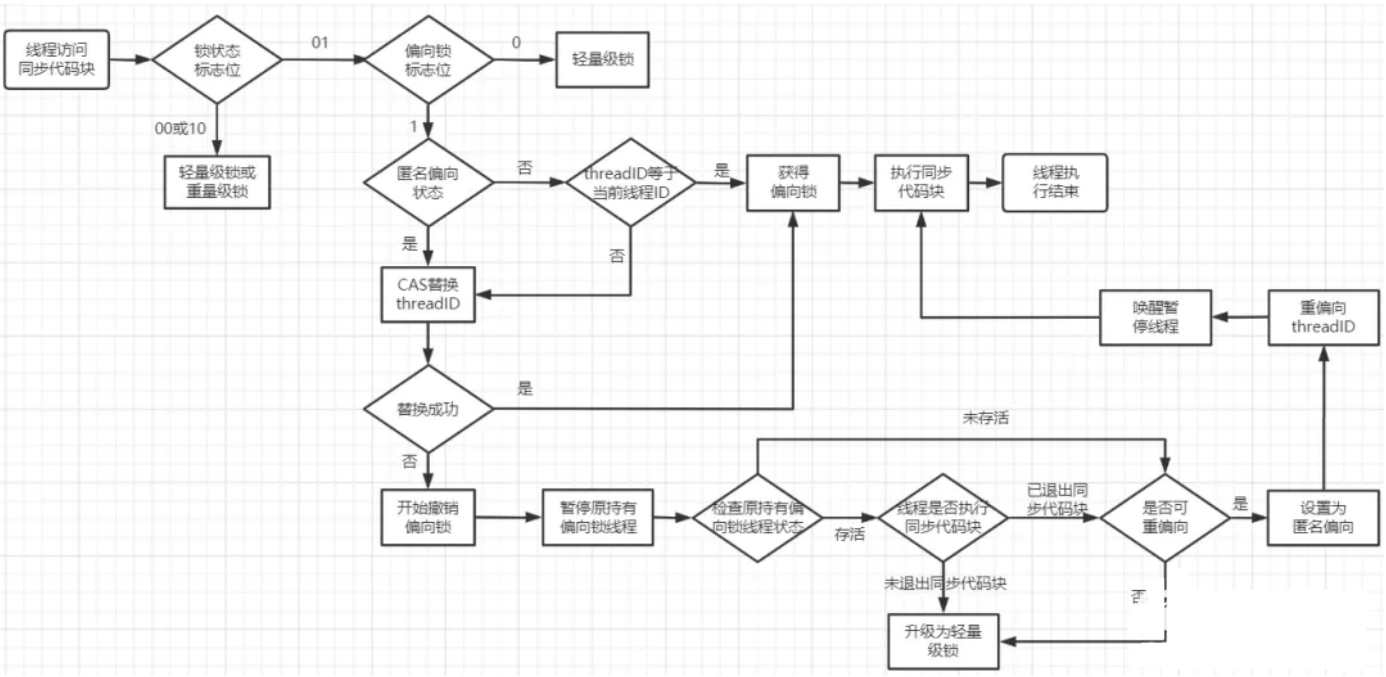
偏向锁对象头将不在存放Hash值，而在此位置上存放线程ID（23bit）+Epoch(2bit),一共25bit,其他部分保持不变。Epoch是一个时间戳，用来判断线程ID是否过时。具体获得锁流程如下：

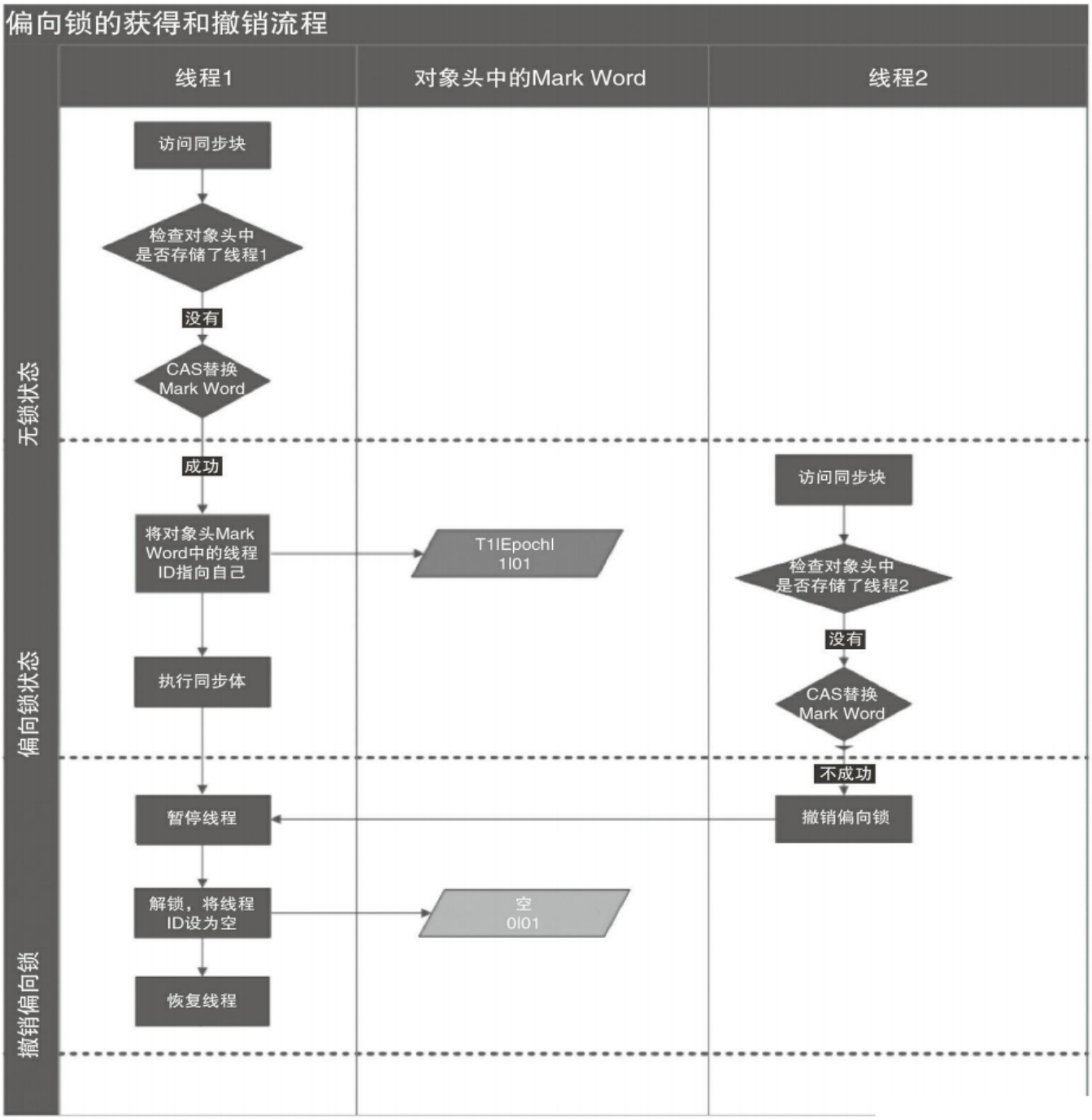
匿名偏向是偏向锁的初始状态，所以先判断锁标志，再判断偏向锁标志位，只有最后三位是101才开始，否则直接走其他的锁。如果是匿名状态，线程ID为0，采用CAS去将当前线程写入，如果成功则获得锁，不成功表示存在竞争。线程ID不为0，此前已经有偏向，判断此值是否和当前线程相同，若一致则表示线程之前就获得了锁，不一致就尝试CAS替换。同意，替换成功获得锁，替换失败存在竞争。未获得锁时将会等待安全点（STW），安全点会进行偏向锁的撤销。

安全点是JVM在进行垃圾GC时为了保证引用关系不会发生变化而设置的安全状态（GC Roots的确定就在此时），STW(STOP THE WORLD)此时将暂停所有线程的工作。

在STW会检测持有偏向锁的线程是否还存活，如果存活则升级轻量级锁，如果线程未存活或者已经退出来同步代码块，将会判断是否可重偏向，否则直接升级为轻量级锁。允许重偏向时会先设置为匿名重偏向，在使用CAS偏向线程。

判断是否可重偏向需要用到Epoch,偏向锁中有一个Epoch，对应的Class类中也有一个Epoch。在进入全局安全点之后，首先会对Class类中的Epoch进行增加，得到新的Epoch_new，然后扫描所有持有Class类实例的线程，根据线程信息判断是否锁住了该对象。如果锁住了说明此对象还在使用，将Epoch_new更新给它，如果未锁住则说明不需要加锁，不进行更新。如果对象的Epoch和类的Epoch相同，则表示它是被更新过的，需要锁，不能重偏向。而如果不相同，则表示已经不需要加锁了，此对象可以重偏向到其他线程。


线程访问
同步代码块
锁状态
标志位
01
偏向锁
标志位
0
轻量级锁
00或10
轻量级锁或
重量级锁
1
匿名偏向
状态
否
threadID等于
当前线程ID
是
获得
偏向锁
执行同步
代码块
线程执
行结束
是
否
CAS替换
threadID
唤醒暂
停线程
重偏向
threadID
替换成功
是
否
未存活
开始撤销
偏向锁
暂停原持有
偏向锁线程
检查原持有偏
向锁线程状态
存活
线程是否执行
同步代码块
已退出同
步代码块
是否可
重偏向
是
设置为
匿名偏向
未退出同步代码块
否
升级为轻量
级锁



9.2、偏向锁的释放

从偏向锁的获取过程可以看到，等到竞争出现的时候才会释放。如果没有出现竞争，它不会去改变Mark Word的相关字段。就算是线程已经执行完同步代码块，不需要加锁了，也不会去修改对象头，那个锁依旧存在，依旧保持偏向。

只是在其他线程需要偏向，出现了竞争的时候会进行判断，如果以前偏向的线程不需要了，那么对象首先会被设置为匿名偏向，然后CAS替换尝试加锁。如果以前偏向的线程还需要加锁，升级为轻量级锁。

所以线程不会主动的将偏向锁设置为匿名偏向状态，不会主动的去释放锁。

9.3、批量偏向与批量撤销

偏向锁有三个参数：

BiasedLockingBulkRebiasThreshold：偏向锁批量重偏向阈值，默认为20次

BiasedLockingBulkRevokeThreshold：偏向锁批量撤销阈值，默认为40次

BiasedLockingDecayTime：重置计数的延迟时间，默认值为25000毫秒（即25秒）

批量重偏向是以class而不是对象为单位的，每个class会维护一个偏向锁的撤销计数器，每当该class的对象发生偏向锁的撤销时，该计数器会加一，当这个值达到默认阈值20时，jvm就会认为这个锁对象不再适合原线程，因此进行批量重偏向。而距离上次批量重偏向的25秒内，如果撤销计数达到40，就会发生批量撤销，如果超过25秒，那么就会重置在[20, 40)内的计数。

当一个线程建立了大量的对象，并对他们都加了偏向锁。而此时若另一个线程也来获取这些对象，此时发生了竞争理论上都会升级轻量级锁。但是因为批量偏向的存在，并不会全部升级。

假设线程A建立了10个对象，全部加偏向锁，随后线程B同样也对这40个对象加锁。线程B对每一个对象进行加锁是，都会导致撤销一次偏向锁，升级为轻量级锁。当这个数值变为20后，JVM会认为其余的对象也不适合线程A，当后面的对象遇到需要同步的时候，会先被重置为可偏向状态，以便快速冲偏向。这样线程B对后面的对象加锁就不会升级为轻量级锁，而是偏向了线程B。

当线程C再来加锁的时候，前20个对象竞争轻量级锁，直接竞争，后20个锁是偏向线程B的，是偏向锁。此时不会触发批量重偏向，所以后20个也升级为轻量级锁。升级为轻量级锁就需要撤销偏向锁，加上之前的20次，一共40次。达到40次撤销偏向锁，会触发批量撤销机制，将偏向锁升级为轻量级锁，并且此类新建的对象都不是无锁不可篇状态，不会出现偏向锁。

9.4、偏向锁的优缺点

优点：

在只有单一线程访问对象的时候，偏向锁几乎没有影响。只有第一次需要CAS操作替换，随后的只要比较线程ID即可，比较方便快速。

缺点：

如果有多个线程访问，就会出现竞争，竞争需要等到安全点时，并且进行一系列分析比较耗费时间。另外，偏向锁存放线程ID和Epoch后，对象头中不存在Hash值，如果程序需要Hash值需要调用HashCode,这会导致偏向锁退出。

如果对象需要调用Object方法，会启用对象的minoter内置锁.。此时会直接由偏向锁退出进入重量级锁。

10、轻量级锁

轻量级锁是相对于使用操作系统互斥量来实 现的传统锁而言的，因此传统的锁机制就被称为“重量级”锁。轻量级锁并不是用来代替重量级锁的，它设计的初衷是在没有多线程竞争的前提下，减少传统的重量级锁使用操作系统互斥量产生的性能消耗。

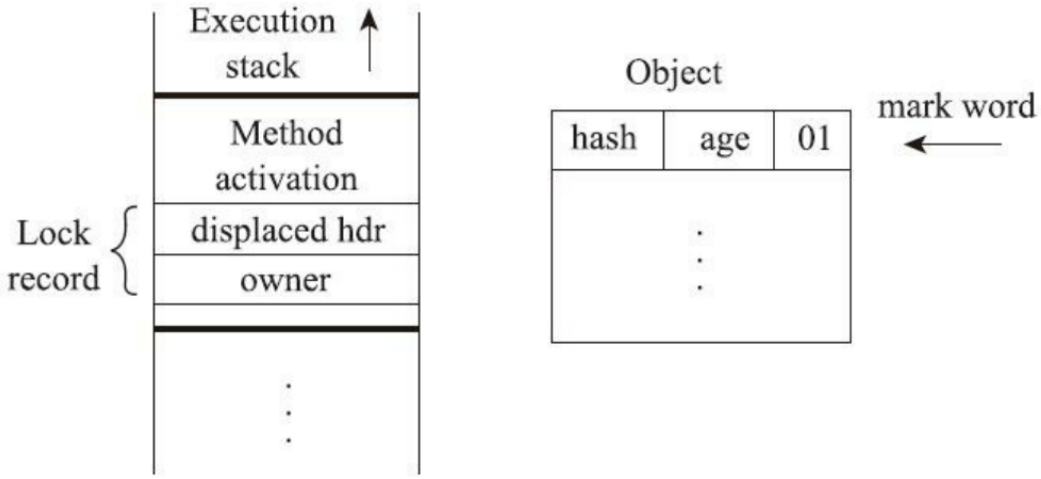
10.1、轻量级锁的获取

虚拟机首先将在当前线程的栈帧中建立一个名为锁记录（Lock Record）的空间，此空间包含两部分：

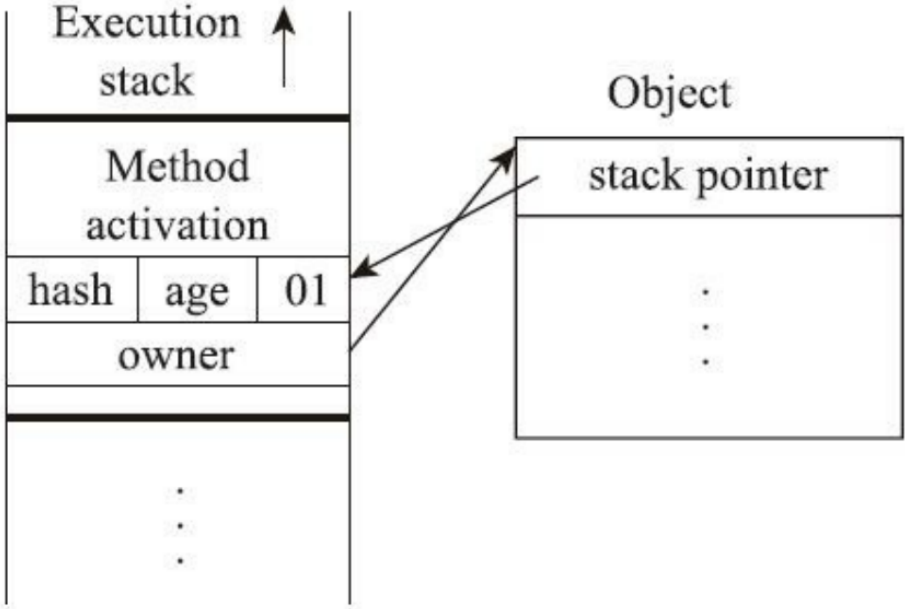
displaced mark word：用于存储锁对象目前的Mark Word的拷贝

owner:指向当前的锁对象的指针

虚拟即首先会将对象的Mark World拷贝到栈帧中的Lock Record，此时owner为空。



然后虚拟机使用CAS操作尝试把对象的Mark Word更新为指向Lock Record的指针（此空间占30bit, 除了最后两位锁标志，Mark Word前30bit均用来存放指针），如果操作成功则代表线程拥有了这个对象锁，对象锁标志位变为00.如失败了则会会判断对象Mark Word存储的指针是否指向自己，如果是则表示拥有锁，执行同步代码，如果不是则意味着其他线程抢占，出现激烈竞争需要升级为重量级锁。



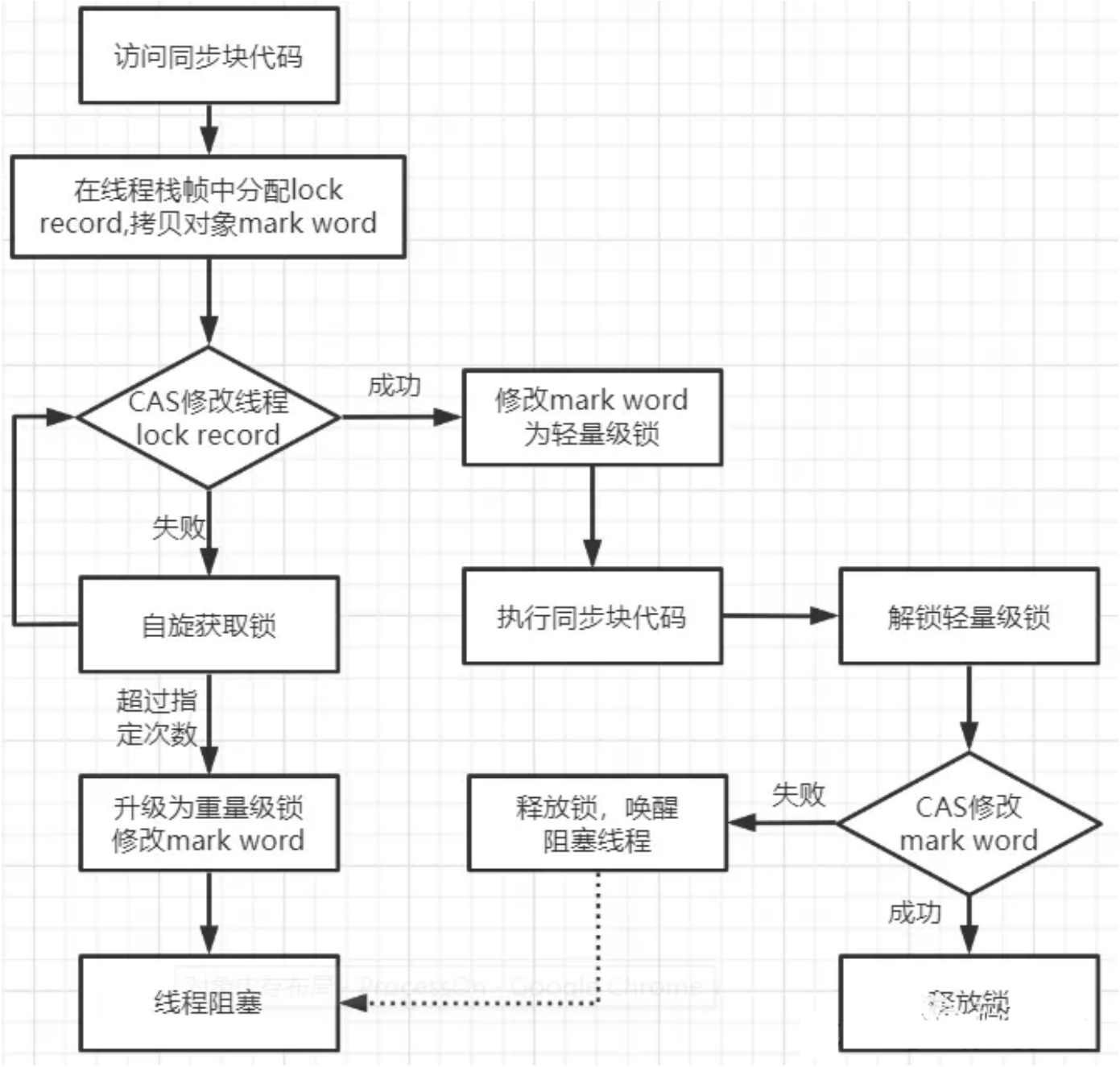
10.2、轻量级锁的释放

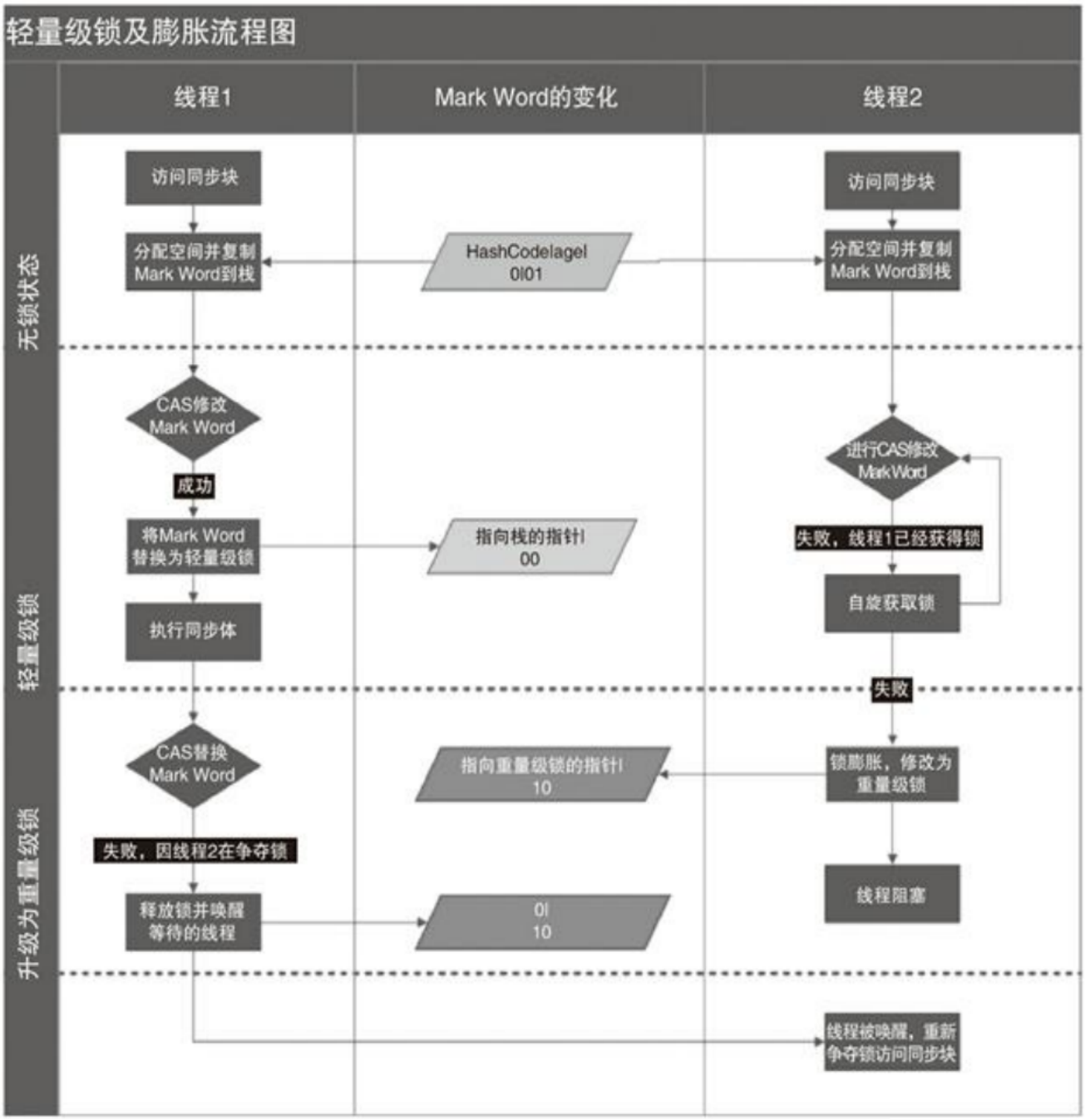
使用CAS尝试将Lock Record中的displaced mark word替换回去，需要检查对象头中的指针是否指向当前线程。

如果替换成功，表示没有竞争，锁成功释放

如果替换失败会进行自旋，如果自旋之后仍未获得锁表示存在竞争并升级为重量级锁。

当其他线程竞争轻量级锁时，并不会对已经持有轻量级锁的线程发送什么，而是对象头的锁标志被修改，同时竞争线程自身被挂起。因此如果CAS替换失败，原本持有锁的线程除了释放锁之外，还需要唤醒被挂起的线程。


访问同步块代码
在线程栈帧中分配lock record,拷贝对象mark word
CAS修改线程 lock record
成功
修改mark word 为轻量级锁
失败
自旋获取锁
执行同步块代码
解锁轻量级锁
超过指定次数
升级为重量级锁 修改mark word
释放锁，唤醒阻塞线程
失败
CAS修改 mark word
成功
线程阻塞
释放锁

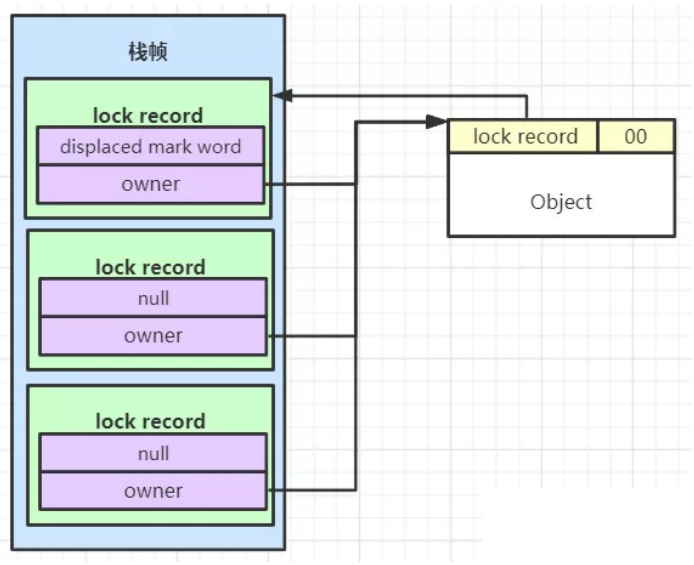


10.3、轻量级锁的重入

轻量锁的每一次重入，都会在栈中生成一个Lock Record。只是只有第一次会拷贝Mark Word，随后的加锁Displaced Mark Word区域为NULL,owner区域统一指向对象头。

线程第一次获得锁时，将对象的Mark Word拷贝到本线程Lock Record，同时将对象的指针指向自己，Lock Record中的owner也指向对象。随后的加锁对象中存储的是指向本线程的指针，并没有Mark Word，也就不需要拷贝。只是将owner指向对象即可。

每加一次锁帧栈中多一个Lock Record，Lock Record的个数也就是加锁的次数。释放锁时也要一个一个释放，只有解锁次数等于加锁次数，才会真正释放锁。释放锁时，如果是重入锁则直接删掉一个Lock Record，如果不是重入则采用CAS替换对象的Mark Word。



10.4、轻量级锁自旋

当轻量级出现竞争以后，会尝试进行自旋。自旋就是CPU空转，线程没有挂起依然在执行，等过一段时间后再去加锁。

这是因为如果升级为重量级锁，是通过操作系统来实现，涉及到内核态和用户态之间的切换，这个操作的比较耗时。

如果竞争没有那么激烈，锁住的同步代码块执行的时间还没有切换上下文花的时间多，反而得不偿失。因此采用自旋锁，出现竞争之后等一等再去尝试，可能前面获得锁的线程已经执行完了，再次加锁。这样就免去了升级重量锁带来的消耗。

自旋不能一直进行。自旋时CPU是空转，这就浪费了处理器资源。上面的情况是竞争不激烈，但假设竞争激烈那么自旋完全是浪费时间，还不如直接升级到重量级锁省资源。以前的自旋次数默认是10，如果10次之后依然不行说明竞争很激烈，需要升级到重量级锁。

JDK1.6以后加入了自适应自旋：

对于某个锁对象，如果自旋等待刚刚成功获得过锁，并且持有锁的线程正在运行中，那么虚拟机就会认为这次自旋也是很有可能再次成功，进而允许自旋等待持续相对更长时间

对于某个锁对象，如果自旋很少成功获得过锁，那在以后尝试获取这个锁时将可能省略掉自旋过程，直接阻塞线程，避免浪费处理器资源。

除此之外，JVM还会根据CPU的负载进行优化：

如果平均负载小于CPUs则一直自旋

如果有超过(CPUs/2)个线程正在自旋，则后来线程直接阻塞

如果正在自旋的线程发现Owner发生了变化则延迟自旋时间（自旋计数）或进入阻塞

如果CPU处于节电模式则停止自旋

自旋时间的最坏情况是CPU的存储延迟（CPU A存储了一个数据，到CPU B得知这个数据直接的时间差）

自旋时会适当放弃线程优先级之间的差异

10.5、轻量级锁自旋

优点：

适合多个线程不同时访问同步对象场景。偏向锁的撤销必须在安全点才可以进行，假设多个线程交替访问某个对象，处理完成后又释放锁，这种情况下偏向锁也可以完成但是效率会很低，每次撤销再加锁必须等到安全点，同时还要进行Epoch的分析，因此偏向锁才会有批量撤销机制，撤销偏向锁次数过多则意味偏向锁不适用，不再新增偏向锁而直接变成轻量级锁。轻量锁则很方便，直接CAS替换就可以，对这种多线程竞争不激烈的场景很适用。锁的自旋也是为此设计。

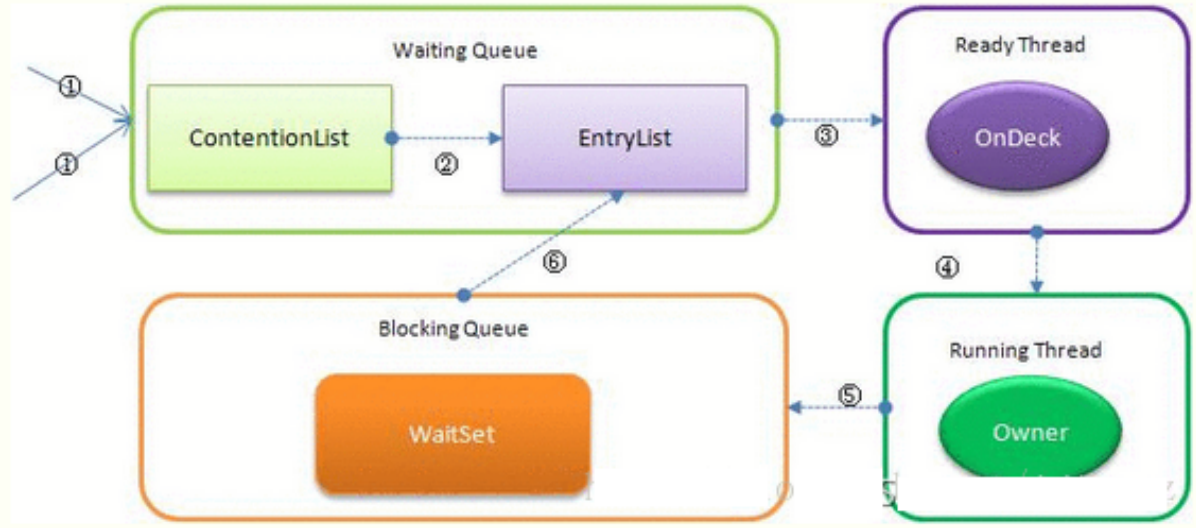
缺点：

竞争激烈的场景下不适用，此时进行自旋就是再浪费CPU资源。竞争激烈时可能进行很多次的自旋都不回获得锁，这种浪费的代价比上下文切换的代价要大，所以引入自适应自旋来裁决。

11、重量级锁

重量级锁的实现依赖于ObjectMonitor，而ObjectMonitor又依赖于操作系统底层的Mutex Lock（互斥锁）实现。

Monitor可以理解为一个同步工具或一种同步机制，通常被描述为一个对象。每一个Java对象就有一把看不见的锁，称为内部锁或者Monitor锁。主要包含以下几部分：



Contention List：竞争队列，所有请求锁的线程首先被放在这个竞争队列中；

Entry List：Contention List中那些有资格成为候选资源的线程被移动到Entry List中；

Wait Set：哪些调用wait方法被阻塞的线程被放置在这里；

OnDeck：任意时刻，最多只有一个线程正在竞争锁资源，该线程被成为OnDeck；

Owner：当前已经获取到所资源的线程被称为Owner；

!Owner：当前释放锁的线程。

count：monitor的计数器，数值加1表示当前对象的锁被一个线程获取，线程释放monitor对象时减1

11.1、重量级锁的获取

锁升级为重量级之后，Mark Word中存储的指针不再指向线程，而是指向ObjectMonitor。当线程访问同步代码块时，每个线程都会被封装成一个ObjectWaiter对象进入monitor。

JVM每次从队列的尾部取出一个数据用于锁竞争候选者（OnDeck），但是并发情况下，ContentionList会被大量的并发线程进行CAS访问，为了降低对尾部元素的竞争，JVM会将一部分线程移动到EntryList中作为候选竞争线程。

Owner线程会在unlock时，将ContentionList中的部分线程迁移到EntryList中，并指定EntryList中的某个线程为OnDeck线程（一般是最先进去的那个线程）。

Owner线程并不直接把锁传递给OnDeck线程，而是把锁竞争的权利交给OnDeck，OnDeck需要重新竞争锁。这样虽然牺牲了一些公平性，但是能极大的提升系统的吞吐量，在JVM中，也把这种选择行为称之为“竞争切换”。

OnDeck线程获取到锁资源后会变为Owner线程，而没有得到锁资源的仍然停留在EntryList中。如果Owner线程被wait方法阻塞，则转移到WaitSet队列中，直到某个时刻通过notify或者notifyAll唤醒，会重新进去EntryList中。

处于ContentionList、EntryList、WaitSet中的线程都处于阻塞状态，该阻塞是由操作系统来完成的（Linux内核下采用pthread_mutex_lock内核函数实现的）。

Synchronized是非公平锁。 Synchronized在线程进入ContentionList时，等待的线程会先尝试自旋获取锁，如果获取不到就进入ContentionList，这明显对于已经进入队列的线程是不公平的，还有一个不公平的事情就是自旋获取锁的线程还可能直接抢占OnDeck线程的锁资源。

为啥采取非公平锁，主要还是考虑到性能。采取非公平锁下一次获得锁线程可能还是原来持有锁的线程，这样就避免了内核和用户态的切换，节省时间。

11.2、重量级锁的重入

线程如果获取到锁，会判断是否重入，如果是重入锁，count计数+1，释放锁时count数值减1。

11.3、重量级锁的优缺点

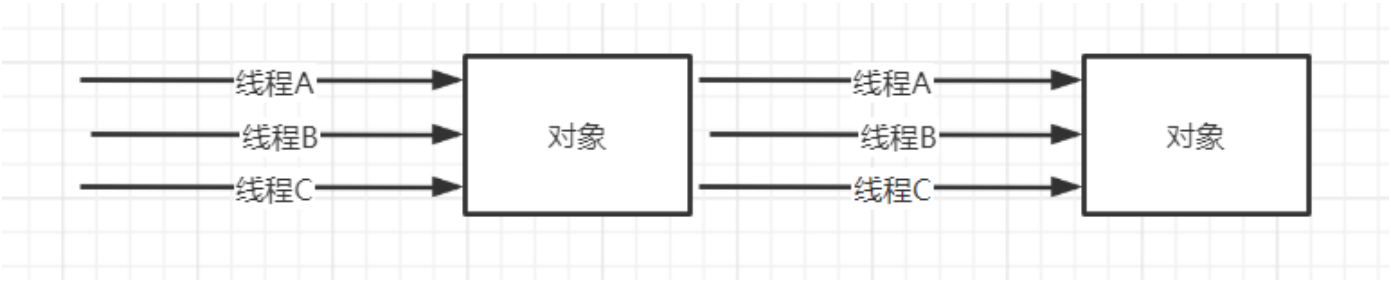
重量级锁需要内核态和用户态的切换，这个代价很大。所以把它放在最后，经过偏向和轻量级之后才是它。

但是只有它能应对竞争激烈的场景，也算是JVM最后的杀手锏了。

Synchronized 反向理解

在总结前文吃透synchronized关键字中，锁升级介绍是由低到高，一步一步的出发。其实也可以反其道而行之，想想怎么锁降级，这样能更好的理解锁升级的思路。

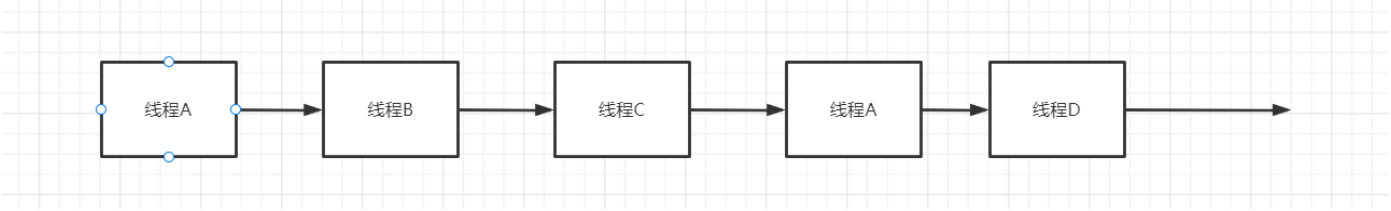
原先synchronized关键字就是现在的重量级锁，不论遇到何种情况都是重量级锁，需要内核态和用户态之间的相互切换。在JDK1.6之后引入了锁升级，分为无锁-偏向锁-轻量级锁-重量级锁。从开发人员角度想，为啥要这么做呢?



无锁状态和重量级锁肯定好理解，一个最轻，一个最重。记住无论何时何地，只要出现两个线程一直争夺同一个锁，必然是用重量级锁。不管原先是无锁、偏向还是轻量级，出现两个线程竞争同一个锁，最后一定是启动重量级锁。但不是任何时候都会有多个线程来竞争同一个锁，像这种竞争激烈场景在全部程序中并不太多。

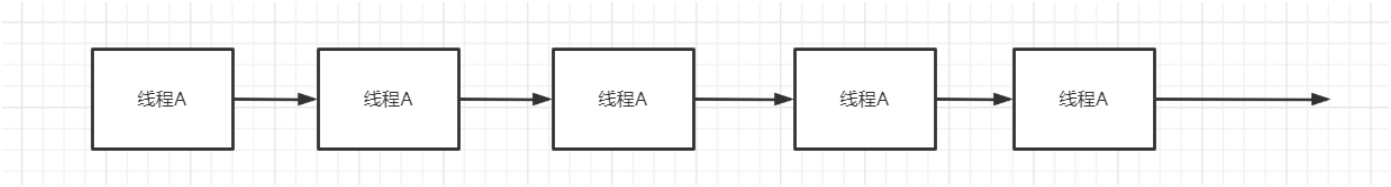
假设同一个对象，一会线程A得到，执行完成之后释放，之后线程B得到，执行完成之后释放，甚至可能还有线程CDEFG。根本没有竞争，只是单纯的加个锁然后释放，这种情况下用重量级锁就浪费了，杀鸡用牛刀，不划算。那就锁简化一下，不用重量级的等待阻塞队列，也不用内核态和用户态切换了。也不需要对象头里指向mionter的指针了，直接把这个存放线程的指针。在线程的帧栈中加个Lock Record，方便轻量级锁的判断。设么？用轻量级锁有些时候还会出现竞争，还只是短暂地？切换线程多不划算，轻量级锁自旋吧，多等一会说不定就没竞争了。万一

这不是短暂地竞争呢？最初是限制自旋次数，又来用适应性自旋，更灵活的判断。



绝大多数连竞争也没有？单个线程访问某个对象，这个对象到死都没见过其他线程。线程每次加锁又释放，费那么大力气建Lock Reocrd，还采用CAS的方式。杀鸡用牛刀，这可太亏了。把锁再简化一点，轻量级里还复制Mark Word，干脆啥都不要了。直接在对象头也写线程ID，线程访问的时候对比一下和自己的一样不一样就行，多简单。锁干脆不释放了，就算不用了也懒得改，等下次那个线程要偏向自己去对象头里把线程ID写成它的就行。

不对啊，单线程访问的省心了，但还是有其他情况啊。万一锁多个对象，其中一部分已经可以释放了，另一部分还必须拿着，偏向锁咋区分这种？简单，给个Epoch值，进入STW的时候看看那些对象还需要，给他们新的Epoch，不需要的就不管。Epoch是不能重偏向，旧的就可以重偏向，哪个线程需要了就可以改过去。万一线程A和线程B交替访问一个对象，或者现场A锁住大量对象后线程B也来锁这些对象，这压根不是偏向锁设计的初衷，引出批量重偏向和批量撤销。



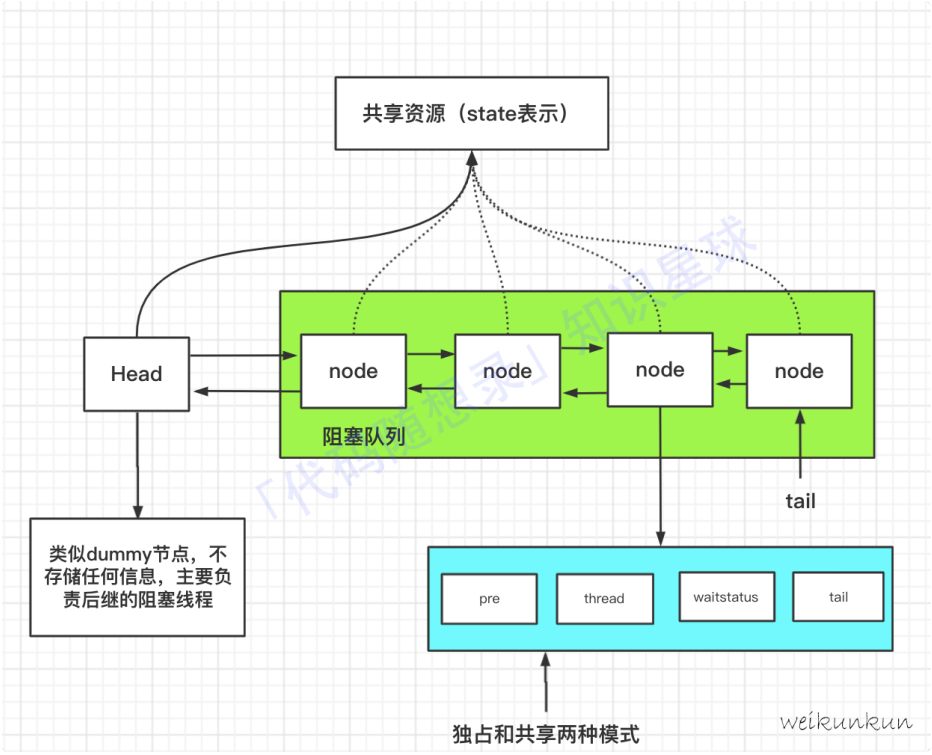
另外，出现竞争系统是怎么判断的？是某个线程想要加锁却被人占有，但是原先持有锁的线程并不知情。原先持有轻量锁的想要释放锁的时候一看，变重量级锁了，已经有一堆线程排着等了。锁释放掉，还得把队列里睡眠都线程唤醒，让他们竞争锁。偏向锁升级轻量级锁，需要到STW才能偏向撤销，还需要回复对象的Mark Word。

AQS

1、概念：

AQS，一个用来构建锁和同步器的框架，许多的经典的同步器都是基于AQS构建出来的，譬如CountDownLatch、Semaphored。

其内部使用同步队列帮我们解决了实现同步器时的一些细节问题，譬如何时阻塞线程，如何按照顺序唤醒线程。



2、核心思想：

AQS通过一个被volatile修饰的int类型的成员变量表示同步状态（共享资源），然后使用cas对这个同步状态进行修改，从而保证线程安全。

如果被请求的共享资源空闲，则将当前的请求线程设置为有效的工作线程，然后将对应的共享资源设置为锁定状态。

如果共享资源被占用，则通过CLH同步队列将暂时用不到的线程封装成一个节点加入到队列中，同时在适当的时候对其进行阻塞和唤醒。

基于AQS实现的应用（目前接触到的）

（1）lock

区别于synchronozed的锁，同时还支持了更加的灵活丰富操作，譬如可定时的尝试获取锁（避免了死锁的发生）、可中断的获取锁。

（2）CountDownLatch

一个或一组操作等其他操作执行结束之后，继续执行。

（3）Semaphore

用于多个共享资源的互斥，或者用于控制并发数量

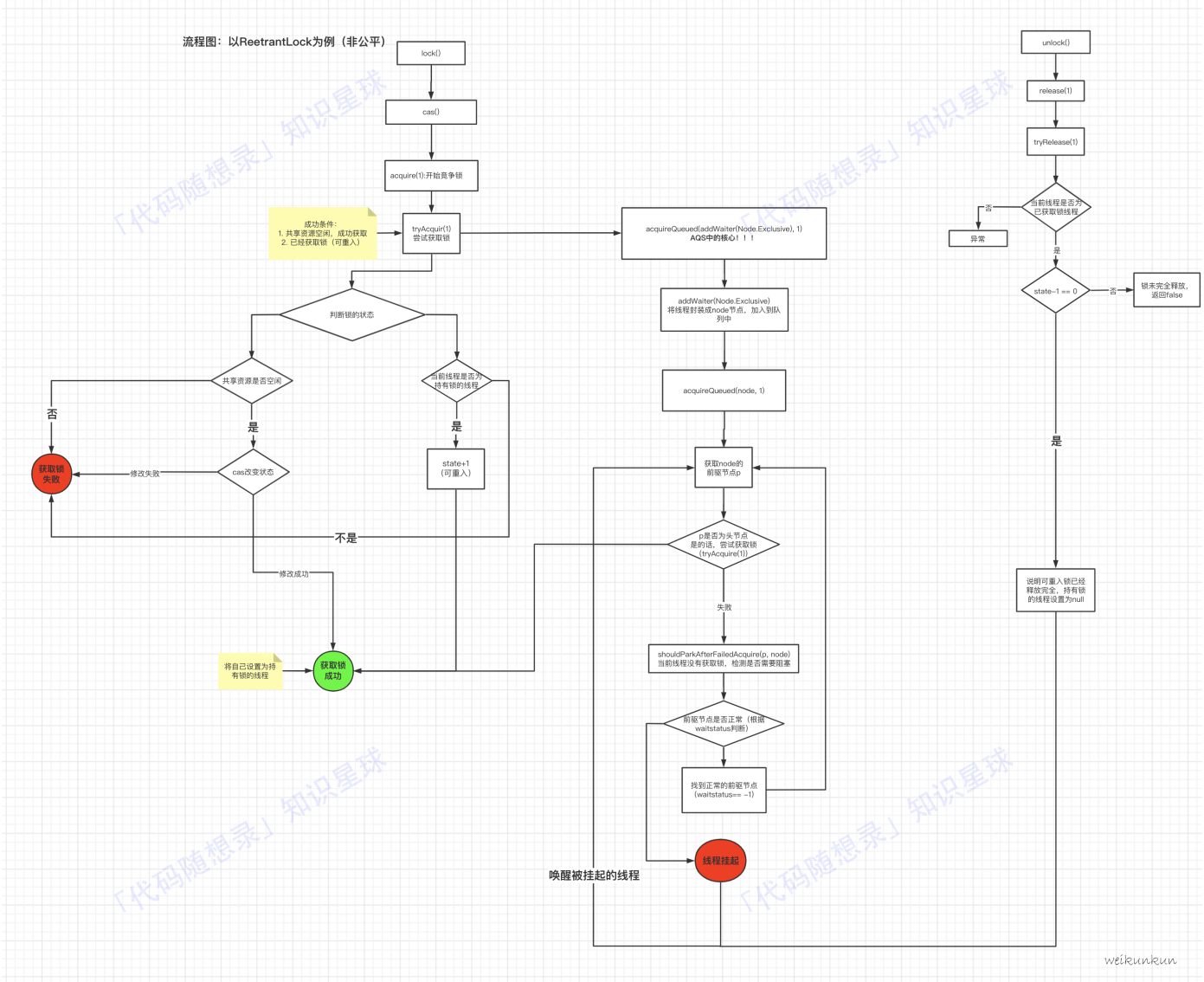
3、内部细节：

1. 目前理解是通过waitStatus来标志线程的阻塞、唤醒等状态，但是每个节点的waitStatus状态不是通过自身设置的，而是通过其后继节点设置的。
2. 内部的同步队列采用的数据结构是双向链表，同时Head节点属于dummy节点，不存储信息，仅表示当前持有锁的线程，同时还会负责后续阻塞线程的唤醒
3. exclusiveOwnerThread变量，标志当前持有锁的线程，防止错误的释放锁，这种思想在Redis分布式锁中也有体现

4. state是用volatile修饰的int型变量，这样其除了能够实现独占锁之外，还能实现Semaphore、CountDownLatch这样的同步工具
5. 对外提供了使用protected修饰的方法tryAcquire、tryRelease等，这样在实现的子类中，能够通过这些方法，实现一些我们自定义的内容，譬如锁的可重入，公平与非公平锁。

基于ReentrantLock的独占式共享锁的整个流程：

原图链接：「代码随想录」知识星球



JDK

java 并发 中的 ThreadLocal

有多个线程(孩子)，争抢同一个共享变量(玩具)，就会产生冲突，而程序的解决办法是加锁(父母说服，讲道理，轮流玩)，但加锁就意味着性能的消耗(父母比较累)。

所以有一种解决办法就是避免共享(让每个孩子都各自拥有一个变形金刚)，这样线程之间就不需要竞争共享变量(孩子之间就不会争抢)。

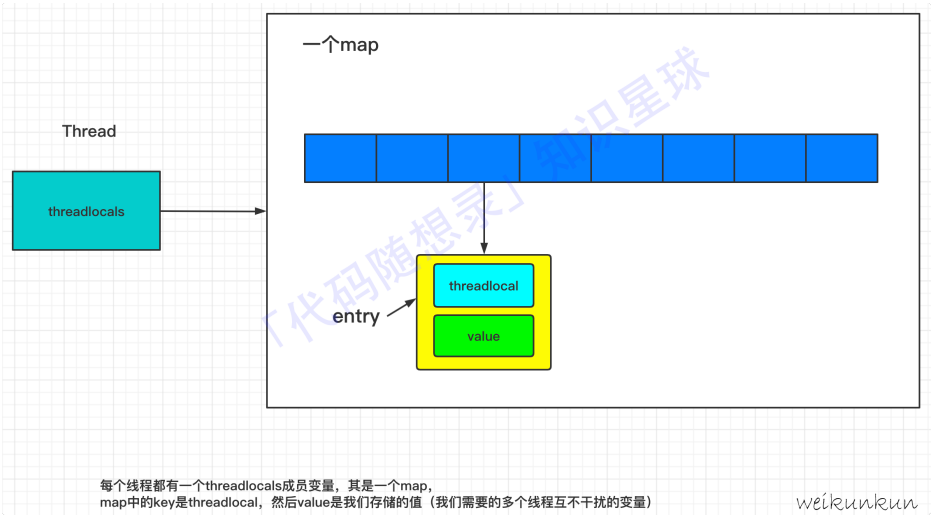
所以 ThreadLocal 的存在就是为了通过本地化资源来避免共享，避免了多线程竞争导致的锁等消耗。

在每个线程内部都有一个名为threadLocals的成员变量，该变量的类型为HashMap，其中key为我们定义的ThreadLocal变量的this 引用，value 则为我们使用set方法设置的值。每个线程的本地变量存放在线程自己的内存变量threadLocals中，如果当前线程一直不消亡，那么这些本地变量会一直存在，所以可能会造成内存溢出，因此使用完毕后要记得调用ThreadLocal的remove方法删除对应线程的threadLocals中的本地变量。

ThreadLocal

概述：

ThreadLocal提供了一种方式，让在多线程环境下，每个线程都可以拥有自己私有的数据结构，进而减少同一个线程内多个函数或者组件之间一些公共变量的传递的复杂度。但因为其自身的实现机制，使用之后记得及时remove，避免内存泄漏。



每个线程都有一个threadLocals变量，该变量的数据结构是threadLocalMap，而map中的每个entry中key是threadLocal，value是我们需要存储的本地变量

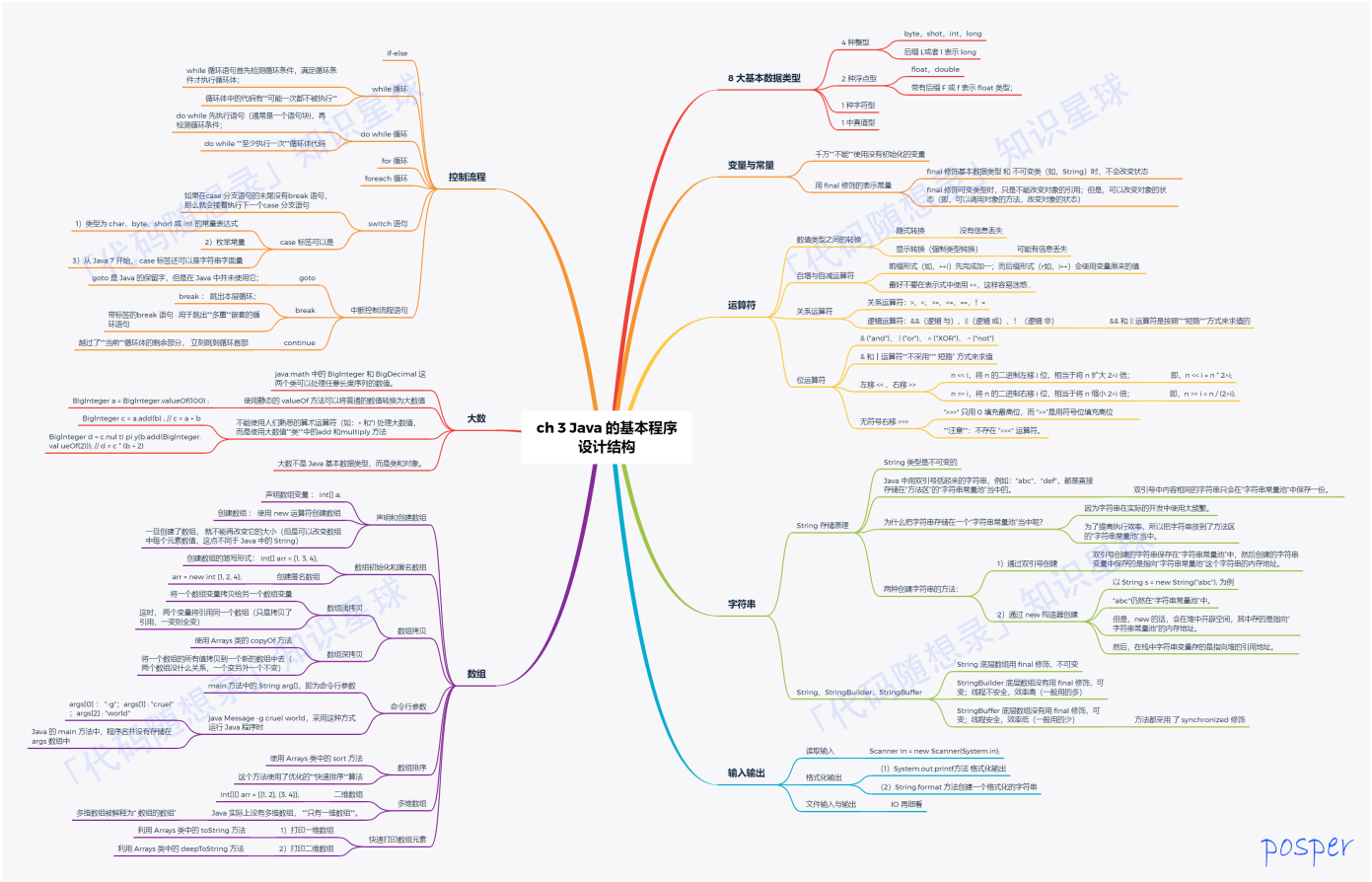
部分细节：

1. get操作，因为threadLocal是一个key，所以每次获取其映射的value时，首先先获取当前线程的threadLocalMap，然后再根据该map和key获取对用的value
2. set操作，同理也需要先根据threadLocal这个key获取map，然后通过map才能真正的将threadLocal变量和value进行映射
3. threadLocalMap和HashMap差不多，使用的数据结构都是hash表，不过对于hash冲突的解决方式，threadLocalMap使用的是线型探测，装载因子采用的是黄金分割点，然后hash算法使用的是斐波那契散列发，扩容方式均为原Entry数组的2倍
4. 因为threadLocalMap的中的key也就是threadLocal是弱引用，所以只要触发GC，key就会被回收，这样ThreadMap中的key就变成了null，但是value被Entry引用，Entry被ThreadLocalMap引用，ThreadLocalMap被Thread引用，这也就说明了只要，线程不终止，value的值一直无法被回收，所以可能会出现内存泄漏的现象，所以使用完之后，要及时调用remove方法。
5. 如果想子线程继承父线程的本地变量，可以采用InheritableThreadLocal，感觉这个在中间件里面用的会很多，没验证过。

思维导图

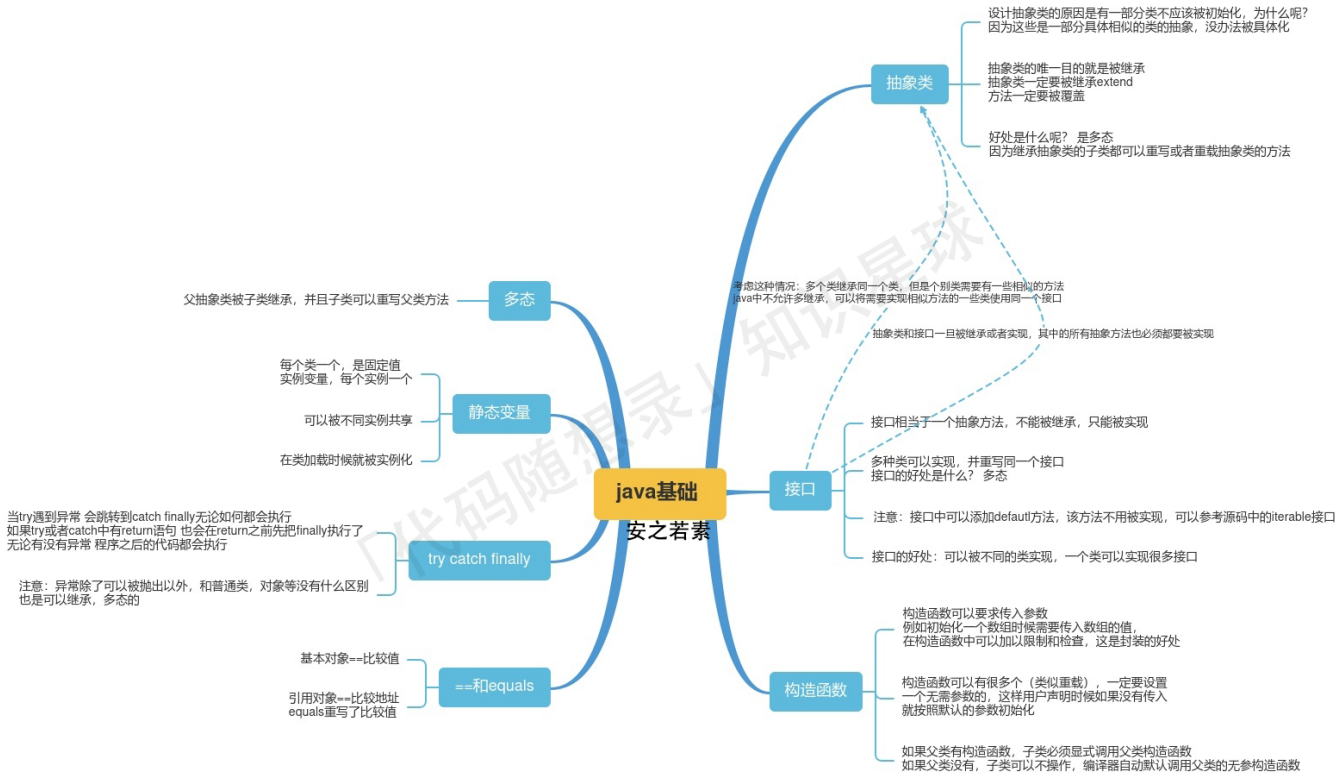
Java程序设计基本结构

原图链接：「代码随想录」知识星球



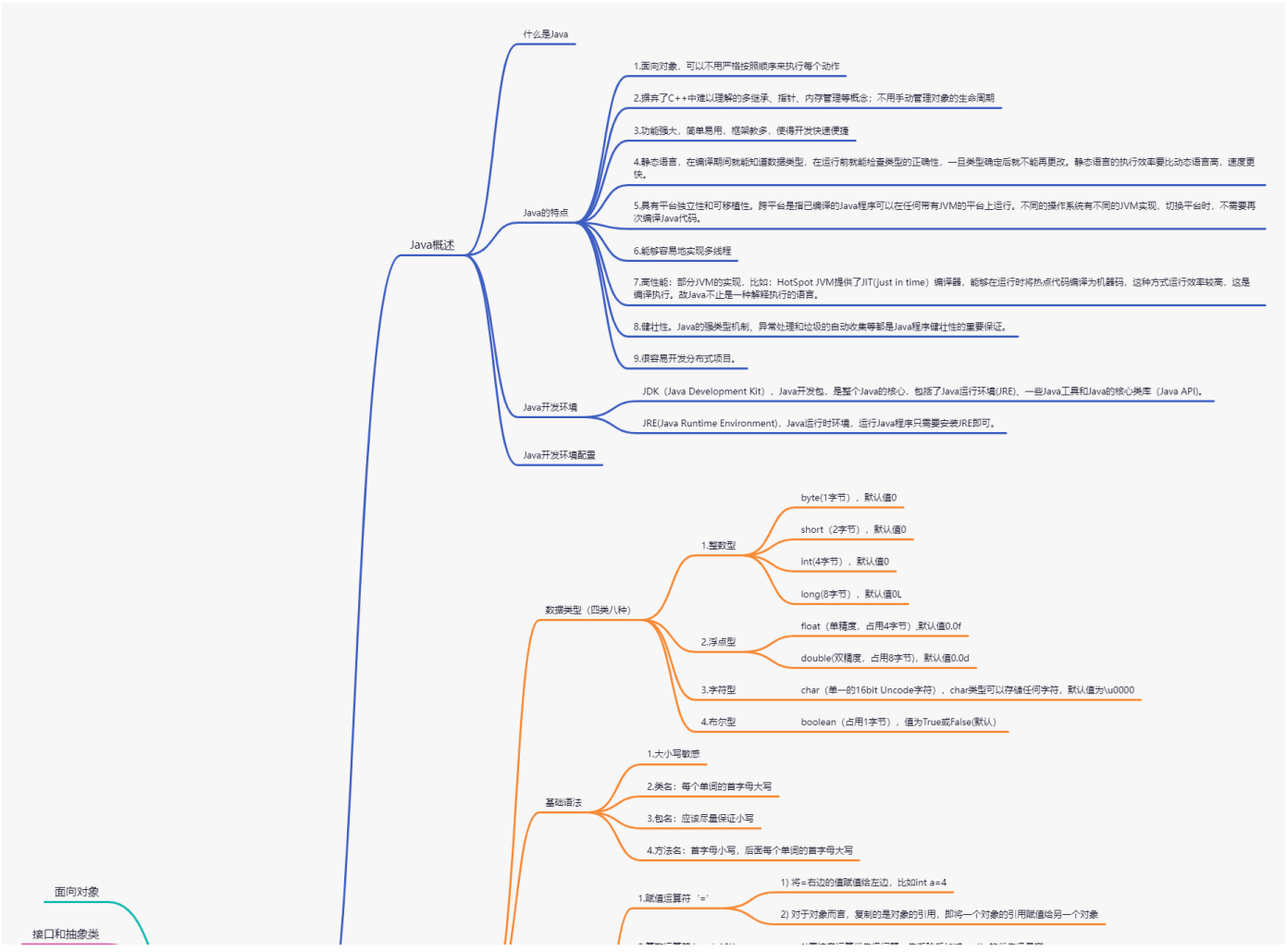
Java 基础总结

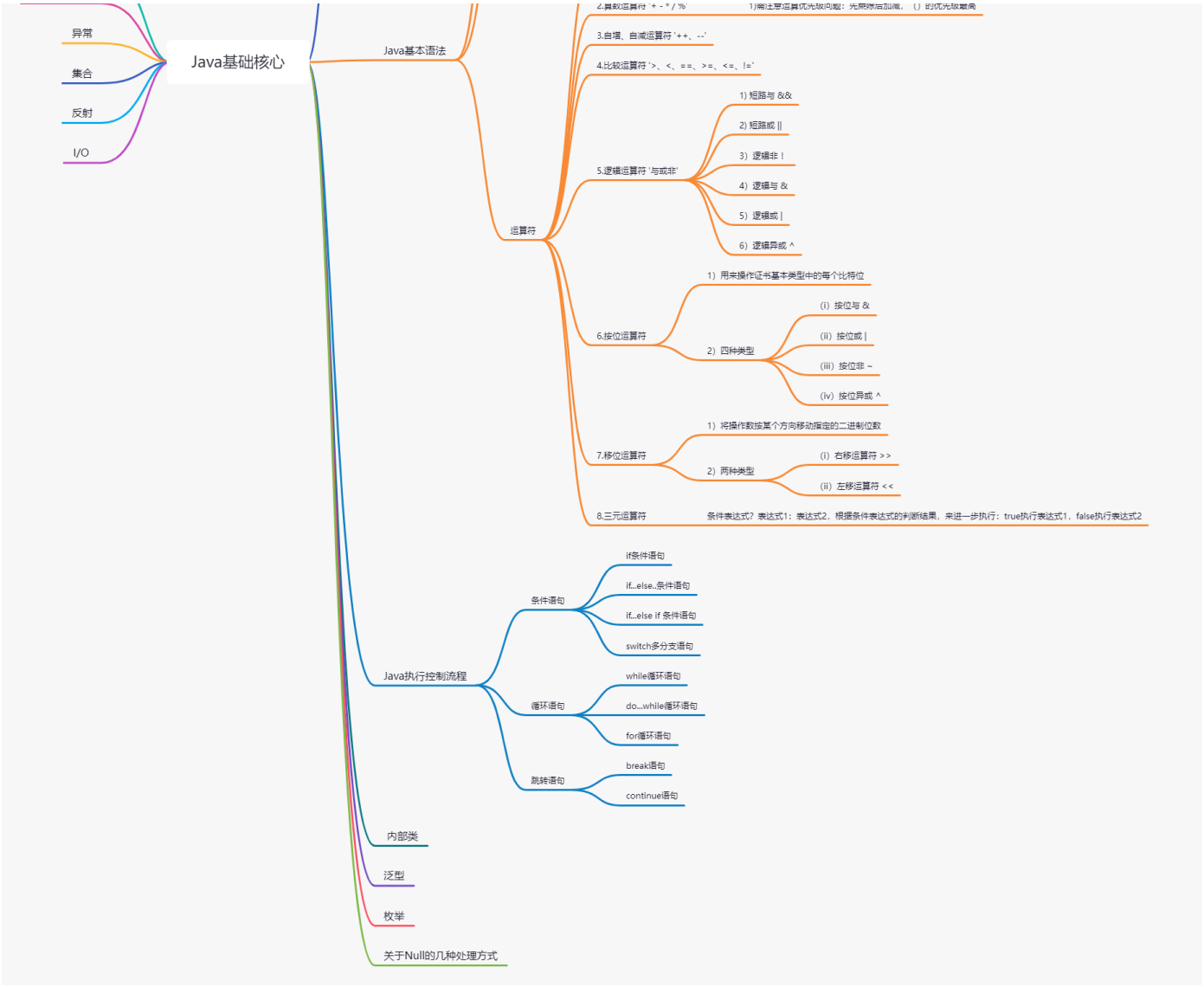
原图链接：「代码随想录」知识星球



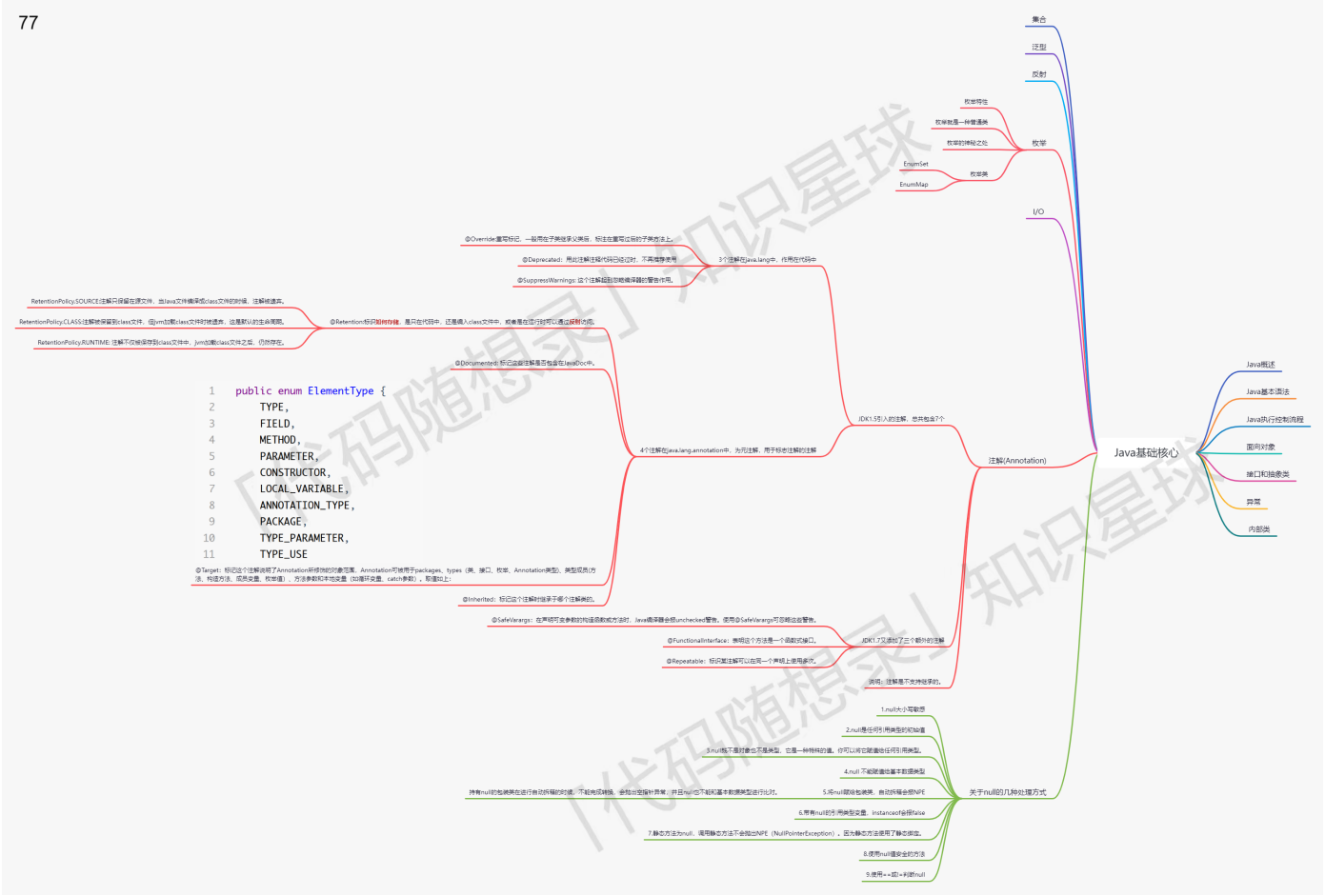
Java基础总结

原图链接：[「代码随想录」知识星球]



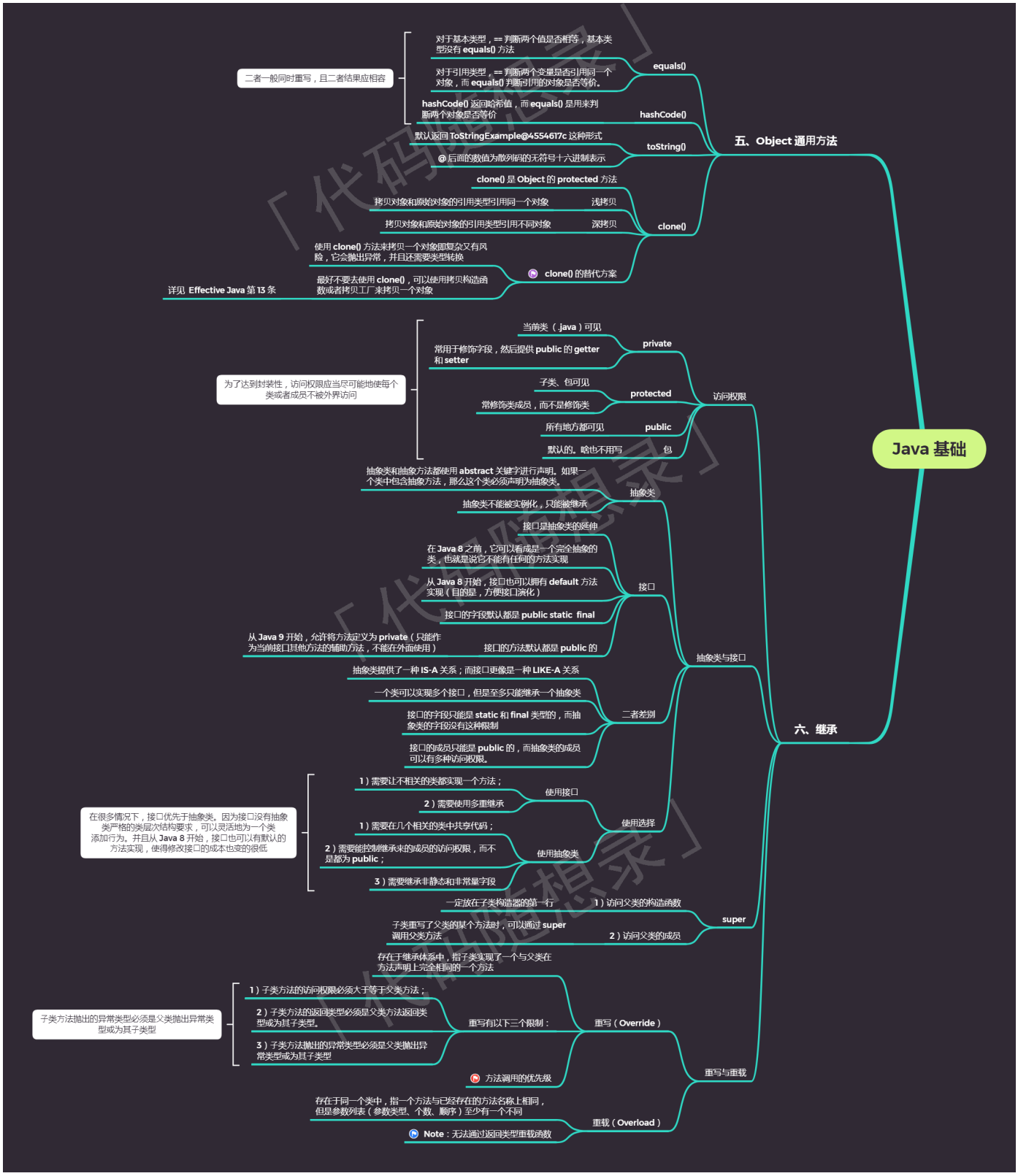


原图链接：[「代码随想录」知识星球]



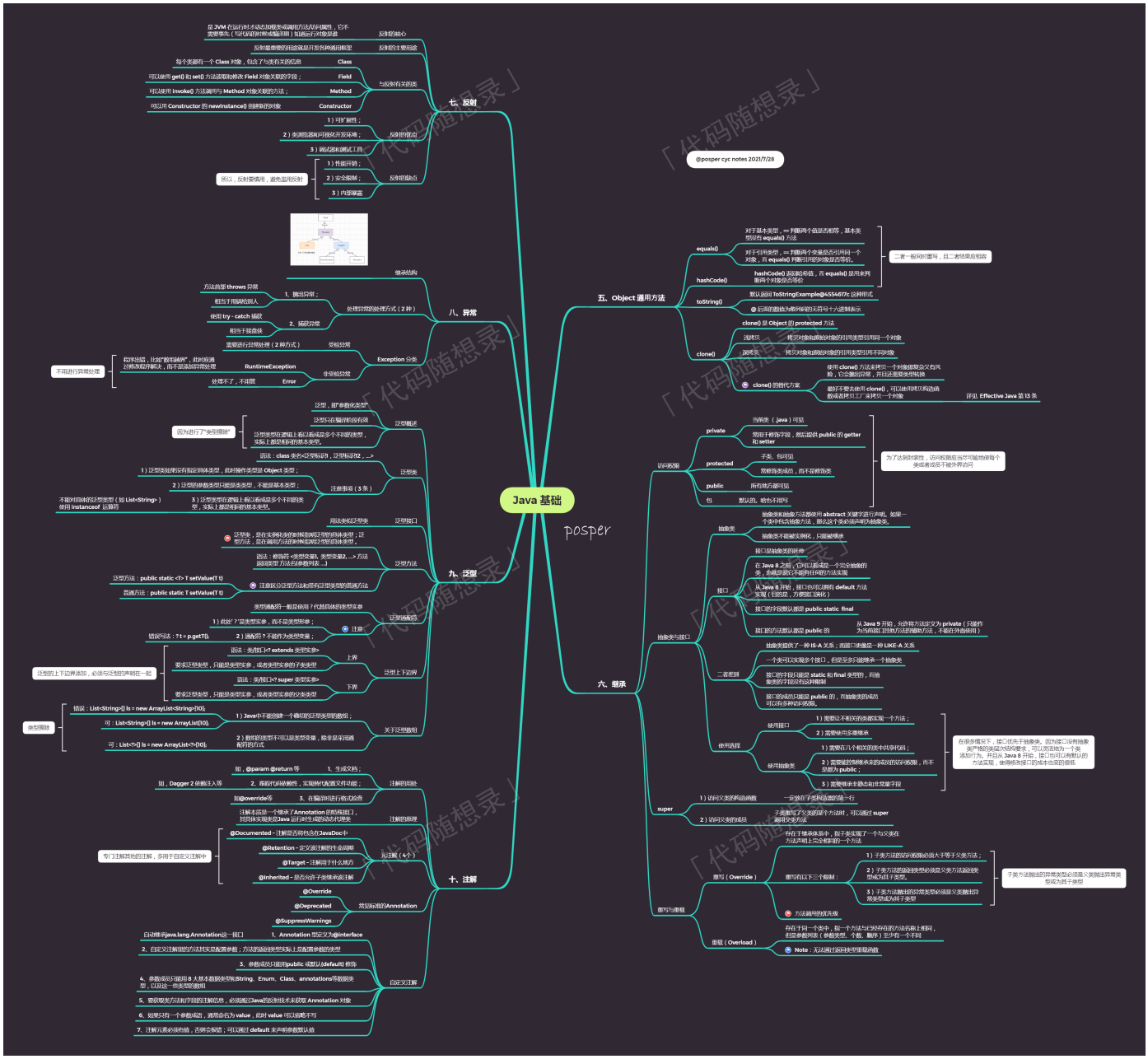
Java基础总结

原图链接：[「代码随想录」知识星球]



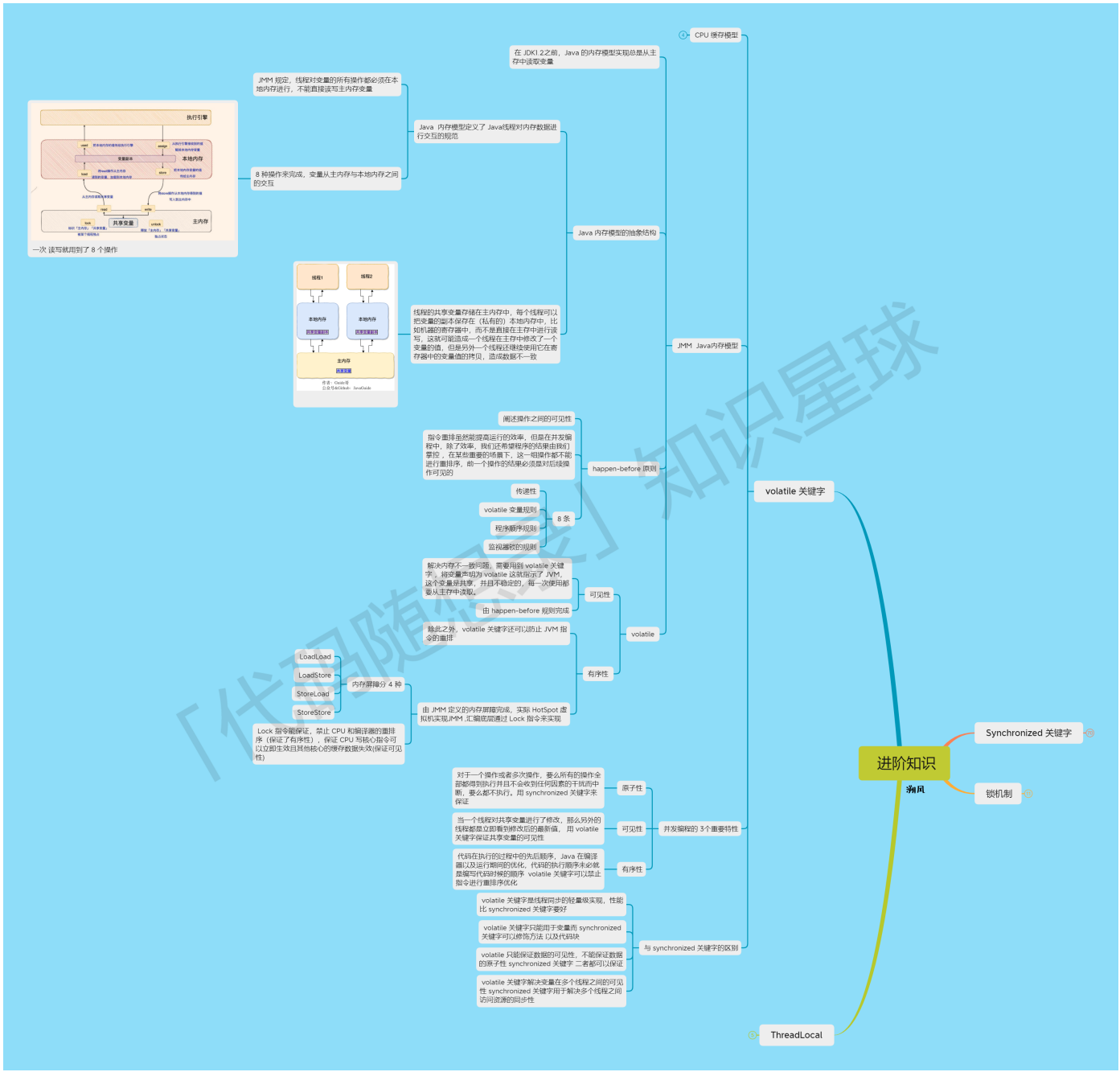
Java基础总结

原图链接：[「代码随想录」知识星球]



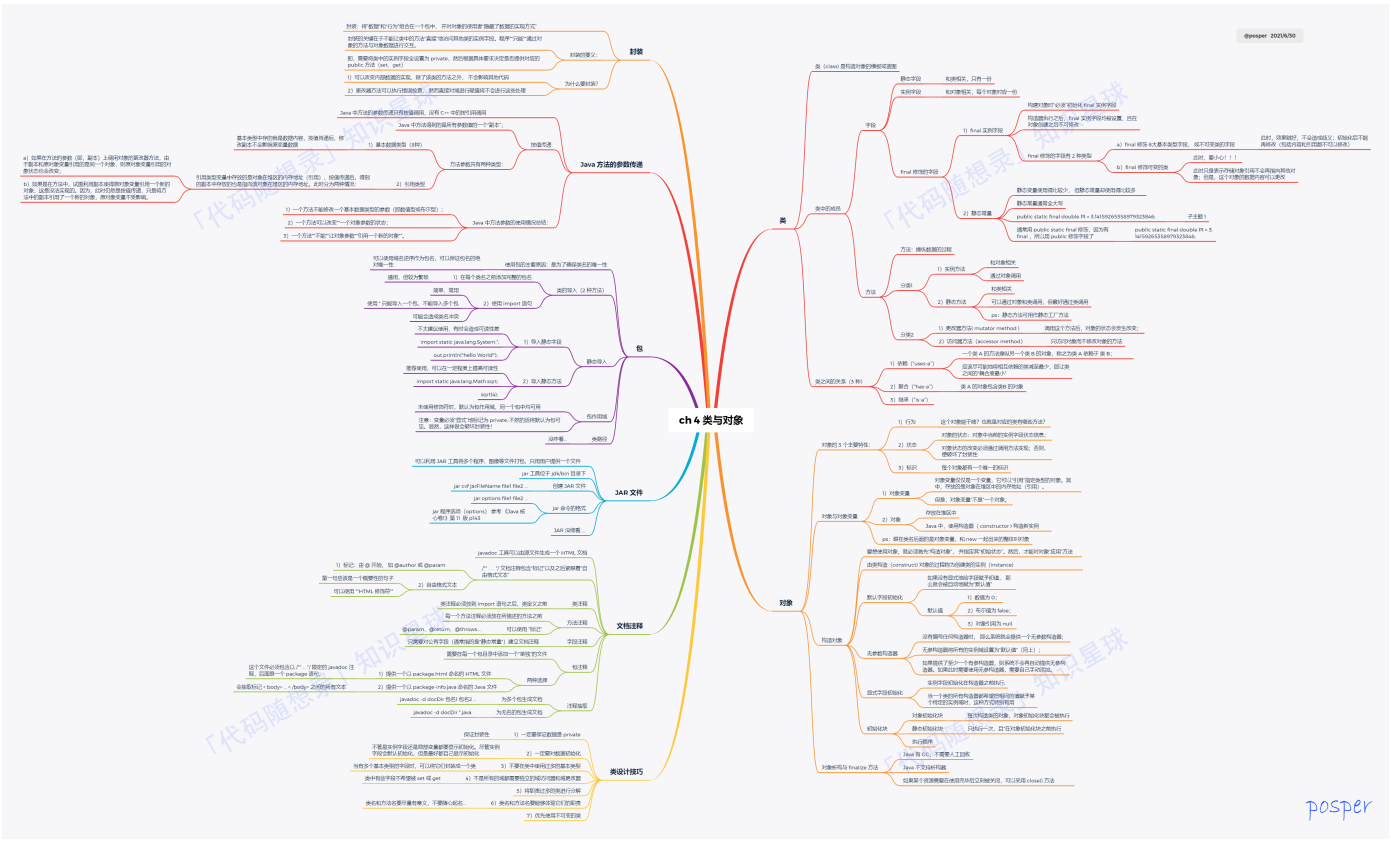
volatile关键字

原图链接：[「代码随想录」知识星球]



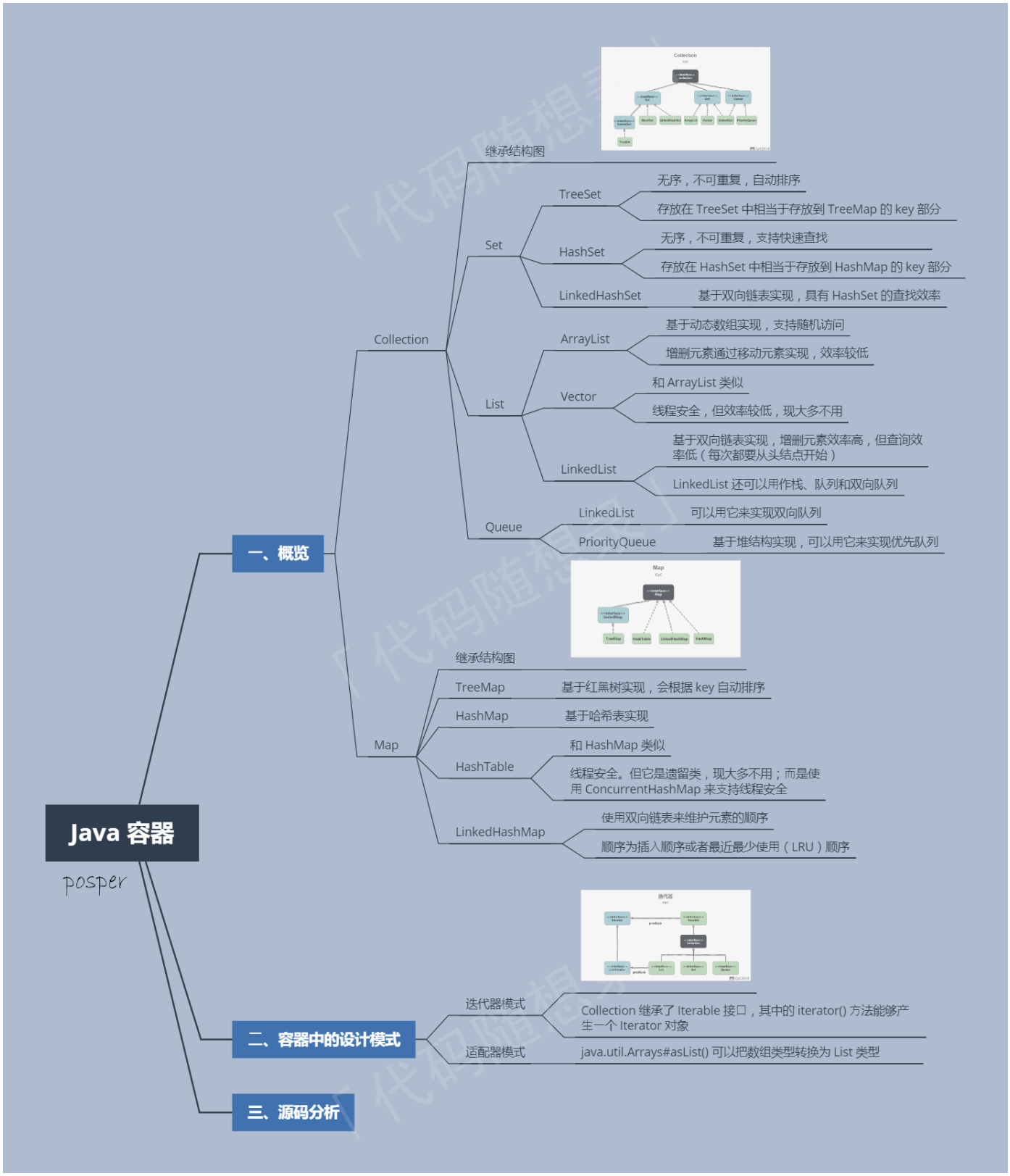
Java类和对象

原图链接：[「代码随想录」知识星球]



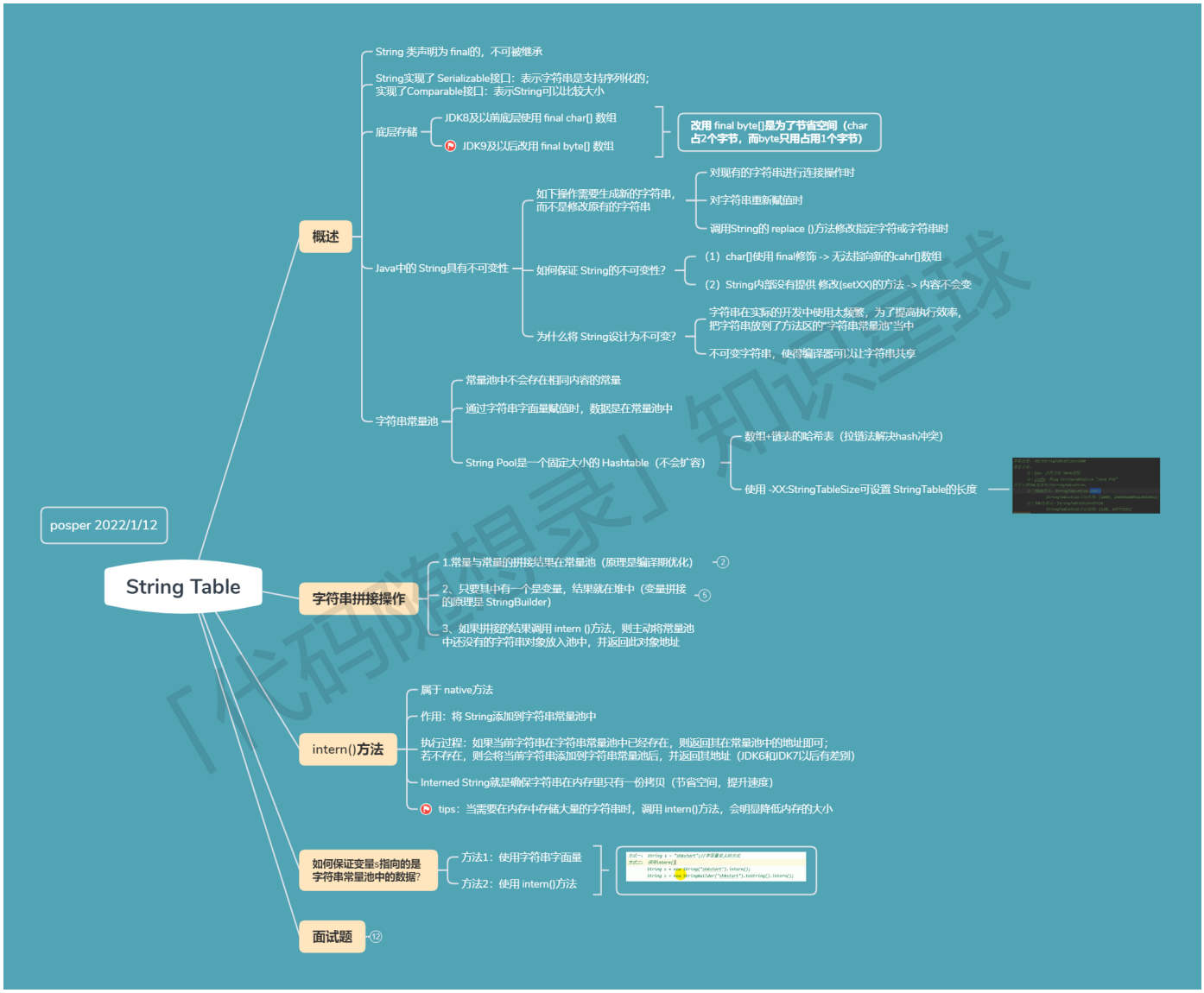
Java容器

原图链接：[「代码随想录」知识星球]



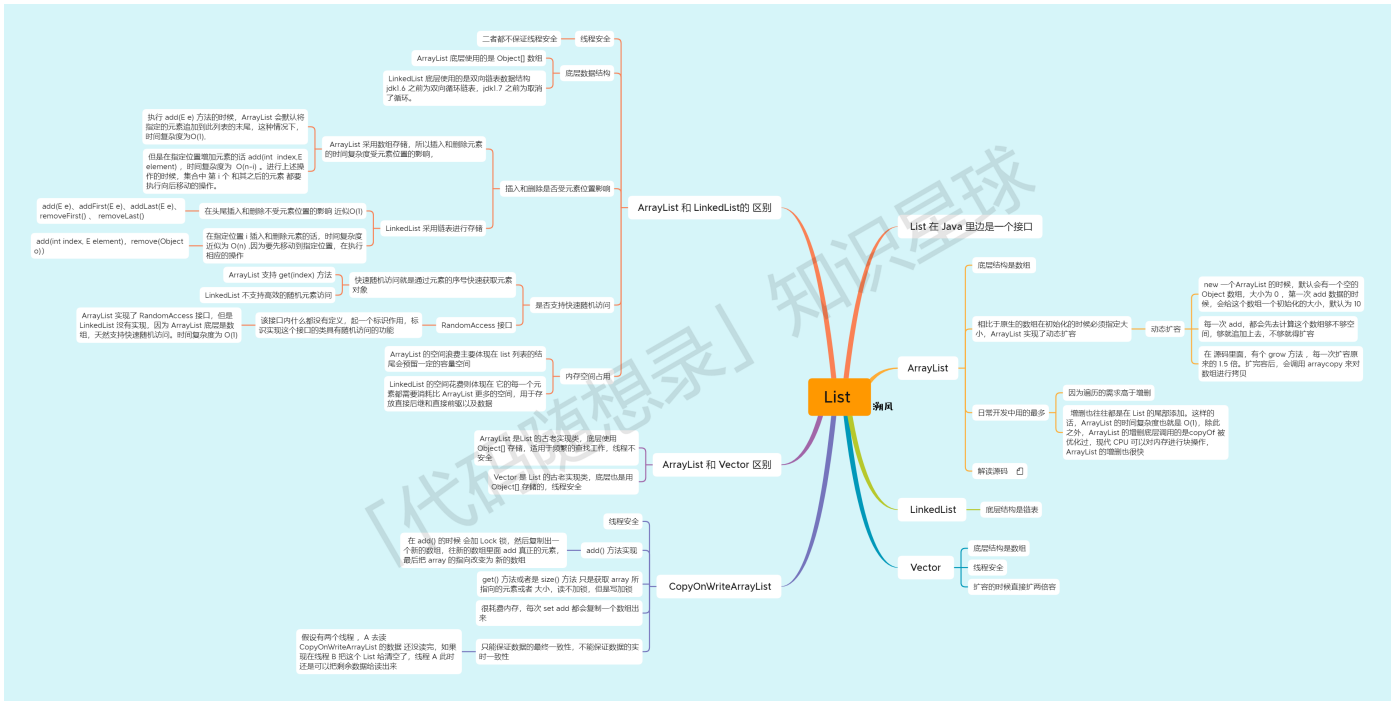
String Table

原图链接：[「代码随想录」知识星球]

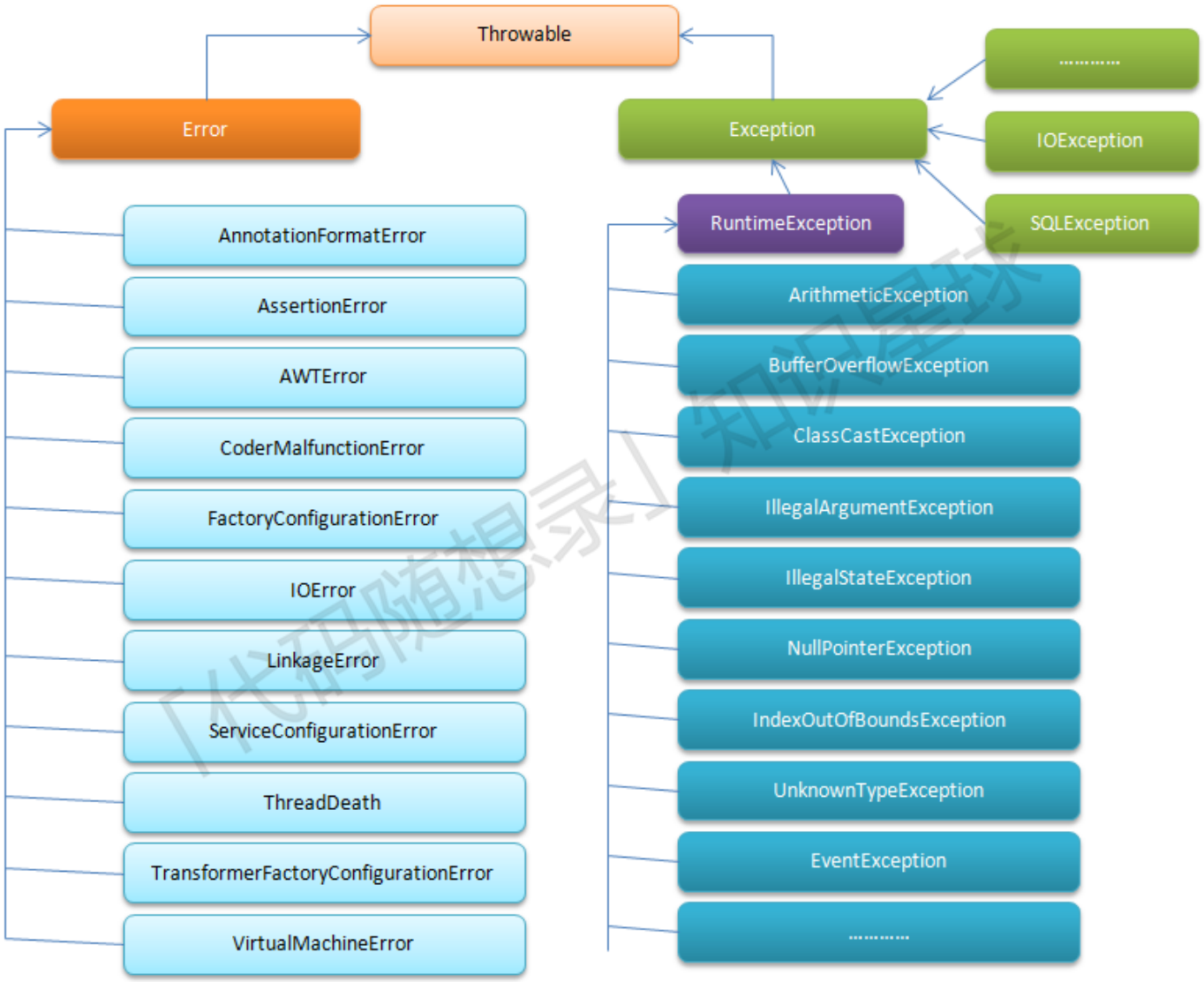


List

原图链接：[「代码随想录」知识星球]

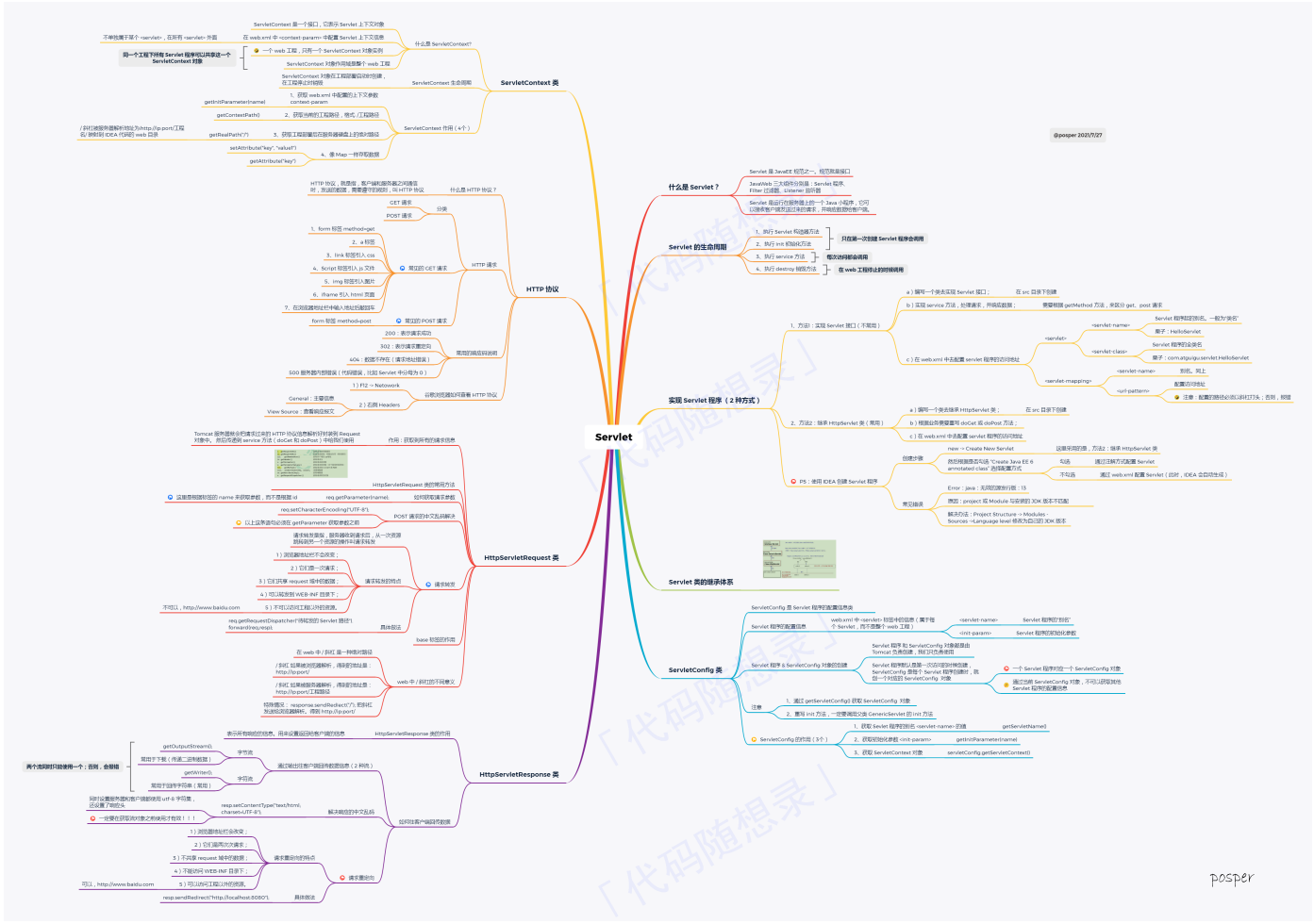

List
List 在 Java 里边是一个接口
ArrayList
底层结构是数组
相比于原生的数组在初始化的时候必须指定大小，ArrayList 实现了动态扩容
动态扩容
new 一个 ArrayList 的时候，默认会有一个空的 Object 数组，大小为 0 ，第一次 add 数据的时候，会给这个数组一个初始化的大小，默认为 10
每一次 add，都会先去计算这个数组够不够空间，够就追加上去，不够就得扩容
在 源码里面，有个 grow 方法，每一次扩容原来的 1.5 倍，扩完容后，会调用 arraycopy 来对数组进行拷贝
日常开发中用的最多
因为遍历的需求高于增删
增删也往往都是在 List 的尾部添加。这样的话，ArrayList 的时间复杂度也就是 O(1)。除此之外，ArrayList 的增删底层调用的是copyOf 被优化过，现代 CPU 可以对内存进行块操作，ArrayList 的增删也很快
解读源码
LinkedList
底层结构是链表
Vector
底层结构是数组
线程安全
扩容的时候直接扩两倍容
ArrayList 和 LinkedList的 区别
二者都不保证线程安全
线程安全
ArrayList 底层使用的是 Object[] 数组
LinkedList 底层使用的是双向链表数据结构 jdk1.6 之前为双向循环链表，jdk1.7 之后为取消了循环。
底层数据结构
执行 add(E e) 方法的时候，ArrayList 会默认将指定的元素追加到此列表的末尾，这种情况下，时间复杂度为O(1)。
但是在指定位置增加元素的话 add(int index,E element) ，时间复杂度为 O(n-i)。进行上述操作的时候，整合中 第 i 个 和其之后的元素 都要执行向后移动的操作。
ArrayList 采用数组存储，所以插入和删除元素的时间复杂度受元素位置的影响。
插入和删除是否受元素位置影响
add(E e)、addFirst(E e)、addLast(E e)、removeFirst() 、 removeLast()
在头尾插入和删除不受元素位置的影响 近似O(1)
add(int index, E element)、remove(Object o))
在指定位置 i 插入和删除元素的话，时间复杂度近似为 O(n) 因为需先移动到指定位置，在执行相应的操作
LinkedList 采用链表进行存储
ArrayList 支持 get(index) 方法
快速随机访问就是通过元素的序号快速获取元素对象
LinkedList 不支持高效的随机元素访问
是否支持快速随机访问
ArrayList 实现了 RandomAccess 接口，但是 LinkedList 没有实现，因为 ArrayList 底层是数组，天然支持快速随机访问。时间复杂度为 O(1)
该接口内什么都没有定义，起一个标识作用，标识实现这个接口的类具有随机访问的功能
RandomAccess 接口
ArrayList 的空间浪费主要体现在 list 列表的结尾会预留一定的容量空间
内存空间占用
LinkedList 的空间花费则体现在 它的每一个元素都需要消耗比 ArrayList 更多的空间，用于存放直接后继和直接前驱以及数据
ArrayList 和 Vector 区别
ArrayList 是 List 的古老实现类，底层使用 Object[] 存储，适用于频繁的查找工作，线程不安全
Vector 是 List 的古老实现类，底层也是用 Object[] 存储的，线程安全
CopyOnWriteArrayList
线程安全
add() 方法实现
在 add() 的时候 会加 Lock 锁，然后复制出一个新的数组，往新的数组里面 add 真正的元素，最后把 array 的指向改变为 新的数组
get() 方法或者是 size() 方法 只是获取 array 所指向的元素或者 大小，读不加锁，但是写加锁
很耗费内存，每次 set add 都会复制一个数组出来
只能保证数据的最终一致性，不能保证数据的实时一致性
假设有两个线程，A 去读 CopyOnWriteArrayList 的数据 还没读完，如果现在线程 B 把这个 List 给清空了，线程 A 此时还是可以把剩余数据给读出来

Java异常


Throwable
Error
Exception
AnnotationFormatError
AssertionError
AWTError
CoderMalfunctionError
FactoryConfigurationError
IOError
LinkageError
ServiceConfigurationError
ThreadDeath
TransformerFactoryConfigurationError
VirtualMachineError
RuntimeException
IOException
SQLException
ArithmeticException
BufferOverflowException
ClassCastException
IllegalArgumentException
IllegalStateException
NullPointerException
IndexOutOfBoundsException
UnknownTypeException
EventException

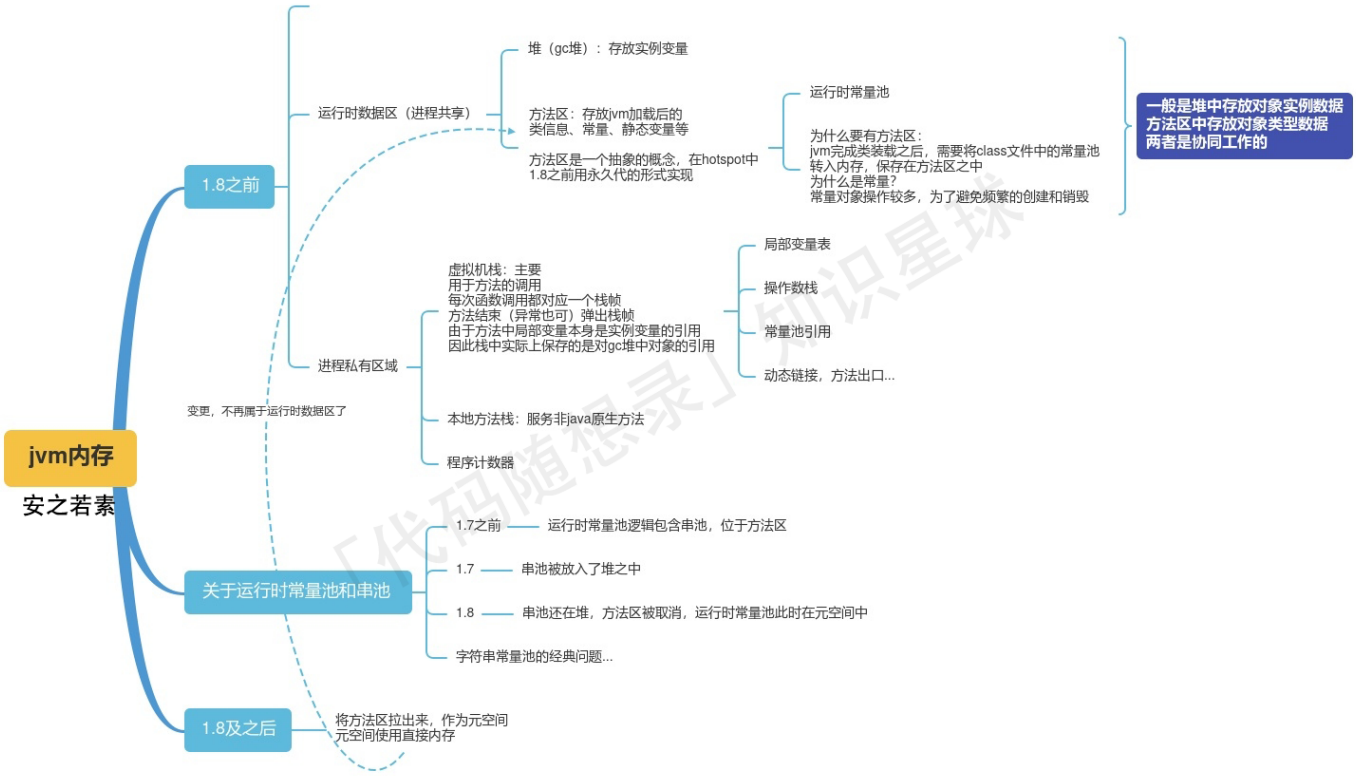
Servlet

原图链接：「代码随想录」知识星球



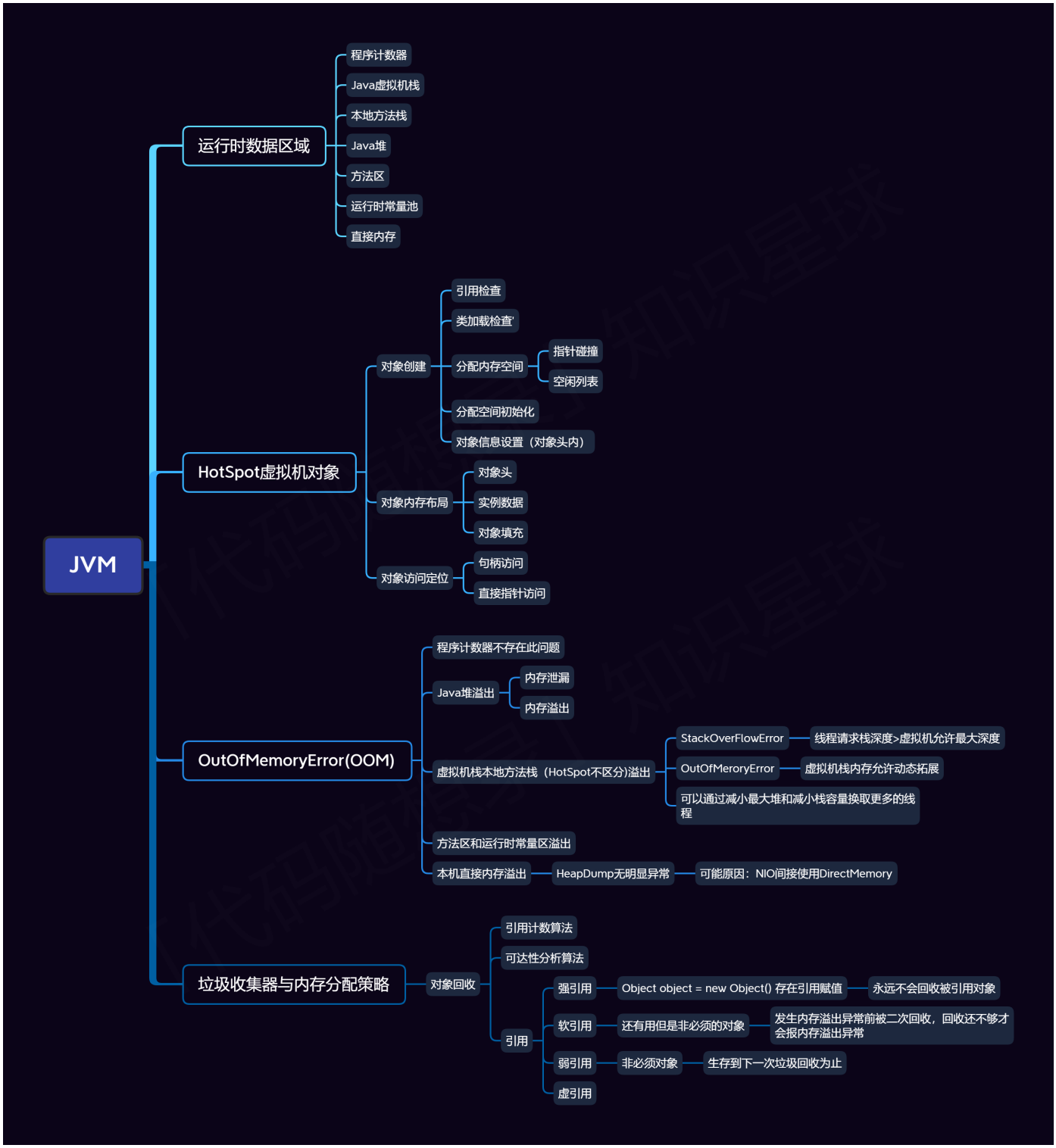
JVM内存

原图链接：「代码随想录」知识星球



JVM内存管理

原图链接：[「代码随想录」知识星球]



JavaWeb基础

原图链接：「代码随想录」知识星球

javaweb 前置

安之若素

web应用

- html css js
- 动态，可以和数据库交互，数据可以持久化
- jsp servlet
  - B/S架构（*）浏览器和服务器
  - C/S架构 客服端和服务器
- java程序
- jar
- properties

web服务器

- tomcat

http

- http1.0 http/1.0 客服端和服务器连接，只能获取一个资源
- http2.0 http/1.1 可以获得多个资源
- https
- 请求
  - 请求行
    - get 携带的参数比较少，大小有限制，url显示数据内容，不安全，但高效
    - post 与get相反
    - head...
  - 消息头
    - accept 浏览器支持的数据类型
    - accep-encoding
    - accep- language 支持的编码格式
    - cache- control
    - connection 断开还是连接
    - host 主机
- 响应
  - refresh 多久刷新一次
  - location 重定位
  - 状态码
    - 200
    - 4** 资源不存在
    - 3** 重定向
    - 5**
      - 服务器代码错误
      - 502 网关错误

面试题

- 本地能访问www.xxx.com吗
- 从浏览器地址栏输入地址，到返回页面，整个过程中发生了什么？

maven

- 在web开发中，使用大量的jar包，需要手动导入，如何让一个文件自动导入配置jar
- 核心思想：约定大于配置，不要违反约束
- 会导入目标jar包依赖的所有jar包
- maven资源导出问题 build中配置resource，防止资源导出失败
- maven父项目可以包括子项目，子项目可以直接调用父项目的所有依赖

servlet

- 开发动态web的技术，是一个接口，只需要两个步骤，编写一个类实现servlet接口，开发好的java类部署在web服务器
- mapping
  - 一个servlet可以指定一个或者多个或者通用前后缀（正则）路径
  - 优先级问题：匹配度高的路径优先级最高
- servlet context
  - sevlet1 url1
  - servlet2 url2
  - servlet3 url3
  - 保存多个servlet数据
  - 请求转发 地址不变，但是显示另一个页面的内容，相当于另一个页面是'代理' 307
  - 重定向 地址跳转 302
  - 读取资源文件

cookie session

- cookie客户端每次访问服务器都需要携带
- session可以保存用户的会话信息，可以把信息放在sesssion中
  - 服务器会对每一个用户（浏览器）创建一个session，一个session独占一个浏览器
  - 用户登陆之后，整个网站都可以访问
- session和cookie的区别
  - cookie把用户的数据写给用户（浏览器）
  - session把用户的数据写到用户独占的session中，服务器保存，只保存重要的信息

jsp：java server pages

- 和serverlet一样，用于开发动态web
- 语法和html差不多，但是jsp页面中可以嵌入java代码
- 服务器内部是如何工作的
  - tomcat中有一个work目录，发现页面转换成了java程序
  - 浏览器不管访问什么资源，都是访问servlet
  - jsp本质就是一个serblet，只要是是java代码，原封输出，如果是html代码，会装饰输出

九大内置对象

- 只在一个页面有效 存东西 pagecontext
- 只在一次请求有效 存东西 request
- response
- 从打开到关闭浏览器 存东西 session
- 服务器开机就一直存在 存东西 application(servlet context)
- config(servlet config)
- out
- 不用 page
- exception

Java Bean

- 实体类，有特定的写法
- 必须要有一个无参数构造
- 属性一定需要私有化
- 必须要有getset方法
- 一般用来和数据库字段做orm
  - java直接操作数据库
    - 表-类
    - 字段-属性
    - 行记录-对象

mvc三层架构

- model
  - 业务处理（逻辑）Service
  - 数据持久CRUD（Dao）
- view
  - 展示数据
  - 提供链接发起servlet请求
- controller（servlet）
  - 接受用户的请求（req，session，，）
  - 交给业务层处理
  - 控制试图跳转
- 架构图

filter

- web服务器调用sevlet时候经过filter，可以指定scope
- 实现权限拦截

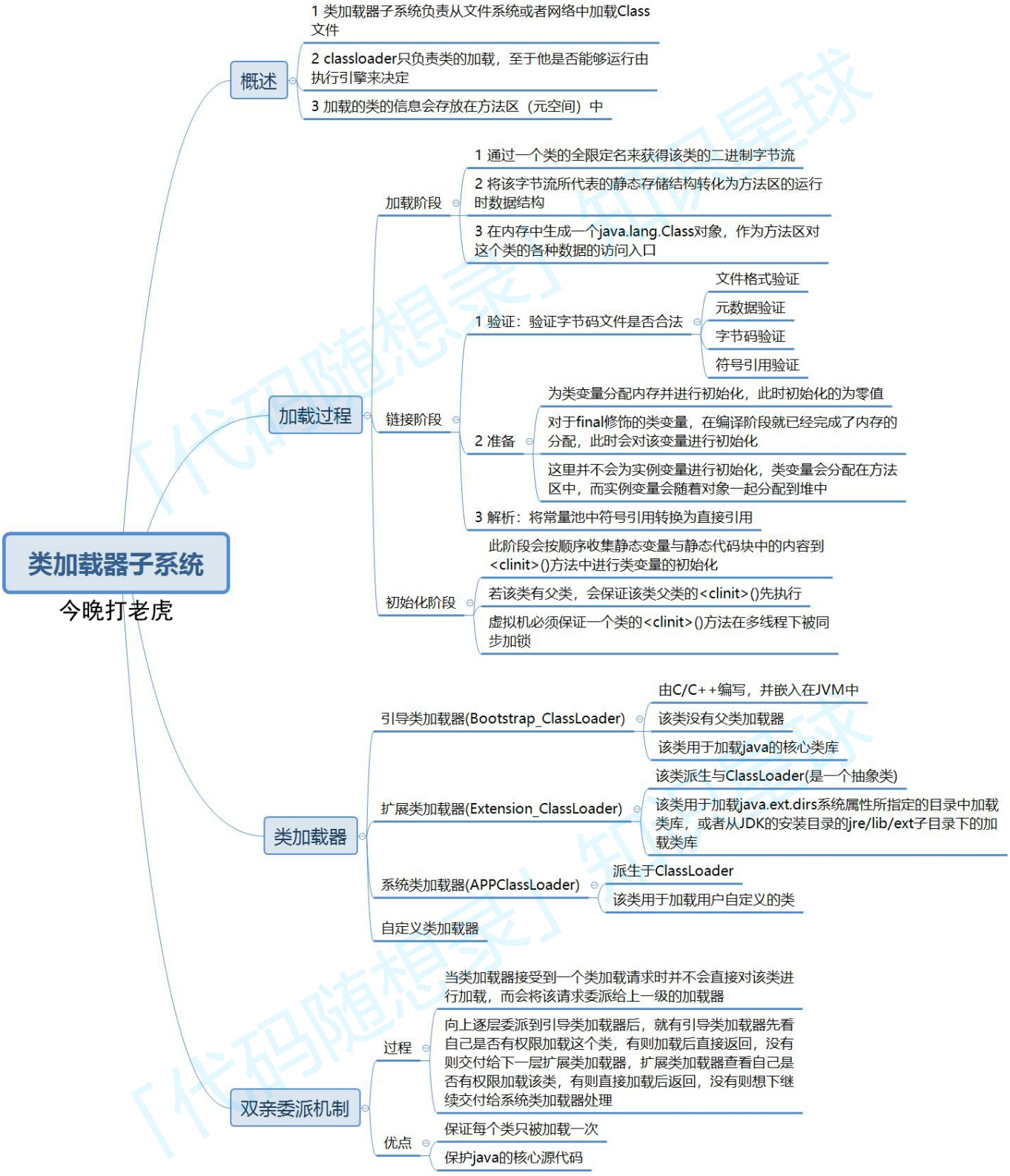
listener

jdbc

- 操作时候注意事务的几大特性ACID

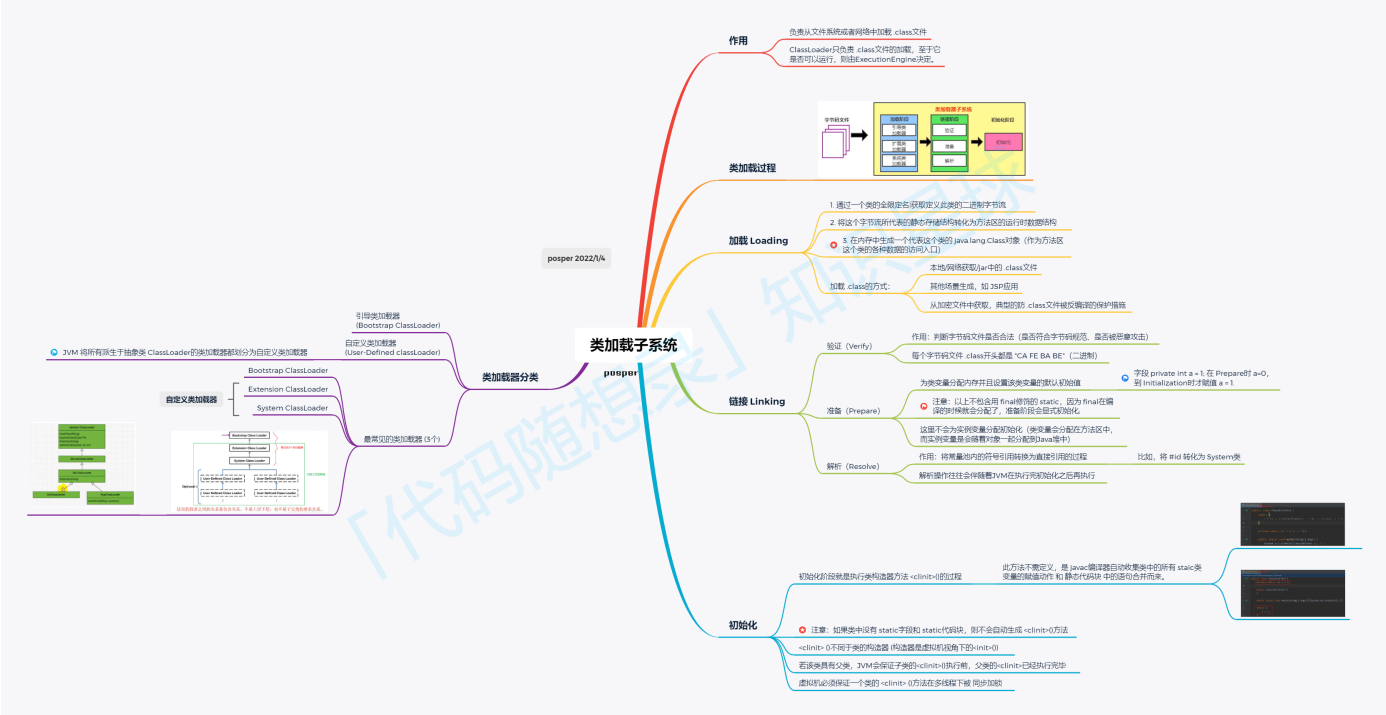
Java类加载子系统（1）

原图链接：「代码随想录」知识星球



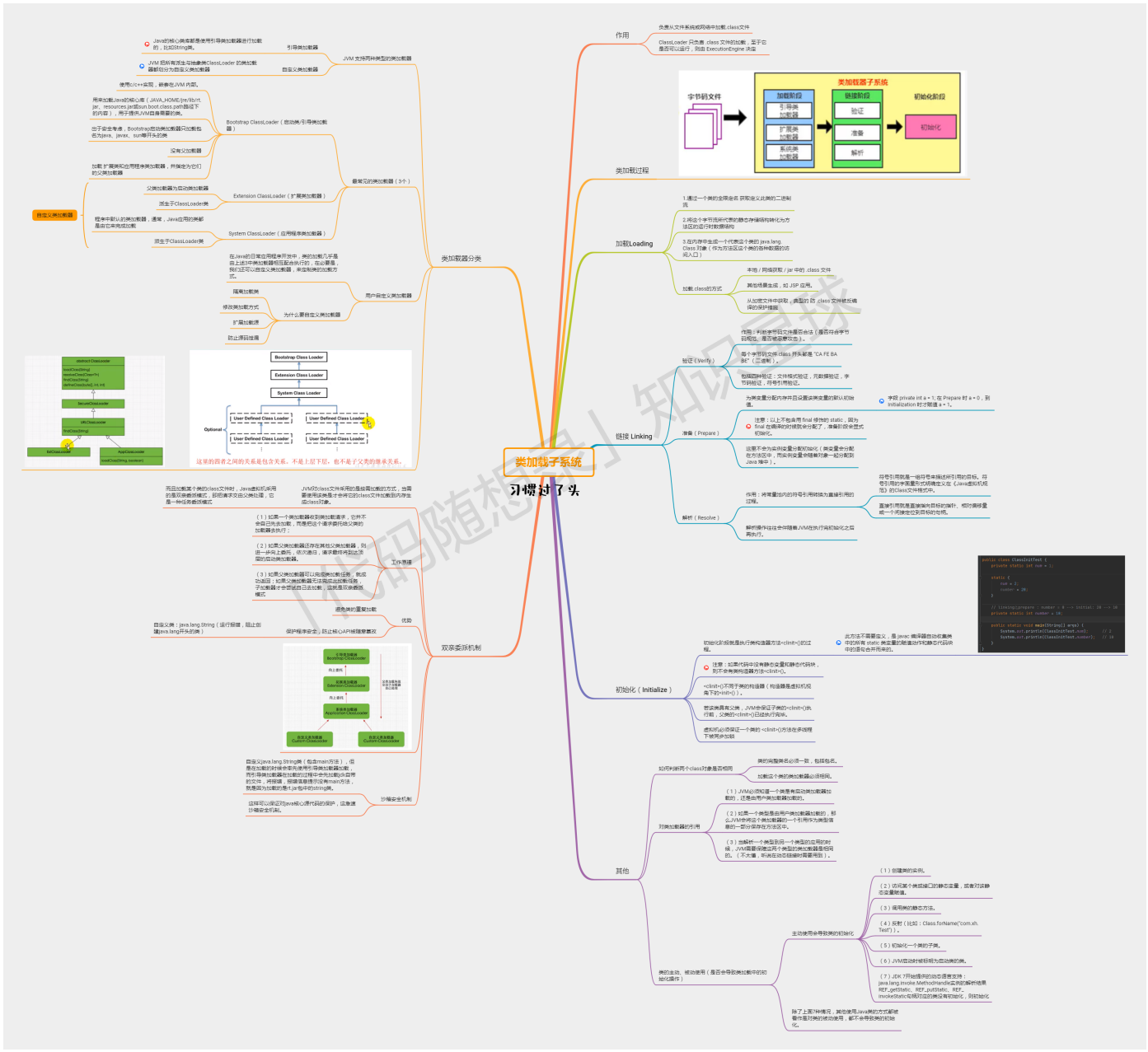
Java类加载子系统（2）

原图链接：「代码随想录」知识星球



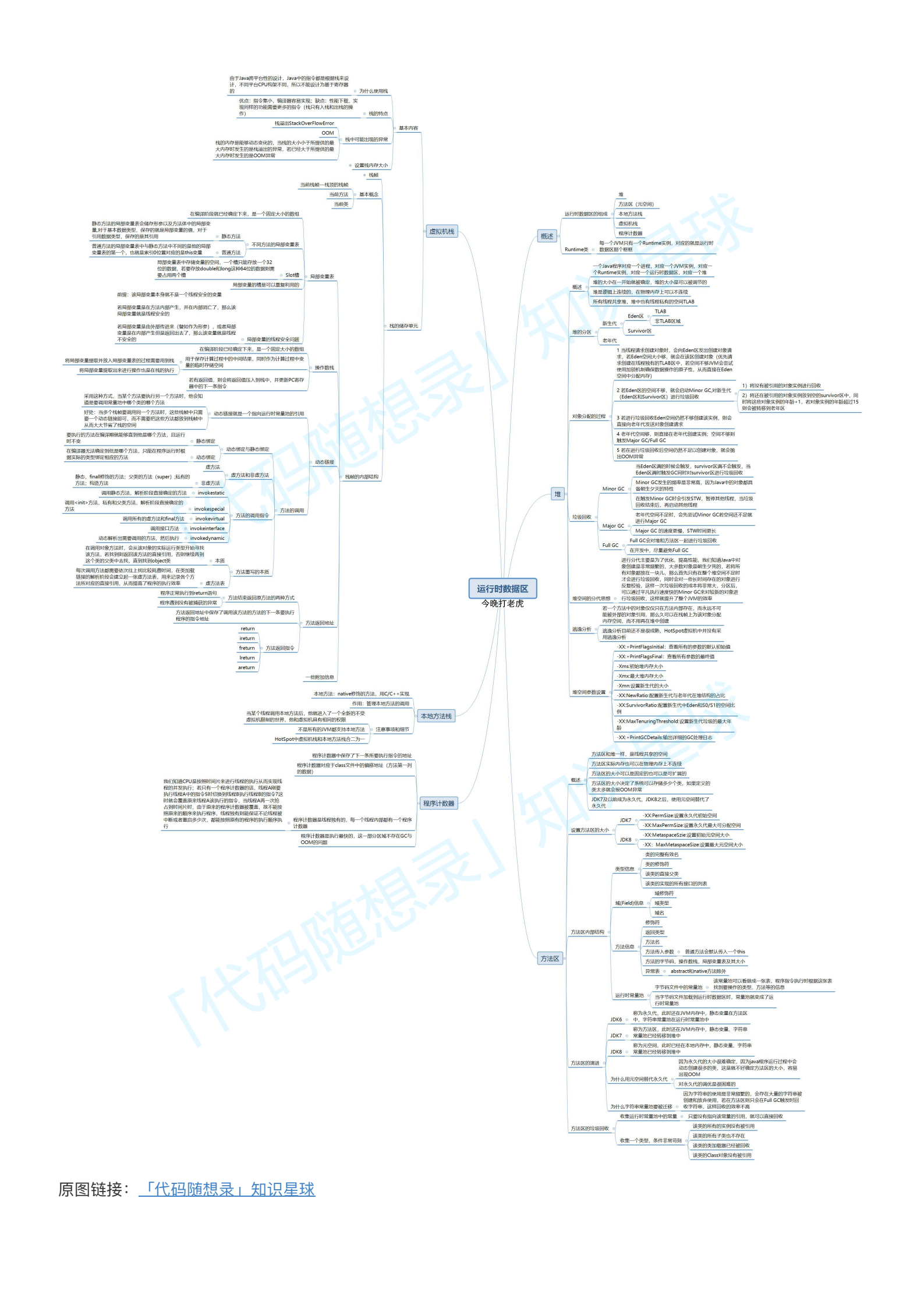
Java类加载子系统（3）

原图链接：「代码随想录」知识星球

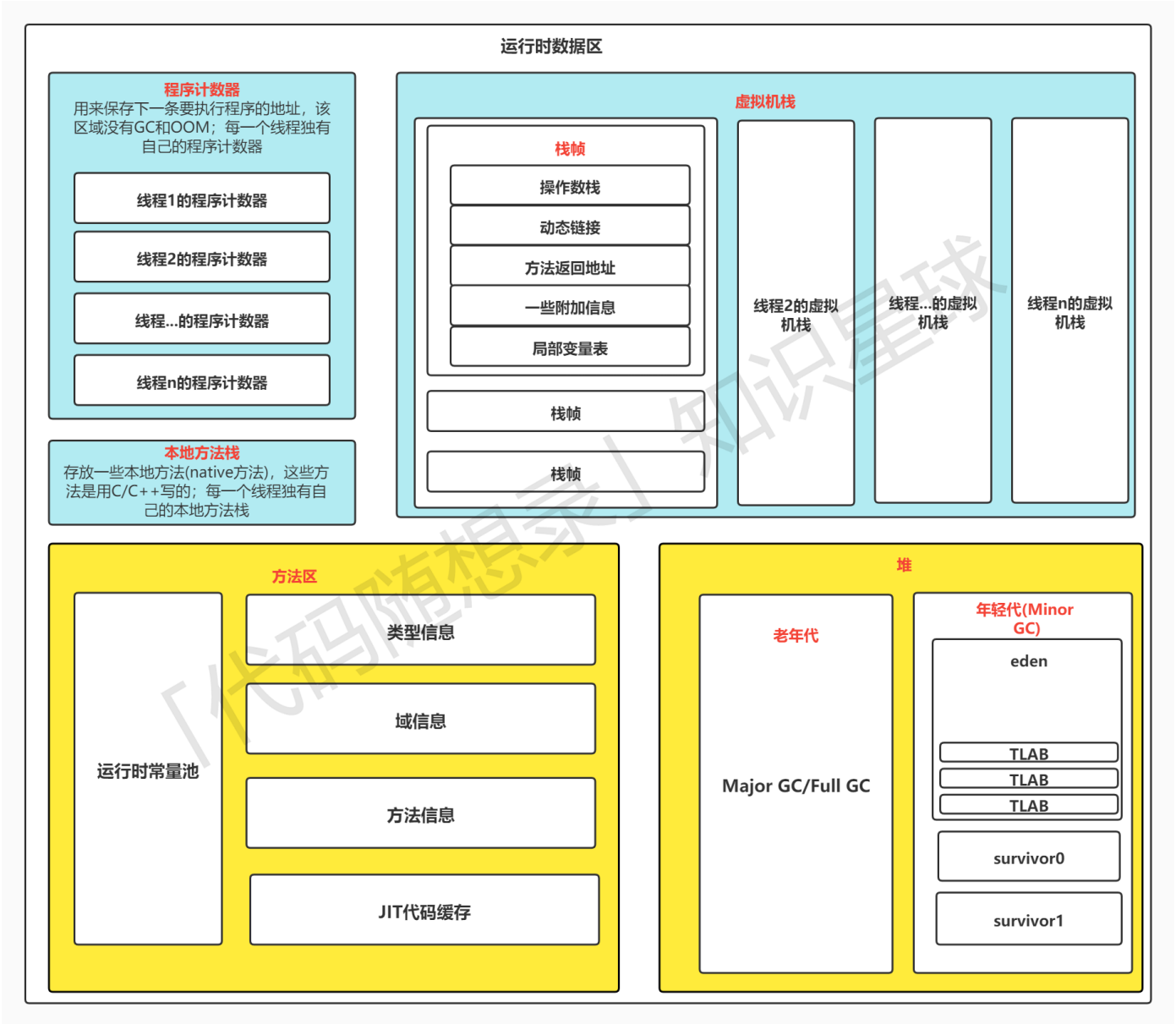


运行时数据区（1）

原图链接：[「代码随想录」知识星球]

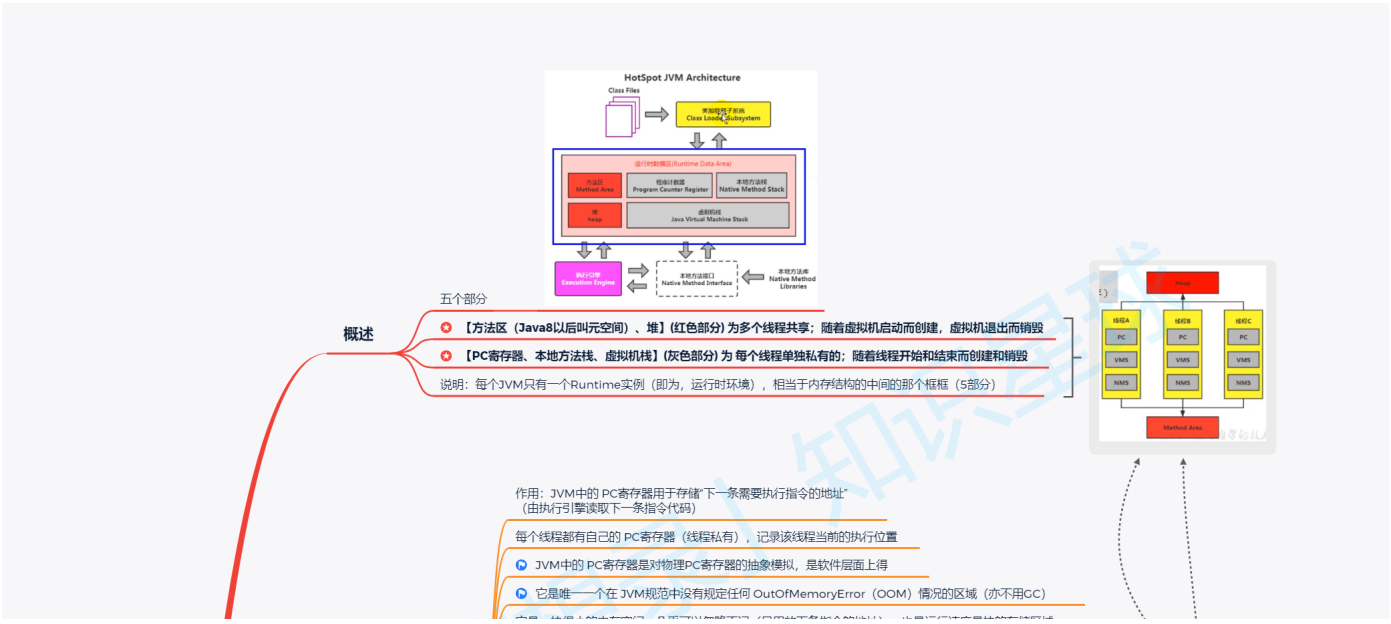


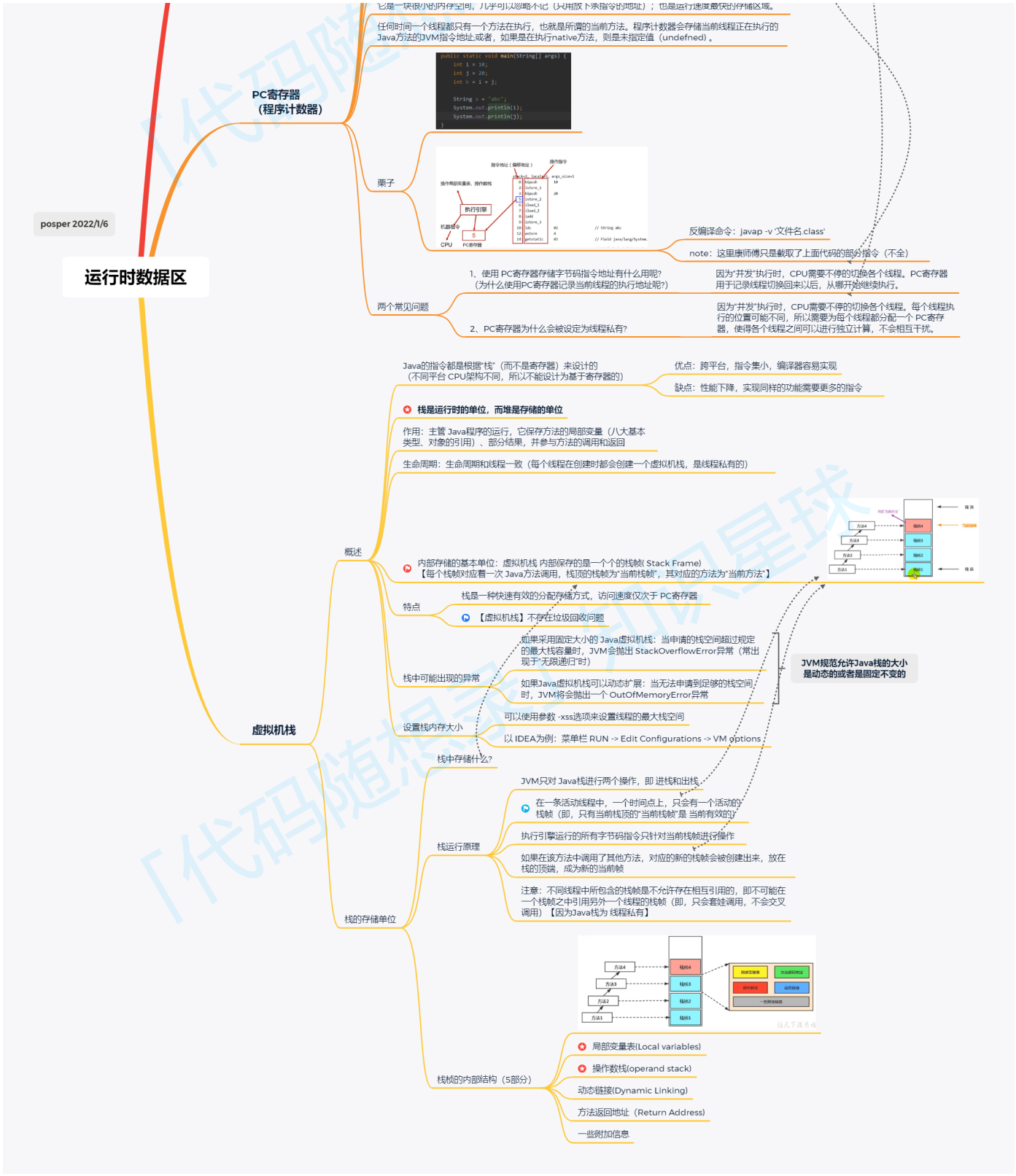
原图链接：[「代码随想录」知识星球]



运行时数据区（2）

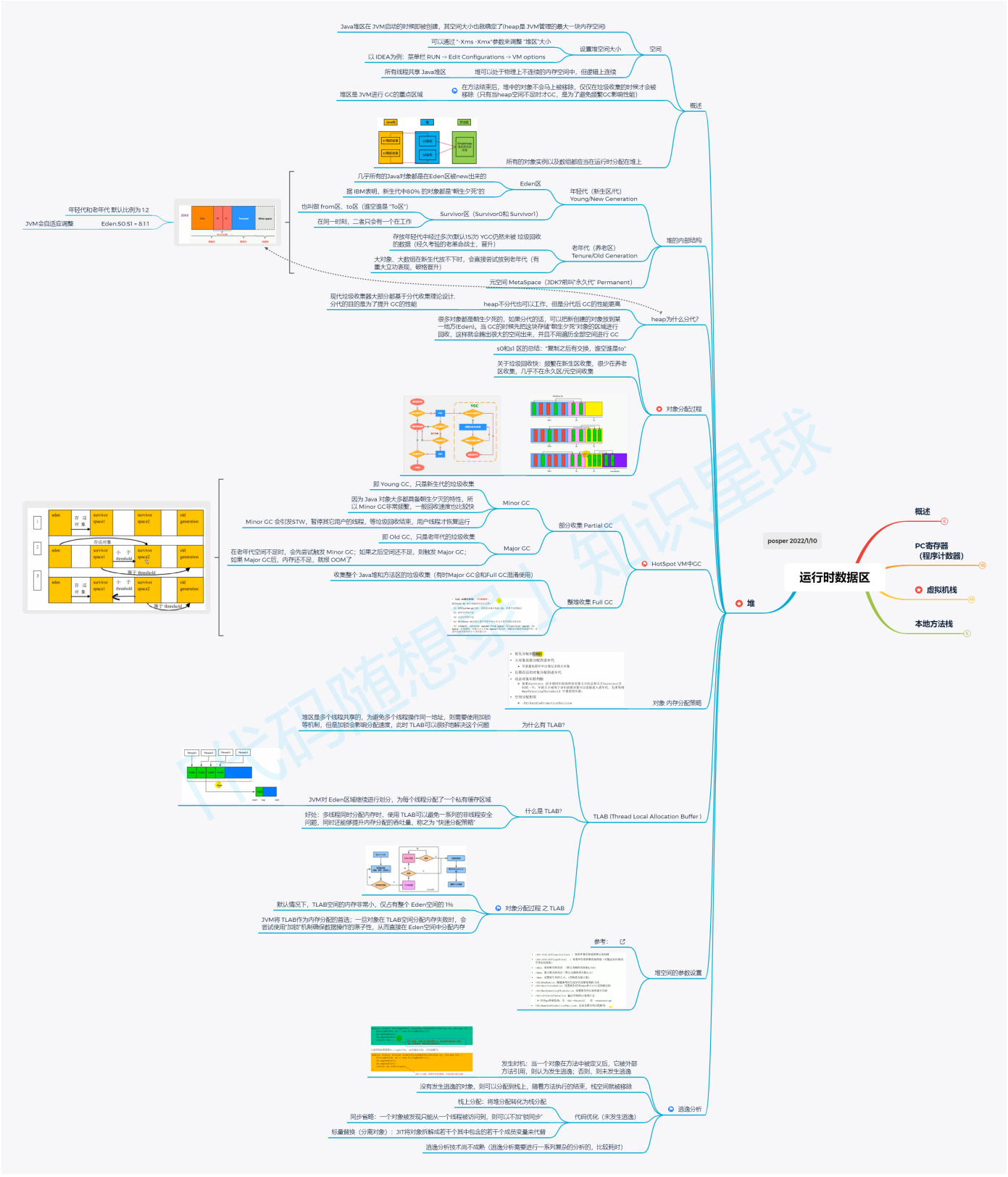
原图链接：[「代码随想录」知识星球]





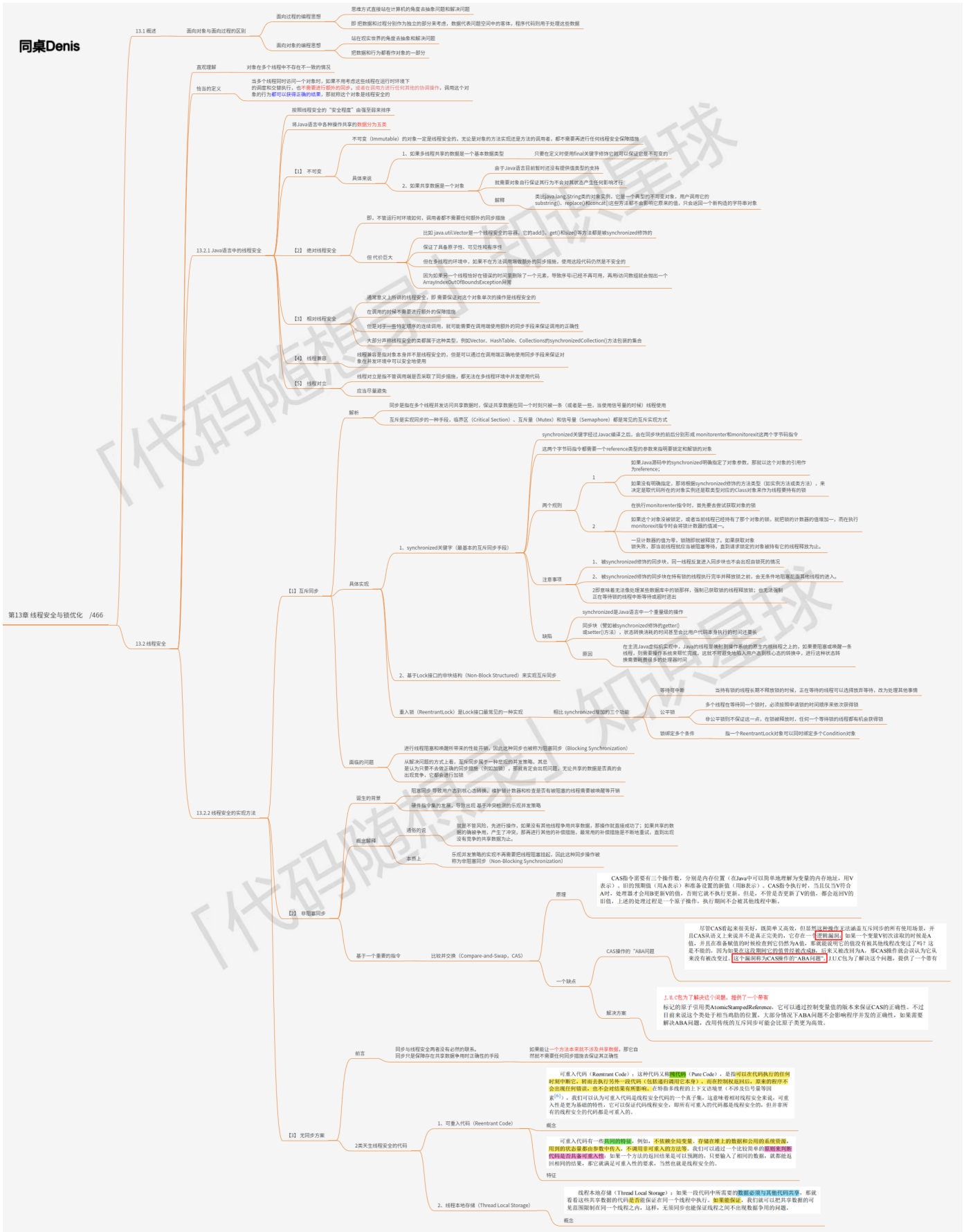
运行时数据区（3）

原图链接：[「代码随想录」知识星球]



线程安全

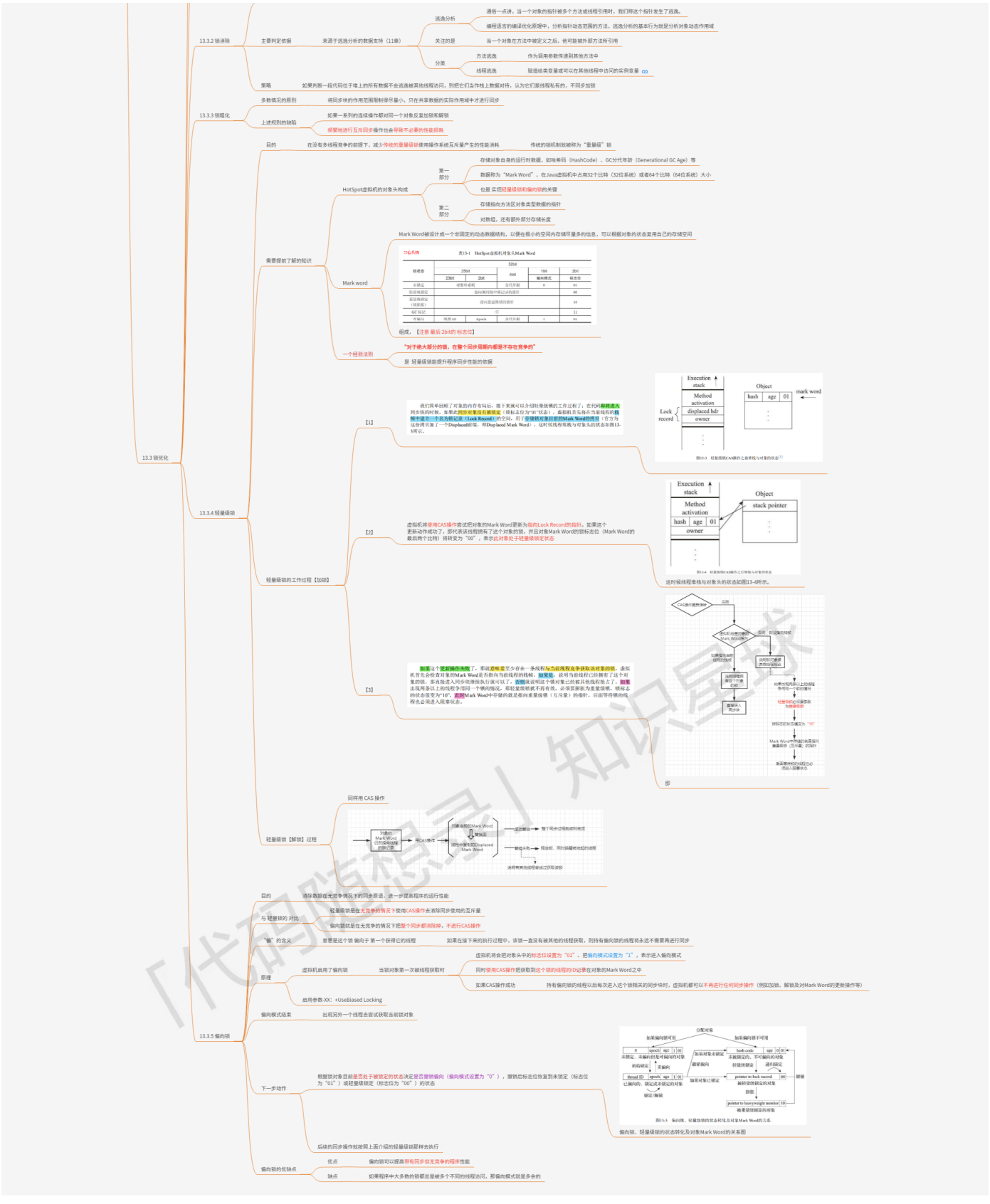
原图链接：「代码随想录」知识星球



原图链接：[「代码随想录」知识星球]

「代码随想录」知识星球

- 第13章 线程安全与锁优化 /466
  - 13.2 线程安全
    - 13.2.1 Java语言中的线程安全
      - 按照线程安全的“安全程度”由强至弱来排序
      - 将Java语言中各种操作共享的数据分为五类
      - 【1】 不可变
        - 不可变（Immutable）的对象一定是线程安全的，无论是对象的方法实现还是方法的调用者，都不需要再进行任何线程安全保障措施
        - 具体来说
          - 1、如果多线程共享的数据是一个基本数据类型
            - 只要在定义时使用final关键字修饰它就可以保证它是不可变的
          - 2、如果共享数据是一个对象
            - 由于Java语言目前暂时还没有提供值类型的支持
            - 就需要对象自行保证其行为不会对其状态产生任何影响才行
            - 解释
              - 类比java.lang.String类的对象实例，它是一个典型的不可变对象，用户调用它的substring()、replace()和concat()这些方法都不会影响它原来的值，只会返回一个新构造的字符串对象
      - 【2】 绝对线程安全
        - 即，不管运行时环境如何，调用者都不需要任何额外的同步措施
        - 但代价巨大
          - 比如 java.util.Vector是一个线程安全的容器，它的add()、get()和size()等方法都是被synchronized修饰的
          - 保证了具备原子性、可见性和有序性
          - 但在多线程的环境中，如果不在方法调用端做额外的同步措施，使用这段代码仍然是不安全的
          - 因为如果另一个线程恰好在错误的时间里删除了一个元素，导致序号i已经不再可用，再用i访问数组就会抛出一个ArrayIndexOutOfBoundsException异常
      - 【3】 相对线程安全
        - 通常意义上所讲的线程安全，即 需要保证对这个对象单次的操作是线程安全的
        - 在调用的时候不需要进行额外的保障措施
        - 但是对于一些特定顺序的连续调用，就可能需要在调用端使用额外的同步手段来保证调用的正确性
        - 大部分声称线程安全的类都属于这种类型，例如Vector、HashTable、Collections的synchronizedCollection()方法包装的集合
      - 【4】 线程兼容
        - 线程兼容是指对象本身并不是线程安全的，但是可以通过在调用端正确地使用同步手段来保证对象在并发环境中可以安全地使用
      - 【5】 线程对立
        - 线程对立是指不管调用端是否采取了同步措施，都无法在多线程环境中并发使用代码
        - 应当尽量避免
    - 13.2.2 线程安全的实现方法
      - 【1】互斥同步
        - 解析
          - 同步是指在多个线程并发访问共享数据时，保证共享数据在同一个时刻只被一条（或者是一些，当使用信号量的时候）线程使用
          - 互斥是实现同步的一种手段，临界区（Critical Section）、互斥量（Mutex）和信号量（Semaphore）都是常见的互斥实现方式
        - 具体实现
          - 1、synchronized关键字（最基本的互斥同步手段）
            - synchronized关键字经过Javac编译之后，会在同步块的前后分别形成 monitorenter和monitorexit这两个字节码指令
            - 这两个字节码指令都需要一个reference类型的参数来指明要锁定和解锁的对象
            - 两个规则
              - 1
                - 如果Java源码中的synchronized明确指定了对象参数，那就以这个对象的引用作为reference；
                - 如果没有明确指定，那将根据synchronized修饰的方法类型（如实例方法或类方法），来决定是取代码所在的对象实例还是取类型对应的Class对象来作为线程要持有的锁
              - 2
                - 在执行monitorenter指令时，首先要去尝试获取对象的锁
                - 如果这个对象没被锁定，或者当前线程已经持有了那个对象的锁，就把锁的计数器的值增加一，而在执行monitorexit指令时会将锁计数器的值减一。
                - 一旦计数器的值为零，锁随即就被释放了。如果获取对象锁失败，那当前线程就应当被阻塞等待，直到请求锁定的对象被持有它的线程释放为止。
            - 注意事项
              - 1、被synchronized修饰的同步块，同一线程反复进入同步块也不会出现自锁死的情况
              - 2、被synchronized修饰的同步块在持有锁的线程执行完毕并释放锁之前，会无条件地阻塞后面其他线程的进入。
              - 2即意味着无法像处理某些数据库中的锁那样，强制已获取锁的线程释放锁；也无法强制正在等待锁的线程中断等待或超时退出
            - 缺陷
              - synchronized是Java语言中一个重量级的操作
              - 同步块（譬如被synchronized修饰的getter()或setter()方法），状态转换消耗的时间甚至会比用户代码本身执行的时间还要长
              - 原因
                - 在主流Java虚拟机实现中，Java的线程是映射到操作系统的原生内核线程之上的，如果要阻塞或唤醒一条线程，则需要操作系统来帮忙完成，这就不可避免地陷入用户态到核心态的转换中，进行这种状态转换需要耗费很多的处理器时间
          - 2、基于Lock接口的非块结构（Non-Block Structured）来实现互斥同步
          - 重入锁（ReentrantLock）是Lock接口最常见的一种实现
            - 相比 synchronized增加的三个功能
              - 等待可中断
                - 当持有锁的线程长期不释放锁的时候，正在等待的线程可以选择放弃等待，改为处理其他事情
              - 公平锁
                - 多个线程在等待同一个锁时，必须按照申请锁的时间顺序来依次获得锁
                - 非公平锁则不保证这一点，在锁被释放时，任何一个等待锁的线程都有机会获得锁
              - 锁绑定多个条件
                - 指一个ReentrantLock对象可以同时绑定多个Condition对象
        - 面临的问题
          - 进行线程阻塞和唤醒所带来的性能开销，因此这种同步也被称为阻塞同步（Blocking Synchronization）
          - 从解决问题的方式上看，互斥同步属于一种悲观的并发策略，其总是认为只要不去做正确的同步措施（例如加锁），那就肯定会出现问题，无论共享的数据是否真的会出现竞争，它都会进行加锁
      - 【2】 非阻塞同步
        - 诞生的背景
          - 阻塞同步 导致用户态到核心态转换、维护锁计数器和检查是否有被阻塞的线程需要被唤醒等开销
          - 硬件指令集的发展，导致出现 基于冲突检测的乐观并发策略
        - 概念解释
          - 通俗的说
            - 就是不管风险，先进行操作，如果没有其他线程争用共享数据，那操作就直接成功了；如果共享的数据的确被争用，产生了冲突，那再进行其他的补偿措施，最常用的补偿措施是不断地重试，直到出现没有竞争的共享数据为止。
          - 本质上
            - 乐观并发策略的实现不再需要把线程阻塞挂起，因此这种同步操作被称为非阻塞同步（Non-Blocking Synchronization）
        - 基于一个重要的指令
          - 比较并交换（Compare-and-Swap，CAS）
            - 原理
              - CAS指令需要有三个操作数，分别是内存位置（在Java中可以简单地理解为变量的内存地址，用V表示）、旧的预期值（用A表示）和准备设置的新值（用B表示）。CAS指令执行时，当且仅当V符合A时，处理器才会用B更新V的值，否则它就不执行更新。但是，不管是否更新了V的值，都会返回V的旧值，上述的处理过程是一个原子操作，执行期间不会被其他线程中断。
            - 一个缺点
              - CAS操作的“ABA问题”
                - 尽管CAS看起来很美好，既简单又高效，但显然这种操作无法涵盖互斥同步的所有使用场景，并且CAS从语义上来说并不是真正完美的，它存在一个逻辑漏洞，如果一个变量V初次读取的时候是A值，并且在准备赋值的时候检查到它仍然为A值，那就能说明它的值没有被其他线程改变过了吗？这是不能的，因为如果在这段期间它的值曾经被改成B，后来又被改回为A，那CAS操作就会误认为它从来没有被改变过，这个漏洞称为CAS操作的“ABA问题”。J.U.C包为了解决这个问题，提供了一个带有
              - 解决方案
                - J.U.C包为了解决这个问题，提供了一个带有标记的原子引用类AtomicStampedReference，它可以通过控制变量值的版本来保证CAS的正确性。不过目前来说这个类处于相当鸡肋的位置，大部分情况下ABA问题不会影响程序并发的正确性，如果需要解决ABA问题，改用传统的互斥同步可能会比原子类更为高效。
      - 【3】 无同步方案
        - 前言
          - 同步与线程安全两者没有必然的联系。同步只是保障存在共享数据争用时正确性的手段
            - 如果能让一个方法本来就不涉及共享数据，那它自然就不需要任何同步措施去保证其正确性
        - 2类天生线程安全的代码
          - 1、可重入代码（Reentrant Code）
            - 概念
              - 可重入代码（Reentrant Code）：这种代码又称纯代码（Pure Code），是指可以在代码执行的任何时刻中断它，转而去执行另外一段代码（包括递归调用它本身），而在控制权返回后，原来的程序不会出现任何错误，也不会对结果有所影响。在特指多线程的上下文语境里（不涉及信号量等因素[51]），我们可以认为可重入代码是线程安全代码的一个真子集，这意味着相对线程安全来说，可重入性是更为基础的特性，它可以保证代码线程安全，即所有可重入的代码都是线程安全的，但非非所有的线程安全的代码都是可重入的。
            - 特征
              - 可重入代码有一些共同的特征，例如，不依赖全局变量、存储在堆上的数据和公用的系统资源，用到的状态量都由参数中传入，不调用非可重入的方法等。我们可以通过一个比较简单的原则来判断代码是否具备可重入性：如果一个方法的返回结果是可以预测的，只要输入了相同的数据，就都能返回相同的结果，那它就满足可重入性的要求，当然也就是线程安全的。
          - 2、线程本地存储（Thread Local Storage）
            - 概念
              - 线程本地存储（Thread Local Storage）：如果一段代码中所需要的数据必须与其他代码共享，那就看看这些共享数据的代码是否能保证在同一个线程中执行。如果能保证，我们就可以把共享数据的可见范围限制在同一个线程之内，这样，无须同步也能保证线程之间不出现数据争用的问题。
    - 13.2.3 互斥同步的实现
      - 目的
        - 高效并发
        - HotSpot虚拟机开发团队在这个版本上花费了大量的资源去实现各种锁优化技术
          - 适应性自旋（Adaptive Spinning）
          - 锁消除（Lock Elimination）
          - 锁膨胀（Lock Coarsening）
          - 轻量级锁（Lightweight Locking）
          - 偏向锁（Biased Locking）
      - 13.3.1 自旋锁与自适应自旋
        - 诞生背景
          - 互斥同步对性能最大的影响是阻塞的实现，挂起线程和恢复线程的操作都需要转入内核态中完成，这些给Java虚拟机带来坏的影响
          - 【问题】共享数据的锁定状态只会持续很短的一段时间，为了这段时间去挂起和恢复线程并不值得
        - 概念解析【上述问题的解决方案】
          - 两个或以上的线程同时并行执行时
          - 让后面请求锁的那个线程“稍等一会”，但不放弃处理器的执行时间
          - 等待持有锁的线程是否短时间内释放锁
        - 开启的参数
          - JDK6之前使用-XX：+UseSpinning参数来开启，后续默认开启
        - 注意事项【一个问题】
          - 自旋等待不能代替阻塞，因为虽然避免切换的开销，但也占用处理器时间，如果持有锁很长的情况下，就会带来性能的浪费
        - 解决方法
          - 超过限定的次数就解锁
            - 即自旋等待的时间必须有一定的限度，如果自旋超过了限定的次数仍然没有成功获得锁，就应当使用传统的方式去挂起线程
            - 默认十次，也可以使用参数-XX：PreBlockSpin来自行更改
        - 自旋锁的优化【自适应自旋】
          - 即 自旋时间不再固定
            - 由前一次在同一个锁上的自旋时间及锁的拥有者的状态来决定
      - 概念
        - 虚拟机即时编译器在运行时，对被检测到不可能存在共享数据竞争的锁进行消除



锁优化

原图链接：[「代码随想录」知识星球]

第13章 线程安全与锁优化 /466

13.3 锁优化

13.3.1 自旋锁与自适应自旋
- 诞生背景
  - 线程的操作都需要转入内核态中完成，这些给Java虚拟机带来许多的影响
  - 【问题】共享数据的锁定状态只会持续很短的一段时间，为了这段时间去挂起和恢复线程并不值得
- 概念解析【上述问题的解决方案】
  - 两个或以上的线程同时并行执行时
  - 让后面请求锁的那个线程“稍等一会”，但不放弃处理器的执行时间
  - 等待持有锁的线程是否短时间内释放锁
- 开启的参数 JDK6之前使用-XX：+UseSpinning参数来开启，后续默认开启
- 注意事项【一个问题】 自旋等待不能代替阻塞，因为虽然避免切换的开销，但也占用处理器时间，如果持有锁很长的情况下，就会带来性能的浪费
- 解决方法 超过限定的次数就解锁
  - 即自旋等待的时间必须有一定的限度，如果自旋超过了限定的次数仍然没有成功获得锁，就应当使用传统的方式去挂起线程
  - 默认十次，也可以使用参数-XX：PreBlockSpin来自行更改
- 自旋锁的优化【自适应自旋】 即 自旋时间不再固定 由前一次在同一个锁上的自旋时间及锁的拥有者的状态来决定

13.3.2 锁消除
- 概念 虚拟机即时编译器在运行时，对被检测到不可能存在共享数据竞争的锁进行消除
- 主要判定依据 来源于逃逸分析的数据支持（11章）
  - 逃逸分析
    - 通俗一点讲，当一个对象的指针被多个方法或线程引用时，我们称这个指针发生了逃逸。
    - 编程语言的编译优化原理中，分析指针动态范围的方法，逃逸分析的基本行为就是分析对象动态作用域
  - 关注的是 当一个对象在方法中被定义之后，他可能被外部方法所引用
  - 分类
    - 方法逃逸 作为调用参数传递到其他方法中
    - 线程逃逸 赋值给类变量或可以在其他线程中访问的实例变量
- 策略 如果判断一段代码位于堆上的所有数据不会逃逸被其他线程访问，则把它们当作栈上数据对待，认为它们是线程私有的，不同步加锁

13.3.3 锁粗化
- 多数情况的原则 将同步块的作用范围限制得尽量小，只在共享数据的实际作用域中才进行同步
- 上述规则的缺陷
  - 如果一系列的连续操作都对同一个对象反复加锁和解锁
  - 频繁地进行互斥同步操作也会导致不必要的性能损耗

13.3.4 轻量级锁
- 目的 在没有多线程竞争的前提下，减少传统的重量级锁使用操作系统互斥量产生的性能消耗 传统的锁机制就被称为“重量级”锁
- 需要提前了解的知识
  - HotSpot虚拟机的对象头构成
    - 第一部分
      - 存储对象自身的运行时数据，如哈希码（HashCode）、GC分代年龄（Generational GC Age）等
      - 数据称为“Mark Word”，在Java虚拟机中占用32个比特（32位系统）或者64个比特（64位系统）大小
      - 也是 实现轻量级锁和偏向锁的关键
    - 第二部分
      - 存储指向方法区对象类型数据的指针
      - 对数组，还有额外部分存储长度
  - Mark word
    - Mark Word被设计成一个非固定的动态数据结构，以便在极小的空间内存储尽量多的信息，可以根据对象的状态复用自己的存储空间
    - 32位系统 表13-1 HotSpot虚拟机对象头Mark Word

锁状态	32bit				
	25bit		4bit	1bit	2bit
	23bit	2bit		偏向模式	标志位
未锁定	对象哈希码		分代年龄	0	01
轻量级锁定	指向栈中锁记录的指针				00
重量级锁定（膨胀后）	指向重量级锁的指针				10
GC 标记	空				11
可偏向	线程 ID	Epoch	分代年龄	1	01

    - 组成，【注意 最后 2bit的 标志位】
  - 一个经验法则 “对于绝大部分的锁，在整个同步周期内都是不存在竞争的”
    - 是 轻量级锁能提升程序同步性能的依据
- 轻量级锁的工作过程【加锁】
  - 【1】
    - 我们简单回顾了对象的内存布局后，接下来就可以介绍轻量级锁的工作过程了，在代码即将进入同步块的时候，如果此同步对象没有被锁定（锁标志位为“01”状态），虚拟机首先将在当前线程的栈帧中建立一个名为锁记录（Lock Record）的空间，用于存储锁对象目前的Mark Word的拷贝（官方为这份拷贝加了一个Displaced前缀，即Displaced Mark Word）。这时候线程堆栈与对象头的状态如图13-3所示。

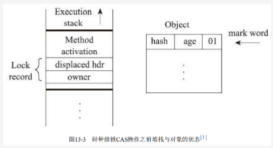


图13-3 轻量级锁CAS操作之前堆栈与对象的状态[1]

  - 【2】
    - 虚拟机将使用CAS操作尝试把对象的Mark Word更新为指向Lock Record的指针。如果这个更新动作成功了，即代表该线程拥有了这个对象的锁，并且对象Mark Word的锁标志位（Mark Word的最后两个比特）将转变为“00”，表示此对象处于轻量级锁定状态

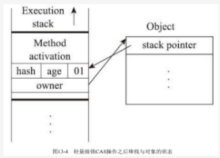
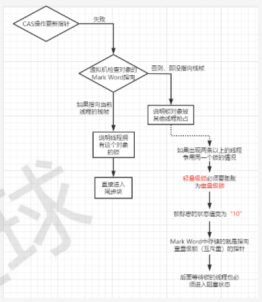


图13-4 轻量级锁CAS操作之后堆栈与对象的状态

这时候线程堆栈与对象头的状态如图13-4所示。

  - 【3】
    - 如果这个更新操作失败了，那就意味着至少存在一条线程与当前线程竞争获取该对象的锁。虚拟机首先会检查对象的Mark Word是否指向当前线程的栈帧，如果是，说明当前线程已经拥有了这个对象的锁，那直接进入同步块继续执行就可以了，否则就说明这个锁对象已经被其他线程抢占了。如果出现两条以上的线程争用同一个锁的情况，那轻量级锁就不再有效，必须要膨胀为重量级锁，锁标志的状态值变为“10”，此时Mark Word中存储的就是指向重量级锁（互斥量）的指针，后面等待锁的线程也必须进入阻塞状态。



即

- 轻量级锁【解锁】过程
  - 同样用 CAS 操作

13.3.5 偏向锁
- 目的 消除数据在无竞争情况下的同步原语，进一步提高程序的运行性能
- 与 轻量锁的 对比
  - 轻量级锁是在无竞争的情况下使用CAS操作去消除同步使用的互斥量
  - 偏向锁就是在无竞争的情况下把整个同步都消除掉，不进行CAS操作
- “偏”的含义 意思是这个锁 偏向于 第一个获得它的线程 如果在接下来的执行过程中，该锁一直没有被其他的线程获取，则持有偏向锁的线程将永远不需要再进行同步
- 原理
  - 虚拟机启用了偏向锁 当锁对象第一次被线程获取时
    - 虚拟机将会把对象头中的标志位设置为“01”、把偏向模式设置为“1”，表示进入偏向模式
    - 同时使用CAS操作把获取到这个锁的线程的ID记录在对象的Mark Word之中
    - 如果CAS操作成功 持有偏向锁的线程以后每次进入这个锁相关的同步块时，虚拟机都可以不再进行任何同步操作（例如加锁、解锁及对Mark Word的更新操作等）
  - 启用参数-XX：+UseBiased Locking
- 偏向模式结束 出现另外一个线程去尝试获取当前锁对象

- 下一步动作 根据锁对象目前是否处于被锁定的状态决定是否撤销偏向（偏向模式设置为“0”），撤销后标志位恢复到未锁定（标志位为“01”）或轻量级锁定（标志位为“00”）的状态

Synchronized关键字

原图链接：[「代码随想录」知识星球]

Java 并发

ThreadLocal

- 简介
  - 解决每一个线程都有自己专属的本地变量，让每个线程绑定自己的值，可以将ThreadLocal 类 比喻成存放数据的盒子，盒子中可以存储每个线程的私有数据
  - 创建了一个 ThreadLocal 变量，那么访问这个变量的每个线程都会有这个变量的本地副本，每个线程可以用 get() 和 set() 方法来获取默认值或者将其值更换为当前线程所存的副本的值，从而避免了线程安全问题
- 举例子
- 原理

Synchronized 关键字

对 Synchronized 关键字的了解

- synchronized 关键字解决的是多个线程之间访问资源的同步性
- synchronized关键字可以保证被它修饰的方法或者代码块在任意时刻只能有一个线程执行。在 Java 1.5 之前中，synchronized 属于重量级锁，效率低下

synchronized 的使用方式

- 修饰实例方法：作用于当前对象实例 this 加锁，进入同步代码前需要先获得该对象实例的锁
  - synchronized void method() {
    //业务代码
    }
- 修饰静态方法：也就是给当前的类的Class 对象加锁，会作用于类的所有对象实例，进入同步代码块之前要获得当前 class 的锁。因为静态成员不属于任何一个实例对象，而是类的成员。
  - 所以如果一个线程 A 调用 一个实例对象的非静态synchronized 方法，而线程 B 需要调用这个实例对象所属类的静态synchronized 方法，并不会发生互斥现象。因为访问静态 synchronized 方法占用的锁是当前类的锁，而访问 非静态synchronized 方法占用的锁是当前实例对象锁
  - synchronized static void method() {
    //业务代码
    }
- 修饰代码块：指定加锁对象，对给定对象/类加锁。synchronized(this|Object) 表示进入同步代码块前要获得给定对象的锁。 synchronized(类.class) 表示进入同步代码块之前要获得当前 class 的锁
  - public void Test(){
    SynchronizedTest test = new SynchronizedTest();
    synchronized (test){
    System.out.println("hello world");
    }
    }

总结：synchronized 关键字加到 static 静态方法和 synchronized(class) 代码块上都是是给 Class 类上锁。synchronized 关键字加到实例方法上是给对象实例上锁。 尽量不要使用 synchronized(String a) 因为 JVM 中，字符串常量池具有缓存功能！

一个不会的面试题：手写单例模式！ 解释一下双重检验锁方式实现单例模式的原理"

构造方法不可以使用 synchronized 关键字修饰，因为构造方法本身就是线程安全的。

底层原理：对象监视器monitor 的获取

- Monitor
  - 。它内置在每一个对象中，任何一个对象都有一个monitor与之关联，synchronized在JVM里的实现就是基于进入和退出monitor来实现的
  - 简单来说可以想象成为一个只能容纳一个对象的房间，然后想要获取对象锁的线程就是想要进入这个房间的客人。
- 修饰同步块 — monitorenter 和 monitorexit 两个字节码指令
  - synchronized 被编译后生成 monitorenter 与 monitorexit 指令，其中 monitorenter 指令指向同步代码块的开始位置，monitorexit 指令则指明同步代码块的结束位置。在 JVM 中实现进入和退出monitor
  - monitorenter，如果当前monitor的进入数为0时，线程就会进入monitor，并且把进入数+1，那么该线程就是monitor的拥有者(owner)。如果该线程已经是monitor的拥有者，又重新进入，则会把进入数再次+1，也就是可重入的。
  - 执行monitorexit 的线程必须是monitor 的拥有者，执行 monitorexit 指令后，monitor 的进入数减一，如果减一后进入数为0 表明锁被释放。其他被阻塞的线程就可以尝试去获取monitor 的所有权 。
- 修饰 方法 — 一个标志位ACC_SYNCHRONIZED
  - 作用就是一旦执行到这个方法时，就会先判断是否有标志位，如果有这个标志位，就会先尝试获取monitor，获取成功才能执行方法，方法执行完成后再释放monitor。在方法执行期间，其他线程都无法获取同一个monitor。

性能问题

- monitor 的下层是依赖操作系统的 Mutex Lock 来实现的。Java 线程事实上是对操作系统线程的映射，所以每次进行挂起或者唤起一个线程都要切换到操作系统的内核态，比较耗费性能

锁机制

- 自旋锁
  - 线程如果没有争取到 monitor 时，就会进入阻塞状态，当持有锁的线程释放了锁之后，才可以再去竞争，这样的话，会消费大量的性能在阻塞和唤醒的切换上（如果线程占用的时间很短，那么切换的时间可能会超出线程执行任务的时间）。
  - 为了避免阻塞和唤醒的切换，在没有获得锁的时候就不进入阻塞，而是不断的循环检测锁是否被释放，这就是自旋。
  - 但是线程一直在循环检测锁的状态，就会占用 CPU资源，如果锁被占用的时间较长，那么自旋的次数就会变多，占用 CPU 时间变长，导致性能变差。
    - 通过参数 -XX: PreBlockSpin 来设置只需自旋的次数，自旋一定的次数后就挂起。
- 自适应自旋锁
  - 自旋次数不固定，而是由前一次在同一个锁上的自旋时间以及锁的拥有者的状态来决定。
  - 如果这一次自旋成功了，便可能下一次也能够成功，于是允许自旋的次数就会更多，反之，自旋的次数就减少。
- 锁消除
  - 在 JVM 编译时，通过对运行时上下文的扫描，去掉不可能存在共享资源竞争的锁，节约无意义的请求时间。比如说：StringBuffer 的 append() 方法
    - public synchronized StringBuffer append(String str) {
      toStringCache = null;
      super.append(str);
      return this;
      }
    - 在实例方法中使用StringBuffer作为局部变量使用 append 方法，此时StringBuffer 不可能存在共享资源竞争的，因此会自动将锁消除
      - public String add(String s1, String s2) {
        //sb属于不可能共享的资源,JVM会自动消除内部的锁
        StringBuffer sb = new StringBuffer();
        sb.append(s1).append(s2);
        return sb.toString();
        }

JDK1.6 之后的优化

- 将多个连续加锁，解锁的操作链接在一起，扩展成为一个范围更大的锁
- 锁粗化
- 锁升级
  - 在 Java 中每个对象都拥有一把锁，这把锁存在于对象头中，记录了当前对象被哪一个线程所占用。对象头中的 Mark Word 就是这个相比
  - 彼此要存在四种状态，依次是：无锁状态、偏向锁状态、轻量级锁状态、重量级锁状态，他们会随着竞争的激烈而逐渐升级。注意锁可以升级不可降级，这种策略是为了提高获得锁和释放锁的效率。
  - 无锁
    - 无需对资源进行任何同步保护，可以给各个线程随意调用
    - 资源会被竞争，但是不使用操作系统同步原语对共享资源进行锁定，而是通过一些其他的机制来控制同步，比如 CAS
  - 偏向锁
    - 为单线程指定下设计，在大多数情况下，锁不仅不存在多线程竞争，而且总是由同一线程多次获得。也就是假设上是多线程的场景，实际上是单线程。
    - Mark Word中，当锁的标志位 为 01，那么判断倒数第二个 bit 是否为 1，为 1 时代表当前对象的锁状态为偏向锁，于是再去读 Mark Word 的前 23 个 bit，也就是线程 ID，通过这个 ID 来判断是否是 同一个线程再次进入，如果是，则不再进行同步操作，省去了大量的锁申请的操作，提高性能
  - 轻量级锁
    - 当有多个线程竞争锁的锁标志位为00。
    - 线程想要获得某个对象的锁，假如看到锁的标志位为 00，那么就知道它是轻量级锁，线程会在自己的虚拟机栈中开辟一块被称为 Lock Record 的空间，用于存放 对象头 的Mark Word 的副本以及Owner 指针。
    - 线程通过 CAS 去获得锁，获得成功
      - 线程会将复制该对象的 Mark Word 到虚拟机栈中的 Lock Record 中 ，并且将 Lock Record 中的Owner 指针指向该对象锁。
      - 对象的 Mark Word 中的前 30 bit 将生成一个指针，指向持有该对象锁的线程的虚拟机栈中的 Lock Record。
    - 其他的线程想要获取这个对象，将会自旋等待，自旋等待的过程完全在用户空间解决，不需要进行系统中断和现场恢复
    - 锁竞争激烈，让线程进入阻塞状态，进入阻塞队列，以减少 cpu 消耗。
  - 重量级锁
    - 一旦有自旋等待的线程数超过 1 个，那么轻量级锁将会升级

在用户空间内去完成锁的获取与释放，避免进入重量级锁的状态，那么将会调用内核空间，产生较大的开销

synchronized 和 ReentrantLock 的区别

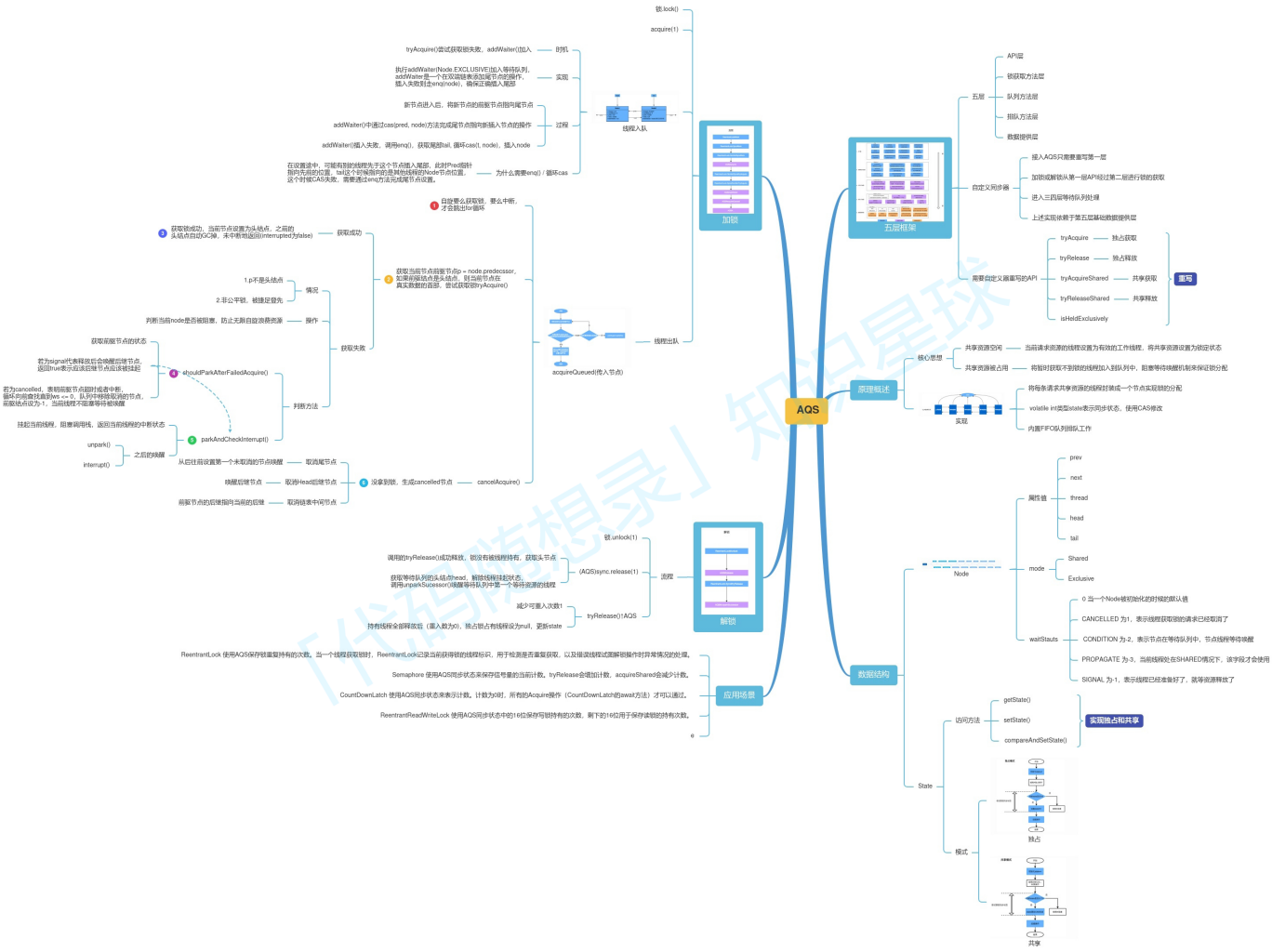
1. 二者都是可重入锁
   - "可重入锁" 指的是自己可以再次获取自己的内部锁。比如一个线程获得了某个对象的锁，此时这个对象的锁还没有释放，当其再次想要获取这个对象的锁的时候还是可以获取的，如果不可锁重入的话，就会造成死锁。同一个线程每次获取锁，锁的计数器都自增 1，所以要等到锁的计数器下降为 0 时才能释放锁
2. synchronized 依赖于 JVM 而 ReentrantLock 依赖于 API
   - synchronized 是依赖于 JVM 实现的，前面我们也讲到了 虚拟机团队在 JDK1.6 为 synchronized 关键字进行了很多优化，但是这些优化都是在虚拟机层面实现的，并没有直接暴露给我们。ReentrantLock 是 JDK 层面实现的（也就是 API 层面，需要 lock() 和 unlock() 方法配合 try/finally 语句块来完成），所以我们可以通过查看它的源代码，来看它是如何实现的。
3. ReentrantLock 比 synchronized 增加了一些高级功能
   - 等待可中断：ReentrantLock提供了一种能够中断等待锁的线程的机制，通过 lock.lockInterruptibly() 来实现这个机制。也就是说正在等待的线程可以选择放弃等待，改为处理其他事情。
   - 可实现公平锁：ReentrantLock可以指定是公平锁还是非公平锁。而synchronized只能是非公平锁。所谓的公平锁就是先等待的线程先获得锁。ReentrantLock默认情况是非公平的，可以通过 ReentrantLock类的ReentrantLock(boolean fair)构造方法来制定是否是公平的。
   - 可实现选择性通知（锁可以绑定多个条件）：synchronized关键字与wait()和notify()/notifyAll()方法相结合可以实现等待/通知机制，ReentrantLock类当然也可以实现，但是需要借助于Condition接口与newCondition()方法。

volatile 关键字

「代码随想录」知识星球

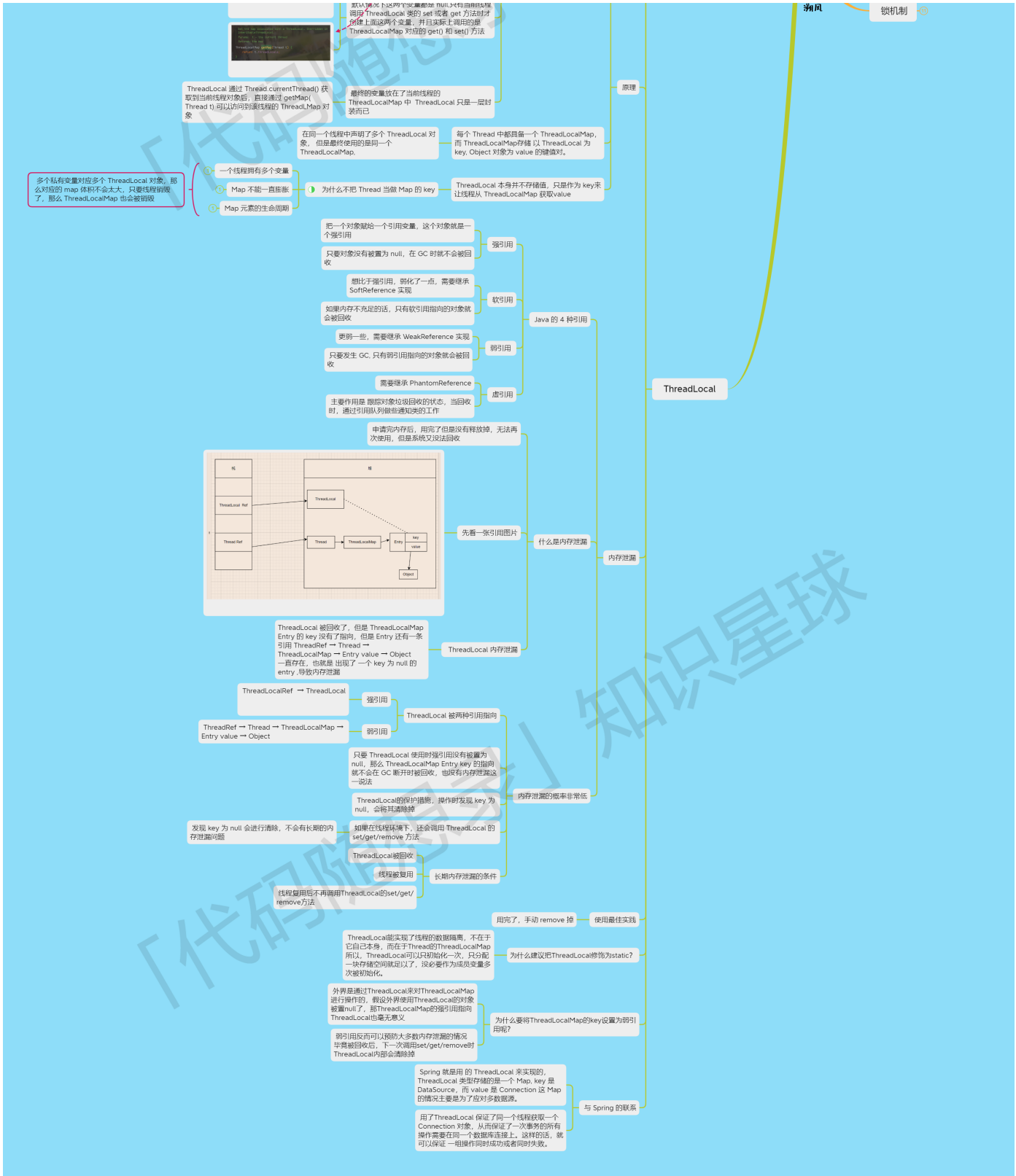
AQS

原图链接：[「代码随想录」知识星球]



ThreadLocal

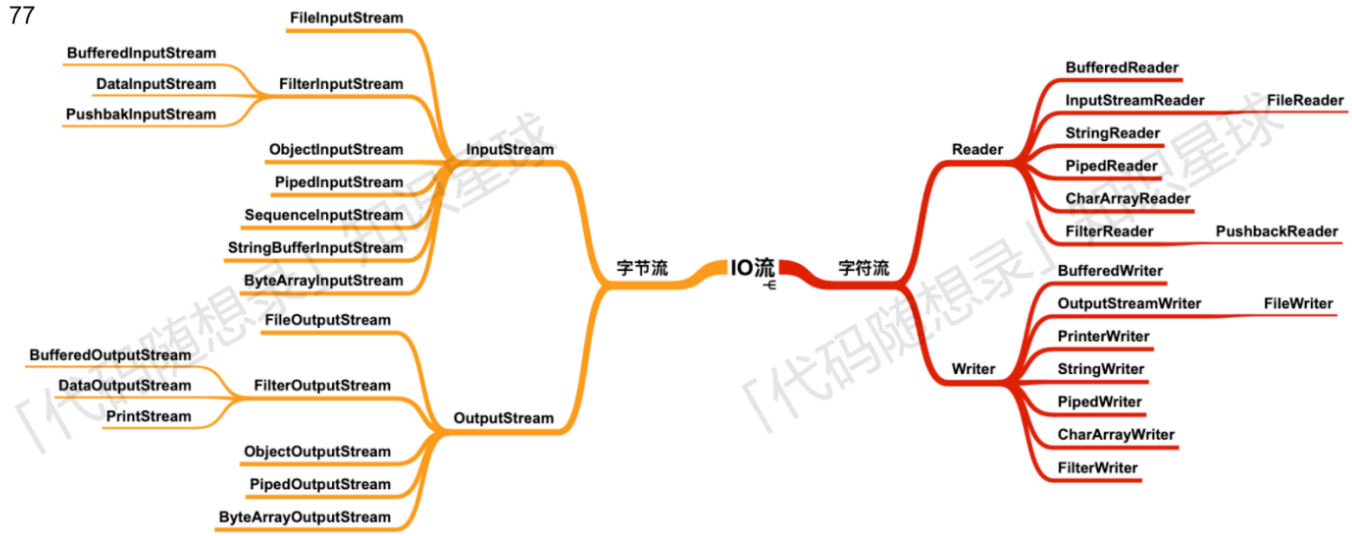
原图链接：[「代码随想录」知识星球]



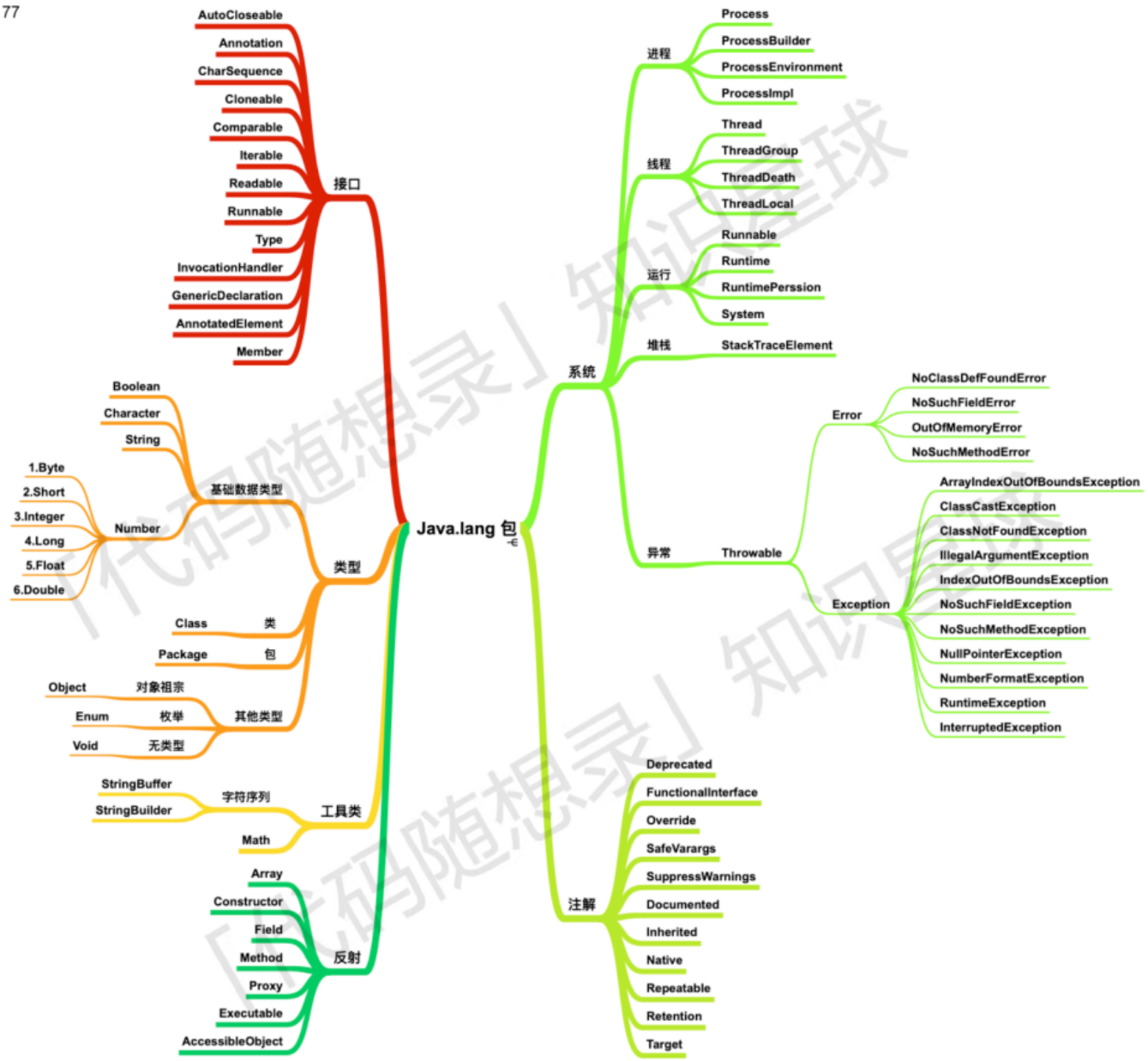
Java工具包

原图链接：[「代码随想录」知识星球]

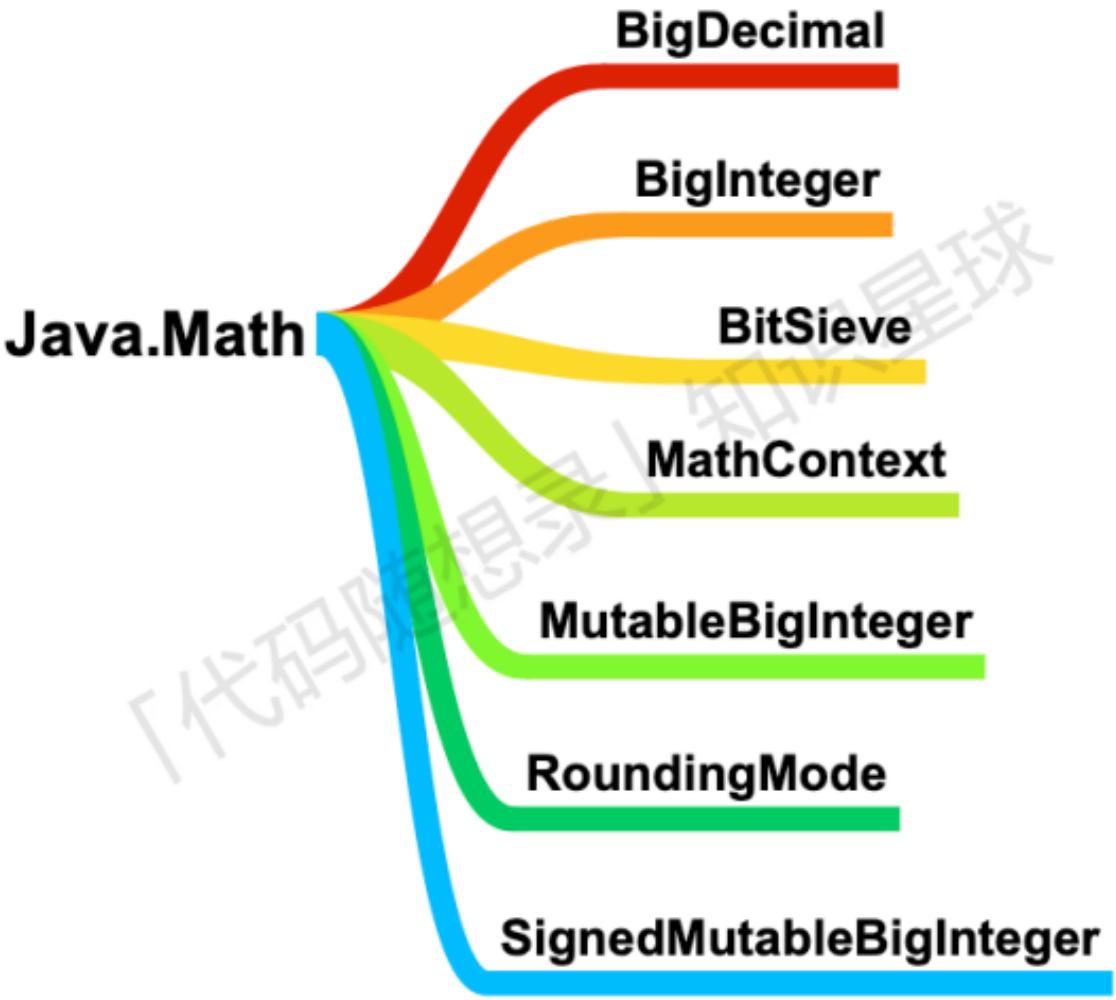
(1) Java.IO



(2) Java.lang

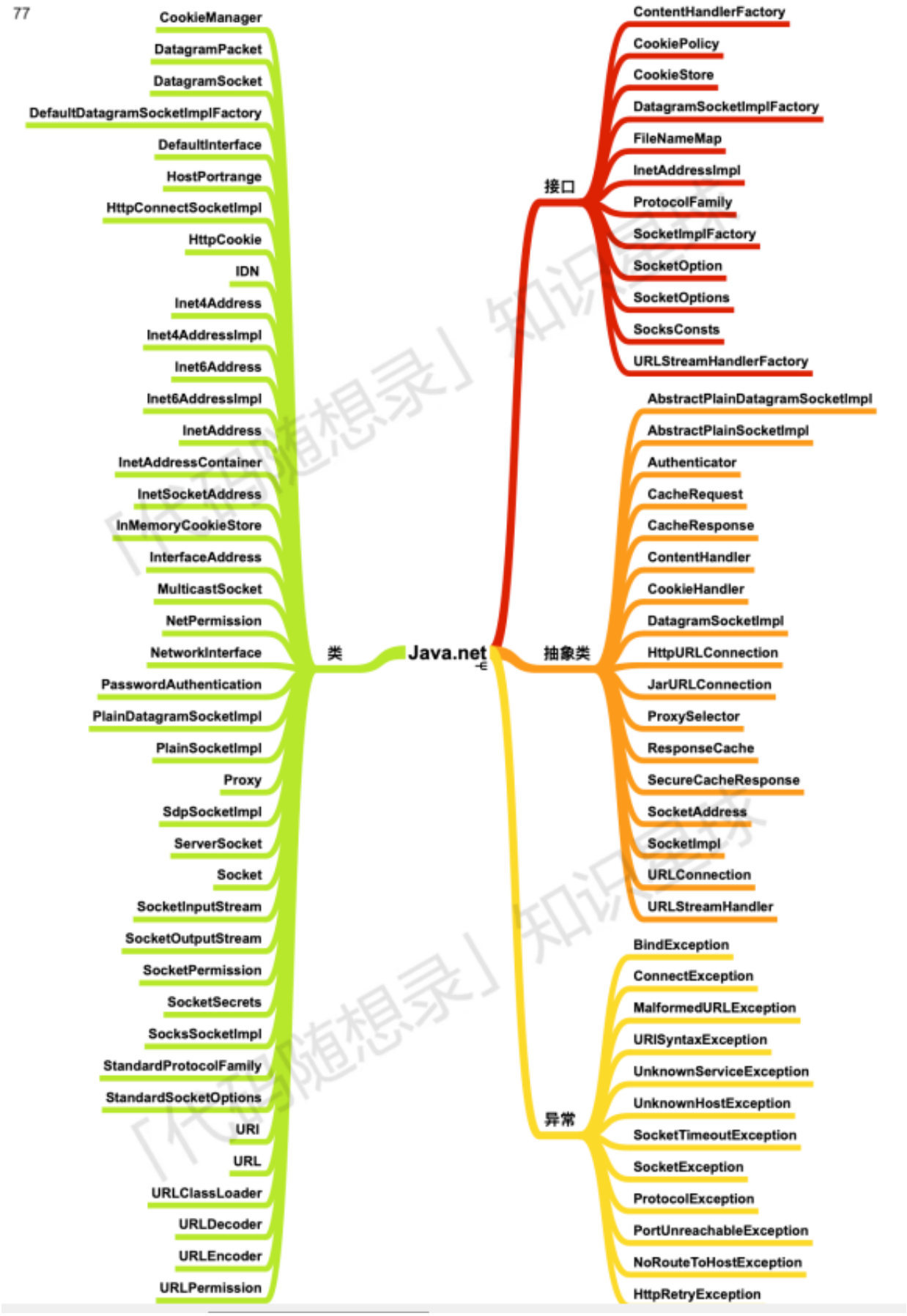


(3) Java.math



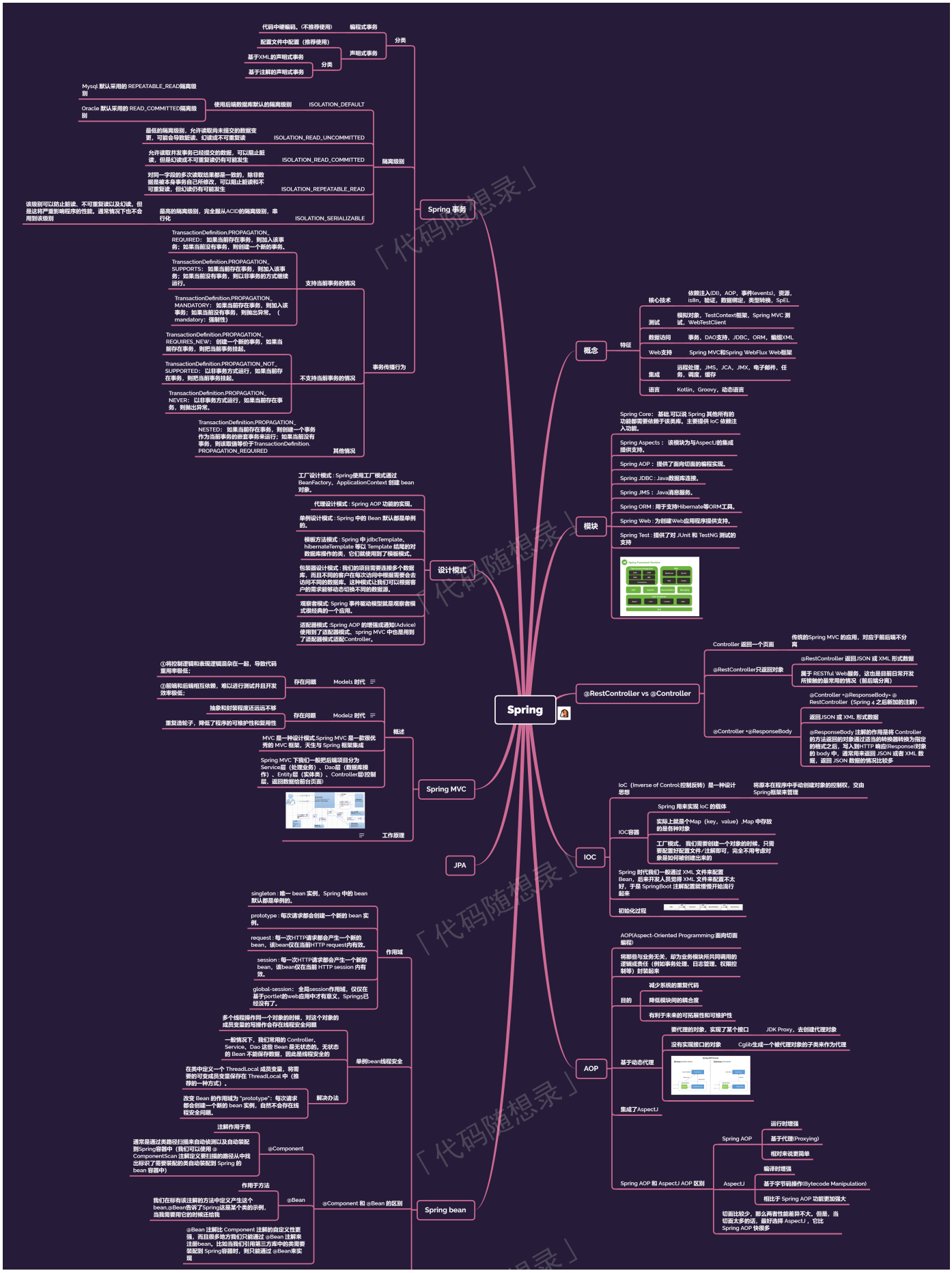
「代码随想录」知识星球

(4) Java.net


Java.net
类
CookieManager
DatagramPacket
DatagramSocket
DefaultDatagramSocketImplFactory
DefaultInterface
HostPortrange
HttpConnectSocketImpl
HttpCookie
IDN
Inet4Address
Inet4AddressImpl
Inet6Address
Inet6AddressImpl
InetAddress
InetAddressContainer
InetSocketAddress
InMemoryCookieStore
InterfaceAddress
MulticastSocket
NetPermission
NetworkInterface
PasswordAuthentication
PlainDatagramSocketImpl
PlainSocketImpl
Proxy
SdpSocketImpl
ServerSocket
Socket
SocketInputStream
SocketOutputStream
SocketPermission
SocketSecrets
SocksSocketImpl
StandardProtocolFamily
StandardSocketOptions
URI
URL
URLClassLoader
URLDecoder
URLEncoder
URLPermission
接口
ContentHandlerFactory
CookiePolicy
CookieStore
DatagramSocketImplFactory
FileNameMap
InetAddressImpl
ProtocolFamily
SocketImplFactory
SocketOption
SocketOptions
SocksConsts
URLStreamHandlerFactory
抽象类
AbstractPlainDatagramSocketImpl
AbstractPlainSocketImpl
Authenticator
CacheRequest
CacheResponse
ContentHandler
CookieHandler
DatagramSocketImpl
HttpURLConnection
JarURLConnection
ProxySelector
ResponseCache
SecureCacheResponse
SocketAddress
SocketImpl
URLConnection
URLStreamHandler
异常
BindException
ConnectException
MalformedURLException
URISyntaxException
UnknownServiceException
UnknownHostException
SocketTimeoutException
SocketException
ProtocolException
PortUnreachableException
NoRouteToHostException
HttpRetryException

Spring

原图链接：[「代码随想录」知识星球]



@Autowired 注解自动装配 bean

@Component：通用的注解，可标注任意类为 Spring 组件。如果一个Bean不知道属于哪个层，可以使用@Component 注解标注

@Repository：对应持久层即 Dao 层，主要用于数据库相关操作

@Service：对应服务层，主要涉及一些复杂的逻辑，需要用到 Dao层

@Controller：对应 Spring MVC 控制层，主要用于接受用户请求并调用 Service 层返回数据给前端页面

类声明为Spring的 bean 的注解

生命周期

「代码随想